



A SYNECHIC AND TOPOKINETIC PROGRAMMING LANGUAGE

Continuity, Motion, and the Design of Jac

JASON MARS

EDITION
1.1.0

A SYNECHIC AND TOPOKINETIC
PROGRAMMING LANGUAGE

Continuity, Motion, and the Design of Jac

A SYNECHIC AND TOPOKINETIC PROGRAMMING LANGUAGE

*Continuity, Motion,
and the Design of Jac*

JASON MARS

Edition 1.1.0 · July 2026

A Synechic and Topokinetic Programming Language
Continuity, Motion, and the Design of Jac

Copyright © 2026 Jason Mars. All rights reserved.

Edition 1.1.0, July 2026. Changes between editions are recorded in Appendix [H](#).
All editions are archived at doi:[10.5281/zenodo.21464040](https://doi.org/10.5281/zenodo.21464040).

Typeset with L^AT_EX (memoir) in Palatino on a 7 in × 10 in trim.
The Jac language and the Jaseci runtime stack are open source:
<https://github.com/jaseci-labs/jac>

ABSTRACT

For seventy years, programming languages have inherited two silent assumptions from the von Neumann tradition. The first is that a program’s semantics end at the edge of its process, so that frontend and backend, managed and native, script and service are separate programs, joined by glue code, marshaling layers, and toolchains the programmer must operate by hand. The second is that computation stands still while data is delivered to it. This book presents Jac, a production language that discards both assumptions, and develops the theory, design, and implementation of the two language classes it defines. The book’s key claim is that both assumptions are conventions rather than necessities, and that discarding them defines two classes of programming language: 1) the *synechic* languages, which present a single continuous semantic medium across the substrates convention treats as separate, and 2) the *topokinetic* languages, in which the locus of computation moves, as a first-class construct, through a topology of data. The two classes are independent, and Jac is the first language that is a member of both.

Jac is a *synechic* language: it presents a single continuous semantic medium across conventionally discrete substrates. One surface syntax compiles directly to Python bytecode, JavaScript, and native machine code, entering each host runtime as a first-class citizen and inheriting the PyPI, npm, and C ecosystems without interop scaffolding. We formalize this property as *substrate transparency*. It generalizes tierless programming from the tiers of a web application to entire foreign ecosystems, and downward through a self-contained toolchain that internalizes compiler, linker, package managers, server, and deployer in a single artifact. The continuity reaches even memory discipline: one checked medium spans garbage collection as Python practices it and ownership with borrowing as Rust enforces it, adopted by degrees and reversible at every step. The runtime completes the continuity claim by presenting a single system image across processes, machines, and users. Programs get scale-invariant semantics from script to planet-scale service without modification, and meaning-typed constructs extend the same continuity to delegation of program logic to large language models.

Jac is a *topokinetic* language: the locus of computation is a first-class, mobile semantic construct. In its Object-Spatial Programming (OSP) paradigm, programs are expressed as walkers traversing a persistent topology of nodes and edges. Computation moves through the shape of the data rather than data streaming to a fixed site of compute. The topology persists by reachability. It serves at once as the program’s data

model and its store, so the database and its mapping layers dissolve into the language. Computation on encounter is declared from either side: a walker states what it does on arrival, and a node states what it does when a walker of a given kind arrives. We give this model a formal semantics, situate it against its ancestors in mobile ambients, actor systems, and agent mobility, and show that a broad class of modern applications is a natural fit for topological expression.

The book draws on the language's evolution across a decade of research and production deployment. It proceeds in six parts, moving from an anatomy of the discontinuities of software, through the theories of synechism and topokinesis and the machinery that realizes them, to empirical evaluation and the design space the two classes open. Jac is the book's existence proof: one production system in which both inherited assumptions are discarded at once. The book is written not just for programming-language researchers and graduate students, but also systems builders, and passionate developers of every kind who want a rigorous account of what programming looks like when the discontinuities are removed.

Keywords: synechic languages · topokinetic languages · Object-Spatial Programming · substrate transparency · single system image · tierless programming · AI-integrated languages

CONTENTS

Abstract	v
Preface	xiii
Acknowledgments	xvii
How to Read This Book	xix
Notation and Conventions	xxiii
The Vocabulary of the Argument	xxv
I THE DISCONTINUITIES OF SOFTWARE	1
1 One Application, Twelve Notations	3
1.1 An Inventory of One Contemporary Application	4
1.2 A Taxonomy of Glue	5
1.3 The Cost of Discontinuity	8
1.4 Two Silent Assumptions of the von Neumann Inheritance	10
1.5 The Theses of This Book	11
1.6 A Note on History	12
II SYNECHISM: A THEORY OF CONTINUITY	15
2 Substrate Transparency	17
2.1 The Classical Transparency Family and Its Ceiling	18
2.2 Substrate Transparency Defined	19
2.3 A Cost Model for Glue	21
2.4 The Limits of Continuity: Lawful Boundaries	23
2.5 Tierlessness as Restricted Synechism	24
2.6 The Stochastic Substrate	25
3 The Design of a Synechic Surface	29

3.1	Supersetting as Strategy	30
3.2	One Syntax, Three Lowerings	31
3.3	Ecosystem Inheritance Without Wrappers	33
3.4	The Discontinuity That Isn't	35
3.5	Language Mechanics of Separation	36
3.6	Design Rules for Synechic Surfaces	37
4	The Toolchain as Hidden Tier	41
4.1	The Build System as a Substrate Boundary in Disguise	42
4.2	Anatomy of the Self-Contained Artifact	43
4.3	Package Management Across Three Ecosystems as One Operation	45
4.4	Building, Serving, and Deploying as Language Operations	47
4.5	Project Kinds: The Product Space of One Program	48
4.6	Reproducibility, Versioning, and Trust in a Fused Binary	50
5	The Memory Discontinuity: Gradual Borrow Checking	53
5.1	Memory Discipline as a Substrate Divide	54
5.2	The Ownership Dial	55
5.3	The Membrane	57
5.4	The Gradual Guarantee for Memory	59
5.5	The Verifiable Endpoint	60
6	The Semantic Discontinuity: Meaning-Typed Programming	63
6.1	The Language Model as Substrate	64
6.2	The Prompt as Glue Code	64
6.3	Delegation Semantics: Implemented by Meaning	65
6.4	Meaning Types and Automatic Prompt Synthesis	67
6.5	Correctness under Stochastic Delegation	69
6.6	The Synechic Medium as Shared Ground	71
III	TOPOKINESIS: A THEORY OF MOBILE COMPUTATION	73
7	Object-Spatial Programming, Informally	75
7.1	Nodes, Edges, and Walkers by Example	76
7.2	Abilities: Computation Triggered by Arrival	77
7.3	Spawning, Visiting, and the Grammar of Traversal	79
7.4	From Control Flow to Path	80
7.5	Three Worked Examples	82
8	The Object-Spatial Calculus	87
8.1	Syntax of the Core Calculus	87
8.2	Operational Semantics of Traversal and Ability Activation	89
8.3	Walker Scheduling, Traversal Order, and Determinism	93
8.4	Encodings and Separations: OSP Against Actors, Ambients, and Vertex-Centric Models	95
8.5	Expressiveness: What Topokinesis Adds, and What It Does Not	96

9	Types and Topology	99
9.1	Typing Nodes, Edges, and Walkers	99
9.2	Edges as Typed Relations	101
9.3	Gradual Typing Against the Python Substrate	104
9.4	Meaning Types Within the Type System	106
10	Persistence by Reachability	109
10.1	The Root Node and the Dissolution of the Database Discontinuity . . .	109
10.2	Persistence as a Topological Property, Not an I/O Operation	111
10.3	Consistency and Transactions over a Living Graph	113
10.4	Schema Evolution: Migrating Topologies That Never Shut Down . . .	115
11	Transience by Ownership: The Region Is the Subgraph	119
11.1	The Unit of Death	120
11.2	Handles, Opens, and the Growth Rule	121
11.3	The Second Membrane	122
11.4	Sendable Subgraphs	123
11.5	The Measured Alignment	124
IV	THE MACHINERY	127
12	The Polypiler	129
12.1	Compilation, Transpilation, Polypilation	129
12.2	Pipeline Architecture	132
12.3	Supersetting Python at the Grammar and AST Level	134
12.4	Program Partitioning: Deciding What Runs Where	135
12.5	Lowering to Python Bytecode	138
12.6	Lowering to JavaScript and WebAssembly	138
12.7	Native Lowering through LLVM and Static Linkage	139
12.8	The Design Space of Polypilation	140
13	The Runtime Core	145
13.1	The Runtime as Substrate Mediator	146
13.2	Graph Memory	148
13.3	The Walker Execution Engine	150
13.4	The Persistence Engine	153
14	Scale Invariance	157
14.1	Definition: Invariance under Deployment-Scale Transformation	157
14.2	From Script to Service: Walkers as Projected Interfaces	158
14.3	User Multiplicity as Root Multiplicity	161
14.4	Distribution: Sharding the Topology, Moving the Walker	162
14.5	The Physical Limits: Latency, Failure, and Cost	165
14.6	Deployment Absorbed	166

V	EVALUATION AND PRAXIS	169
15	Case Studies	171
15.1	A Full-Stack Social Application: One Codebase, Zero Glue, Measured .	171
15.2	A Conversational AI Product in Production	175
15.3	A Native-Compiled Game Delivered as WebAssembly	177
15.4	A Four-Tier Showcase and Its Production Twin	179
15.5	Longitudinal Notes from a Decade of Deployments	181
16	Quantitative Evaluation	183
16.1	Glue Elimination: Lines, Artifacts, and Dependency Surface	184
16.2	Boundary Experiments on the Showcase Pair	187
16.3	Performance: Traversal, Lowering, Service, and Scale	189
16.4	Developer Studies: Time to First Application and Defect Rates	193
16.5	Threats to Validity	195
17	Objections and Limits	197
17.1	The Lock-In Objection: What It Costs to Leave	197
17.2	Debugging Dissolved Boundaries	199
17.3	The Leaky-Abstraction Objection, Steelmanned	201
17.4	Performance Ceilings and the Sufficiently Smart Runtime	202
17.5	When Not to Use Jac	203
VI	THE LANGUAGE CLASSES	205
18	A Taxonomy of Synechic and Topokinetic Languages	207
18.1	The Two Axes as a Design Space	207
18.2	Necessary and Sufficient Machinery for Membership	211
18.3	Design Principles for Successors	213
18.4	Open Problems: Formal, Systems, and Human	215
19	Coda: Continuity as a Design Ethic	219
19.1	Peirce’s Synechism and the Metaphysics of the Unbroken	220
19.2	Discontinuity and the Economics of Software in the Era of AI Authorship	221
19.3	What Programming Looks Like When the Discontinuities Are Gone . .	222
	Irrational not to	227
A	The Jac Language Reference	229
A.1	Lexical Structure and Module Layout	229
A.2	Declarations	231
A.3	Object-Spatial Constructs	235
A.4	Statements and Expressions	237
A.5	The Client Tier and Python Interoperation	240
B	The Object-Spatial Calculus: Rules and Proofs	243
B.1	The Full Syntax	243

B.2	The Complete Small-Step System	244
B.3	The Type System	248
B.4	Metatheory	250
B.5	What Is Proved and What Is Conjectured	256
C	Glossary of Terminology	257
D	A Historical Timeline of Jaseci and Jac	267
D.1	The Jaseci Runtime Era, circa 2019 to 2023	267
D.2	The jaclang Rewrite, circa 2023 to 2024	268
D.3	The Meaning-Typed Era, circa 2024 to 2025	268
D.4	The Fused-Binary Era, 2025 to the Present	269
D.5	Reading the Record	270
E	Artifact and Reproducibility Guide	273
E.1	Obtaining the Toolchain	273
E.2	Running the Case Studies	274
E.3	Reproducing the Measurements	275
E.4	Version Pinning and Where the Artifacts Live	276
F	A History of Attempts at Continuity	279
F.1	Portability by Imposition	280
F.2	Portability by Inversion	281
F.3	Tierless and Multitier Programming	281
F.4	Distribution Transparencies	283
F.5	Single System Images at the Operating System Layer	284
F.6	The Partial-Continuity Trap	284
G	A History of Mobile Computation	289
G.1	Backus’s Turing Lecture and the von Neumann Bottleneck	290
G.2	Actors, CSP, and Message Passing	291
G.3	Process Calculi with Location	291
G.4	Mobile Agents: The Rise and Fall of Traveling Computation	292
G.5	Thinking Like a Vertex: Traversal as Trapped Topokinesis	294
G.6	The Missing Synthesis	295
H	Revision History	297
H.1	Edition 1.1.0 (July 2026)	297
H.2	Edition 1.0.0 (July 2026)	299
	Bibliography	301

PREFACE

This book argues that two of the oldest habits in software construction are conventions rather than necessities, and it presents a production language in which both are discarded. The first habit is to assemble an application from disjoint programs joined by glue. The second is to hold computation still while data is carried to it. The chapters that follow make that argument with definitions, semantics, machinery, and measurement. A preface owes the reader something different: an account of where the ideas came from, and of why the moment to press the argument is now.

For years my research had run at the systems level rather than the language level. Around the end of 2019 the idea for Jac and Jaseci arrived, and I began building it as a hobby project. Collaborators joined over time. The inquiry started from a blunt observation. A modern application of even modest ambition demands a dozen technologies: a frontend framework, a backend framework, a database, a cache, a queue, an API layer, a build system, a deployment apparatus. Each has its own conceptual model, and each is joined to its neighbors by code that exists only to cross a boundary. The original Jaseci stack (Mars 2022; Mars et al. 2023) was a first answer, a runtime that absorbed the scale-out machinery of production applications beneath a programming model in which computation traverses a graph of persistent data. It was built to make production AI applications fast to stand up, and it did that work in the field. In retrospect its deeper contribution was diagnostic. Building and operating it taught us which boundaries in the conventional stack were essential and which were merely inherited.

Two of the stack’s ideas outgrew their origins as systems conveniences and became language semantics. The first is the walker. When data lives in a topology, the natural unit of computation is a mobile locus that moves through that topology, carrying its state with it. Formalizing that observation produced Object-Spatial Programming (Mars 2025), the paradigm developed in Part III of this book. The second is the meaning type. Delegating a function’s implementation to a large language model is a typing and semantics problem, not a prompt-engineering problem, and that observation matured into Meaning-Typed Programming (Dantanarayana et al. 2025). Around these two ideas the language itself was rebuilt: a superset of Python, with additional lowerings to the web substrate and to native machine code, and with the compiler, package managers, type checker, server, and deployer fused into a single self-contained artifact. What began as a research stack roughly a decade ago is now a language with commercial deployments, including production conversational AI

systems, developed in the open by a community that no longer fits in one room.

The harder question is why the ideas demand a book now. For most of the period this book surveys, the discontinuities of software were a tolerable tax. Glue code was tedious and boundary defects were real, but human engineers paid the cost in human time. They carried in their heads the correspondences that the discontinuities severed: this JSON field is that database column, this endpoint is that client stub. Two developments have changed the ledger, and together they define what I call the AI construction era. The first is that the language model has become a component inside programs, an executor to which logic is delegated, joined to the rest of the program today by prompt strings, which are glue code in its purest form. The second, and the more consequential for this book's urgency, is that machines now write a growing share of software. A coding agent amplifies whatever its medium makes cheap. In a continuous medium it amplifies working systems. In a discontinuous one it amplifies the defects that were always concentrated at the discontinuities, now produced at machine speed and reviewed by fewer human eyes. The tacit correspondence knowledge that let human engineers hold a fragmented stack together resides in no model's context. The durable remedy is not better recall but fewer discontinuities: a medium in which interface, logic, data, and deployment are one legible whole for human and machine authors alike. Discontinuities that were tolerable when humans wrote software are untenable when machines do.

The vocabulary of the book is deliberate. *Synechism* is Peirce's name for the doctrine that insists on continuity as of prime importance (Peirce 1892). I borrow it because the book's first thesis is precisely an insistence on continuity, engineered rather than metaphysical. A language is *synechic* when it presents one continuous semantic medium across conventionally discrete substrates. It is *topokinetic* when the locus of computation moves, as a first-class construct, through a topology of data. The stationary-computation habit that topokinesis inverts is as old as the stored-program machine, and the discomfort with it is nearly as old. Backus gave that discomfort its canonical statement (Backus 1978). I have chosen throughout to name classes rather than promote a product. Each adjective names a class of language, not a feature of one design, and the classes are independent: a language can be *synechic* and hold its computation still, or *topokinetic* inside a discontinuous stack. Jac matters in these pages as evidence that one production system can be a member of both classes at once, and the ambition of the book is a pair of language classes of which Jac is the first member, not the last. Precision about the claims includes precision about their limits. The places where discontinuities lawfully resurface, the costs of the approach, and the arguments against it are treated at length, and Chapter 17 is given entirely to the objections I find strongest.

A word on what this book is not. It is not a manual. The complete grammar appears in Appendix A, and the language's reference documentation and playground live online, but the chapters are organized around an argument, not around features. It is written for programming-language researchers who want the definitions and the semantics, for systems builders who want the engineering record, and for practitioners and graduate students who want to understand what programming looks like when the discontinuities are removed. The language itself is a living artifact and will continue to move beneath the prose. For that reason the book's empirical claims are

pinned to a specific open-source version of the system, and Appendix [E](#) describes how to obtain, run, and verify it.

Finally, acknowledgments. A book about a language is unavoidably a book about the community that built the language. Jac and the Jaseci stack are the work of the Jaseci open-source community: the contributors who wrote and reviewed compiler passes and runtime internals, the maintainers of the documentation and tooling, the researchers who sharpened the paradigm in papers and seminar rooms, the students who learned the language when it was rough and left it less so, and the production adopters whose deployments tested every claim in this book long before I wrote any of them down. My debt to them runs through every chapter. The errors that remain, in the prose and in the argument, are mine.

ACKNOWLEDGMENTS

To Lingjia, my wonderful wife. Without her, none of this work is possible. She is my biggest intellectual skeptic and my greatest champion, both at once, and every idea in this book met her doubt before it met a reader. Without her, my ideas are coal. With her, they become diamonds. And it has been a eudaimonic indulgence to create, together, our two little SSNs and our five-and-counting EINs.

And to Skylar and Xayden, who watch me work (and play) every day. You think of me as one who inspires you guys. Little do you know that you are the ones that indeed inspire me. Every time I look into your eyes, I know I have to do my best. This book, and the journey it represents, is one such piece. Kids, this is what it looks like. This is a worthy struggle. This is a pain that fulfills.

HOW TO READ THIS BOOK

This book is a single argument developed across six parts, and the front-to-back order is the intended reading. Few readers arrive with identical purposes, and the parts are engineered, like the language they describe, to admit more than one path through them. This chapter maps the dependencies among the chapters and marks out three routes, one for each of the audiences named in the preface.

The Shape of the Argument

Part I is diagnostic. Chapter 1 performs an anatomy of a contemporary application, defines the discontinuity that the rest of the book answers, and states the book's two theses informally. The prior literature of portability, tierlessness, distribution, and mobile computation is a record of partial attempts at what this book pursues whole, and it is surveyed in the historical appendices, Appendix F and Appendix G. Part II develops synechism, the theory of continuity: Chapter 2 gives the central definition, and Chapter 3, Chapter 4, Chapter 5, and Chapter 6 carry it across the language surface, the toolchain, the memory-discipline divide, and the stochastic substrate of the language model. Part III develops topokinesis, the theory of mobile computation, moving from the informal (Chapter 7) to the formal (Chapter 8 and Chapter 9) to persistence by reachability (Chapter 10) and its transient complement, the region discipline of Chapter 11. Part IV is the engineering record: compiler, runtime, and scale invariance. Part V evaluates the claims against production experience and states the strongest objections, and Part VI generalizes from Jac to the two language classes it inhabits.

The dependencies are few but real (Figure 1). Chapter 1 is assumed everywhere and is the one chapter no reader should skip. Within Part II, Chapter 2 supplies the definitions on which the rest of the part rests and on which Chapter 14 later draws. Appendix F enriches it but is not prerequisite to it. Within Part III, Chapter 7 is the informal foundation for everything after it, while the calculus of Chapter 8 is strictly required only by Chapter 9 and Appendix B. A reader who defers it loses rigor, not plot. Chapter 10 needs only Chapter 7, and Chapter 11 needs Chapter 7 together with the ownership surface of Chapter 5. Part IV presumes the vocabulary of Parts II and III but is readable with Chapter 3 and Chapter 7 alone, with Chapter 14 additionally leaning on Chapter 10. Parts V and VI presume the whole, though the case studies of Chapter 15 are accessible to anyone who has read Chapter 7.

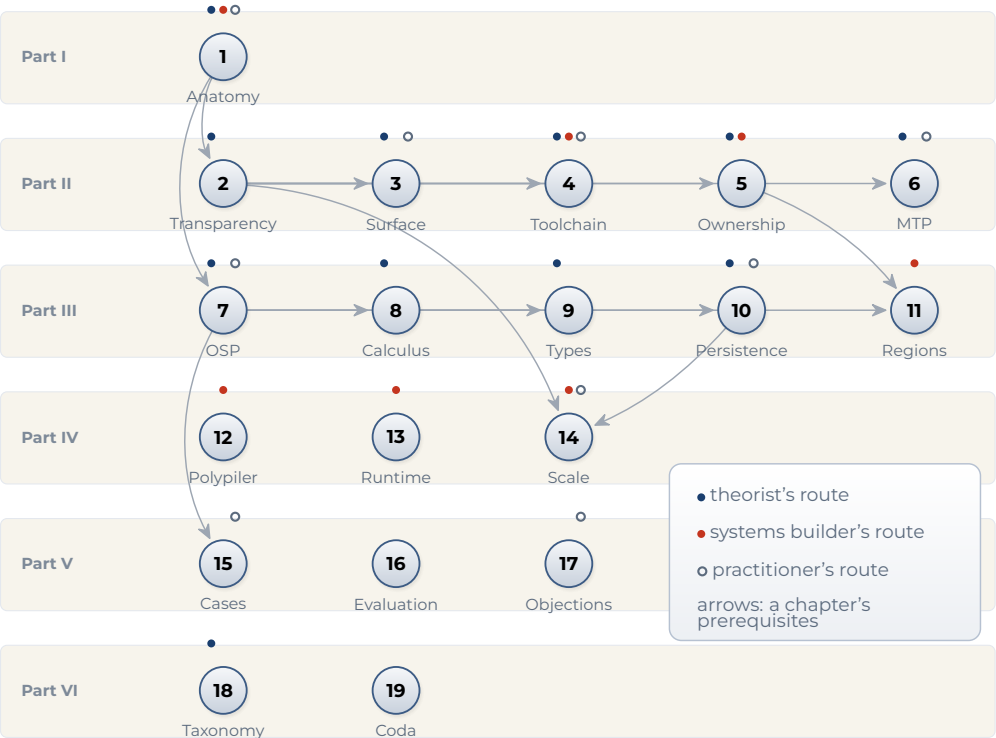


Figure 1: The chapters, their few real dependencies, and the three reading routes. Chapter 1 is assumed everywhere; Parts V and VI presume the whole; the historical appendices enrich but never gate. A dot above a chapter marks it as part of that route.

Three Paths

The programming-language theorist should read Chapter 1 for the theses, then Parts II and III in order (Chapter 2 through Chapter 10), where the definitions, the calculus, and the type system live. From there this reader turns to Appendix A and Appendix B for the complete grammar and the full rules and proofs. Chapter 18 is this reader's payoff: the two classes assembled into a design space with open problems attached. The formal core of the book is the sequence formed by Chapter 2, Chapter 8, and Chapter 9.

The systems builder should read Chapter 1, then the historical appendices Appendix F and Appendix G, which are, at bottom, systems history. From there this reader proceeds directly to Part IV: the polypiler in Chapter 12, the runtime core in Chapter 13, and scale invariance in Chapter 14. We recommend Chapter 4, Chapter 5, and Chapter 11 en route, since the self-contained toolchain and the two memory disciplines are where language design and systems design meet most visibly. This reader can return to the theory of Parts II and III as interest dictates, treating it as the specification that Part IV implements.

The practitioner and the student should read Chapter 1 for the problem, with Appendix F to deepen its history. They should then read Chapter 3, Chapter 4,

Chapter 6, and Chapter 7 for the language as it is written and used, Chapter 10 for the model of data that never touches a database, and Chapter 14 and Chapter 15 for what happens between the laptop and production. Chapter 17 completes this path: knowing a tool's limits is part of knowing the tool. This route yields a working command of the language and its rationale while deferring the formal chapters entirely, and it pairs naturally with the online playground described in Appendix E.

The Appendices

The seven appendices are reference strata, not a seventh part. Appendix A gives the complete grammar of the language. Appendix B gives the object-spatial calculus in full, including the proofs elided from Chapter 8. Appendix C collects every defined term in the book with its definition and etymology, and it is the arbiter whenever a terminological question arises. Appendix D records the historical development of Jaseci and Jac. Appendix E is the artifact and reproducibility guide, connecting the book's claims to the open-source implementation and the online companion. Appendix F and Appendix G carry the full historical surveys, of the pursuit of continuity and of mobile computation respectively, summarized at the close of Chapter 1. Readers who want the conventions governing code, mathematics, and citation before starting should read the chapter on notation and conventions that follows this one. Everyone else may treat it as a reference and begin with Chapter 1.

NOTATION AND CONVENTIONS

This chapter records the conventions used throughout the book. It is meant to be consulted rather than studied, and the details below are also restated where they first matter.

Terminology

Coined terms are set in italics at first use. The book has six central coinages: *synechic*, *substrate transparency*, *topokinetic*, *Object-Spatial Programming*, *scale invariance*, and *meaning types*. Each receives exactly one canonical definition, set in a shaded definition box in the chapter that introduces it formally, and Appendix C collects all of them. The register of the coinages is kept strict. *Synechism* names the doctrine of continuity, *synechic* is the property adjective applied to languages, and *synechistic* is reserved for the design ethic taken up in Chapter 19. Likewise *topokinesis* names the phenomenon, computation in motion over a topology of data, and *topokinetic* is the corresponding adjective. The prose is first-person plural and present tense, and the language under study is named Jac.

Code

Inline code is set in a monospaced face: `walker`, `visit`, `root`, and identifiers drawn from real programs, such as `to_profile_view`, appear exactly as they do in source. Displayed code is set in numbered listings with the caption beneath, numbered within each chapter and cross-referenced by number. Jac listings are real code, not pseudocode: statements end with semicolons, blocks are delimited by braces, and every excerpt is drawn from or checked against the implementation version documented in Appendix E. Where a fragment is deliberately schematic, the surrounding prose says so. Listings in Python, shell commands, and plain text also appear and are identifiable from context and caption. A representative Jac listing follows.

```
node Profile {
  has username: str = "";
}

walker greet {
  can start with Root entry {
```

```

    visit [-->[?:Profile]];
  }
  can hello with Profile entry {
    print("hello " + here.username);
  }
}

```

Listing 1: A minimal object-spatial program, shown here to illustrate the display conventions for Jac code.

Formal Notation

The formal development is concentrated in Chapter 8, with full rules and proofs in Appendix B. Other chapters borrow its notation only lightly. We write $\mathcal{G}$ for a topology, $\mathcal{N}$ and $\mathcal{E}$ for its nodes and edges, and $\mathcal{W}$ for a collection of walkers. A configuration $\langle s \mid \mathcal{G} \rangle$ pairs a program state s with the topology it inhabits, and execution is a small-step relation written $\longrightarrow$, so that a single step of traversal or ability activation takes the form $\langle s \mid \mathcal{G} \rangle \longrightarrow \langle s' \mid \mathcal{G}' \rangle$. Inference rules are displayed in the standard form, premises above a horizontal line and conclusion below. Metavariable conventions, and the name of the core calculus itself, are fixed at the point of use in Chapter 8.

Definitions, theorems, lemmas, propositions, claims, principles, examples, and remarks share a single counter numbered within each chapter, so Definition 4.1 may be followed directly by Theorem 4.2. The shared counter makes any numbered statement easy to locate. The shaded boxes that carry the canonical definitions of the six coinages sit outside this numbering and are referenced by the term they define.

Citations and Cross-References

Citations follow author-year style. A work cited in subject position appears as Backus (1978), while a parenthetical citation appears at the close of a claim (Waldo et al. 1994). Every chapter of the main text ends with an unnumbered section of Bibliographic Notes that weaves the chapter’s sources into a guided-reading narrative. The consolidated bibliography appears at the back of the book. Cross-references name the unit they point to, whether a chapter, an appendix, a listing, or a table, and chapters are always referenced as wholes when cited from other chapters, so that each chapter remains readable on its own terms.

THE VOCABULARY OF THE ARGUMENT

This book introduces vocabulary as it introduces claims, and a reader deserves both in one place before either begins. This chapter lists the concepts the book coins or repurposes, one line apiece, grouped in the order the argument builds them, each with the location that carries its formal statement, motivation, and caveats. A vermillion star ★ marks the nine terms coined in this book. The full alphabetical ledger, with etymologies and the supporting vocabulary of the Jac language, is Appendix C, and the defining chapters remain the arbiters; nothing here supersedes them.

— THE DIAGNOSIS

DISCONTINUITY A boundary at which the representation of meaning must change and over which no verifier has jurisdiction; the site at which glue accrues and defects pool. → Definition 1.1

GLUE The marginal expression a boundary crossing demands: the program text required to realize a call, data flow, or import across substrates, less what the same edge would cost were its endpoints co-located. → Chapters 1 and 2

BOUNDARY CROSSING An edge in a program's reference structure whose two endpoints are assigned to different substrates; the unit at which glue is counted. → Chapter 2

SUBSTRATE A triple of runtime, ecosystem, and toolchain: the executor that gives artifacts meaning, the libraries importable at it, and the apparatus that builds and ships for it. → Definition 2.1

JURISDICTION The reach of a verifier: the set of program points and crossings a compiler, type checker, or other static instrument can examine and reject. → Chapters 2 and 19

LAWFUL BOUNDARY An essential boundary (latency, partial failure, cost, stochasticity) that a language must surface as declared, typed semantics rather than dissolve. → Chapter 2

— SYNECHISM: THE THEORY OF CONTINUITY

SYNECHISM The theory developed in Part II: substrate transparency, the cost model

for glue, and the lawful-boundaries principle, taken together as an account of continuity across substrates. → Chapter 2

SYNECHIC ♦ The property of a language that presents a single continuous semantic medium across conventionally discrete ecosystems, tiers, and toolchains, with no glue at any crossing. → Chapter 2

SUBSTRATE TRANSPARENCY ♦ The property that the identity of the runtime, ecosystem, and toolchain at any program point has no bearing on how the program is expressed. → Chapter 2

GRADUAL BORROW CHECKING ♦ Memory discipline as a continuum within one language: managed semantics by default, ownership adoptable one declaration at a time, and a checked boundary mediating every crossing between the regimes. → Chapter 5

OWNERSHIP DIAL The four-position surface through which gradual borrow checking is adopted, per module: managed, annotated, enforced, headerless, with guarantees strengthening monotonically. → Chapter 5

MEMBRANE The closed boundary between the owned and managed memory regimes, admitting exactly three crossings: sealing, reboxing, and exceptional abort. → Chapter 5

MEANING TYPES ♦ Semantic annotations from which prompts are automatically synthesized, making delegation of program logic to large language models (by `llm()`) a typed language feature rather than string engineering. → Chapter 6

— TOPOKINESIS: THE THEORY OF MOTION

TOPOKINESIS Motion of the locus of computation over a topology of data; the noun names the phenomenon, the adjective the language property. → Chapter 7

TOPOKINETIC ♦ The property of a language in which the mobile locus of computation over a topology of data is a first-class semantic construct, inverting the convention of streaming data to a fixed site of compute. → Chapter 7

OBJECT-SPATIAL PROGRAMMING (OSP) ♦ The paradigm realizing topokinesis: programs expressed as walkers traversing a persistent topology of nodes and edges, with abilities triggered by arrival. → Chapters 7 and 8

WALKER The mobile locus of computation: an archetype carrying state, declaring location-triggered abilities, spawned onto a node and moved by `visit`. → Chapter 7

NODE The archetype declaring an object that occupies a location in a topology: a place that knows when it is visited and can react. → Chapter 7

EDGE The archetype declaring a first-class, typed relationship between two nodes, with data fields and abilities of its own. → Chapter 7

ABILITY The unit of dispatch: a block of computation bound to an event of arrival or departure rather than to a caller's invocation. → Chapters 7 and 8

TOPOLOGY The live graph of typed nodes and edges over which walkers travel: at once the program's data model and its store. → Chapter 7

ROOT NODE The distinguished node anchoring every topology; reachability from it confers persistence, and each served user is issued a root of their own. → Chapters 7 and 10

PERSISTENCE BY REACHABILITY The persistence rule of the topology: a datum is durable exactly while a path connects it to a root, the same reachability a tracing collector consults for liveness. → Chapter 10

TOPOLOGY-ALIGNED REGIONS ♦ Reclamation as a property of place: an ownable handle owns an arena, construction under its open and traversal through its residents grow it, and the handle's death reclaims the whole subgraph at once. → Chapter 11

GROWTH RULE The placement rule of topology-aligned regions: an allocation inside an ability anchors to the region of the node being visited, so a walker grows the region it stands in. → Chapter 11

BORROW-COLLAPSE The escape discipline of the region boundary: a region-rooted value flowing to a binding that cannot outlive the handle is registered as a shared borrow of the handle, and the borrow law does the rest. → Chapter 11

SENDABLE SUBGRAPH A region whose handle has no live borrows; provably without external references, it may cross a concurrency boundary as a constant-time move of the handle. → Chapter 11

— THE MACHINERY AND THE CLASSES

POLYPILER ♦ A compiler whose unit of compilation is the whole polyglot application, whose targets are ecosystems rather than instruction sets, and whose optimization surface includes the boundaries between them; *polypilation* names the process. → Chapter 12

SELF-HOSTING FRONTIER The division of the toolchain artifact into components written in Jac and external engines carried in binary form for feature completeness; the design goal is for every component above the operating system to cross to the Jac side. → Chapter 4

SCALE INVARIANCE ♦ The property that program semantics are invariant under deployment-scale transformation: one user to N users, one machine to M machines, transient to persistent. → Chapter 14

SINGLE SYSTEM IMAGE The presentation of a collection of processes, machines, and users as one continuous machine: the same semantics at service scale as in a single-process script. → Chapter 14

ZERO-RC COMPILATION The native compilation mode whose emitted artifact contains no reference counting and no collector, with the absence machine-checked rather than claimed. → Chapters 5 and 12

COMPOSITION THESIS The claim that the synechic and topokinetic classes are independent in definition and compounding in value: the deepest dissolutions require a member of both at once. → Chapters 1 and 18

PART I



THE DISCONTINUITIES OF SOFTWARE

ONE APPLICATION, TWELVE NOTATIONS: AN ANATOMY OF BEST PRACTICE

We should have some ways of coupling programs like garden hose—screw in another segment when it becomes necessary to massage data in another way.

M. D. McIlroy, 1964

Modern software has fragmented into a patchwork of languages, package ecosystems, and configuration dialects. This chapter counts that patchwork and measures what it costs. The book advances two claims about programming languages, and this chapter lays the ground for both. The first claim is that the discontinuities of modern software are contingent: they are not laws of computation. The second is that they are artifacts of two assumptions we inherited from the earliest stored-program machines and stopped noticing. Neither claim can be earned until the discontinuities have been seen plainly, counted, and measured. That is the work of this chapter, and we do it by analyzing one unremarkable application.

The example is deliberately ordinary. We do not analyze a pathological legacy system or a strawman assembled to lose the argument. We analyze the stack that a competent team, following current best practice, would assemble today for a modest product: a single-page web frontend, a Python API backend, a relational database, a cache, a task queue, a large-language-model feature, containers, continuous integration, and infrastructure-as-code. Every component in the example is the recommended choice of its category. The pathology is not a failure to follow best practice. It *is* best practice.

The chapter moves from observation to diagnosis. We first inventory the application and count what a developer must hold in mind. We then classify the glue into six

Table 1.1: The inventory of one contemporary application. Each layer is the mainstream choice of its category. Each brings its own notation, and most bring their own package ecosystem.

Layer	Representative choice	Notations introduced
Frontend	React with TypeScript	TypeScript, JSX, CSS
Backend API	FastAPI on CPython	Python
Relational store	PostgreSQL	SQL, migration scripts
Cache	Redis	client calls, server config
Task queue	Celery over Redis	Python, broker config
AI integration	hosted LLM API	prompt strings, JSON schemas
Frontend packages	npm	JSON manifest, lockfile
Backend packages	pip	TOML manifest, lockfile
Secrets and environment	dotenv	key-value files
Container build	Docker	Dockerfile, compose YAML
Continuous integration	GitHub Actions	YAML, embedded shell
Infrastructure	Terraform	HCL
Orchestration	Kubernetes	YAML manifests

recurring categories and argue that its costs concentrate at boundaries, where no tool sees the whole and no type system reaches. Finally we trace the fragmentation to its source, two silent assumptions carried forward from the earliest stored-program machines, and state informally the two theses this book develops in response.

1.1 An Inventory of One Contemporary Application

This section takes the inventory. Our example is a task-management product with an AI assistant: users maintain lists of tasks in a browser, and a language model drafts, categorizes, and prioritizes tasks on request. Nothing about it is exotic. A product of this shape is the contemporary equivalent of the textbook payroll program: the application every framework tutorial builds and every startup builds first.

The frontend is a single-page application written in TypeScript with React: components in JSX, styling in CSS, state management in the framework’s idiom. The backend is a Python API service exposing REST endpoints, FastAPI or its equivalent. Durable state lives in PostgreSQL, reached through an object-relational mapper and evolved through a chain of migration scripts. A Redis instance caches hot reads and doubles as the broker for a task queue, where background workers execute the slow work: calling the hosted language model, sending email, recomputing derived data. The AI feature itself is a client library, a set of prompt templates interpolated with user data, and parsing code that coaxes structured output from the model’s replies. The whole assembly is built into container images by Dockerfiles, wired together for local development by a compose file, tested and shipped by a continuous-integration pipeline described in YAML, and provisioned onto a cloud by Terraform, with the production topology described a second time in Kubernetes manifests.

Now we count, and Table 1.1 keeps the ledger. The repository contains four languages in which logic is written: TypeScript, Python, SQL, and the shell scripts threaded through the CI pipeline. It contains two presentation notations: the HTML

dialect embedded in JSX, and CSS. It contains six configuration dialects: JSON for the npm manifest and the TypeScript compiler, TOML for the Python project, YAML in three unrelated schemas (compose, CI, Kubernetes), the Dockerfile language, HCL for Terraform, and the key–value dotenv format for secrets. That is twelve distinct notations a maintainer must be able to read, and at least eight in which a routine feature might require an edit. The count is charitable. It omits the prompt-template hybrid of the AI feature and the ORM’s query dialect.

The count continues past syntax. There are five package ecosystems in play: npm for the frontend, PyPI for the backend, a container registry for base images, the CI platform’s marketplace of reusable actions, and the Terraform registry of providers and modules. Each has its own resolver, its own versioning conventions, its own security-advisory channel, and its own failure modes. Two lockfiles, one per language ecosystem, freeze dependency graphs that the other cannot see. The build produces half a dozen distinct artifacts: a minified JavaScript bundle, an installed Python environment, at least two container images, a migration history that is itself a versioned artifact, and a Terraform state file that must be treated with the care of a database because it is one.

Hold the inventory in mind from the perspective of a single developer tracing a single feature. When a user clicks “prioritize my tasks,” the click handler (TypeScript) calls a generated client (TypeScript, generated from an OpenAPI document) that issues an HTTP request (JSON) to a route (Python) that validates the body against a schema (Python), enqueues a job (Python, serialized through the broker), which a worker dequeues to assemble a prompt (an English-and-JSON hybrid), call the model, parse the reply, write results through the ORM (Python classes mirroring SQL tables), and invalidate a cache key (a string convention shared, by discipline alone, between two codebases). To follow one user intention end to end, the developer crosses no fewer than eight representational boundaries (Figure 1.1). Not one of the crossings is checked by any compiler that can see both sides. Crossing 1 is the treacherous case that proves the rule: the handler’s call into the generated client appears checked, but both sides of what the type checker sees are the client’s restatement of the server, manufactured from an OpenAPI document that no tool verifies against its source. The eight numbered crossings are also this book’s ledger. The chapters ahead dissolve them one by one, and the coda settles the account: Chapter 19 redraws this figure with none of the eight remaining and only the world’s own boundaries left standing.

1.2 A Taxonomy of Glue

The crossings of the previous section share a structure, and it deserves a name before we classify what it costs.

Definition 1.1 (Discontinuity). A *discontinuity* is a boundary at which the representation of meaning must change and over which no verifier has jurisdiction.

Both clauses are necessary. For example, a function calling a function in the same language crosses a boundary (between modules, perhaps between authors), but every value passes with its representation intact and one compiler sees both sides. Nothing must be restated, so nothing can silently drift. The eight crossings of the inventory

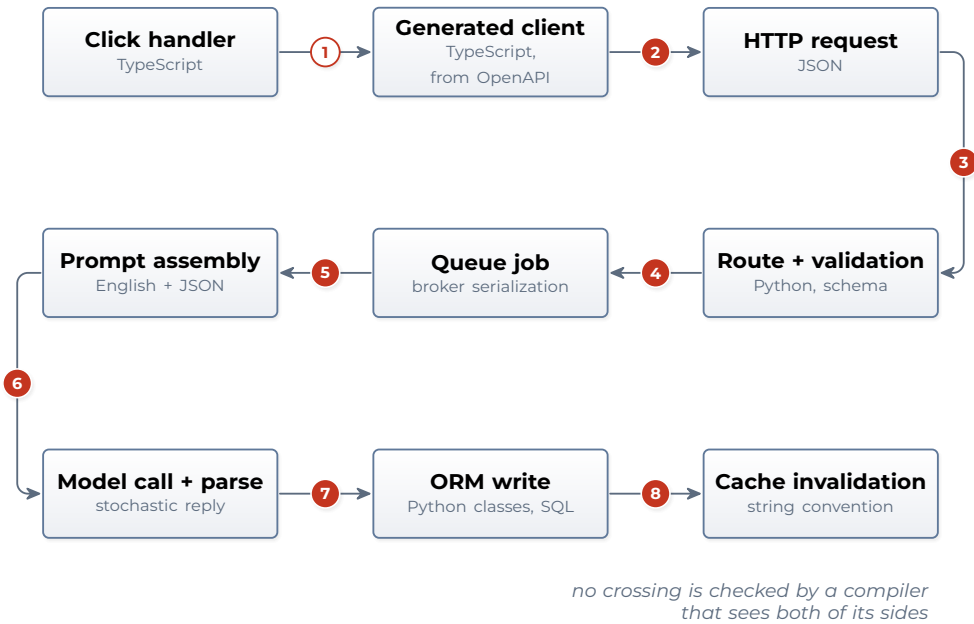


Figure 1.1: One user intention, “prioritize my tasks,” traced end to end through the archetypal application of Table 1.1. The intention crosses eight representational boundaries (numbered), each a change of notation authored and maintained by hand. Crossing 1 is drawn hollow because it is the treacherous one: the call into the generated client appears checked, but both sides of what the type checker sees are the client’s own restatement of the server, manufactured from a document no tool verifies against its source.

fail on both counts: at each one, meaning must be re-encoded in a notation the far side demands, and no tool checks the encoding against its source. Glue is what discontinuity costs, and we use that word in a narrow sense as well: glue is code, configuration, or generated scaffolding whose sole purpose is to carry meaning across a discontinuity, and which adds no domain behavior of its own. It restates decisions already made elsewhere: a type, a route, a schema, a topology. Viewed through these definitions, the inventory of the previous section resolves into six recurring categories.

Foreign-function interfaces and bindings. The oldest category predates the network: two languages sharing one process. The numerical and machine-learning core of the Python ecosystem is C, C++, Rust, and Fortran wearing a Python mask, and every mask is a binding layer: handwritten or generated shims that restate each function signature in the calling convention of the guest. Bindings are version-coupled isomorphisms maintained by hand: when the native side changes a struct, the binding must change in sympathy, and the failure mode of a stale binding is not a type error but a segmentation fault. Our example inherits this category silently, in the compiled wheels beneath its ORM driver and its tokenizer, and pays for it whenever a wheel is missing for a platform and a C toolchain must be summoned to build one.

Serialization and marshaling. Where FFI crosses a language boundary within

a process, marshaling crosses a process boundary, and the classic account of its machinery (stubs that flatten structured data for the wire and reconstitute it on the far side) is as old as remote procedure call itself (Birrell and Nelson 1984). In the example, JSON is the lingua franca, and every hop pays an encode–decode toll: browser to API, API to queue, worker to model, model back again. The deeper cost is not the cycles but the contract. A wire format is a type signature that no compiler enforces. Producer and consumer honor it by convention, and drift between them is discovered at runtime, in production, by whoever is on call.

API scaffolding. Between frontend and backend sits a layer whose entire content is restatement: route tables that name functions a second time in URL vocabulary, request and response schemas that mirror types already declared on both sides, an OpenAPI document generated from the backend and a client generated from the document, authentication middleware, pagination envelopes, and version prefixes. None of this expresses what the application does. All of it expresses that the application is split in two. The scaffolding exists because the function call, the primitive the language already provides, cannot cross the tier boundary. So the industry rebuilt the function call out of HTTP, one convention at a time.

Object-relational mappers. The relational model earned its permanence (Codd 1970), but the program that consumes it speaks in objects, and the ORM is the standing treaty between the two type systems. The treaty is expensive to administer. Model classes restate DDL. Migration scripts restate diffs between model versions. Query methods restate SQL with the corners rounded off. The mapper’s leaks (the N+1 query, the session detached at the wrong moment, the lazy load that fires inside a template) form a folklore so standardized that interviewers use it as a shibboleth. The ORM does not remove the boundary between object and relation. It concentrates the boundary into a library and bills for it in expertise.

Build orchestration. The example contains a second program that few would name when asked what the application is: the program that builds it. Bundler configuration, transpiler settings, multi-stage Dockerfiles, compose files, and CI pipelines together form a substantial codebase written in some of the least disciplined languages in computing. YAML with embedded shell is the worst of them, and the build rests on it. This second program has its own bugs, its own performance work, and its own specialists, yet it computes nothing about tasks or users. It exists to shepherd artifacts across toolchain boundaries, and it is rewritten from scratch, by convention, in every repository.

Infrastructure-as-code. Finally the topology of the deployment (what runs where, what talks to what, what scales when) is stated outside the program, in HCL and in Kubernetes manifests, descendants of the cluster-management lineage that gave us Borg and its successors (Burns et al. 2016). The serverless movement promised to absorb this layer, and its own retrospective is candid about the remainder (Jonas et al. 2019). Note what infrastructure-as-code restates: the application’s own call graph. The compose file and the manifests re-describe, in a third and fourth notation, which components the code already imports and invokes. When the program changes shape, a human must remember to change its shadow.

The categories cooperate. Listing 1.1 shows the same three-field record as it actually appears in our example’s repository: once as DDL, once as an ORM model,

once as an API schema, once as a frontend type.

```
CREATE TABLE users (                -- migrations/0004_users.sql
    id UUID PRIMARY KEY,
    email TEXT NOT NULL UNIQUE,
    display_name TEXT NOT NULL
);

class User(Base):                    # app/models.py (ORM)
    __tablename__ = "users"
    id: Mapped[UUID] = mapped_column(primary_key=True)
    email: Mapped[str] = mapped_column(unique=True)
    display_name: Mapped[str]

class UserOut(BaseModel):            # app/schemas.py (API)
    id: UUID
    email: str
    display_name: str

export interface User {              // web/src/types/user.ts
    id: string;
    email: string;
    displayName: string;
}
```

Listing 1.1: One record shape maintained in four notations within a single repository.

Four copies, three type systems, and one quiet landmine: the fourth copy renames `display_name` to `displayName` to satisfy the frontend’s casing convention, and the translation lives in a serializer configuration that no reviewer has looked at since it was pasted in. Adding one field to this record is a four-file, three-language, two-lockfile change followed by a migration, a regenerated client, and a cache-key bump. Not a single line of the diff implements behavior. Every line is glue.

1.3 The Cost of Discontinuity

This section measures what the discontinuities cost. They matter beyond tidiness, because discontinuities are where software breaks, and glue is discontinuity made permanent.

Consider first where guarantees end. Within TypeScript’s jurisdiction, the compiler enforces coherence: rename a field and every stale use is a build error. Within Python’s, type hints and tests approximate the same discipline. *Between* the two jurisdictions there is nothing. No compiler sees both sides of the HTTP boundary, of the ORM treaty, or of the prompt. The strongest tool available at the discontinuities of our example is textual search across the repository, which is to say that the whole-program type checker of the modern stack is `grep`. Every discontinuity is a place where a routine refactoring stops propagating and becomes a coordination problem between representations, and often between deploy schedules: the API must tolerate old clients, the queue must tolerate old message shapes, the database must tolerate the code that has not rolled out yet. Where coherence is maintained by convention rather than

checked by tools, defects do not distribute uniformly across the codebase. They pool at the discontinuities. We defer measurement to Chapter 16 and rest the argument here on structure rather than statistics: defects concentrate at discontinuities because a discontinuity is, by Definition 1.1, a program point where the machinery that prevents defects has no jurisdiction.

The second cost is paid in attention. Each of the twelve notations of Section 1.1 carries its own semantics, its own tooling, its own failure lore, and its own release calendar. A developer moving through one feature does not merely switch syntax. She switches evaluation models, from the browser's event loop to HTTP's request-response, to the queue's at-least-once delivery, to SQL's transactional semantics, to the stochastic contract of a language model. The switching cost recurs on every feature, forever. Teams pay it in onboarding weeks, in the wiki page enumerating the steps to run the application locally, and in the specialist roles (frontend, backend, DevOps, data) whose very existence maps one-to-one onto the discontinuities of the stack. The org chart is a picture of the glue.

This diagnosis has a distinguished pedigree. Brooks (1987) divided the difficulties of software into essence, the irreducible complexity of the problem being solved, and accident, the complexity contributed by our tools. He argued that high-level languages had by then removed so much accident that no order-of-magnitude gain remained within a single program. We accept the taxonomy and report a development Brooks did not forecast. In the decades since, the accident grew back, not inside programs but between them. Glue is accidental complexity *par excellence*: it restates decisions already made, in notations history happened to choose, and every line of it is work the problem never asked for. The four copies of User in Listing 1.1 contain one decision and three echoes.

Nor is the glue an aberration that better engineering hygiene will drain away. It is an institution. Bernstein (1996) could already survey, in 1996, an entire industry layer whose product category is the occupied boundary: middleware, the services whose reason for existence is that applications cannot reach each other directly. The intervening decades renamed the offerings (enterprise service bus, API gateway, integration platform, orchestrator) without changing the address. Meanwhile McIlroy (1968), proposing mass-produced software components at the very conference that named software engineering, imagined catalogs of interchangeable parts. Half of that vision arrived beyond his asking, in registries of millions of packages. What did not arrive is the other half, the one his 1964 epigraph to this chapter presumes: couplings that fit. We manufacture components industrially and join them artisanally. And the joints do not stay sealed. Spolsky (2002) observed that all non-trivial abstractions leak, which for our purposes compounds the bill, since each layer of glue must be understood twice: once as the abstraction it presents, once as the machinery that shows through when it fails. A stack of a dozen leaky layers is not a dozen conveniences. It is two dozen obligations.

One objection holds that this is merely the price of decomposition, that discontinuities are what modularity looks like from below. The objection deserves a careful answer, and it receives a full chapter (Chapter 17). The short answer is that modularity requires *interfaces*, and interfaces require boundaries between *modules*. But nothing in the logic of decomposition requires that the boundary also be a change of language,

of type system, of package ecosystem, of serialization regime, and of deployment unit all at once. Modules need boundaries. Nothing requires that a boundary be a discontinuity. The conventional stack bundles one good idea with five contingent ones and sells them as a package. This book is about unbundling them.

1.4 Two Silent Assumptions of the von Neumann Inheritance

This section traces the fragmentation to its source. It is not the result of any single decision. No committee chose twelve notations. No architect designed the four-copy record. The fragmented stack is what precipitates, over decades, out of two assumptions so old and so successful that they stopped being visible as assumptions at all. Both are present in the founding document of the field.

The 1945 EDVAC report (Neumann 1945) describes the organization that still underlies essentially every commercial machine: a memory holding both program and data, a control unit stepping through instructions, and an arithmetic organ past which data is streamed to be transformed. Two commitments are latent in that picture. The first: *computation is stationary*. The site of processing is fixed, and data travels to it: from memory to processor, and by later extension from disk to memory, from database to application, from server to client. The second: *the machine is the program's world*. The report describes one store, one control, one device. The semantics of a program extend exactly to the edge of its memory and no further. Whatever lies beyond the store is not the program's concern, and by inheritance it is not the programming language's concern either.

Backus (1978) named the first commitment's cost from inside the machine: the "von Neumann bottleneck," the word-at-a-time tube between processor and store, and, more damning, the intellectual bottleneck it induced, with languages designed as elaborations of the machine's traffic pattern and programs conceived as schedules for moving data past a fixed point of computation. Backus was concerned with expressiveness within one machine, but his diagnosis scales up uncannily well. Replace *word* with *row*, *JSON document*, or *prompt*, and replace *tube* with *query connection*, *REST call*, or *model API*, and the modern stack appears as the bottleneck writ large. The database ships rows to the compute tier because computation cannot go to the data. The cache exists to disguise the cost of that shipping. The queue ships work descriptions to wherever the stationary workers are. Sixty years of architecture scaled the machine out to planetary size and kept the traffic pattern.

The second commitment had consequences only when machines multiplied. So long as the program's world truly was one machine, a language that ended at the memory boundary was complete. But client-server, then the web, then the cloud made every serious application span machines, runtimes, and administrative domains. Languages, faithfully inheriting the 1945 worldview, continued to define semantics that end at the process boundary. Everything between processes was therefore left to be arranged outside the language, and it was: by protocols, formats, middleware, and manifests, the six categories of Section 1.2. What the language declines to govern, glue must. Neither commitment is a law of computation. They are engineering defaults from a report about a machine with one memory, and they hardened into the invisible background of language design.

1.5 The Theses of This Book

Against those two assumptions this book sets two theses. We state them here informally, as claims still to be earned. Their formal development is the business of Parts II and III.

The first thesis answers the assumption that the machine is the program's world, and we call the property it names *synechic*, from the Greek *synecheia*, continuity. It is the same root Peirce chose for his doctrine that continuity is the fundamental character of the real (Peirce 1892). A language is synechic, informally, when it presents one continuous semantic medium across the substrates that convention treats as separate ecosystems: when a program spans browser and server, managed runtime and native code, one machine and many, without any program point requiring glue, marshaling, or interop scaffolding to make the crossing. In a synechic language the inventory of Section 1.1 collapses. No framework hides the twelve notations. The discontinuities that demanded them are absent from the semantics to begin with. Even the inventory's oldest category dissolves this way: the two-languages-in-one-process boundary is drawn where memory discipline changes, parting the Python kind of language from the Rust kind, and Chapter 5 renders that line as a gradient within one checked medium, walked by degrees, never crossed. The record of Listing 1.1 is declared once, and the compiler bears the burden of every representation the substrates require. We make this precise in Chapter 2 as *substrate transparency*, give it a cost model, and, because the claim requires it, map the residue no language may dissolve. Latency, partial failure, and cost are physics. A synechic language must surface them lawfully.

The second thesis answers the stationary assumption, and we call the property it names *topokinetic*, from *topos*, place, and *kinesis*, motion. A language is topokinetic, informally, when the locus of computation is itself a first-class, mobile construct: programs are written as computation that travels through the shape of its data rather than as a fixed site to which data is fetched. In the paradigm this book develops, data lives as a persistent topology of nodes and edges, and the unit of program is a *walker*: a locus that moves along the topology, carrying state, triggering computation on arrival. Where the von Neumann inheritance asks "how do I bring the data here?", the topokinetic program asks "how does the computation get there?" The second question, we will argue, is the natural shape of an application whose data is already a graph of users, tasks, sessions, and memories. We introduce this paradigm, Object-Spatial Programming, by example and argument in Chapter 7.

The two theses are independent, and each names a class of language rather than a feature of one design. A language can be synechic and hold its computation still. A language can be topokinetic inside a discontinuous stack. The historical record realized each class partially and separately, never both, and Chapter 18 places that record on a single plane. To the two theses we therefore add a third, and we call it the *composition thesis*: the two classes are independent in definition and compounding in value. Continuity gives motion somewhere durable and unbroken to happen. Motion gives continuity its natural unit of computation. And the deepest dissolution this book performs, the disappearance of the database into the language, requires both at once (Chapter 10). This book presents all three claims through a single existence proof: Jac, a production language that is a member of both classes and in which both

properties carry the design. Jac grew out of a decade of research and production deployment in the Jaseci lineage (Mars et al. 2023). Its spatial paradigm has been formalized (Mars 2025), and its treatment of the newest discontinuity (the boundary between deterministic code and large language models, today bridged by the prompt string, the newest category of glue) is the subject of Chapter 6 (Dantanarayana et al. 2025). We will be equally explicit about what is not claimed. We do not claim that the discontinuities physics imposes disappear, that every application should be a graph, or that one language suits all software. Chapter 17 steelmans the case against us. The claim of this chapter is narrower and, we think, already defensible: the fragmented stack is not a law of nature. It is the long shadow of a 1945 machine description, maintained by habit and paid for in glue. A shadow is something a field can choose to step out of.

1.6 A Note on History

Neither thesis is without ancestors, and the reader deserves an account of them. Two appendices carry the full surveys, and a summary here keeps the arc of the argument in one place. The pursuit of continuity is nearly as old as the fragmentation itself. It runs through portability regimes that impose a common runtime on every machine and their inversions that carry the program's world along with the program, the tierless and multitier languages that compiled one program into the browser-server split, the distribution transparencies of the RPC lineage, and the single-system-image operating systems that promised one computer where there were many. Appendix F reads that literature as a record of partial successes and diagnoses the trap common to them: each dissolved some discontinuities and left the remainder standing. A medium that is continuous only in part forfeits most of the value of continuity, because the developer must still staff every boundary that survives. The pursuit of mobile computation is younger but no less persistent: from Backus's indictment of stationary compute, through the message-passing models of actors and CSP, the process calculi that made location formal, the mobile-agent systems of the 1990s that sent whole programs traveling, and the vertex-centric analytics frameworks that preserved a fragment of the same idea inside batch processing. Appendix G surveys that lineage and argues that what it lacked was not vision but synthesis: mobility was demonstrated repeatedly and never joined to a continuous medium, persistent data, and a type discipline that could make it safe. Both appendices are reference strata, not prerequisites. The reader who proceeds directly to Part II loses no thread of the argument.

Bibliographic Notes

The framing of glue as accidental complexity descends from Brooks (1987), whose essence-accident distinction remains the sharpest instrument for asking what our tools contribute to our difficulties. This chapter argues that the accidental share, which Brooks judged largely spent within programs, was reconstituted between them. The component vision against which we measure the fragmented stack is McIlroy (1968), presented at the 1968 NATO conference, and the chapter's epigraph is

McIlroy's earlier garden-hose memorandum, whose image of standardized couplings became Unix pipes: an early demonstration that a shared medium (there, the byte stream) is what makes glue-free composition possible. Bernstein (1996) documents the institutionalization of the boundary as an industry layer, and Spolsky (2002) supplies the law of leaky abstractions that compounds its cost.

The machinery of the six categories has its own literature. Marshaling and stub generation enter with remote procedure call (Birrell and Nelson 1984). The relational model that the ORM treaty answers to is Codd (1970). The cluster-management lineage behind contemporary orchestration is recounted in Burns et al. (2016), and the serverless movement's candid self-assessment appears in Jonas et al. (2019). The two silent assumptions are read from Neumann (1945) and from the critique in Backus (1978), whose Turing lecture named the von Neumann bottleneck and anticipated, at the scale of a single machine, the traffic pattern this chapter finds at the scale of the data center. The term *synechic* is coined after the synechism of Peirce (1892), to which Chapter 19 returns. The Jac lineage documented here (the Jaseci stack, the object-spatial formalism, and meaning-typed programming) is presented in Mars et al. (2023), Mars (2025), and Dantanarayana et al. (2025) respectively.

PART II



**SYNECHISM: A THEORY OF
CONTINUITY**

SUBSTRATE TRANSPARENCY

The purpose of abstracting is not to be vague, but to create a new semantic level in which one can be absolutely precise.

Edsger W. Dijkstra, 1972

Part I stated the problem and stopped short of a fix. Chapter 1 counted the artifacts of one routine application and found that most of them compute nothing. They exist to carry values between the components that do. The survey in Appendix F traces seventy years of partial remedies (virtual machines, polyglot runtimes, tierless languages, distribution middleware, single-system-image operating systems). Each dissolved one discontinuity and left the others intact or freshly hardened. That history does not lack engineering effort. It lacks a definition: a statement, precise enough to check against a language, of the property a language must have for the discontinuities to be gone. This chapter supplies that definition.

This chapter builds the definition in five moves. We begin where the prior art came closest, the classical family of distribution transparencies, and show that its ceiling lies at the boundary it declined to cross. The transparencies constrain how a program *behaves* across machines and hold fixed how it is *expressed*. We then introduce the two terms that govern the rest of this book: *synechic* for the first of the book's two language classes, and *substrate transparency* for the property that admits a language to it. We develop the definition into three instruments. The first is a formal statement of transparency as observational equivalence of program expression. The second is a cost model that renders the glue burden of any stack measurable, put to work in Chapter 16. The third is a principle identifying the boundaries that no language may hide. We close by placing the tierless literature as a restriction of the general property, and by extending the definition to executors that are not even deterministic, in preparation for Chapter 6.

2.1 The Classical Transparency Family and Its Ceiling

This section examines the classical transparency family and locates the ceiling that stops it short of continuity. The distributed-systems community was the first to make hiddenness a design objective with a name. The Reference Model of Open Distributed Processing standardized eight *distribution transparencies* (access, failure, location, migration, persistence, relocation, replication, and transaction). Each is a promise that some operational fact about a distributed system will not disturb the logic of programs running on it (Raymond 1995). The family's founding gesture predates its codification. Remote procedure call set out to make the invocation of a procedure on another machine indistinguishable from a local call (Birrell and Nelson 1984), and CORBA industrialized the ambition into an interoperability substrate spanning languages and vendors (Vinoski 1997). Bernstein (1996) gave the resulting layer its lasting name: middleware, a stratum of services interposed between application and platform so that applications need not speak of platforms.

The shared form of these promises is also their shared limit, so it pays to state the form exactly. Each transparency asserts that the *observable behavior* of a fixed program is invariant under some transformation of its deployment. An object relocates, a replica fails and recovers, an access path changes, and the program computes the same results from the same unmodified text. The quantifier ranges over runtime configurations. The program text is held constant on both sides of the equation. This is why middleware could deliver the family at all: a property that never varies the source code never needs the language's cooperation. One member of the family is the exception that clarifies the rule. Persistence transparency, pursued at the language level, became orthogonal persistence, the doctrine that an object's longevity should be independent of how the program manipulates it (Atkinson and Morrison 1995). In doing so it changed what programmers write, not merely what runtimes do. It is the ancestor of Chapter 10, and a first hint that the interesting transparencies are the ones that reach the text.

The classical program also drew its definitive internal critique. Waldo et al. (1994) argued that four facts distinguish distributed from local computing (latency, a different memory access model, partial failure, and concurrency), and that no amount of interface polish can mask them, because they are not artifacts of representation but properties of the world. The unified-object vision, in which local and remote objects are interchangeable behind one interface, founders on these four rocks. The critique is often read as counsel of despair about transparency in general. We read it more narrowly, as a list of which facts of distribution are *essential*, and therefore as a specification of which boundaries a language must surface rather than dissolve. We return to that list in Section 2.4.

Grant the classical family everything it promises, and the fragmented stack of Chapter 1 still stands. For example, a JavaScript frontend and a Python backend can enjoy flawless access and location transparency through some RPC framework and still require two languages, two package managers, two build systems, two dependency trees, and an interface-description artifact suspended between them. The discontinuity between browser and server is not, at bottom, a fact about where computation happens. It is a fact about the medium in which computation is written.

The classical transparencies quantify over the sites of execution and say nothing about the substrates of expression. That silence is the ceiling. Behavioral transparency saturates at the language boundary, because behavior is the only thing middleware can reach. To go further, transparency must become a property of the program text itself.

2.2 Substrate Transparency Defined

This section states the two definitions on which this book rests. The first names the *synechic* class of languages. The second names the property that admits a language to it. The adjective derives, through Peirce’s *synechism* (the doctrine that continuity is of prime importance in philosophy (Peirce 1892)), from the Greek *synecheia*, continuity. We defer the philosophical resonance to Chapter 19 and use the term here in a strictly technical sense. Throughout the book, *synechism* names the theory developed in this part, and *synechic* is the property adjective for languages.

SYNECHIC

A language is *synechic* if it presents a single continuous semantic medium across conventionally discrete ecosystems, tiers, and toolchains, such that no program point requires glue code, marshaling, or interop scaffolding to cross a substrate boundary.

SUBSTRATE TRANSPARENCY

A language exhibits *substrate transparency* when the identity of the underlying runtime, ecosystem, and toolchain at any program point has no bearing on how the program is expressed.

The first definition is an engineering ideal stated in engineering vocabulary. The second is the property that makes it precise. To check the property against a language we need three prior notions: what a substrate is, what it means to cross between two of them, and what expression counts as glue.

Definition 2.1 (Substrate). A *substrate* is a triple $s = (R, E, T)$ consisting of a *runtime* R , the executor that gives meaning to compiled artifacts; an *ecosystem* E , the corpus of existing libraries and services importable at that runtime; and a *toolchain* T , the apparatus that produces, links, packages, and ships artifacts for R .

The triple is chosen so that the conventionally distinct kinds of boundary become instances of one kind. The managed tier of an application is a substrate: the Python bytecode interpreter, the PyPI corpus, and the Python packaging apparatus. The web tier is another: a browser JavaScript engine, the npm corpus, and a bundler pipeline. The native tier is a third: the bare machine reached through LLVM (Lattner and Adve 2004), the C library corpus, and a compiler-linker chain. A tier is a substrate, and a foreign ecosystem is a substrate. A toolchain, as Chapter 4 develops, is a component of every substrate and a discontinuity in its own right.

Definition 2.2 (Placement and boundary crossing). Let P be a program with program points $\text{pts}(P)$, and let $\mathcal{S}$ be a family of substrates. A *placement* is a map $\sigma : \text{pts}(P) \rightarrow \mathcal{S}$ assigning each point the substrate on which it executes. An edge (p, q) in the program’s reference structure—a call, a data flow, or an import—is a *boundary crossing* under σ if $\sigma(p) \neq \sigma(q)$ in any component of the triple.

Definition 2.3 (Glue). The *glue* of a crossing (p, q) under σ is the marginal expression it demands: the program text required to realize the edge when $\sigma(p) \neq \sigma(q)$, less the text required to realize the same edge when p and q are co-located. A language is *glue-free* over $\mathcal{S}$ when this margin is empty for every crossing.

Definition 2.3 makes the taxonomy of Chapter 1 formal. A foreign-function binding, a serializer, an interface-description file, a generated API client, an object-relational mapping: each is expression whose entire justification is that some $\sigma(p)$ differs from some $\sigma(q)$. None would survive co-location. That counterfactual is how the definition identifies them.

With these pieces the central definition can be stated. Write $\mathcal{E}_L(\varphi, s)$ for the set of expressions in language L that realize a specification φ when executed on substrate s , where s is *admissible* for φ (capable, in principle, of executing some realization of it).

Definition 2.4 (Substrate transparency, formal). A language L is *substrate-transparent* over a substrate family $\mathcal{S}$ if both of the following hold. (i) *Expression-invariance*: for every specification φ and all $s, s' \in \mathcal{S}$ admissible for φ , $\mathcal{E}_L(\varphi, s) = \mathcal{E}_L(\varphi, s')$. (ii) *Null crossing cost*: for every program, placement, and edge, the glue of the crossing is empty—composition across a boundary is expressed by the same constructs as composition within a substrate.

The definition has an equivalent reading as an observational equivalence, and the reading explains the chapter’s title. Call two placements σ_1, σ_2 of a program *expressionally equivalent* when the same computation text realizes the program under both. Substrate transparency holds exactly when every pair of admissible placements is expressionally equivalent: the source is a fixed point of placement change, and an observer who sees only the computation text can recover nothing about σ . Where the classical transparencies made deployment invisible to the program’s *behavior*, substrate transparency makes the substrate invisible in the program’s *expression*. The quantifier has moved from configurations of the runtime to texts of the language, and that movement is the entire content of the generalization.

Remark 2.5. Substrate transparency does not forbid the programmer from directing placement. A language may provide annotations that assign fragments to substrates—in Jac, a `cl` block routes its declarations to the web substrate and a `na` block routes its own to native code (Chapter 3)—without violating Definition 2.4, because such an annotation is a declaration *attached to* expression, not a change *within* it. The compiler may equally infer the placement from what a fragment uses, and Chapter 3 shows that it does. Inference and annotation are two spellings of the same declaration. The computation text on either side of the marker is the same language, the same types, the same idioms. What the definition forbids is that expression bend around the substrate: that a crossing demand marshaling code, a second dialect, or a wrapper. Declared placement returns in Section 2.4 as the mechanism by which lawful boundaries surface.

2.3 A Cost Model for Glue

This section turns the definition into a measurement. Definition 2.4 is binary, but real languages approximate it by degrees, and the argument of this book obliges us to measure the approximation. We therefore define a metric on stacks, deliberately coarse and deliberately countable from a repository, which Chapter 16 applies to real codebases. The unit of measurement is the passage of a single value through a system, because that is where glue accumulates. A value is born in one representation, and every discontinuity it crosses exacts expression from the programmer.

Definition 2.6 (Boundary-crossing metric). Let A be a stack (the set of artifacts a team authors and maintains) and let Π be a family of end-to-end paths, each tracing one value from its source of truth to its point of final use. For a path $\pi \in \Pi$: (i) the *authored representation shifts* $\rho(\pi)$ count the stages at which the value’s encoding changes *and* the change is effected or configured by authored text—re-encodings performed invisibly by a runtime do not count; (ii) the *artifact dialects* $\delta(\pi)$ count the distinct languages and formats among the artifacts that must be authored for π to function; (iii) the *crossing cost* of the path is $\chi(\pi) = \rho(\pi) + (\delta(\pi) - 1)$, so that a path confined to one dialect with no authored shifts costs zero. For the stack as a whole, the *coherence obligations* $\kappa(A)$ count the pairs of artifacts in distinct dialects that encode overlapping facts and are kept consistent only by discipline, no compiler or checker spanning the pair. The *glue burden* of the stack relative to Π is

$$G(A, \Pi) = \sum_{\pi \in \Pi} \chi(\pi) + \kappa(A).$$

A stack authored in a substrate-transparent language attains $G = 0$ on every path family: one dialect, no authored shifts, and no pair of artifacts left to drift apart. The restriction to *authored* shifts is essential to what the metric measures. A synechic system still serializes values over HTTP and still lays objects out differently in a browser heap than in a native one. Physics is not repealed. But those re-encodings are the compiler’s and runtime’s concern, invisible in expression, just as register allocation is invisible in a garbage-collected language. The metric measures what programmers write and maintain, because that, per the definition, is what substrate transparency governs.

A worked illustration (Figure 2.1) fixes intuition. The counts here describe the archetypal stack surveyed in Chapter 1 and are illustrative rather than measured. Consider adding one field, a user’s biography, to the profile record of a conventional single-page application. The value’s path from source of truth to pixels traverses roughly five dialects: a SQL column and its migration, an ORM model and a serializer in the server language, an OpenAPI entry in YAML, a client-side type in TypeScript, and a component binding in JSX. The authored shifts number about three: the row-to-object mapping, the object-to-JSON serializer, and the JSON-to-typed-object parse on the client. The crossing cost of this one path is then $\chi = 3 + (5 - 1) = 7$, and the change leaves behind at least four coherence obligations (column against model, model against serializer, serializer against schema, schema against client type) for a contribution to G of eleven. Every one of those eleven units is a place where the next engineer can introduce a defect by editing one artifact and forgetting its twin.

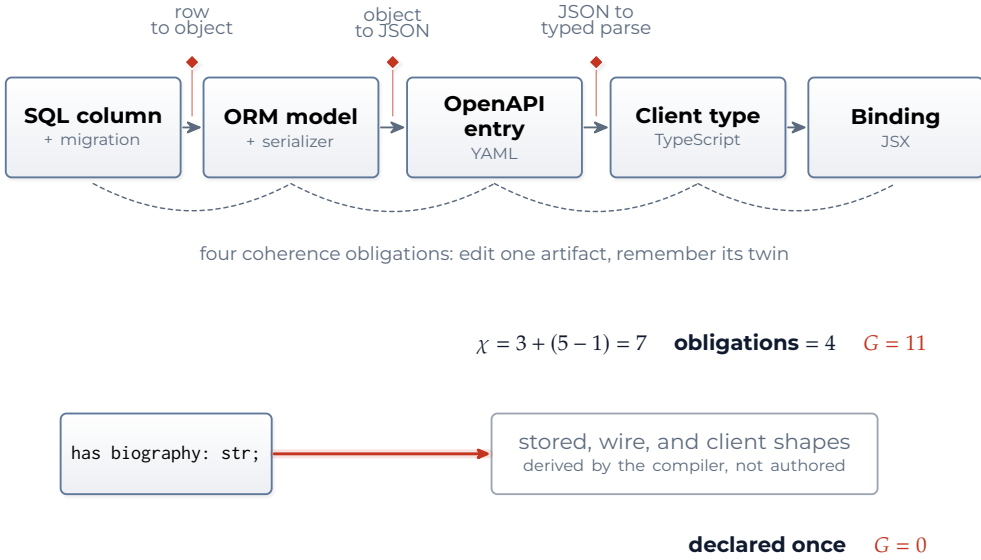


Figure 2.1: The cost of one field. Adding a user’s biography to a conventional single-page application touches five dialects, authors three representation shifts, and leaves four coherence obligations behind, a contribution of eleven to the glue account G . The same field in Jac is one declaration whose other shapes are the compiler’s obligation.

The same record in Jac is a single declaration in a single dialect. Listing 2.1 shows the profile node of the littleX social application developed in Chapter 15, with its methods elided. The fields exist here and nowhere else, and the client-side code that renders them is written in the same language against the same type.

```
node Profile {
  has username: str = "",
    bio: str = "",
    created_at: str = "";
}
```

Listing 2.1: The complete authored artifact for the profile record in the littleX social application. The fields exist in one declaration and one dialect; adding a field is a one-line change with no coherence obligations.

On this path $\delta = 1$, $\rho = 0$, and no coherence pair is created: $\chi = 0$. The metric is crude: it weighs all shifts equally and says nothing about the difficulty of any single one. But it has the virtues we need: it is language-agnostic, monotone in stack growth, and computable by inspection of a repository. Chapter 16 applies it to full codebases and discusses its threats to validity.

The cost model measures what transparency removes. Its deepest consequence, however, is not a removal but an extension, and it deserves to be stated as a claim of its own.

Proposition 2.7 (Jurisdiction). *In a substrate-transparent language, every boundary crossing lies within the jurisdiction of the language’s verifiers: the instruments that check composition*

within a substrate—the type checker first among them—check composition across one, because by Definition 2.4 the two are the same expression.

The proposition is nearly definitional, and it is the half of the book’s argument that expression-invariance alone does not deliver. The definition of discontinuity in Chapter 1 has two clauses: a representation of meaning that must change, and a verifier with no jurisdiction over the change. The cost model answers the first clause. ρ and δ count what must be restated, and a transparent language drives both to their floor. The proposition answers the second. When a cross-substrate call is written as a call, the checker that types calls types this one. It sees caller and callee against a single declaration and rejects the drift that a conventional stack discovers at runtime, in production, by whoever is on call. The coherence obligations $\kappa(A)$ are the pairs no verifier spans, and substrate transparency takes κ to zero by re-describing the system until nothing lies outside the checker’s sight. For example, renaming a field of Listing 2.1 is a compile-time error at every stale use (server, client, or native), where the same rename in the conventional stack is a textual search whose misses ship. This is Chapter 1’s observation that the whole-program type checker of the modern stack is `grep`, run in reverse. Two qualifications bound the claim. First, jurisdiction is reach, not depth. What the checker proves within its reach is the type system’s affair, and the gradual discipline forced by Python containment, together with the static rules that keep the dynamic type from silently widening, is the subject of Chapter 9. Second, jurisdiction does not abolish the lawful boundaries of the next section: an awaited cross-tier call is checked as an expression while remaining visible as physics.

2.4 The Limits of Continuity: Lawful Boundaries

This section marks the limits of continuity. A theory that admired only invisibility would repeat the mistake it set out to correct. The four differences of Waldo et al. (1994) (latency, memory access, partial failure, concurrency) are not representation artifacts that a better compiler can dissolve. They are properties of physically separated computation. The CAP theorem sharpened one of them into an impossibility result: a service cannot be simultaneously consistent, available, and partition-tolerant (Gilbert and Lynch 2002). The mature reading of the result is a design discipline: partitions are rare, and when they occur, the system must choose, explicitly, what to sacrifice (Brewer 2012). Brooks (1987) supplies the vocabulary that organizes the chapter’s claim: the boundaries cataloged in Chapter 1 divide into the accidental and the essential. The accidental ones are the discontinuities: dialects, marshaling, toolchains, all artifacts of how we happen to represent computation. One of the cataloged boundaries deserves its filing made explicit, because the field treats it as a fact about kinds of language. That boundary is the memory-discipline divide between managed languages and systems languages, whose cost the inventory recorded as the bindings category. We file it with the accidental, and Chapter 5 makes the case, exhibiting memory discipline as a continuum within one checked medium, not a choice between languages. No representation can remove the *essential boundaries*: latency, partial failure, and cost.

Principle 2.8 (Lawful boundaries). A synechic language must dissolve every discontinuity and must surface every essential boundary as a semantic construct. At

each program point where latency, partial failure, or cost can manifest, the language provides expression—declared, typed, and handleable—through which it appears, and never permits it to arrive solely as undeclared runtime behavior.

The principle inverts the RPC bargain. Remote procedure call hid the essential in order to sell sameness, and the Waldo critique showed the bill coming due at runtime, where hidden latency became mysterious slowness and hidden partial failure became undebuggable inconsistency. The law of leaky abstractions generalizes the complaint: every abstraction over an essential difference eventually leaks it (Spolsky 2002). The synechic response is a refusal to abstract over essence at all. The boundary is admitted into the semantics, where it can be typed, checked, and handled. In the Jac surface this takes concrete forms that Chapter 3 develops. For example, a client-tier call into server-side logic is an awaited asynchronous expression, so latency is visible at the call sites where it exists, even as the discontinuities of representation and dialect around those call sites are gone. How the runtime surfaces failure and cost beneath the single system image is the subject of Chapter 14, and the consistency choices a living graph forces are the subject of Chapter 10. Results such as the CALM theorem, which characterizes when coordination can be avoided entirely (Hellerstein and Alvaro 2020), are usable by a runtime because the relevant boundaries remain semantically visible to it. The leaky-abstraction objection receives its steelman in Chapter 17. The position defended there is the one stated here: a synechic language leaks less because it declines to abstract over the four differences that cannot bear it.

Note the division of labor between principle 2.8 and Remark 2.5. Declared placement is the syntax of lawful boundaries: a `cl` block or an awaited call is the programmer acknowledging an essential boundary, in the language, at the point where it lies. Substrate transparency and lawful boundaries are therefore not in tension but complementary halves of one discipline: expression never varies with the substrate’s identity, and expression always registers the substrate’s physics.

2.5 Tierlessness as Restricted Synechism

This section places the tierless languages within the definition. One research lineage did move transparency into the program text, and it is the closest ancestor of the property defined in this chapter. Links compiled a single source program into client code, server code, and database queries (Cooper et al. 2006). Hop unified the two halves of a web application in one Scheme dialect (Serrano et al. 2006). Ur/Web gave the client-server-database triple a statically typed, whole-program treatment (Chlipala 2015). Eliom brought the discipline to OCaml (Radanne et al. 2016). ScalaLoci generalized the assignment of code to places into placement types (Weisenburger et al. 2018). The multitier survey of Weisenburger et al. (2020) organizes the field along dimensions (placement specification, communication abstraction, formalization) that map recognizably onto the components of Definition 2.1. These languages were the first for which program expression was invariant under tier assignment, and our definitions locate them.

Definition 2.9 (Restricted substrate transparency). A language exhibits substrate transparency *restricted to* a family $S_0 \subseteq S$ when the conditions of Definition 2.4 hold with all quantifiers ranging over S_0 .

In these terms, a tierless language is one that is substrate-transparent over a family of *tiers*: substrates that differ in their runtime component while sharing, in effect, a single closed ecosystem and a single external toolchain. The restriction has three independent axes. First, the family is fixed and finite, chosen by the language designer and closed to extension: client, server, and sometimes database, the tiers of one web application. Second, the ecosystem is closed. The program lives within the language’s own libraries, and crossing into a foreign corpus (PyPI, npm, the C world) is out of scope. Whole-program compilers such as Ur/Web turn that closure to advantage, since analyses can assume they see everything. Third, the toolchain is external: the compiler is transparent across tiers, but the apparatus of building, packaging, and deploying remains outside the language, in the sense that Chapter 4 argues makes it a hidden tier.

Synechism, as this book construes it, is the removal of all three restrictions. The substrate family is open: managed, web, and native substrates in Chapter 3, the toolchain substrate in Chapter 4, and, in the next section, substrates that are not deterministic at all. The ecosystems are foreign and inherited. The toolchain is internalized until it, too, satisfies the invariance condition. Tierlessness is thus substrate transparency over the smallest interesting family. The placement is meant as genealogy: the multitier results survive intact as the special case, and the generalization inherits their central lesson, that invariance of expression must be designed into a language rather than retrofitted around one.

2.6 The Stochastic Substrate

Every definition so far quietly assumed that executors are deterministic. $\mathcal{E}_L(\varphi, s)$ presumes that whether an expression realizes a specification on a substrate is a settled fact. The assumption deserves to be surfaced, because the most consequential new executor of the current era violates it. Large language models (Vaswani et al. 2017) behave as general-purpose executors of tasks specified in natural language (Brown et al. 2020), and contemporary practice attaches them to programs through hand-built prompt strings, parsing scaffolds, and retry loops. That is glue, the marshaling layer of the AI era. If the model is a substrate, the definition of substrate transparency ought to say what continuity means across its boundary.

Definition 2.10 (Stochastic substrate). A *stochastic substrate* is a substrate whose runtime component is a *stochastic executor*: a conditional distribution $M(y \mid x, a)$ over outputs y given an input x and semantic annotations a . A stochastic substrate is admissible for a specification φ when realizations adequate to φ carry sufficient probability mass under M ; the quantitative standard of adequacy is a matter for the delegation semantics of Chapter 6.

The instructive observation is that Definition 2.4 extends to this executor class without alteration, and the reason is the chapter’s central design decision. Behavioral

transparency, the classical kind, cannot even be stated for a stochastic executor. Observational equivalence of behavior fails between two runs of the same call, so any definition quantifying over behavior founders immediately. Expressional transparency is indifferent to the executor's determinism, because it quantifies over program text. Applied to the stochastic substrate, expression-invariance requires two things. First, delegation to a model must be written with the language's ordinary abstraction constructs: a function, with a signature, a return type, and unremarkable call sites. Second, the difference must be confined to a declared implementation clause. That is the lawful-boundary discipline of principle 2.8 applied to a substrate whose essential boundaries are stochasticity, latency, and monetary cost. Listing 2.2, drawn from the day-planner application examined in Chapter 15, shows the shape this takes in Jac.

```
enum Category { WORK, PERSONAL, SHOPPING, HEALTH, FITNESS, OTHER }

def categorize(title: str) -> Category by llm();
sem categorize = "Categorize a task based on its title";

node Task {
  has title: str,
  done: bool = False,
  category: str = "other";
}

def:pub add_task(title: str) -> Task {
  category = str(categorize(title)).split(".")[1].lower();
  task = root ++> Task(title=title, category=category);
  return task;
}
```

Listing 2.2: A function implemented by a language model. The signature and call site are those of an ordinary typed function; the delegation and its semantic annotation are declared clauses, not string engineering.

The `by llm()` clause is the surfaced boundary. It declares, at the definition, that this function's executor is a model, just as a placement annotation declares that a fragment's executor is a browser. The call inside `add_task` is expressed as any typed call is expressed. No prompt is authored and no output is parsed by hand, and the enum return type constrains what the executor may produce. The `sem` annotation supplies the meaning from which the runtime synthesizes the model's instructions, a mechanism this book develops in Chapter 6 under the name of meaning-typed programming (Dantanarayana et al. 2025), together with the contracts, validation, and retry semantics that make delegation to a distribution dependable rather than hopeful.

With stochastic substrates admitted, the family $\mathcal{S}$ now contains every executor this book requires: the managed, web, and native substrates of Chapter 3, the toolchain of Chapter 4, the distributed runtime of Chapter 14, and the model. The theory of synechism is, at this point, complete as a definition and a discipline: one invariance condition, one cost model, one principle marking its lawful limits. What remains is to show that a production language satisfies it, and that demonstration begins in the next chapter.

Bibliographic Notes

The classical transparency program is best entered through Raymond (1995), which introduces the RM-ODP reference model and its eight transparencies. The founding engineering document is the remote procedure call paper of Birrell and Nelson (1984). Its industrial form is surveyed by Vinoski (1997), and Bernstein (1996) frames the middleware stratum in which the family was delivered. The indispensable critique is Waldo et al. (1994), whose four essential differences this chapter promotes from objection to specification. Orthogonal persistence, the one classical transparency pursued inside language semantics, is comprehensively treated by Atkinson and Morrison (1995). On the essential boundaries themselves, Gilbert and Lynch (2002) proves the CAP conjecture, Brewer (2012) tempers its folklore reading, Hellerstein and Alvaro (2020) characterizes when coordination is avoidable, Brooks (1987) supplies the essence-accident distinction, and Spolsky (2002) states the leaky-abstraction objection that Chapter 17 steelmans.

The tierless lineage that Section 2.5 places as restricted synechism runs from Links (Cooper et al. 2006) and Hop (Serrano et al. 2006) through Ur/Web (Chlipala 2015) and Eliom (Radanne et al. 2016) to the placement types of ScalaLoci (Weisenburger et al. 2018). The survey of Weisenburger et al. (2020) maps the field’s dimensions and is the natural companion to this chapter’s definitions. The term synechism originates with Peirce (1892), whose philosophical program Chapter 19 revisits. For the stochastic substrate, Vaswani et al. (2017) introduces the transformer architecture, Brown et al. (2020) establishes language models as few-shot executors of natural-language task specifications, and Dantanarayana et al. (2025) presents the meaning-typed abstraction whose full development occupies Chapter 6.

THE DESIGN OF A SYNECHIC SURFACE

Simple things should be simple, complex things should be possible.

Alan Kay

The previous chapter defined substrate transparency as a property a language may or may not possess: the identity of the runtime, ecosystem, and toolchain at any program point has no bearing on how the program is expressed. A definition of this kind states what must be true. It does not state what to write. Between the property and a working language sits a long sequence of design decisions: what the surface syntax admits, where a new construct may appear, how a foreign library enters scope, and what a boundary crossing looks like on the page. This chapter records those decisions as Jac makes them, and argues that the decisions generalize into a discipline for building continuous surfaces.

This chapter stays close to real code. The running artifact is littleX, a complete social application shipped in the Jac distribution's example suite: a persistent social-graph backend, a browser frontend with reusable components, authentication, channels, and search, all under one language, one manifest, one entry point. Every Jac listing here is drawn from that application or from the language reference, and we cite file names so the reader can check the claims against the artifact (Chapter 15 measures the same application quantitatively). We proceed through five design commitments: supersetting an incumbent language, lowering one syntax to three substrates, inheriting three ecosystems through ordinary imports, dissolving the client-server discontinuity, and separating declaration from implementation. We close by extracting design rules for synechic surfaces that do not depend on Jac in particular.

3.1 Supersetting as Strategy

The first design commitment is to superset an incumbent language whole. A new language faces a cold-start problem in three currencies at once: it has no libraries, no idioms, and no programmers. The classical response is to bootstrap all three slowly and hope to survive the interval. The synechic response is different: choose an incumbent ecosystem and contain its language semantically, so that the new surface begins life already possessing the incumbent's libraries, idioms, and programmers. Jac's incumbent is Python. The containment is not an interoperability bridge bolted onto a foreign language, but an embedding. Jac's expression language, data model, and standard-library semantics are Python's, the server lowering emits ordinary Python bytecode, and modules cross the boundary in both directions: a Python file imports a `.jac` module with an ordinary `import` statement, and a `.jac` file imports Python packages the same way. For any Python program there is a Jac program with the same meaning, obtained by mechanical respelling. Where even respelling is unwanted, an inline `::py::` block admits Python text verbatim.

The strategic precedent is TypeScript, whose relationship to JavaScript Bierman et al. (2014) analyze in just these terms: a syntactic superset whose additions (a static type discipline, erased before execution) ride on top of an unchanged runtime semantics. TypeScript demonstrated that containment converts migration into adoption: an existing codebase is already a valid program in the new language, and every library in the incumbent ecosystem works on day one. Jac accepts the lesson and extends it along a different axis. Where TypeScript's superset adds annotations, Jac's superset adds *semantics*: archetypes for topology and mobile computation (Chapter 7), abilities dispatched by arrival rather than by call, codespace tags that place a declaration on a substrate (Section 3.2), persistence rooted in reachability (Chapter 10), and delegation of function bodies to language models (Chapter 6). These constructs have no Python spelling, and they are not local sugar over Python. Lowering a walker touches event dispatch, persistence, and service projection at once, a global transformation of the kind Felleisen (1991) uses to distinguish genuine expressiveness from abbreviation. A type-only superset yields a better checker for the same programs. A semantic superset yields new programs.

Containment on the page is best shown by the parts of littleX that use none of the new constructs. The excerpt in Listing 3.1 is the opening of the littleX backend: a Python standard-library import, a helper function, and two plain value objects. A Python programmer reads this without instruction. The differences are concrete syntax, not semantics: braces for blocks, semicolons for statements, `has` for field declarations, and types required on signatures. The `datetime` call chain, the string concatenation, and the default field values all mean what they mean in Python.

```
import datetime;

def _now -> str {
    return datetime.datetime.now(datetime.UTC).replace(tzinfo=None).isoformat() +
        "Z";
}

obj ProfileBundle {
```

```

    has profile: Profile,
        followers: list[Profile],
        following: list[Profile],
        tweets: list[Tweet];
}

obj ChannelBundle {
    has channel: Channel,
        member_count: int,
        is_member: bool,
        posts: list[Tweet] = [];
}

```

Listing 3.1: Python-shaped Jac: the opening of the littleX backend module. A standard-library import, a function, and two value objects, with no construct beyond Python’s semantic reach.

The containment is graded. A project may be pure Jac. It may embed Python in `:py::` blocks. It may be mostly Jac importing Python utilities, or mostly Python importing Jac modules for the constructs Python lacks. At the conservative extreme it may remain pure Python and use the Jac runtime as a library. Typing is graded the same way: values arriving from untyped Python enter Jac as any and must be narrowed or cast before they flow into typed positions, in the gradual-typing tradition (Siek and Taha 2006; Siek et al. 2015). We treat the type system’s side of this bargain in Chapter 9. The effect of the whole arrangement is the one Steele (1999) argues a language must plan for: growth. Jac did not have to grow a general-purpose core, an ecosystem, or a community from seed. It grew new semantics from a stock that already had all three.

3.2 One Syntax, Three Lowerings

The second commitment is that one surface syntax lowers to three substrates (Figure 3.1). Jac calls the substrate assignment of a piece of code its *codespace*. Server code compiles to Python bytecode and lives among PyPI packages. Client code compiles to JavaScript and lives among npm packages. Native code compiles through LLVM (Lattner and Adve 2004) to machine code with a C ABI, linked by the toolchain itself with no external compiler or linker (Chapter 4). The compiler assigns codespaces by inference: code declares its substrate by what it uses. A function that returns JSX or imports an npm package is client code. A module whose imports are C-ABI foreign declarations is native code. Archetypes, served endpoints, and Python-ecosystem imports pin a module to the server. A module that needs none of the server’s machinery compiles native by default, and when the native lowering cannot carry it, the compiler falls back to the server lowering and says so, so the preference never costs a program (a `jac.toml` knob restores the server default for a project that wants it). Explicit placement remains available and overrides inference everywhere: a file suffix (`.cl.jac` for client, `.sv.jac` for server, `.na.jac` for native), a braced `cl { }`, `sv { }`, or `na { }` block, or a single-statement prefix such as `sv import`. In littleX, the server tier is the social-graph module and the client tier is a set of `.cl.jac` files (`frontend.cl.jac` and a

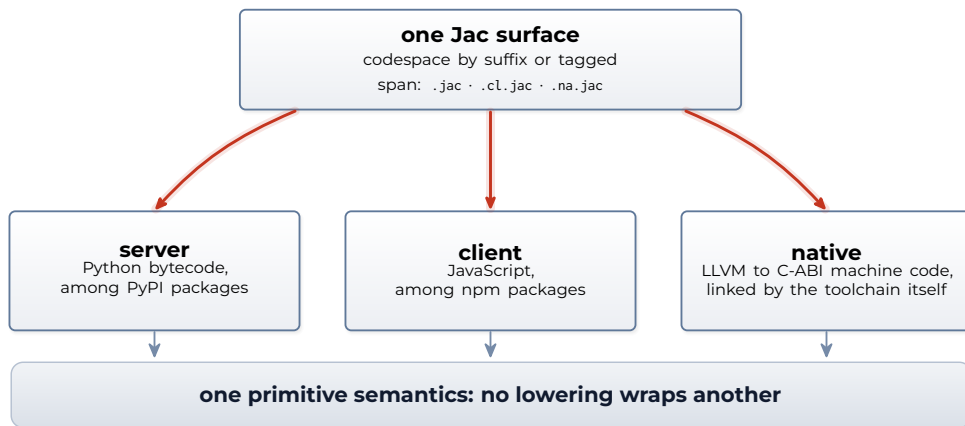


Figure 3.1: One syntax, three lowerings. A declaration’s codespace selects its substrate, each lowering targets its substrate’s ecosystem directly, and all three rest on the same primitive semantics. No lowering is implemented as a wrapper over another.

directory of components) compiled together under one manifest. The application’s entire entry point appears in Listing 3.2.

Chapter 12 names the machinery that discharges this commitment: the surface below is compiled by a *polypiler*, one compiler whose unit is the whole polyglot application and whose targets are ecosystems rather than instruction sets.

```

"""Little X - Entry point combining backend and frontend."""

cl {
  import from .frontend { app as ClientApp }

  def:pub app -> JsxElement {
    return <ClientApp/>;
  }
}

```

Listing 3.2: The littleX entry point, verbatim. A client-codespace block re-exports the frontend component. The social-graph module beside it carries no marker at all: its archetypes and served walkers pin it to the server codespace by inference.

This arrangement inverts the portability strategies surveyed in Appendix F. The virtual-machine tradition, from p-code to the JVM (Lindholm and Yellin 1999), made many machines look like one substrate and asked every program to live on it. The polyglot-runtime tradition made many languages meet on one substrate. Jac holds the surface fixed and enters each substrate as a first-class citizen. The server lowering produces ordinary Python bytecode, indistinguishable to the interpreter from compiled Python source. The client lowering produces the JavaScript a web toolchain expects, JSX expressions included. The native lowering produces a standalone executable or a C-ABI shared library. No lowering wraps another. There is no interpreter in the native binary and no Python process behind the browser.

A shared surface keeps its promise only if a semantic contract sits beneath it. The compiler defines a primitive-semantics interface (the fixed meanings of `int`, `str`, `list`, `dict`, the operators, and the builtins) that every backend must implement, so that an expression means the same thing wherever it lands. Where a substrate genuinely cannot support a construct (the native codespace currently excludes `async` and `Python-ecosystem imports`), the compiler rejects the program at compile time rather than emulating the feature or failing at runtime. When the native placement was the compiler’s own inference rather than the programmer’s marker, the rejection softens to a fallback: the module recompiles for the server, and behavior is preserved either way. Continuity is never counterfeited. The surface is uniform exactly where the semantics are. The machinery of the three lowerings (partitioning, code generation, and the interop stubs between codespaces) is the subject of Chapter 12, and we keep it offstage here.

3.3 Ecosystem Inheritance Without Wrappers

The third commitment is that foreign libraries enter through the ordinary import path. A language’s ecosystem story can be evaluated with a one-question audit that asks what the programmer must write to use a foreign library. For most languages the answer is a taxonomy item from Chapter 1: a binding generator, an FFI declaration file, a wrapper package maintained by a third party, or a serialization layer. For a synechic surface the answer must be one thing: an import statement. The first line of the littleX backend, already visible in Listing 3.1, is `import datetime`;. A Python standard-library module enters a Jac file with no wrapper, no stub generation, and no configuration, because the surrounding codespace lowers to the bytecode world where that module natively lives. Third-party PyPI packages enter identically: the littleX manifest declares `numpy` and `scikit-learn` as dependencies, and they are imported like any Jac module.

The same audit passes on the other two substrates. Listing 3.3 collects the three doorways, and Figure 3.2 draws the arrangement: three corpora, one doorway apiece, and no wrapper stratum between corpus and program. In the client codespace, littleX imports the npm package `lucide-react` by name and destructures four icon components from it. The manifest’s npm section sits beside its PyPI section in the same `jac.toml`, and one toolchain operation resolves both (Chapter 4). In the native codespace, a C shared library is imported with type signatures written in Jac syntax: fixed-width ABI types such as `f64` appear only inside the import declaration, and the compiler coerces at the boundary. A logical library name (`import from raylib`) resolves to the platform-correct `.so`, `.dylib`, or `.dll`, just as a linker’s `-l` flag would. An explicit path pins a versioned soname when required. The direction also reverses: compiling a native module with `-shared` exports its `:pub` functions as a C-ABI library that C, Python, or Rust hosts can load, so Jac is a citizen of the C ecosystem in both directions.

```
# Server codespace: a Python standard-library import
import datetime;

# Client codespace: npm packages, imported by name
```

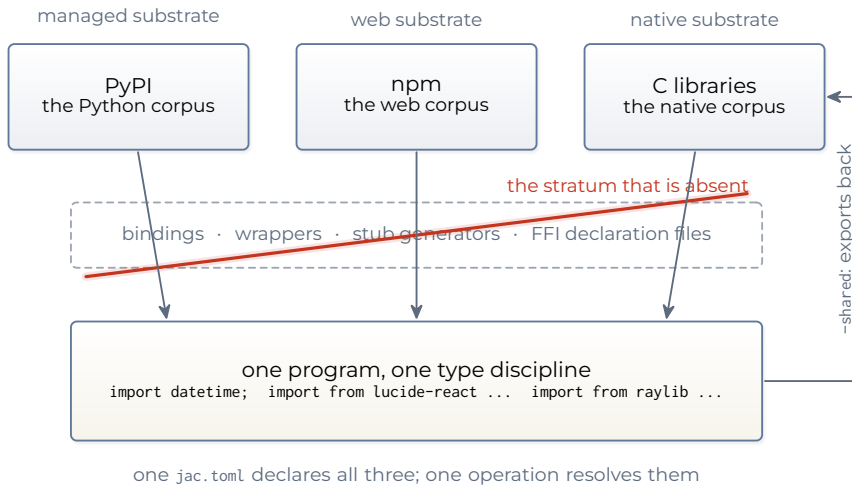


Figure 3.2: Ecosystem inheritance without wrappers. Each foreign corpus enters through the ordinary import statement of the codespace that lowers to its world, so the wrapper stratum a conventional stack maintains between program and corpus has nothing to do and does not exist. The doorway also reverses: a native module compiled shared exports its public functions to the C world, making the program a citizen of the corpus rather than a client of it.

```
import from "lucide-react" { Feather, Sun, Moon, Pencil }
import from "@jac/runtime" { jacSignup, jacLogin, jacLogout, jacIsLoggedIn }

# Native codespace: a C shared library, with ABI signatures
import from "/usr/lib/libm.so.6" {
  def sqrt(x: f64) -> f64;
  def pow(base: f64, exp: f64) -> f64;
}
```

Listing 3.3: Three ecosystems through one doorway. The first fragment is from the littleX backend, the second from the littleX frontend, the third from the language reference's native-compilation chapter.

Note what is absent. There is no interface definition language, no generated binding package to version and maintain, and no marshaling code at any call site. Only the C path requires declarations at all, and those declarations are Jac type signatures serving the role a header file serves for a C compiler: they describe the foreign symbol. They do not wrap it. This is the wrapper tax of Chapter 1 refunded, and it is also the closest a production system has come to the component economy McIlroy (1968) imagined: a routine written for one ecosystem consumed from another as a catalog part, with the cost of crossing driven to the cost of naming.

3.4 The Discontinuity That Isn't

The fourth commitment dissolves the client-server discontinuity. The classical web application keeps its most expensive discontinuity in the middle. A frontend and a backend, written in different languages, agree about their shared data only through discipline: route tables, JSON schemas, duplicated DTO classes, and client SDKs that drift from the servers they mirror. littleX has the same architecture, a browser client and a persistent server, and none of the artifacts. Listing 3.4 shows how. The frontend imports the server's walkers and archetypes directly, with the `sv` prefix marking a client-side import of server-codespace declarations. A client handler then invokes a walker with `root spawn`, an expression that reads as a local spawn and lowers to an authenticated HTTP call. The walker's `has` fields are the request payload. The values it reports come back in the result's `.reports` list. Walkers themselves are the subject of Chapter 7. For this chapter's purpose they are typed, spawnable units of server computation, and the service surface they project is developed in Chapter 14.

```
# frontend.cl.jac -- client codespace, importing the server surface
sv import from .social_graph { create_tweet, load_feed, get_profile }
sv import from .social_graph { Profile, Tweet, ProfileBundle, ChannelBundle }

# frontend.impl.jac -- a client handler invoking server walkers
impl app.postTweet -> None {
  if not composerText.strip() {
    return;
  }
  text = composerText;
  composerText = "";
  root spawn create_tweet(content=text);
  feedResult = root spawn load_feed(search_query="");
  tweets = feedResult.reports[0] if feedResult.reports else [];
  profileResult = root spawn get_profile();
  myProfile = profileResult.reports[0] if profileResult.reports else myProfile;
}
```

Listing 3.4: Cross-tier invocation in littleX, abridged from the frontend declaration and implementation files. The client imports server walkers and shared archetypes, then spawns walkers as if they were local.

The decisive element is the second import line. `ProfileBundle` and `ChannelBundle`, the objects of Listing 3.1, are declared once, in the server module, and imported into the client alongside the `Profile` and `Tweet` nodes they carry, where the application's state fields have types such as `list[Tweet]` and `ProfileBundle | None`. There is exactly one declaration of what a tweet looks like in transit (the `Tweet` node itself), and the compiler owns its representation on both sides of the wire. The data is isomorphic across tiers not because two schemas are kept in agreement but because there is nothing to disagree: no OpenAPI document, no mirror classes, no serializer registry. Even the access-control boundary is absorbed into the declaration surface: a spawned walker requires the caller's authenticated root by default, and marking a walker `walker:pub` (as littleX does for its cross-user profile directory) makes it an open endpoint. Authorization is a modifier on a declaration, not a middleware stack.

This design invites the objection Waldo et al. (1994) raised: the RPC tradition’s habit, running from Birrell and Nelson (1984) onward, of dressing a network call as a local one and hiding latency and partial failure until they strike. The surface answers the critique, along the lines Chapter 2 laid down for lawful boundaries. Every littleX handler that crosses the tier is declared `async`. The crossing is visible in the type of the operation, where the program’s control flow must acknowledge it, not in marshaling code, which acknowledges nothing and merely adds weight. What is dissolved is the discontinuity (the duplicated types, the route strings, the hand-run serialization), not the physics. Placed against the tierless lineage of Links, Hop, Ur/Web, Eliom, and ScalaLoc (Chlipala 2015; Cooper et al. 2006; Radanne et al. 2016; Serrano et al. 2006; Weisenburger et al. 2018), the design reads as a generalization. Those systems unified the tiers of a web application within one language and its own libraries. Here the unified tiers are also full citizens of two foreign ecosystems: the client tier composes npm’s React component world, and the server tier composes PyPI. The contract extends downward into persistence, because the graph the walkers traverse outlives the request without an object-relational layer (Chapter 10). The survey criteria of Weisenburger et al. (2020) classify multitier languages by degrees of placement and communication transparency. A synechic surface asks those criteria to range over ecosystems as well as tiers.

3.5 Language Mechanics of Separation

Continuity does not abolish structure. A surface that dissolves boundaries between substrates still needs boundaries within programs (between data and behavior, between interface and body), and Jac’s are chosen so that they compose with the substrate story. Two mechanisms carry the weight: the archetype system and the declaration–implementation split.

Archetypes are the declaration forms of Jac’s nominal types. `obj` declares a plain object (fields introduced by `has`, a constructor derived from them, methods by `def`) and is the form Python’s classes map onto. `node` and `edge` declare the vertices and typed relations of a topology, and `walker` declares a mobile unit of computation whose methods include *abilities*: bodies prefixed `can ... with` Type entry, dispatched not by call but by arrival at a location of the named type. The listing in Listing 3.5 shows the server side of the invocation in Listing 3.4: a walker with one typed field, an ability that runs at the caller’s root and steers toward the profile, and an ability that runs on arrival to attach a new node. The design point, ahead of the paradigm’s full development in Chapter 7, is uniformity: a Python-shaped `obj` and a spatial walker are declared in the same form, typed by the same checker, and separated by the same mechanics. The new semantics extend the surface rather than fork it.

```
walker create_tweet {  
  has content: str;  
  
  can run with Root entry {  
    visit [-->[?:Profile]];  
  }  
}
```

```

    can make with Profile entry {
      new = here +>:Post():+> Tweet(
        content=self.content, author_username=here.username, created_at=_now()
      );
      grant(new, level=WritePerm);
      report new;
    }
  }
}

```

Listing 3.5: A walker from the littleX backend, verbatim. Fields are declared with `has`; abilities are dispatched by arrival at a node of the named type.

The second mechanism separates what a construct is from how it works. Any method or function may be declared as a bare signature, its body attached elsewhere by an `impl` block: in the same file, or by convention in a sibling `.impl.jac` file. littleX uses the split at full scale: the app component in `frontend.cl.jac` declares some two dozen handlers as one-line signatures (Listing 3.6), and `frontend.impl.jac` holds every body. We have already read one in Listing 3.4. The split composes with codespaces through variant modules: files sharing a base name (`mymod.jac`, `mymod.sv.jac`, `mymod.cl.jac`, `mymod.impl.jac`, an `impl/` subdirectory) are discovered and compiled together, with implementations matched to declarations across variants. The convention is not decorative. The self-hosted compiler (Chapter 12) is written under the same discipline: the core modules in `jaclang/jac0core/` keep their bodies in an `impl/` directory (`archetype.impl.jac`, `compiler.impl.jac`, `runtime.impl.jac`), so the split scales to the system’s own foundations.

```

# frontend.cl.jac -- inside the app component: interface only
async def loadData -> None;
async def postTweet -> None;
async def handleLike(tweetId: str) -> None;
def navigate(tab: str) -> None;

```

Listing 3.6: Interface without implementation: handler declarations inside the littleX frontend component. Bodies live in the sibling implementation file.

In a single-substrate language, declaration–implementation separation is a code-organization preference. In a synechic language it does architectural work. The declaration is the cross-substrate face of a construct: the part a client needs in scope to type-check a root `spawn`, and the part the checker needs to verify a contract. The implementation is substrate-local, free to change without disturbing anything across the boundary. The discontinuity between tiers, dissolved as glue code, reappears in healthy form as a boundary: an interface written in the language itself.

3.6 Design Rules for Synechic Surfaces

The commitments of this chapter are Jac’s, but nothing in them requires Jac. We close by restating them as rules a designer of any synechic surface can apply, in the spirit of the design space mapped in Chapter 18.

Principle 3.1 (Contain an incumbent, whole). A synechic surface should semantically contain an existing ecosystem language, such that every program of the incumbent has

a meaning-preserving spelling in the new surface and modules cross the boundary in both directions.

Containment is what makes ecosystem inheritance real. An interoperability layer grants access to an ecosystem at a per-call price. An embedding grants membership. The test is monotonicity: adopting the surface must never require abandoning working code, and partial adoption must be a stable resting point, not a transitional embarrassment.

Principle 3.2 (Add semantics, not only discipline). The superset’s additions should include constructs the incumbent cannot express, chosen to dissolve the discontinuities the incumbent forces its programmers to bridge by hand.

A type-only superset improves the checking of existing programs. That is worthwhile, and TypeScript proved its market. But a surface earns the synechic name by what it makes expressible (placement of code on substrates, computation over persistent topology, delegation to stochastic executors), because these are the concerns that otherwise leak out of the language and into glue.

Principle 3.3 (Make the substrate a modifier, not an architecture). Where code runs should be declared at the granularity of a declaration — a tag, a block, a file suffix — inside one program, never at the granularity of a project.

When the unit of tier assignment is the repository, moving a function across the boundary is a rearchitecture. When it is the declaration, it is an edit. Everything else in the synechic argument (shared types, glue-free invocation, one toolchain) follows more easily once the boundary is small enough to be spelled next to a name.

Principle 3.4 (One doorway for every ecosystem). Foreign libraries from every supported ecosystem should enter through the language’s ordinary import path, with their manifests unified, and with declarations required only where an ABI must be described.

The audit question of Section 3.3 is the enforcement mechanism: if using a package requires generating, vendoring, or maintaining a wrapper, the surface has delegated its continuity to a cottage industry. A manifest that names PyPI, npm, and C dependencies in one file is the toolchain-level expression of the same rule (Chapter 4).

Principle 3.5 (One declaration per contract; lawful boundaries as effects). Every datum that crosses a dissolved boundary should have exactly one declaration, with representation owned by the compiler on all sides; the boundaries that lawfully remain — latency, partial failure, cost — should surface as typed effects at the crossing, never as marshaling code.

This is the rule that answers Waldo. The failure mode of transparent distribution was never the absence of ceremony. It was concealment. A surface that deletes the duplicate schema but keeps the `async` on the handler deletes the discontinuity, which was accidental, and keeps the boundary, which is physical.

Five rules do not make a language, and Chapter 4 will immediately complicate the picture: beneath every one of these surface commitments sits a toolchain that must

build, serve, link, and package across the same three substrates the syntax spans. The surface can only be as continuous as the machinery beneath it. The toolchain, the hidden tier, is the next subject.

Bibliographic Notes

The supersetting strategy is best studied through its precedent: Bierman et al. (2014) give the definitive analysis of TypeScript’s containment of JavaScript, including the deliberate unsoundness that adoption economics forced, and Steele (1999) remains the essential statement of language growth as a design constraint. The gradual-typing foundations that make a typed superset of a dynamic language tractable are due to Siek and Taha (2006), with the refined criteria in (Siek et al. 2015). Their role in Jac’s type system is taken up in Chapter 9. For the substrate side, Lindholm and Yellin (1999) document the virtual-machine strategy this chapter inverts, and Lattner and Adve (2004) describe the compilation framework beneath Jac’s native lowering.

The dissolved client–server discontinuity descends from two literatures. Multitier programming begins with Links (Cooper et al. 2006) and Hop (Serrano et al. 2006), reaches typed maturity in Ur/Web (Chlipala 2015) and Eliom (Radanne et al. 2016), extends to general distributed placement in ScalaLoci (Weisenburger et al. 2018), and is surveyed by Weisenburger et al. (2020). Chapter 2 formally places tierlessness as restricted synechism. The remote-invocation tradition runs from Birrell and Nelson (1984) to the critique of Waldo et al. (1994), which principle five of this chapter is designed to satisfy rather than evade. McIlroy (1968) supplies the component-economy vision behind ecosystem inheritance. On mechanics of separation, Kiczales et al. (1997) represents the broader research program of letting programs be organized by concern rather than by control structure, of which Jac’s declaration–implementation split is a deliberately conservative relative. The system context for littleX and its ancestors in the Jaseci stack is given by Mars et al. (2023).

THE TOOLCHAIN AS HIDDEN TIER

The definition of substrate transparency in Chapter 2 names three identities that must have no bearing on how a program is expressed: the runtime, the ecosystem, and the toolchain. Chapter 3 showed how Jac’s surface dissolves the first two: one syntax entering the Python, JavaScript, and native substrates as a first-class citizen, inheriting each ecosystem without wrappers. The third identity is the least discussed in the programming-languages literature and, we will argue, the most corrosive in practice, because it hides. The runtime discontinuities of Chapter 1 announce themselves: an HTTP request has a URL, a foreign function call has a binding, a serialization format has a schema. The toolchain discontinuity is crossed before the program runs, so its cost is billed everywhere but to the language: to “setup,” “operations,” or “developer experience.”

This chapter makes the case that the build system is a substrate boundary like any other, and that a language claiming synechic continuity must therefore internalize its toolchain or watch the continuity claim fail at the edge of every project. We then descend beneath the language surface to the artifact that carries it: the jac binary, a native launcher stub with the entire runtime (a private CPython, the sealed compiler, a statically linked LLVM, a JavaScript runtime, a linker, and the C floor it links against) appended to the same file. The design is not an installer convenience. It is the engineering that lets one language subsume the package managers of three ecosystems under one verb, and lets building, serving, and deploying become operations of the language. We then assemble the catalog those operations produce: twelve project kinds, from command-line tool to mobile application, all projections of one program.

We close with the objection the design invites. A fused binary is large, and forty years of engineering conscience (Wirth’s plea for lean software above all) counsels suspicion of large artifacts. We confront the tension directly: the binary is large because the stack it replaces is larger, and the quantity it actually eliminates is not measured in bytes.

4.1 The Build System as a Substrate Boundary in Disguise

In this section, we show that the build system is a tier of the system like any other, hidden only because no diagram draws it. A tier, in the vocabulary of Chapter 1, is a territory of a system with its own languages, its own runtime, its own failure modes, and its own specialists, joined to its neighbors by glue. The build system qualifies on every count. Its languages are Make and CMake, the JSON of `package.json`, the TOML of `pyproject.toml`, the plugin-and-loader configuration of the bundler, Dockerfiles, and the YAML dialects of continuous-integration pipelines. Each is a genuine programming language, with evaluation order, scoping quirks, and failure semantics of its own. Its runtime is the developer's shell and the CI executor. Its specialists are build and release engineers, a profession that exists because this tier is real. Its glue is shell scripts, environment variables, and lockfiles. The only thing that distinguishes it from the tiers on the architecture diagram is that nobody draws it.

The disguise works because toolchain discontinuities are crossed at a different time than runtime discontinuities, not at a different kind of boundary. Marshaling glue mediates between two components executing simultaneously in different substrates. Toolchain glue mediates between the same component at two moments: between source and artifact, between the author's machine and a colleague's, between the toolchain as configured today and as it will have drifted in six months. It is glue in time rather than in space. Version skew is the temporal analogue of a serialization mismatch: two parties exchanging an artifact under incompatible assumptions about its encoding. "Works on my machine" is a marshaling failure whose wire format is the filesystem and whose parties are separated by weeks.

Under the definition of substrate transparency, the two failures are literally the same defect. Adding a dependency is a program-meaningful act: it changes what `import` denotes. Yet in a conventional stack it is expressed outside the program, in a foreign dialect, once per ecosystem. For example, one new dependency becomes a line in `requirements.txt` or `pyproject.toml`, another in `package.json`, a system package or a submodule for the C library, then a lockfile per manifest, each with its own resolution semantics. Choosing a compilation target, provisioning a server, or deploying a service likewise requires authoring artifacts in dialects that are about the program but not of it. At each such point the identity of the toolchain visibly bears on expression, which is what substrate transparency forbids.

The history surveyed in Appendix F repeats the pattern at this boundary. The JVM unified the execution artifact (write once, run anywhere) but left dependency resolution, building, and deployment outside the abstraction, where an ecosystem of Ant, Maven, and Gradle grew to fill the gap (Lindholm and Yellin 1999). The multitier languages unified the expression of client and server, but the projects that hosted them were assembled with conventional toolchains. The program was tierless while the repository was not (Weisenburger et al. 2020). McIlroy (1968) imagined mass-produced software components with the machinery of industrial distribution behind them. What eventually arrived was the package manager: one per ecosystem, mutually unintelligible, each a separate dialect of the same idea. The result is a large share of what Brooks (1987) called accidental complexity, concentrated in the one tier that no language had claimed responsibility for. A synechic language must claim it.

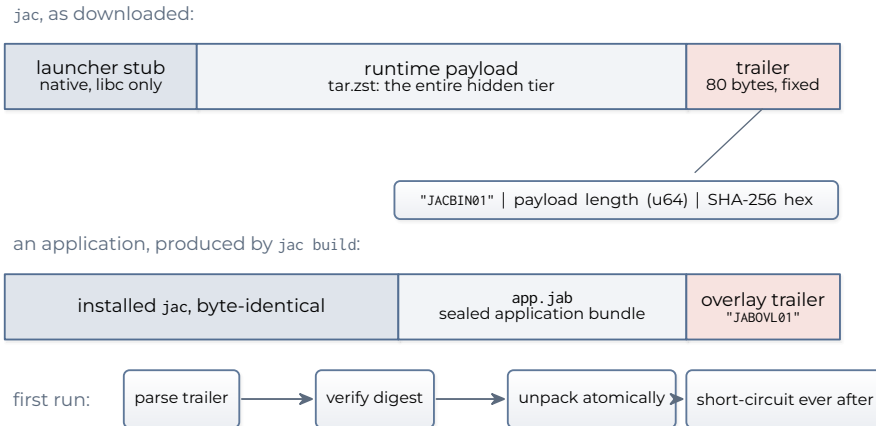


Figure 4.1: On-disk anatomy of the self-contained toolchain. The downloaded binary is a native launcher stub with the runtime payload appended and an 80-byte trailer addressing it; an application executable appends a sealed bundle to a byte-identical copy of the same file under a second trailer. First run verifies and unpacks once; every later start touches only the marker.

The remainder of this chapter is the record of what claiming it requires.

4.2 Anatomy of the Self-Contained Artifact

The jac binary a user downloads is one file with two parts. The first is a small native launcher stub: machine code linked against libc only, with no dependency on any Python, Node, or compiler installation. Indeed, no Python is linked at the stub's own build time. The second is the runtime payload, a compressed archive appended to the same file and addressed through an 80-byte trailer at the end of the file: an eight-byte format marker, the payload length, and the SHA-256 digest of the payload in hex. Listing 4.1 shows the layout, drawn in Figure 4.1, together with the shape of an application executable derived from it, to which we return in Section 4.4.

```

jac = [ launcher stub ][ runtime payload (tar.zst) ][ trailer ]
trailer = "JACBIN01" | payload length (u64) | sha256 hex
          (80 bytes, fixed, at end of file)

app = [ installed jac, byte-identical ][ app.jab ][ overlay trailer ]
overlay trailer = "JABOVL01" | length | sha256 hex (same codec)
  
```

Listing 4.1: On-disk shape of the jac binary, and of an application executable produced by appending a sealed app bundle to a copy of it.

On first run the launcher parses the trailer, verifies the payload against its digest, and unpacks it into a per-version cache directory. The directory's name is derived from the first sixteen hex characters of that digest, folded together with a hash of the binary's own path, so two installed versions coexist and two co-located checkouts with identical payloads get distinct trees. The extraction is atomic (a temporary directory

Table 4.1: Contents of the jac artifact, the host tooling each component displaces, and each component’s constitution: written in Jac, or carried as an external engine. The design goal stated in this section is for every row above the operating system to eventually read Jac.

Component carried in the artifact	Host tooling displaced	Constitution
Native launcher stub (links libc only)	—	bootstrap seed carried
Private CPython 3.14 (pinned, optimized, stripped)	Python, pyenv, conda	
Jac compiler and runtime, sealed as precompiled JIR	server, deploy, and LLM frameworks	Jac
Bun executable, invoked by absolute path	Node.js, npm, npx, yarn	carried
LLVM 22, statically linked behind one shared library	gcc, clang	carried
Linker plus static libc, crt, musl, and wasm libc	ld, lld, make, cmake, emscripten	Jac; libc carried
Package manager (pip in-payload; npm via Bun)	pip, pipx, uv, poetry, virtualenv	carried
Type checker with vendored typeshed stubs	mypy, pyright, tsc	Jac
Formatter, test runner, language server, agent tooling	black, ruff, pytest, jest	Jac
Editor (pinned Neovim fork, linked into the stub)	a separate editor and LSP setup	carried

renamed into place, with a completion marker guarding against partial unpacks), and stale versions are garbage-collected. Every subsequent run short-circuits on the marker. After the one-time unpack, startup does not touch the payload again.

What the payload contains is the entire hidden tier. Table 4.1 gives the inventory alongside the host tooling each component displaces, condensed from the project’s own accounting.

Several rows deserve elaboration. The interpreter is a private build of CPython 3.14 (a pinned python-build-standalone release, optimized with profile-guided and link-time optimization and then stripped) that the launcher loads with `dlopen` at startup and drives in-process. The system Python is never consulted. The launcher pins the interpreter’s notion of its own executable back to the jac binary itself, so that anything which re-spawns the interpreter (multiprocessing pools, test-runner workers) lands back inside the same artifact, never on whatever `python3` happens to be on the path. The interpreter is pointed at its private tree through its initialization configuration rather than through environment variables, so that child processes do not inherit a distorted view of the world. The launcher takes the hygiene of the substrate boundary seriously even across process spawns.

The compiler and runtime ship precompiled to JIR, the compiler’s intermediate representation (Chapter 12), as a sealed image: a manifest maps module names to their compiled forms, several hundred modules resolve from the manifest without touching the filesystem or invoking the compiler, and a cold start performs no live compilation. Sealing is a trust decision as much as a performance one: the runtime fails closed on a manifest or format mismatch rather than degrading to compiling whatever source it finds. But it is not concealment. The sources ship alongside the compiled image so that tracebacks and introspection render faithfully. The JavaScript substrate is served by the actual Bun executable, carried inside the payload and invoked by absolute path,

never placed on the user’s path. The native substrate is served by LLVM statically linked into a single shared library (Lattner and Adve 2004), and by the toolchain’s own linker, which writes ELF, Mach-O, PE, and WebAssembly images directly (Haas et al. 2017). The static libc and C-runtime archives, a musl runtime for Linux targets, and WebAssembly libc bitcode ride in the payload, so “the C toolchain” is not an external assumption but cargo. Even the editor is aboard: a pinned fork of Neovim is statically linked into the launcher stub itself, so the editing verb boots in milliseconds without initializing the interpreter at all.

The launcher’s own build discipline mirrors what it ships. The tool that assembles the payload fetches each component (the CPython tree, the typedshd stubs at a pinned commit, the LLVM archives) over HTTP and verifies every one against pinned cryptographic digests before it is staged, so the supply chain of the artifact is anchored at build time, not trusted at install time. We return to what this buys in Section 4.6.

The constitution column of Table 4.1 records a design position that deserves explicit statement. The components of the payload divide into two groups. The first group is written in Jac: the compiler and runtime (roughly 190,000 lines, Chapter 12), the type checker, the formatter and test tooling, and the linkers. The second group consists of external engines carried in binary form: the CPython 3.14 interpreter, the Bun JavaScript runtime, LLVM 22, the static C runtime archives, and the editor. The external engines are included for one reason: feature completeness. Each provides, today, a level of compatibility and performance that a reimplementation would not, and shipping the established engine is preferable to shipping a partial imitation of it. The division is not permanent, however. Our ultimate design goal is to replace each carried engine with a Jac implementation, so that the entire artifact above the operating system is written in the language it ships. The goal rests on two facts: 1) each carried engine is a discontinuity in the sense of Chapter 1, reached across an ABI, built by a foreign toolchain, and outside the jurisdiction of the language’s own verifier, and 2) gradual borrow checking (Chapter 5) removes the reason these components were written in C and C++ in the first place, since its strongest position produces native code with no memory-management runtime. Digest pinning (Section 4.6) manages the boundary each engine introduces. A Jac implementation removes it. We refer to the division between the two groups as the *self-hosting frontier*, and the record so far shows it moving in one direction only: the compiler was rewritten in Jac first, the linkers followed (displacing `ld` and `lld`), and no component has moved the other way. Figure 4.2 draws the frontier through the payload: the rows above the line are settled, the rows below it are carried, and the dashed line at the bottom is the boundary the artifact concedes rather than claims. We make no claim about schedule. Two components bound the goal: the operating system, which Section 4.6 concedes as the floor, and the cryptographic primitives, which will be replaced only when a Jac implementation can be validated to the same standard.

4.3 Package Management Across Three Ecosystems as One Operation

Chapter 3 established that a Jac program imports PyPI, npm, and C-level functionality without wrappers. The toolchain half of that claim is that acquiring those dependencies

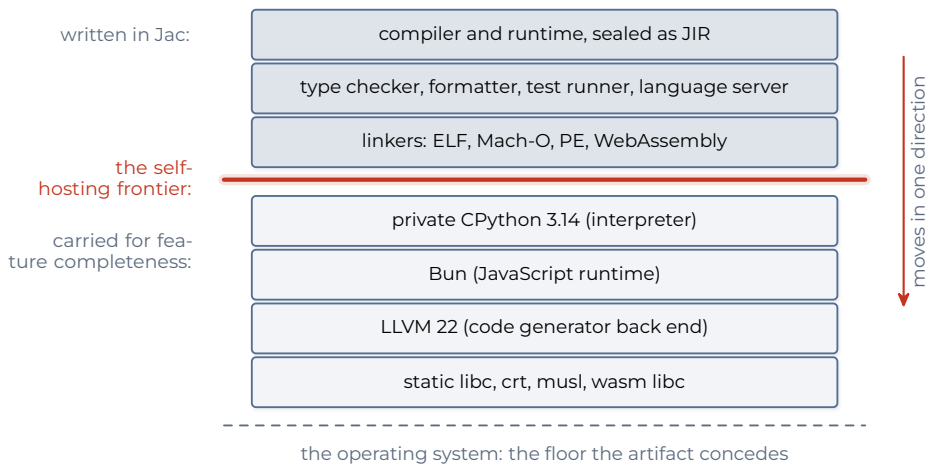


Figure 4.2: The self-hosting frontier across the payload of Table 4.1. Components above the line are written in Jac. Components below it are external engines carried in binary form for feature completeness, digest-pinned but outside the verifier’s jurisdiction. The frontier has moved only downward: the compiler crossed first, then the linkers. The design goal is for the frontier to reach the operating-system line.

is one operation against one manifest. A project declares its Python and npm dependencies side by side in `jac.toml`, as in Listing 4.2, drawn from the littleX application studied in Chapter 15.

```
[project]
name = "little-x"
version = "1.0.0"
entry-point = "main.jac"

[dependencies]
numpy = ">=1.24.0"
scikit-learn = ">=1.3.0"

[dependencies.npm]
react = "^18.2.0"
react-dom = "^18.2.0"

[dependencies.npm.dev]
typescript = "^5.3.3"

[dev-dependencies]
watchdog = ">=3.0.0"
```

Listing 4.2: A project manifest declaring Python and npm dependencies side by side, excerpted from the littleX example application.

A bare `jac install` synchronizes the project environment to this manifest: Python packages, git dependencies, and npm packages resolve in one command, into a

project-scoped virtual environment the tool creates and owns. Naming a package (`jac install numpy`) installs it and records a compatible-release constraint back into the manifest. Flags select the ecosystem and scope (`-npm` for client-side packages, `-global` for tools installed once into the binary’s own site and importable from any project, `-plan` to preview resolution without installing). The flag surface matters less than what executes underneath: `pip` runs inside the payload’s private interpreter, and `npm` resolution runs through the carried `Bun`. No host Python, `pip`, Node, or `npm` is consulted at any point, so the operation behaves identically on a machine with seventeen Pythons and a machine with none. The version-skew discontinuity of Section 4.1 is not managed. It is removed, because there is no second toolchain whose version could skew.

The same unification extends to running what was installed. `jac x` resolves a command-line tool from either ecosystem (a Python console script from the project environment or the global site, an `npm` tool from the project’s `node_modules`) and runs it under the correct substrate automatically, Python tools in-process under the bundled interpreter and `npm` tools through `Bun`. It is one verb where the conventional stack has `pipx`, `npx`, and a shell-activation ritual.

The third ecosystem, C, is inherited on different terms, because C has no central registry to manage. What a conventional stack must provision from the C world is the toolchain itself (compilers, linkers, headers, and runtime libraries), and that provisioning is what the payload’s LLVM, linker, and vendored `libc` floor internalize. The C ecosystem’s “package manager” is, in effect, the anatomy of Section 4.2. And the relationship runs both ways: the build projections of the next section emit wheels for PyPI, tarballs for `npm`, and C-ABI shared libraries callable from any language with a foreign-function interface, making Jac a producer into each of the three ecosystems as well as a consumer. This bidirectional membership is the third requirement of the polypiler of Chapter 12, and the toolchain is where its publish half lives.

4.4 Building, Serving, and Deploying as Language Operations

In a conventional stack, “build,” “serve,” and “deploy” name practices: sequences of foreign-tool invocations that the engineer composes, encodes in scripts and CI configuration, and maintains as the tools drift. In Jac they are operations: verbs of the toolchain whose semantics are defined against constructs of the language. Listing 4.3 shows the lifecycle.

```
jac run main.jac      # execute directly
jac dev               # hot-reload development loop
jac start main.jac    # serve: REST API, auth, docs, client
jac start --scale      # deploy the same program to Kubernetes
jac build              # type-check gate, then a sealed .jab bundle
jac build --as binary  # one self-contained app executable
jac build --as native  # small LLVM-compiled native binary
```

Listing 4.3: The lifecycle of a Jac program expressed as toolchain verbs.

The claim that these are language operations rests on where their meaning comes from. `jac start` does not wrap a web framework. It projects the program’s own

declarations into a service. A walker marked public becomes an HTTP endpoint whose request body maps onto the walker's fields and whose reports become the JSON response. Interactive documentation and authentication are generated from the same declarations, and each user's traversals are isolated to their own persistent graph. The projection is semantic (Chapter 14 develops it into the claim that program meaning is invariant under deployment scale), and it is only expressible because the server lives inside the language's own toolchain, where the compiler's knowledge of the program is available to it.

`jac build` likewise begins in the type system: it runs the whole-program type check as a fail-closed gate and refuses to emit an artifact that does not pass. What it then emits is a choice of projection over one program. The default is a sealed application bundle: the same manifest-plus-JIR image format the toolchain itself boots from, compressed into a single file that any `jac` runs or serves with zero live compilation. The binary projection appends that sealed bundle onto a byte-identical copy of the running launcher, using the overlay trailer of Listing 4.1, yielding one executable that carries the full runtime to a machine with nothing installed. At boot, the launcher detects the overlay, steps over it to find its own payload, and hands the application image to the same loading path as any bundle. The native projection compiles the restricted native subset ahead of time through the embedded LLVM into a small, dependency-free executable. The `wheel` and `npm` projections emit the program into the packaging formats of the neighboring ecosystems. One program yields a family of artifact shapes, each selected by a flag, where the conventional stack maintains a separate build system for each.

Deployment receives the same treatment. `jac start --scale` builds the container images, provisions the datastore and cache the runtime needs, and deploys to Kubernetes with health checks, all from one flag. The runtime can do this because it knows what a Jac service requires, whereas a generic orchestrator cannot know what any given application requires and therefore must ask, in YAML, at length (Burns et al. 2016). The industry's prior attempt to absorb operations, serverless computing, moved the burden into the platform and thereby traded a configuration discontinuity for a vendor one (Jonas et al. 2019). Absorbing it into the language's own toolchain keeps what replaces that discontinuity under the same roof as the semantics. The deeper argument, that the program's meaning is invariant under this change of deployment scale, belongs to Chapter 14. Here the point is narrower but essential: nothing about deploying required leaving the language.

4.5 Project Kinds: The Product Space of One Program

The previous section ended on a claim worth pressing: one program yields a family of artifact shapes, each selected by a flag. In this section, we assemble the family into a catalog and name its organizing concept. The toolchain calls each shape a *project kind*: a declaration, under `[project]` in `jac.toml`, of what the project is for, from which one verb derives everything else it must know. A kind composes the codespaces of Chapter 3 and applies one of four delivery treatments to the result: the artifact is executed, served, packaged, or wrapped in a native shell. There are twelve, and Table 4.2 factors each into its composition and its treatment.

Table 4.2: The twelve project kinds. Each kind composes the three codespaces under one of four delivery treatments, so the catalog is a small space of combinations rather than twelve modes of the language.

Kind	Composes	Treatment	Product
cli	server	executed	command-line program
cli-native	native	executed	compiled program, run in place
native-binary	native	packaged	standalone zero-dependency executable
native-lib	native	packaged	C-ABI shared library
service	server	served	headless REST API
service-mesh	server, replicated	served	microservices, providers auto-started
py-package	server	packaged	wheel for PyPI
js-package	client	packaged	tarball for npm
web-app	server + client	served	full-stack application
web-static	client + native	served	in-browser page, native code as WebAssembly
desktop	server + client	served, shelled	application in the operating system's webview
mobile	client	packaged, shelled	mobile bundle for Android and iOS

Read down the composition column of Table 4.2 and the catalog collapses into arithmetic. A full-stack application is a service plus a client. The web-static kind swaps the server out of that sum and puts native code, compiled to WebAssembly, in its place, so the heavy loop runs in the browser rather than behind it. A desktop application is the full-stack application handed to the operating system's webview. The mobile kind ships the client half to a phone in a webview shell and reaches its server over HTTP, and a React Native variant of the same shell replaces the webview with platform-native views. Microservices are the service replicated, with each `sv` import marking a boundary (Chapter 14). And the three packaged library kinds are the producer half of the bidirectional membership established in Section 4.3: a wheel into PyPI, a tarball into npm, a shared library into the C world. Nothing in the table is a mode of the language. Every row is a placement of the same program's parts under one of four treatments.

Most projects never declare their kind. The toolchain assigns kinds the way the compiler assigns codespaces: by inference, from what the program is. An entry point in the native codespace makes the project `cli-native`. An entry point marked `server` (`main.sv.jac`) makes it a service. A client entry point, or a client dependency in the manifest, makes it a `web-app`. A markerless entry is a `cli`. An explicit kind in the manifest overrides the inference, exactly as a file suffix overrides codespace inference, and is needed only where the entry point underdetermines the intent (a `web-static` project, whose entry looks like any other client module). The parallel is the point, and Chapter 12 states it in full: `substrate`, `tier`, `service`, and `kind` are one placement decision computed at four granularities. Principle 3.3 required that where code runs be declared at the granularity of a declaration, never at the granularity of a project. The kind extends the same rule one level up: what the project becomes is a projection over the program, selected by one line and usually by none, never an architecture the repository is arranged around. For example, adding one client component to a headless service changes the inferred kind from `service` to `web-app`, and the next `jac` start builds and serves a client alongside the walkers it already served. No file moved.

The verbs of Section 4.4 read the kind rather than asking the engineer to remember it. A bare `jac run` executes the runnable kinds, serves the served kinds, and builds the artifact kinds, and `jac run -show` prints the resolved plan without running it. A bare `jac build` type-checks and then emits the kind's own artifact: a sealed bundle for the applications, a wheel or a tarball for the packages, a native executable for the binaries. The kind also has a birth verb. `jac create -kind service` stamps the manifest and emits the scaffold, and the scaffold of every core kind is one entry file plus the manifest itself. The near-emptiness is the observation worth keeping. A conventional starter template is large because it must pre-generate the glue its architecture will demand: build configuration, client-server wiring, a deployment skeleton. A kind has no glue to pre-generate. The scaffold is small because the architecture it would otherwise encode has moved into the language.

The edges of the catalog deserve the same precision as its rows. The desktop and mobile kinds are the youngest of the twelve. They work, the toolchain's own documentation grades them beta, and they are the only rows that need toolchains the artifact does not carry: a system webview library on Linux, the Android or iOS toolkits for the phones. We claim no more for them here than the table does. The remaining rows are exercised by the case studies of Chapter 15: littleX is a web-app, the shooter of that chapter ships as a native-binary and again as web-static, one game delivered two ways, and the four-tier showcase composes served, packaged, and native rows in one repository. The eject projection weighed in Chapter 17 belongs to the same family, one more thing a program can become. And the claim recorded in Appendix D, that every project kind builds through one command into a sealed application image, is this section's table restated as history.

4.6 Reproducibility, Versioning, and Trust in a Fused Binary

Fusing the toolchain into one artifact changes the answers to three questions that every engineering organization must ask: whether it can rebuild what it ran, whether its members agree on what they are running, and why it should believe that the thing it downloaded is what it thinks it is.

Reproducibility and versioning collapse into a single number. In a conventional stack, "the version of the toolchain" is a vector (interpreter, package manager, JS runtime, bundler, compiler, linker, orchestrator), and reproducing an environment means reproducing the vector, which is why lockfiles, version managers, and container images exist. In the fused design the vector has one component: the `jac` version pins the interpreter patch release, the Bun release, the LLVM major, the typeshed commit, and the compiler itself, all sealed under one payload digest. The per-version cache keyed by that digest means upgrading is installing a new file and rolling back is running the old one. Two versions coexist without a version manager, because the artifact is its own version manager. The digest-keyed cache is a modest instance of an idea that Unison carries much further (identifying code by content rather than by name and hoping the names still resolve (Unison Computing 2024)), applied here to the toolchain, not to user definitions.

Trust follows the same chain of custody. The payload is verified against its trailer digest before first use. The compiler image boots from a sealed manifest and fails

closed on any mismatch, which moves the trusted step from every startup to the one build that produced the manifest. Application bundles are hash-verified at load, and the components of the toolchain itself were digest-pinned when the artifact was assembled. None of this is exotic cryptography. It is the ordinary discipline of content addressing, applied uniformly because one party owns the whole artifact. The conventional stack cannot apply it uniformly, because no one owns the whole stack.

And yet the artifact is large. It carries an interpreter, a JavaScript runtime, a statically linked LLVM, a linker, C runtime archives, type stubs, and an editor. By no measure is that small, and Wirth (1995) taught the field to treat such girth as a symptom of decadence. The charge deserves a direct answer, and the answer has two halves. The first is that the unit of account is wrong. Wirth's complaint was leveled at software whose mass grows without corresponding function: fat in the organism. The fused binary's mass is not new function piled onto a language. It is the same stack the developer would otherwise install piecewise (Python and its version manager, Node and its package tree, compilers, linkers, build tools, and the frameworks the language absorbs), relocated into one file with one owner. Measured as a system, the fused artifact is the smaller configuration: fewer installed components, and, more importantly, exponentially fewer reachable states, because the version vector has collapsed to length one. The binary is large because the stack it replaces is larger, and what the fusion eliminates is not bytes but combinatorics. Complexity, not disk, was always the substance of Wirth's plea. Note also that the mass concentrates in the carried engines of Table 4.1: the components written in Jac contribute a small fraction of the bytes, and each crossing of the self-hosting frontier converts carried binary into checked source without growing the artifact.

The second half is a concession, because the first half is not the whole truth. Fusion means everyone carries everything: the user who never compiles natively still downloads LLVM, and a monolithic release couples the upgrade cadence of components that upstream ships separately. The abstraction also leaks at its floor, as abstractions do (Spolsky 2002): the binary internalizes the toolchain but not the operating system, so system libraries, drivers, and platform quirks remain a discontinuity, relocated to a single, owned, testable place but not abolished. And a language that absorbs its toolchain invites the lock-in objection with unusual force, which is why the toolchain itself carries the counterweight: the source projection of `jac build` ejects an application as an editable server-and-JavaScript source tree containing no Jac files at all. We weigh that objection, and the others this chapter has earned, in Chapter 17.

Bibliographic Notes

The intellectual frame for this chapter (components, accident, and the cost of girth) comes from three classics. McIlroy (1968) proposed mass-produced software components at the 1968 Garmisch conference, a vision whose eventual realization as per-ecosystem package managers is discussed in Section 4.1. Brooks (1987) supplied the essence-and-accident vocabulary in which we class toolchain labor as accident. And Wirth (1995) is the conscience against which Section 4.6 measures the fused

binary. The claim that prior unifications stopped at the toolchain boundary can be checked against the primary sources: the Java Virtual Machine specification (Lindholm and Yellin 1999) for the portability-by-artifact tradition, and the multitier survey of Weisenburger et al. (2020) for the tierless languages, whose treatment in this book continues in Appendix F and Chapter 2.

On the machinery itself, Lattner and Adve (2004) describes the LLVM infrastructure that the artifact links statically for its native pathway, and Haas et al. (2017) the WebAssembly format its linker emits for the browser substrate. The deployment discussion draws on Burns et al. (2016) for the lineage from Borg to Kubernetes (instructive because it documents how much configuration a general-purpose orchestrator must externalize) and on the Berkeley view of serverless computing (Jonas et al. 2019) for the platform-side attempt to absorb operations. Spolsky (2002) names the law under which we concede the operating-system discontinuity, and Unison (Unison Computing 2024) is the nearest contemporary in identifying software by content hash, a discipline the launcher applies to its runtime payload. The primary sources for the anatomy of Section 4.2 are the repository’s launcher sources and their documentation, which the artifact guide of Appendix E describes how to obtain and verify. The kind catalog of Section 4.5 is checkable the same way: the registry of the twelve kinds ships in the toolchain’s project subsystem, and its documentation carries the user-facing composition matrix.

THE MEMORY DISCONTINUITY: GRADUAL BORROW CHECKING

A LISP programmer knows the value of everything, but
the cost of nothing.

Alan J. Perlis, 1982

Part II has carried the continuity claim across an escalating series of substrates: the managed, web, and native runtimes of Chapter 3, and the build, serving, and deployment machinery of Chapter 4. Every boundary dissolved so far has separated a language from its infrastructure. One boundary cataloged in Part I is different in kind, because the field services it with a second language. It is the divide between the managed languages and the systems languages, drawn where memory discipline changes: on one side, reclamation is the runtime's affair and no loop can be certified free of it, and on the other, reclamation is the program's affair and the discipline is levied on every line. Chapter 1 counted the junction between the two worlds as the inventory's oldest glue category, the foreign-function interfaces and bindings. Chapter 2 then filed the divide itself among the accidental boundaries, on a promise of demonstration (Section 2.4). This chapter redeems that promise.

The structure of the argument is parallel to the chapters before it. We first establish that memory discipline marks a substrate divide in the full sense of this book's definitions, and that the boundary programs cross to reach the disciplined world is a discontinuity of the kind cataloged in Chapter 1 (Section 5.1). We then present the language's answer in three moves. A small annotation surface, the ownership dial, introduces affine ownership one declaration at a time over the managed language (Section 5.2). A closed boundary, the membrane, mediates every crossing between the owned and managed worlds (Section 5.3). A pair of guarantees makes the dial trustworthy, and a verifiable endpoint makes its strongest position a checkable

property of the shipped binary rather than an optimizer’s achievement (Sections 5.4 and 5.5).

5.1 Memory Discipline as a Substrate Divide

In this section, we state the divide and measure it against the definitions of Chapter 2. Every mainstream language answers the memory-management question once, for every program it will ever compile. Rust asks the programmer to accept ownership discipline everywhere: every binding is affine, every alias is policed, and the borrow checker must be satisfied before the first line prints (Matsakis and Klock 2014). The reward is real, and it is the reward systems builders leave managed languages to obtain: a runtime with no collector, deterministic destruction, and memory safety established at compile time (Jung et al. 2018). Python, JavaScript, and their managed kin ask nothing, and can promise nothing. No loop, however hot and however carefully written, can be certified to run without reference-count traffic or collector pauses, because the memory manager is woven into every object the program touches.

Because the choice is whole-program, an application that needs both disciplines must be written in two languages, and the seam between them is the discontinuity this chapter dissolves. The ritual is familiar enough to have a folklore: profile the managed application, find the hot loop, rewrite it in Rust or C, and bind the result back through a foreign-function layer. Every element of the ritual is glue in the strict sense of Chapter 1. The rewritten kernel restates logic the managed side already expressed. The binding restates every signature in the calling convention of the guest, as a version-coupled isomorphism maintained by hand. And the failure mode of a stale binding is not a type error but a segmentation fault, because no compiler sees both sides of the seam. The divide, in other words, satisfies our definition of a discontinuity exactly, and its cost recurs at every crossing. What makes it remarkable is its standing: the field does not treat it as a boundary within programming but as a fact about kinds of language, deep enough to sort languages into families.

The field has dismantled a wall of this shape before. Twenty years ago, statically and dynamically typed languages were separate worlds, and the accepted wisdom was that a program picked a side. Gradual typing dismantled that wall (Siek and Taha 2006). It gave one language a dial, let annotations be added one declaration at a time, and articulated the criteria that make the dial trustworthy, chief among them the *gradual guarantee*: adding a correct annotation never changes what a program computes (Siek et al. 2015). The ecosystem consequences were enormous, and every major dynamic language now carries a gradual type system in production use. No corresponding account exists for memory management. The phrase “gradual ownership” has appeared in the literature once, and precision requires saying what it meant: Sergey and Clarke (2012) gradualized the *ownership types* of Clarke et al. (1998), a discipline that constrains who may point at whom for the sake of encapsulation. That work moves, borrows, and frees nothing. Same adjective, different noun. The property it gradualizes is aliasing topology, not reclamation. The construct this chapter defines takes the other noun: what it makes gradual is the affine discipline Rust’s borrow checker enforces, the discipline of when memory dies.

The goal of this chapter is *continuity of memory discipline*: one checked medium

in which the managed and ownership regimes are positions on a dial rather than kinds of language, adopted by degrees and reversible at every step. The construct that delivers it is a named extension of the synechic medium, the first of two this part introduces, and we define it now.

GRADUAL BORROW CHECKING

Gradual borrow checking is memory discipline as a continuum within one language: unannotated code retains fully managed semantics, ownership annotations introduce affine values with moves, borrows, deep immutability, and deterministic destruction, adoptable one declaration at a time, and a closed, checked boundary mediates every value that crosses between the two regimes.

5.2 The Ownership Dial

This section presents the annotation surface as a set of design decisions with rationale. The surface is called the *ownership dial*, and the name is meant mechanically: the dial has four positions, a program's modules may sit at different positions, and the guarantees strengthen monotonically as the dial turns (Figure 5.1). At the first position, *managed*, nothing is annotated and the checker tracks nothing. Allocation, sharing, and reclamation behave exactly as in the pre-ownership language. At the second, *annotated*, chosen bindings carry ownership markers and the checker enforces the discipline for the annotated names only. At the third, *enforced*, a module's entire interface with the world must be owned, as hard errors. At the fourth, *headerless*, an enforced module compiles to native code with no memory-management runtime at all, and the absence is machine-checked. This section develops the second position. Section 5.5 develops the third and fourth.

One annotation buys the discipline. Listing 5.1 shows a packet pipeline whose churn a profile has flagged, with a single `own` marker added to the hot binding. An `own` binding is the unique owner of its value. Four syntactic positions consume it (assignment to another binding, argument passing, return, and storage into a field or subscript), and after any of them the source binding is dead until reassigned. Reading a dead binding is a compile-time error, and the checker's diagnostics fire only on names the programmer tagged. Notice what did not change: parse and transform kept their signatures, and callers that never annotate are untouched.

```
obj Packet { has payload: str = "", size: int = 0; }

def parse(raw: str) -> Packet { return Packet(payload=raw, size=len(raw)); }

def transform(p: Packet) -> Packet {
    p.payload = p.payload.upper();
    return p;
}

with entry {
    for raw in feed() {
        p: own Packet = parse(raw);
```

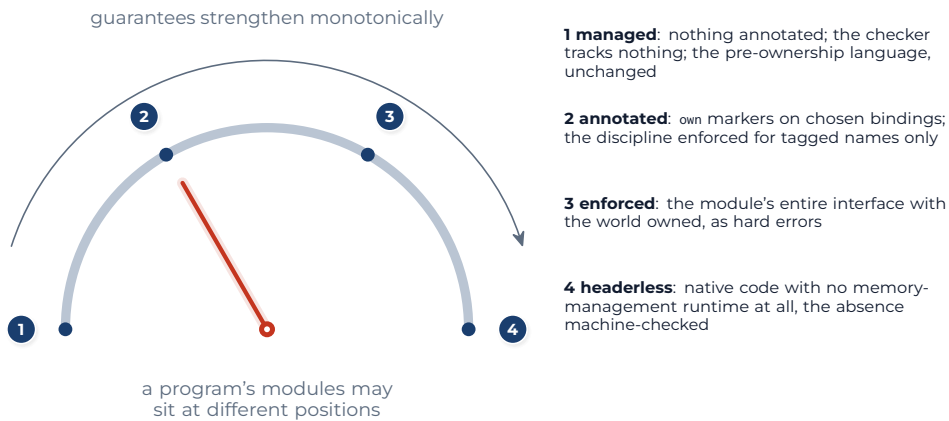


Figure 5.1: The ownership dial. Four positions, one per notch of strengthening guarantee, from the untouched managed language to a machine-checked zero-GC native artifact. The needle stands at the second position, the one this section develops; Section 5.5 turns it to the third and fourth. The fourth position is the one a crossing of the self-hosting frontier requires (Chapter 4).

```

    q = transform(p); # moves out of p
    audit(p);         # error[E1301]: use of p after it was moved
    emit(q);
  }
}

```

Listing 5.1: The ownership dial at its second position. The own annotation makes `p` the unique owner of each parsed packet; passing `p` to `transform` moves the value, and the later use of `p` is rejected at compile time. The unannotated functions are untouched.

One principle governs the whole surface: ownership is a property of bindings, not of types or objects. A `Packet` is not an owned type. *This binding* owns *this* packet. The principle is what makes the system gradual. No type ever changes meaning when a caller annotates a local, so annotated and unannotated code compose without wrappers, coercions, or witness objects, and turning the dial on one function never forces a neighbor's signature to change. A second decision serves the same end: ownership here is affine rather than linear. A binding that is never consumed is silently dropped at its last use, so every annotation is a pure restriction on the code that exists, never an obligation to write code that does not.

Borrows complete the annotated position. The expression `&x` takes a shared, read-only borrow of an owner, and `&mut x` takes a mutable one, under the familiar exclusion law: any number of live shared borrows, or exactly one mutable borrow, never both, and no borrow may outlive its owner. The consequential divergence from Rust is that these borrows are *second-class* in the sense of Osvald et al. (2016). A borrow may be passed down the stack but never returned upward, stored into a field, or captured so as to outlive its creating scope. Second-class borrows eliminate lifetime parameters, the piece of Rust's surface with the steepest learning curve, because a borrow that cannot

escape needs no name for its extent. The cost is expressiveness: no borrow-holding data structures, and no iterator-returning-references idiom. In a gradual medium the cost is tolerable for a structural reason. The managed world remains available, and the data structure that wants to hold references simply holds managed references. Rust must make first-class borrows work because there is nowhere else to stand. A gradual language gets to spend the complexity budget where it pays. Two further markers round out the surface. An `imm` binding is deeply, transitively immutable, and immutability is what concurrency needs: a payload may cross a concurrency boundary only if it is statically race-free, either an `imm` value that no one can write or an `own` value moved into the task that no one else retains. And an archetype may declare a drop ability, run exactly once at the value’s death, before its fields are torn down.

The timing of that death is the commitment the surface stands on. A used binding is destroyed at the exit of its scope, in declaration order, and the timing is *mode-invariant*: for a uniquely owned value, the reference-count death under the managed modes and the statically placed destructor call under the headerless mode occur at the same program point, so a program observes the same drops in the same order at every dial position. That identity is not free. It is a proof obligation and a testing obligation, and holding it is what entitles the system to call the dial semantics-preserving, as Section 5.4 develops. On every backend, meanwhile, the annotations themselves are erased rather than reified: `&x` compiles to exactly `x`, there is no runtime representation of ownership, and a module with no annotations short-circuits the analysis entirely. Unannotated code is not analyzed, not slowed down, and not laid out differently. We state this as the design’s pay-as-you-annotate clause, and it is precisely the clause that sound gradual type systems historically violated, when wrappers and casts imposed costs on the unannotated side (Takikawa et al. 2016).

5.3 The Membrane

In this section, we specify the boundary between the owned and managed worlds, because gradual systems live or die at their boundary. Gradual typing’s boundary vocabulary of casts, wrappers, and blame does not transfer to memory management. The property at stake is not a value’s shape but its lifetime, and lifetime obligations cannot be wrapped around a value after the fact. The design instead adopts a construct from the object-capability literature, where a *membrane* is a boundary that mediates every reference passing between two object graphs so that authority cannot leak across it (Miller 2006). We keep the name and change what it mediates. This membrane governs memory discipline rather than access, and it admits exactly three crossings, each chosen so that neither world can violate the other’s invariants (Figure 5.2).

The first crossing is *sealing*, and it runs from owned to managed. Storing an owned value into any managed location, an object field or a subscript slot, seals it: the value moves across the membrane, the managed world assumes responsibility for its lifetime, and the source binding is consumed, exactly as in Listing 5.2. Sealing is one-way. Reading the field back yields an ordinary managed value, never an owned binding, because no operation takes ownership of a managed field’s contents. One-wayness is what keeps the checker modular: it never needs an alias analysis of the managed heap,

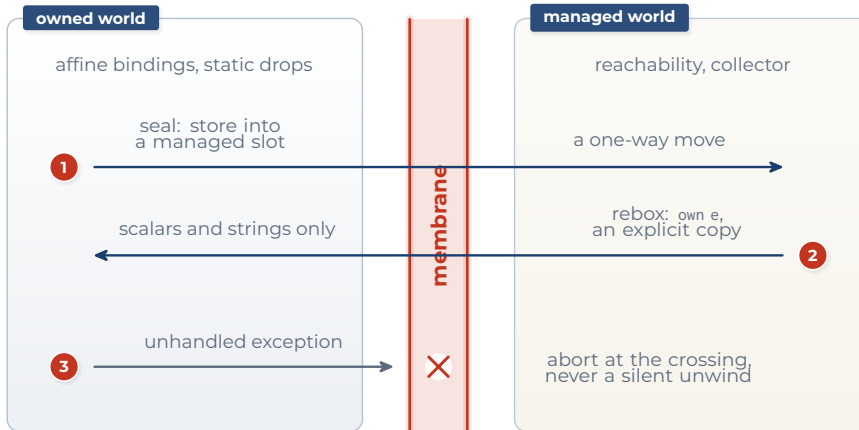


Figure 5.2: The membrane’s three crossings. Sealing moves an owned value into the managed world, one-way, consuming its binding. Reboxing mints fresh owned storage by explicit copy, restricted to values whose uniqueness copying can guarantee. And exceptional control does not cross at all: an unwind that reaches the membrane aborts there, the one named observable of the gradual guarantee, rather than leaking or double-freeing frames it cannot see.

because ownership facts never depend on managed structure. Sealing also happens silently at unannotated call boundaries. An owned local passed to a plain parameter seals at the call, the callee sees a normal value, and the annotation stays where it was written instead of spreading through the program. The alternative, rejecting such calls, would make every annotation infectious and the dial unturnable in real code.

```
a: own Buffer = Buffer();
h = Holder();
h.ref = a;    # sealed: moved into managed storage
print(a);    # error[E1301]: use of a after it was moved
```

Listing 5.2: Sealing at the membrane. Storing the owned buffer into a managed field moves it across the boundary; the managed world owns it thereafter, and the source binding is dead.

The second crossing, *reboxing*, runs the other way, and it is explicit and copying: `own e` mints fresh owned storage from a scalar or string value. Restricting the crossing to values with copy semantics is a soundness decision, because copying is the only way to produce a value whose uniqueness is beyond doubt. The design deliberately provides no operation that adopts a managed object as owned. The managed world cannot certify that no alias survives, and an adoption primitive would convert every surviving alias into a potential use-after-free, which is exactly the unsoundness gradual typing’s critics feared, transplanted to memory.

The third crossing is exceptional control, and it is the membrane’s hardest case. Owned frames carry static destruction obligations. An unwinder that pops such a frame without running its drops leaks, and one that runs them without understanding borrow state may free twice. The design adopts the abort discipline: an exception

with no handler in the same enforced frame, or a managed exception arriving at the membrane inbound, terminates the program at the crossing rather than unwinding into frames that have no unwind metadata. In-frame handling is fully supported, with drops emitted on the handler path, and code that uses exceptions for cross-frame control flow simply keeps those frames at the managed positions of the dial. The choice is deliberately austere, and it conceals nothing. The alternative, silently leaking or heroically unwinding, would falsify the coming guarantee in exactly the cases users cannot debug. Instead the abort appears in the guarantee itself, as its one named observable.

5.4 The Gradual Guarantee for Memory

We now state what entitles the documentation to say “annotate incrementally” without a footnote of exceptions. The dial’s license is a pair of properties, established over a core calculus with two heaps and an instrumented semantics that counts reference-count operations, and stated here in prose. The first is *erasure*: deleting every ownership annotation from a well-typed program never changes its behavior at all, including the observable order of drop hooks, because the static drop of an unaliased value and the reference-count death of that same value fall on the same program point. The second is *annotation monotonicity*, the gradual guarantee analog for memory: adding annotations to a program either preserves its computed answer or is rejected statically, and the sole new observable an annotated program can exhibit is the membrane’s abort. Its shape follows the refined criteria of Siek et al. (2015), with the one deviation surfaced in the statement rather than in folklore. Adding annotations never yields a *different* value. The static direction holds for a simple reason: unannotated names are simply untracked, so there is no analog of a dynamic cast that could fail later.

The guarantee has an executable witness, and the witness runs in the compiler’s test suite on every commit. One source, compiled under the headerless, reference-counting, and cycle-collecting modes, must produce byte-identical output, and an executable implementation of the erasure function deletes every annotation from a benchmark kernel and checks that the erased program’s output matches the annotated program’s in every mode. The instrument has teeth. What it catches is precisely the class of defect nothing else can see: miscompiles that are masked in the managed modes because retains and releases balance there, and that surface as memory corruption only when the collector is gone. The differential harness has caught real code-generation bugs of exactly this shape, each one silently correct at small scale and invisible to every unit test. A gradual system’s guarantee theorems are also its best test oracles.

The guarantee also has a measured slope, because the sobering record of sound gradual typing demands the measurement. When gradual types were implemented by reifying casts and wrappers at boundaries, mixed configurations paid order-of-magnitude penalties, and the finding reshaped the field’s expectations (Takikawa et al. 2016). The ownership dial’s annotations are erased, never reified, so no wrapper, cast, or boundary object exists whose cost could spike at a mixed point. The measurement bears the design out: an exhaustive sweep of one kernel’s annotation lattice, from bare to fully annotated, is statistically flat, with medians spanning less than three percent, and the fully erased kernel and its fully annotated twin compile, under

the managed mode, to intermediate representation with identical reference-count machinery. Annotations are free where they do not pay. Where they do pay is the endpoint, and the payment is the next section’s subject.

5.5 The Verifiable Endpoint

This section develops the dial’s third and fourth positions, where ownership turns from a per-binding discipline into a per-module guarantee with an artifact-level witness. A module declared *enforced* must keep every heap-typed interface position, its parameters, returns, and fields, in the owned world. Violations are hard errors that block code generation, and each reports the provenance of the offending obligation: which declaration, import, or inference step put the heap type in the interface position. In gradual migration the actionable question is always why a given position needs ownership, and the diagnostic answers it. Heap values leave an enforced module only through one explicit builtin, the membrane made visible in the source.

An enforced module compiled for the native substrate under `-gc none` then sheds the last of the management machinery. Objects lose their reference-count headers entirely. Construction is a bare allocation call. Destruction is a direct call to a synthesized destructor at the statically determined drop point, which runs the user’s drop hook, recursively drops owned fields, and frees. Owned containers are monomorphized per element type, because a type-erased list of headerless elements is a contradiction, and we accept the code-size cost as the cost of keeping the mode’s promise: boxed elements with hidden headers would silently reintroduce the machinery the mode promises to eliminate.

The endpoint’s defining artifact is the assertion. A build mode, `-assert-no-rc`, scans the emitted intermediate representation and fails the build if any reference-count or collector machinery survives, with an exact acceptance criterion: zero retains, zero releases, no collector entry points, no header initialization. Listing 5.3 shows the mode’s output on the pipeline module. The distinction we insist on is between an optimizer and a guarantee. Optimizing compilers have long elided reference counts when analysis permits, and Perceus is the state of the art (Reinking et al. 2021). But elision is best-effort by nature. Its output is a faster binary, not a checkable property, and a regression in an inliner or a new aliasing pattern silently restores the traffic. Under enforcement the causality is reversed. The type system rejects programs whose lifetimes the compiler cannot place statically, so the absence of a collector is a consequence of typing, witnessed per build by the assertion. When a later compiler change breaks the property, the build fails, which is the regression instrument working.

```
$ jac nacompile pipeline.jac --enforce-nogc --gc none --assert-no-rc
  assert-no-rc ok: 0 __rc_* symbols . 0 trace fns . no roots buffer
  Binary written to pipeline (280896 bytes).
```

Listing 5.3: The endpoint as a checkable fact. The assertion mode fails the build if any reference-count or collector machinery survives in the emitted code; here it certifies the enforced module clean and links it.

The endpoint pays where lifetime management is the workload. Across the language’s ownership benchmark suite, whose kernels are compiled from one source per kernel under three memory modes, the managed builds contain 1,791 reference-count references in their intermediate representation and every enforced build contains zero, certified by the assertion at link time. The pointer-churn kernels show the payoff: the binary-trees kernel runs $1.67\times$ faster than its collector build in half the peak memory, and a clone-heavy symbolic-differentiation kernel, the workload most hostile to reference counting, runs $2.49\times$ faster in a third of the memory. The other kernels are the unflattering half of the story, and reporting them is part of the discipline. Kernels whose time is arithmetic or hash-table internals sit at parity, and a red-black tree written in an index-based idiom regresses slightly, because its parallel scalar arrays engineered the collector out before the dial arrived. The endpoint’s value is workload-dependent, and the design never promised otherwise.

The requirement is also the toolchain’s own. Chapter 4 identifies the components of the jac artifact that remain external engines: the interpreter, the JavaScript runtime, the code generator back end, and the compression libraries beneath them. These are the components conventionally written in C and C++, and the reason is memory discipline: an interpreter’s dispatch loop or a codec’s inner loop cannot carry a collector or reference-count traffic and remain competitive. The stated goal of replacing each engine with a Jac implementation therefore depends on this chapter’s machinery. A replacement engine is written as an enforced module and compiled at the fourth dial position, headerless, with the absence of management machinery certified by the assertion. Note that not every replacement waits on the dial: the linkers were rewritten in Jac on the managed substrate, where their performance requirements are met without it. The dial exists for the components whose requirements are stricter.

Because enforcement is per-module, one binary freely mixes the dial’s positions: enforced kernels compiled headerless, annotated middle layers on the managed floor, and untouched script code, with each module’s memory mode checked at link time and the membrane governing every crossing. This is the demonstration Chapter 2 filed the divide against. The line between the managed kind of language and the systems kind, deep enough that the field names language families after it, is exhibited as a continuum within one checked medium, walked by degrees in either direction, and the boundary the conventional stack services with a second language and a bindings layer is, in this medium, a dial. One discontinuity remains on Part II’s map. Every substrate crossed so far, runtime, toolchain, and now memory discipline, has been deterministic. The most consequential new substrate of the present era is not, and Chapter 6 completes the continuity claim by crossing it.

Bibliographic Notes

The two worlds this chapter joins are each well documented. Rust is the modern statement of whole-program ownership discipline (Matsakis and Klock 2014), with its semantic foundations secured by Jung et al. (2018), and it occupies the dial’s far endpoint, everywhere, for every program. The ownership-types lineage descends from Clarke et al. (1998), whose gradualization by Sergey and Clarke (2012) shares this chapter’s adjective while addressing aliasing topology rather than reclamation.

The gradual framework itself is Siek and Taha (2006), its trustworthiness criteria are refined in Siek et al. (2015), and the performance record that shaped this design’s erasure-not-reification commitment is Takikawa et al. (2016). Second-class values, the basis of the borrow design, are developed by Osvald et al. (2016). The membrane construct is adapted from the object-capability tradition of Miller (2006). Perceus (Reinking et al. 2021) is the strongest representative of reference-count elision as optimization, the modality against which Section 5.5 defines the guarantee alternative.

The compilation machinery that discharges this chapter’s claims on the native substrate, including the emission modes and their shared program meaning, is presented in Chapter 12, and the runtime’s uniform treatment of value death across substrates in Chapter 13. The benchmark suite, the differential-identity harness, and the annotation eraser ship with the language’s open-source artifact, and Appendix E describes how to reproduce every number reported here.

THE SEMANTIC DISCONTINUITY: MEANING-TYPED PROGRAMMING

The programmer, like the poet, works only slightly removed from pure thought-stuff.

Frederick P. Brooks, Jr., 1975

Part II has argued that a language can present one continuous semantic medium across substrates, and the preceding chapters have carried that argument across a particular set of them: the managed, web, and native runtimes of Chapter 3, the toolchain of Chapter 4, and the memory-discipline divide of Chapter 5. Every substrate so far has been deterministic. Given the same program and the same inputs, the Python interpreter, the browser, and the linked native binary produce the same results, and the theory of Chapter 2 was developed first for that setting. But the definition of substrate transparency was extended, at the close of that chapter, to executors that are not deterministic at all. The extension was not a formal indulgence. The most consequential new execution substrate of the present era is the large language model, and programs increasingly delegate real work to it.

This chapter completes Part II by carrying the continuity claim across the deterministic–stochastic boundary. The structure of the argument is deliberately parallel to the chapters before it. We first establish that the language model is a substrate in the full sense of our definition, and that the boundary programs cross to reach it is a discontinuity of the kind cataloged in Chapter 1 (Section 6.1). We then examine the glue code that has accreted at this discontinuity (the hand-built prompt) and show that it recapitulates, in prose, every pathology of the marshaling layers that came before it (Section 6.2). The remainder of the chapter presents the language’s answer in four moves. A delegation construct, by `llm()`, makes a language model the implementation of an ordinary typed function (Section 6.3). Semantic annotations and automatic prompt synthesis eliminate the hand-written marshaling text (Section 6.4).

A contract discipline recovers a meaningful notion of correctness under stochastic execution (Section 6.5). And finally, the same continuous medium serves the reverse traffic, machines writing programs, and closes Part II where the era demands it be closed (Section 6.6).

6.1 The Language Model as Substrate

Recall the working notion of a substrate developed in Chapter 2: an execution environment with its own invocation protocol, its own data representations, its own failure modes, and its own cost profile, such that crossing into it from program text conventionally requires scaffolding. The Python runtime qualifies. So do the browser, the C ABI, and, as Chapter 4 argued, the build and deployment machinery itself. The large language model qualifies on every count. It is an executor: presented with a specification of intent and a set of input values, it produces output values. It has an invocation protocol, typically an HTTP interface accepting and returning token sequences. It has a data representation (free text) that matches no type discipline in the calling program. It has failure modes (refusals, truncation, malformed output, rate limits) and a cost profile (latency in seconds, marginal cost per token) that differ from a local function call by several orders of magnitude.

What makes this substrate genuinely new is not the protocol but the execution model. The transformer architecture (Vaswani et al. 2017) yielded, at scale, systems that can be directed to perform a task by describing the task in their input (Brown et al. 2020). The program, in the classical sense, is partially relocated into the data: a frozen model plus a task description behaves as an implementation of that task. Instruction-tuned successors sharpened this into a de facto calling convention in which natural language is the instruction set (OpenAI 2023). The result is an executor whose “machine code” is prose and whose evaluation is sampled: the same input may lawfully produce different outputs on different invocations.

The synechic stance toward such a substrate follows directly from the definitions of Chapter 2, and it is worth stating both halves. First, the identity of the executor should have no bearing on how the program is expressed: a function implemented by a language model should be declared, typed, called, and composed like any other function. Second, and equally, the lawful differences of the substrate must not be papered over. The lesson of Waldo et al. (1994) applies with more force here than it did to the network: an abstraction that pretends a stochastic, slow, costly executor is a fast deterministic one will fail its users when it matters most. Latency, cost, and nondeterminism must surface, but they must surface as semantics (types, parameters, exceptions), not as expression-level scaffolding smeared across the codebase. The rest of this chapter is an exercise in holding both halves of this stance at once.

6.2 The Prompt as Glue Code

Consider how a conventional application invokes a language model today. Somewhere in the codebase there is a template: a block of prose with interpolation holes, into which the program’s state is serialized. The template restates, in English, what the surrounding code already declares in types (“you will be given a task title; respond

with one of the following categories”) and appends formatting instructions: respond only with JSON, use these exact keys, do not include commentary. On the return path there is a parser that extracts structure from the model’s text, a validator that checks the extraction against expectations the type system never sees, and, in mature codebases, a retry loop that re-invokes the model with a corrective message when parsing fails. Around the whole assembly sit few-shot examples, escaping logic, and version-pinned model identifiers.

Every element of this list has a name among the glue categories of Chapter 1, because every element is marshaling. The prompt template serializes arguments. The parser deserializes results. The retry loop is failure handling at a boundary. This is the anatomy of an RPC stub (Birrell and Nelson 1984), reconstructed by hand for a wire format that happens to be natural language. The practice has acquired a flattering name (prompt engineering), but structurally it is glue code, and it is glue code of an unusually degraded kind. Classical marshaling layers have schemas, and their duplication of program structure is at least mechanically checkable: an IDL compiler will object when the stub and the interface disagree. The prompt has no schema. It duplicates the program’s type declarations in prose, and no compiler checks that the duplication is faithful. Rename a field, add an enum member, change a return type, and the prompt silently rots. The failure appears later, probabilistically, as misparsed output in production.

The diagnosis of Chapter 2 therefore applies without modification. The boundary-crossing burden of the LLM discontinuity is paid at every call site, in artifacts the language does not see, by developers acting as human compilers from type structure to English and back. And the decomposition Brooks made canonical tells us which part of this burden is real (Brooks 1987). The essence of a delegated call is small: the intent of the function, the input values, and the type of result the program requires. Everything else (the template, the format instructions, the parser, the retry choreography) is accident, and accidents of the sort a language can absorb. The language needs to provide only two things so that the essence alone remains in the program text: a delegation operator and a place to attach meaning.

6.3 Delegation Semantics: Implemented by Meaning

Jac’s answer begins with the `by` operator. In its general form, `by` delegates the work named on its left to the executor named on its right. The operator is deliberately generic, and the language permits plugins to define delegation targets. Its standard and most consequential use binds a function declaration to a language model: a function may be declared with a full signature and no body, with `by llm()` in the body’s place. The declaration asserts that an implementation exists. The delegation clause names the substrate that will supply it at runtime.

The examples in this section and the next are drawn from the day planner, a small full-stack application in the language’s example suite whose entire backend, AI integration, persistent state, and browser interface occupy a single file. Its task-categorization path appears, verbatim in the essentials, in Listing 6.1.

```
enum Category { WORK, PERSONAL, SHOPPING, HEALTH, FITNESS, OTHER }
```



```
def categorize(title: str) -> Category by llm();
sem categorize = "Categorize a task based on its title";

node Task {
  has title: str,
    done: bool = False,
    category: str = "other";
}

def:pub add_task(title: str) -> Task {
  category = str(categorize(title)).split(".")[1].lower();
  task = root ++> Task(title=title, category=category);
  return task;
}
```

Listing 6.1: Delegated categorization in the day planner. The function `categorize` has no body; its implementation is supplied by a language model at runtime, and its call site is an ordinary call.

Three observations carry the semantic weight. First, the call site is indistinguishable from an ordinary call. Inside `add_task` (itself a public endpoint, per the projection semantics of Chapter 14) the expression `categorize(title)` involves no client object, no prompt, no parsing, and no awareness that a network, a model, and a sampling procedure lie behind it. This is substrate transparency in the sense defined in Chapter 2: the identity of the executor has no bearing on how the program point is expressed. Second, the declaration is fully typed. The function accepts a `str` and returns a `Category`: not a string that is hoped to name a category, but a member of a closed enumeration of six values. The type does double duty: it is the contract the caller relies on, and, as the next section develops, it is also the specification from which the model’s output format is derived. The returned member flows onward like any enum value. The endpoint stores its lowercased name on a persistent `Task` node connected to `root`, the reachability root whose semantics Chapter 10 develops. Third, the delegation target is a configured value, not a hard-coded dependency. The expression `llm()` accepts arguments (a model name, a temperature, retry bounds) and in their absence resolves the model from project configuration: the day planner’s `jac.toml` names its default model in a `[byllm.model]` stanza, placing model identity with the rest of the project’s declared intent under the toolchain of Chapter 4, where it can vary by environment without touching program text.

Consider what it means for a function to be “implemented by meaning”. The declaration in Listing 6.1 carries more information than a conventional signature is credited with. Its name is an English verb describing an operation. Its parameter is named `title`, not `x`. Its return type enumerates the acceptable answers. Well-written code has always carried this semantic load for the benefit of human readers (it is why naming is taken seriously), but conventional execution discards it: the machine never reads the names. Delegation to a language model inverts this. The stochastic executor is an engine that acts on meaning, and the latent specification that good code already contains becomes sufficient input for an implementation. This is the central move of what Dantanarayana et al. (2025) name Meaning-Typed Programming (MTP): treat the semantic content of ordinary program structure (names, types, and targeted

annotations) as the specification of delegated behavior, and make the language, not the programmer, responsible for the translation.

6.4 Meaning Types and Automatic Prompt Synthesis

The translation is where the glue of Section 6.2 went. It has not disappeared. It has been compiled. When a delegated function is invoked, the runtime synthesizes the prompt mechanically from the program's own structure: the function's name, its parameter names and the argument values bound to them, the declared return type rendered as an output schema, and a fourth ingredient: explicit semantic annotations attached with the `sem` keyword. A `sem` statement binds a string of meaning to a declaration path: to a function, a parameter, an object field, or an enumeration member. The day planner's second delegated function, shown in Listing 6.2, returns a list of structured objects, and its field-level annotations sharpen the parts of the structure that names alone leave ambiguous.

```
enum Unit { PIECE, LB, OZ, CUP, TBSP, TSP, BUNCH }

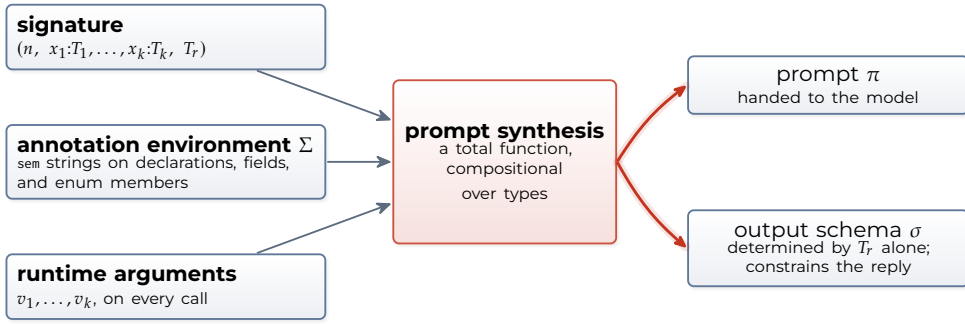
obj Ingredient {
  has name: str,
    quantity: float,
    unit: Unit,
    cost: float,
    carby: bool;
}

sem Ingredient.cost = "Estimated cost in USD";
sem Ingredient.carby = "True if this ingredient is high in carbohydrates";

def generate_shopping_list(
  meal_description: str
) -> list[Ingredient] by llm();
sem generate_shopping_list =
  "Generate a shopping list of ingredients needed for a described meal";
```

Listing 6.2: Semantic annotations in the day planner. The `sem` declarations attach machine-readable meaning to a function and to individual fields; the runtime incorporates them into the synthesized prompt together with the type structure of `Ingredient`.

The synthesis is compositional over types. Because `generate_shopping_list` returns `list[Ingredient]`, the runtime derives the output schema from the declaration of `Ingredient` itself, recursively: each field with its name and type, the closed member set of `Unit`, and the field-level `sem` strings. So the model is told, without any programmer writing a prompt, that `cost` is an estimate in US dollars and `carby` flags high-carbohydrate items. Abstracting from the mechanism (Figure 6.1), prompt synthesis is a total function from a delegated declaration d with signature $(n, x_1:T_1, \dots, x_k:T_k, T_r)$, an annotation environment Σ mapping declaration paths to `sem` strings, and runtime arguments $v_1, \dots, v_k$, to a pair (π, σ) of a prompt and an output schema, where σ is



the programmer authors the declaration and fragments of Σ ; the language authors π and σ , on every call, from the current state of the program

Figure 6.1: Prompt synthesis as a compiled function. A delegated declaration, the sem environment, and the call’s arguments map to a prompt and an output schema; the schema comes from the return type alone, so the model’s reply is constrained by what the program already declares. The prompt cannot drift from the program because it is derived from the program.

determined by T_r alone. The programmer authors d and fragments of Σ . The language authors π and σ , on every call, from the current state of the program. This is the construct the book’s terminology names.

MEANING TYPES

Meaning types are semantic annotations from which prompts are automatically synthesized, making delegation of program logic to large language models (by `llm()`) a typed language feature rather than string engineering.

The consequences follow from the single-source-of-truth structure. The prompt cannot drift from the program, because the prompt is derived from the program: rename a field, add an enum member, or change a return type, and the next synthesized prompt reflects the change, with the same fidelity a recompiled stub reflects a changed interface. The annotations live where the declarations live, in the style of the declaration–implementation separation of Chapter 3, and they are deliberately distinguished from docstrings: a docstring addresses the human reader and is excluded from synthesized prompts, while a `sem` string is meaning intended for machine consumption. The language, in other words, distinguishes the audiences of documentation, a small design decision that keeps the human channel and the machine channel from corrupting each other. The construct also has an empirical record: Dantanarayana et al. (2025) evaluate MTP against contemporary prompt-pipeline frameworks across a benchmark suite and report accuracy on par with or exceeding hand-engineered integrations, with substantially less code, lower token cost, and measurable robustness to degraded naming quality. This is evidence that the latent semantics of ordinary program structure are a sufficient specification in practice, not merely a convenience. How meaning types sit within the type system

proper (their interaction with checking, inference, and the gradual discipline inherited from the Python substrate) is taken up in Chapter 9.

6.5 Correctness under Stochastic Delegation

A function whose implementation is sampled cannot be correct in the way a sorting routine is correct, and a treatment that dodged this point would forfeit the discipline that Chapter 2 demands of a synechic language. The workable notion is older than the problem: design by contract (Meyer 1992). A delegated declaration is a contract between the program and the substrate. Some of its clauses are semantic and cannot be checked mechanically: that the category returned for “dentist appointment” is the *right* category is a judgment the runtime cannot render. But one clause is both checkable and central: the result inhabits the declared type. The design commits the runtime to enforcing that one clause, and to surfacing violations at exactly one program point.

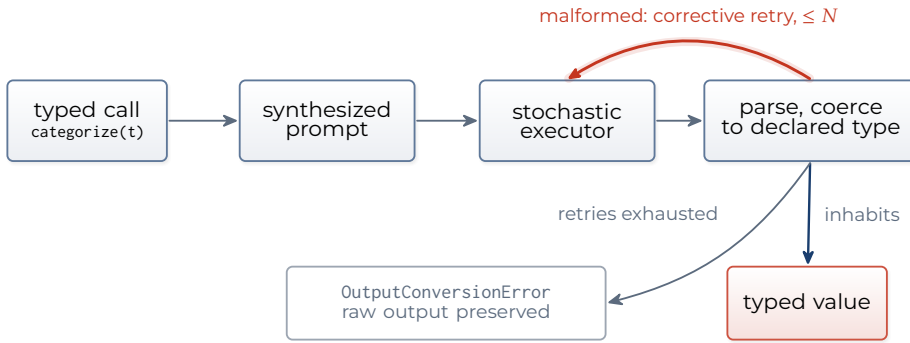
Enforcement proceeds as a ladder. When a delegated call with a structured return type receives the model’s text, the runtime first attempts to parse and, where reasonable, coerce it into the declared type: a quoted “5” may become the integer 5. If the output is empty, malformed, or unconvertible, the runtime does not fail. It regenerates. The conversation is reset to the original synthesized prompt, and a corrective message is injected that states what was wrong, echoes the rejected output back to the model, re-presents the output schema, and instructs the model to respond with the schema alone. This repeats up to a bounded number of retries (configurable per call as by `llm(max_output_retries=5)`, per project in `jac.toml`, defaulting to three), after which the runtime raises a typed error, `OutputConversionError`, that reports the attempt count and preserves the rejected text on a `raw_output` field for diagnosis. A plain `str` return is never retried, since any string satisfies it. The upshot is a dichotomy the caller can reason about: a delegated call either returns a value that inhabits its declared type or raises a typed exception at the call site. Malformed model output does not flow onward as corrupt data. Figure 6.2 traces the ladder end to end, from the call to the dichotomy.

Nondeterminism also yields to the contract discipline at the point where engineering most needs determinism: testing. The runtime ships a mock substrate, `MockLLM`, that replays scripted responses through the same parsing and retry machinery as a real model. The test in Listing 6.3 is complete and runnable (it passes under `jac test` as printed) and scripts a truncated JSON response followed by a well-formed one. It asserts not only the recovered value but the shape of the retry protocol itself, via the mock’s record of prompts seen.

```
import from jactlang.byllm.lib { MockLLM, MockRawResponse }

obj Person {
  has name: str,
    age: int;
}

glob llm = MockLLM(
```



the caller's dichotomy: a value that inhabits the declared type, or a typed error at the call site; malformed output never flows onward

Figure 6.2: The runtime half of delegation: enforcement as a ladder. The model's text is parsed and coerced toward the declared return type; a malformed reply triggers a corrective retry that echoes the rejection back to the model, bounded per call; and exhaustion raises a typed error that preserves the raw output for diagnosis. Figure 6.1 shows how the prompt is made; this figure shows why the caller may trust what comes back.

```

model_name="mockllm",
config={"outputs": [
    MockRawResponse(content="{\"name\": \"Ada\", \"age\": \"\"},
    MockRawResponse(content="{\"name\": \"Ada\", \"age\": 36}")
  ]}
);

def get_person -> Person by llm();

test "malformed output is retried and recovered" {
  person = get_person();
  assert person.name == "Ada" and person.age == 36;
  assert len(llm.seen_prompts) == 2;
}

```

Listing 6.3: A deterministic test of the typed-output retry protocol. The first mocked response is malformed and triggers one corrective retry; the second parses into the declared type. The test asserts on both the value and the number of attempts.

It is worth pausing on what this design does and does not claim. The boundary has not been abolished. It has been given a lawful surface. The stochasticity of the substrate appears in the language as a temperature parameter, a retry bound, and a typed exception. These are semantics, not scaffolding, and the abstraction's leaks (Spolsky 2002) are collected at one point rather than denied. What remains outside the mechanical contract is semantic adequacy, and it remains there on principle: whether a categorization is apt or a shopping list plausible is application-level correctness, expressible as guards, assertions, and tests over values that are, thanks to the typed

boundary, already well-formed. The contract discipline does not make a stochastic executor deterministic. It makes the program's exposure to the stochasticity explicit, bounded, and typed. That is all that correctness under delegation can mean.

6.6 The Synechic Medium as Shared Ground

Everything to this point has concerned traffic in one direction: deterministic programs delegating fragments of their logic to a stochastic executor. The construction of software in the present era runs heavily in the other direction as well: stochastic authors, in the form of coding agents, writing deterministic programs. The two flows cross the same boundary, and the closing observation of Part II is that the synechic property serves both.

Consider what a coding agent faces when it generates a conventional application. It must reproduce the fragmented stack of Chapter 1 in full: two or three languages, a schema duplicated across database, backend, and frontend, configuration files whose mutual consistency no compiler checks, and glue at every junction. Its context window must hold the discontinuities as well as the substance, and its errors concentrate at them for the same reason human errors do: a discontinuity is where structure goes unchecked. A synechic medium changes the terms of the task. The day planner of this chapter is a whole application (delegated AI functions, a persistent graph, public endpoints, and a reactive browser interface) in one file of fewer than three hundred lines, in one language, checked by one compiler. The entire system fits comfortably in an agent's working context, and every cross-tier invariant that a conventional stack leaves to convention is, here, a compile-time property. The economics of machine authorship favor continuity even more strongly than the economics of human authorship did: an author with no tacit knowledge and a bounded context benefits most when there is one medium to know and one checker to satisfy.

The language meets this reader halfway, deliberately. The `jac` binary of Chapter 4 ships a Model Context Protocol server (invoked as `jac mcp`) that exposes the toolchain to coding agents as machine-callable operations: validating and type-checking `Jac` source, checking syntax, formatting, explaining compiler errors, transpiling, executing code, and visualizing the resulting graph, alongside machine-readable resources that include the grammar specification, curated guides, and the example corpus. The documentation itself is published in agent-consumable form, and a built-in terminal agent (`jac ai`) closes the loop for interactive use. The design intent is stated plainly in the project's own description: the language is built for humans and AI to construct software together. Structurally, what the MCP surface provides is a deterministic oracle for a stochastic author (generate, validate against the real compiler, repair), which the reader will recognize as the mirror image of Section 6.5. There, a deterministic program bounded a stochastic executor with typed validation and corrective retries. Here, a stochastic author is bounded by deterministic validation with corrective diagnostics. Both are the same synechic move. The boundary between the deterministic and the stochastic is brought inside the language, and checkable structure is attached to both of its faces.

The deepest version of the point concerns the program text itself. A `sem` annotation is read three ways: the prompt synthesizer reads it as specification, the human

maintainer reads it as documentation, and the coding agent reads it as context. One artifact serves all three because the language made meaning a first-class, placed, machine-readable part of the program rather than an out-of-band convention. This is what it means for the synechic medium to be shared ground: human and machine co-construct one continuous text, and the same semantics that let the program delegate downward to a model let a model contribute upward to the program. With that, the continuity claim of Part II is complete across every substrate this book set out to cross: ecosystems, tiers, toolchain, memory discipline, and now the stochastic executor itself. What remains is the book's second thesis. Continuity concerns where a program may reach without crossing a discontinuity. Chapter 7 turns to motion, where computation itself may go.

Bibliographic Notes

The substrate this chapter absorbs has a short and well-marked lineage: the transformer architecture of Vaswani et al. (2017), the demonstration by Brown et al. (2020) that scaled models can be directed by task descriptions in their input (the observation that makes “implementation by meaning” coherent), and the instruction-following systems typified by OpenAI (2023) that turned natural-language task specification into common engineering practice. The central scholarly reference for this chapter is Dantanarayana et al. (2025), which introduces Meaning-Typed Programming, details the prompt-synthesis and typed-output machinery that Sections 6.4 and 6.5 present in language form, and reports the comparative evaluation against hand-engineered prompt pipelines summarized here. The Jaseci stack described by Mars et al. (2023) is the production lineage in which this integration of AI models into language runtimes was first worked out.

The framing devices are older. The reading of the prompt as a marshaling layer leans on the anatomy of remote procedure call given by Birrell and Nelson (1984), and the insistence that the substrate's stochasticity must surface as semantics rather than be abstracted away restates, for a new boundary, the argument of Waldo et al. (1994). Spolsky (2002) supplies the popular form of the same caution. The essence–accident decomposition used to locate what a language can absorb is from Brooks (1987), and the chapter's epigraph is from the same author's earlier essays (Brooks 1975). The contract discipline of Section 6.5 is Meyer's (Meyer 1992), applied to an executor its author did not anticipate but whose treatment his framework accommodates with little strain.

PART **III**



**TOPOKINESIS: A THEORY OF
MOBILE COMPUTATION**

OBJECT-SPATIAL PROGRAMMING, INFORMALLY

A language that doesn't affect the way you think about programming, is not worth knowing.

Alan J. Perlis, 1982

The first half of this book made the case for continuity. Chapter 1 argued that the dominant cost of modern software lies in the discontinuities between its parts, and Chapters 2 to 6 demonstrated that a language can dissolve those discontinuities by presenting one semantic medium across substrates. Part III turns to the second silent assumption named at the outset: that computation stands still while data is delivered to it. Part II named the first of this book's two language classes, the *synechic*, and the property that admits a language to it. This part names the second. A language belongs to the *topokinetic* class when the locus of computation is a first-class mobile construct over a topology of data. Where a *synechic* language concerns where a program can *be*, a *topokinetic* language concerns where a computation can *go*.

This chapter introduces Object-Spatial Programming the way we believe a paradigm is best introduced: through working programs rather than definitions. Our running example is *littleX*, a small social platform written in Jac and maintained in the language's own repository as its reference application. Every construct we show is drawn from that codebase or from the language reference. We deliberately withhold the formal statement of the paradigm until Section 7.4, after the reader has watched its parts move, and we defer the operational semantics entirely to Chapter 8. The chapter's ancestors (actors, ambients, mobile agents, vertex-centric graph processing) were surveyed in Appendix G. In this chapter, we are concerned only with what it feels like to program this way, because the paradigm's claim, in the end, is Perlis's: it changes how one thinks.

7.1 Nodes, Edges, and Walkers by Example

Object-Spatial Programming rests on three archetypes beyond the ordinary objects and functions that Jac inherits from its Python semantics. A *node* is an object that lives at a location in a graph. An *edge* is a first-class, typed relationship between two nodes, and may carry data of its own. A *walker* is a mobile unit of computation that travels across nodes and edges, carrying state with it. A fourth construct, *root*, is not an archetype but a distinguished node: every program has one in scope at all times, and every graph a program builds is anchored to it. The deeper consequence of that anchoring, persistence by reachability, is the subject of Chapter 10.

The heart of littleX’s data model appears in Listing 7.1. The declarations read like class definitions because they are class definitions: `has` introduces typed fields with defaults, and nodes may carry ordinary methods. What distinguishes them is where their instances live. A *Profile* is not an entry in a table or a value in a heap reachable only through program variables. It is a vertex in a persistent graph. The edges *Follow* and *Post* are declared with empty bodies because, for this application, the relationship’s type is itself the information: who follows whom, who posted what. Edges may equally carry fields of their own. For example, the language reference gives a *Friend* edge a *since* field, turning statements like “Alice has known Bob since 2020” into single declarations.

```

edge Follow {}
edge Post {}

node Profile {
  has username: str = "",
    bio: str = "";
}

node Tweet {
  has content: str = "",
    author_username: str = "",
    created_at: str = "",
    likes: list[str] = [];
}

with entry {
  alice = Profile(username="alice");
  bob = Profile(username="bob");
  root ++> alice;
  root ++> bob;
  alice +>:Follow():> bob;
  alice +>:Post():> Tweet(content="first post",
    author_username="alice");
}

```

Listing 7.1: The littleX data model: two edge types, two node types, and a graph wired together at the root.

The `with entry` block constructs a graph. Instances are created the way any object is, and the connection operators place them. The generic form `++>` draws an untyped directed edge. The typed form `+>:Follow():+>` draws a `Follow` edge, instantiating it like any other object. After this block runs, the program's data is not a set of variables but a shape: a root with two profiles hanging from it, a follow relation between them, and a tweet attached to its author (Figure 7.2).

Computation over that shape belongs to walkers, and Listing 7.2 shows `littleX`'s `get_profile` in full. A walker declares state with `has` (here a typed reports list) and declares *abilities* with `can`: blocks of code bound not to a method name that callers invoke but to an *arrival*. The clause `with Root entry` means “run this when I arrive at a node of type `Root`.” The clause `with Profile entry` means the same for profiles. Inside an ability, `here` names the node currently stood upon, `visit` extends the walk, and `report` sends a value back to whoever launched the walker. Nothing in Listing 7.2 executes until the walker is spawned somewhere. A walker's declaration is an itinerary, not a transaction script.

```
walker get_profile {
  has reports: list[ProfileBundle] = [];

  can run with Root entry {
    visit [-->[?:Profile]];
  }

  can give with Profile entry {
    report here.to_bundle();
  }
}
```

Listing 7.2: A complete `littleX` walker: spawn at the root, walk to the profile, report it bundled with its edge-derived context.

Three constructs, then: data as located objects, relationships as typed values, computation as a traveler. The rest of the chapter is an unpacking of the consequences.

7.2 Abilities: Computation Triggered by Arrival

The ability is the paradigm's replacement for the method call as the unit of dispatch. A method executes because a caller names it. An ability executes because a walker and a node have come together: the runtime matches the type of the arriving walker, the type of the node arrived at, and the phase of the encounter (entry or exit), and runs whatever code has been declared for that combination. Dispatch, in other words, is by *location* rather than by *invocation*.

What makes this more than a stylistic inversion is that the code for an encounter can be declared from either side of it. We have seen the traveler's perspective in Listing 7.2: the walker `get_profile` declares `can give with Profile entry`, stating what *it* does upon reaching a profile. The data's perspective is equally available. A node may declare an ability whose `with` clause names a *walker* type, stating what the place does when a certain kind of traveler arrives. The example in Listing 7.3, adapted from the language reference, makes the room, not the inspector, own the clearance

Table 7.1: Spatial references and their meaning at each declaration site.

Reference	In a walker ability	In a node ability
self	the walker instance	the node instance
here	the node being visited	—
visitor	—	the walker visiting
root	the graph root	the graph root

check, and `disengage` (which terminates a walker’s traversal outright) lets the location expel a visitor that fails it.

```
node SecureRoom {
  has clearance_required: int;

  can check_clearance with Inspector entry {
    if visitor.clearance < self.clearance_required {
      disengage;
    }
  }
}
```

Listing 7.3: A node-held ability: the location reacts to the arriving walker, referenced as `visitor`.

The two perspectives share a small vocabulary of spatial references, summarized in Table 7.1. In a walker ability, `self` is the walker and `here` is the ground beneath it. In a node ability, `self` is the node and `visitor` is the walker that triggered it. Both sides can always see `root`. When both sides declare abilities for the same encounter, both run. The precise interleaving is part of the operational semantics we give in Chapter 8, and nothing in this chapter depends on it.

Encounters have two phases. Entry abilities run the moment a walker lands. Exit abilities are deferred: they run only after everything the walker queued from that node has been fully visited, unwinding in last-in, first-out order like a call stack. This gives the paradigm its most common idiom, *accumulate then deliver*: gather state at each node during entry, and report the aggregate in an exit ability at the spawn point. `littleX` uses the idiom throughout: its feed walker collects tweets in `can gather` with `Tweet` entry and sorts and reports them once in `can deliver` with `Root` exit, as we will see in Section 7.5.

Abilities do not displace ordinary methods. The two divide labor. `littleX`’s `Profile` node carries a plain method, `to_profile_view`, that assembles a typed view of the profile (its followers, its following, its tweets) from the edges around it. The node knows how to describe itself, and walkers decide when to ask. This division keeps walkers thin (they are choreography) and keeps knowledge of a node’s local structure where it belongs (in the node). It is the same separation of data and behavior that object-oriented design has always urged, with one adjustment: behavior that spans many objects is no longer a method on some awkwardly chosen owner or a free function threading through collections, but a walker whose path *is* the span.

7.3 Spawning, Visiting, and the Grammar of Traversal

A walker begins its life as an ordinary value and starts moving when placed on the graph with `spawn`. The expression `root spawn get_profile()` constructs the walker, sets it down at the root (firing whatever entry abilities match), and runs it to completion. Its value is the walker instance itself, whose accumulated reports the caller may read. Any node reference can serve as the spawn site. For example, `littleX`'s mobile variant spawns `like_tweet` directly on the tweet in question, skipping the walk entirely. Spawning is also where the paradigm meets the outside world: when a Jac program is served rather than run, its public walkers become HTTP endpoints, each request spawning a fresh walker on the caller's own root, a projection we examine in Chapter 14.

Movement itself is requested with `visit`, and the statement is quieter than it looks. It does not transport the walker. It *enqueues* destinations. When the current ability finishes, the walker proceeds to the next node in its queue, so a traversal is breadth-first by default, and the queue is programmable: `visit :0: [->]` inserts at the front for depth-first behavior. Traversal is also always explicit: an ability with no `visit` strands the walker deliberately, making termination the default, not an accident. Two statements complete the control vocabulary: `disengage` halts the whole walker immediately, and `skip` returns from the current ability alone, letting queued destinations proceed.

The argument to `visit` (and the paradigm's everyday workhorse) is the *edge reference expression*, a bracketed path notation that is legal anywhere an expression is. Its elements compose a small query algebra. Direction: `[->]` names the nodes across outgoing edges, `[<-]` incoming, `[<->]` both. Edge type: `[->:Follow:->]` crosses only Follow edges. Node filter: a trailing `[?:Profile]` keeps only profiles. Edge predicates filter during the crossing, as in the reference's `[->:Friend:since > 2020:->]`. The forms chain into multi-hop paths and take an explicit starting anchor, so `littleX` computes a profile's followers as `[self<-:Follow:<-[?:Profile]]` (profiles reachable *backward* over follow edges) and its feed sources as `[me->:Follow:->[?:Profile]->[?:Tweet]]`, a two-hop path from a profile through everyone it follows to their tweets. Prefixing edge yields the edges themselves rather than the nodes beyond them: `littleX`'s unfollow operation is `del` applied to `[edge me->:Follow:->here]`. One notation, then, serves both sides of the paradigm: assigned to a variable it is a query, handed to `visit` it is an itinerary. Figure 7.1 anatomizes the notation's five elements and the path they trace through the `littleX` graph.

```
walker setup_profile {
  has username: str = "";

  can run with Root entry {
    visit [-->[?:Profile]] else {
      fresh = here ++> Profile();
      visit fresh;
    }
  }

  can apply with Profile entry {
```

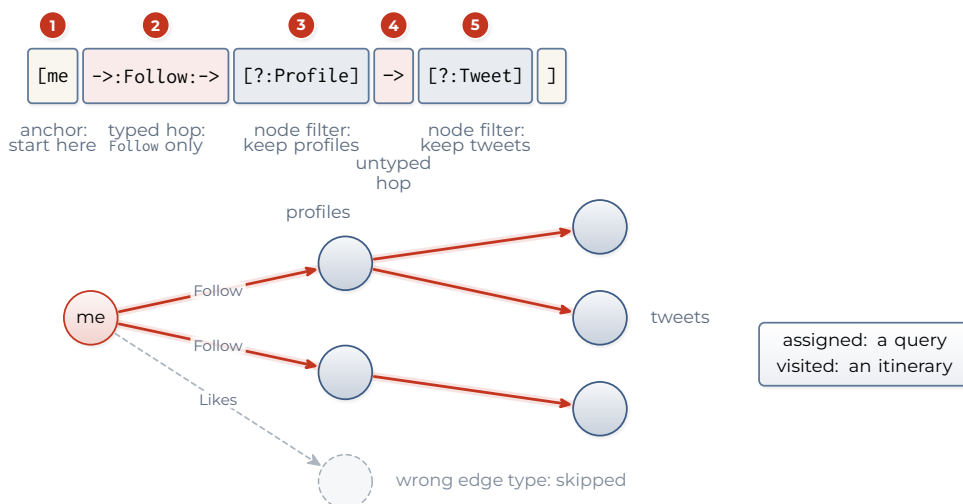


Figure 7.1: Anatomy of an edge reference expression, the paradigm’s everyday workhorse. Five composable elements read left to right: an anchor, a typed hop, a node filter, an untyped hop, a second filter. Below, the path the expression traces through the littleX graph: Follow edges to profiles, then onward to their tweets, with non-matching edges skipped during the crossing. The same expression is a typed query when assigned and an itinerary when handed to visit.

```

    if self.username {
      here.username = self.username;
    }
    report here.to_bundle();
  }
}

```

Listing 7.4: Traversal with a fallback: visit the profile if one exists, otherwise create it and walk into it. Lightly trimmed from littleX.

The pieces come together in Listing 7.4, littleX’s `setup_profile`, which shows one further grace note: `visit ... else`. The `else` block runs when the edge reference matched nothing (here, when no profile exists yet), so find-or-create, ordinarily a query followed by a conditional insert, becomes a single traversal with a fallback that builds the missing destination and walks into it. Whichever branch runs, the walker arrives at a Profile and the same `apply` ability finishes the job. The topology absorbed the branch.

7.4 From Control Flow to Path

We can now state the inversion plainly. In the inherited model of computation (the von Neumann machine and every mainstream language built in its image) the site of computation is fixed, and data is moved to it: fetched from memory, deserialized from disk, queried from a database, delivered over a wire. Backus (1978) named

the resulting traffic jam the von Neumann bottleneck and indicted the “intellectual bottleneck” that came with it, a style of thought organized around shepherding words of data, one at a time, to a stationary processor. Fifty years of abstraction have padded the trip without questioning its direction. Object-Spatial Programming questions the direction: the data stays where it is, arranged in a topology that mirrors its meaning, and the computation (small, stateful, typed) travels through it.

TOPOKINETIC

A language is *topokinetic* if the mobile locus of computation over a topology of data is a first-class semantic construct, inverting the von Neumann convention of streaming data to a fixed site of computation.

OBJECT-SPATIAL PROGRAMMING

Object-Spatial Programming (OSP) is the concrete paradigm realizing topokinesis: programs expressed as walkers traversing a persistent topology of nodes and edges, with abilities triggered by arrival.

Each clause of the definitions is doing work. *First-class* distinguishes the walker from a design pattern: its location, state, and itinerary are language semantics, checked by the type system (Chapter 9) and given meaning by an operational calculus (Chapter 8), not conventions layered on a library. *Persistent* distinguishes the topology from a data structure: the graph outlives the process that builds it (Chapter 10), so the walker’s world is durable ground rather than transient heap. And *triggered by arrival* distinguishes the ability from a callback: the unit of dispatch is a typed encounter between traveler and place, declarable from either side. The paradigm is developed formally in Mars (2025). Its production lineage, a decade of the Jaseci stack in research and deployment, is recounted in Mars et al. (2023).

Each ancestor from Appendix G realizes a fragment of this and withholds the rest. Actor systems (Agha 1986; Hewitt et al. 1973) made the units of computation autonomous and stateful, but left them stationary: actors are fixed addresses, and it is the messages that move. Mobile Ambients (Cardelli and Gordon 2000) made locations themselves mobile, a calculus of places moving through places, but offered no persistent data topology for them to inhabit. The mobile-agent systems of the 1990s (Telescript and Aglets (Lange and Oshima 1998; White 1997)) came closest in spirit, sending stateful programs to remote sites, but as middleware bolted onto host languages, not as semantics, and Appendix G argued that this is why they died. Vertex-centric processing (Malewicz et al. 2010) taught a generation to “think like a vertex,” yet trapped the thought inside a batch-synchronous framework, while graph traversal languages (Francis et al. 2018; Rodriguez 2015) put path-thinking behind a query boundary: expressive within the database, mute beyond it. OSP is the synthesis the pieces were waiting for: agent mobility as language semantics, over the persistent typed topology of a graph database, inside a general-purpose program.

In practice, to *think topologically* is to shift three habits. First, relationships stop being encodings and become declarations: where an object-oriented design stores a list of follower identifiers and a relational design a join table, an object-spatial

design draws a Follow edge, and the design diagram one would have sketched on a whiteboard *is* the program's data model. Second, queries become paths: "the tweets of everyone this user follows" is not a join to compose but a route to name, `[me->:Follow:->[?:Profile]->[?:Tweet]]`. Third, algorithms become itineraries: instead of a procedure that branches on the kind of thing it holds, one writes a walker whose abilities say what to do at each kind of place, and lets arrival do the dispatch. Control flow does not disappear. Inside an ability, Jac is a full imperative language. Control flow relocates, leaving code within an encounter and shape between encounters. The program counter has become a place.

The claim has a boundary, and we mark it now. OSP is a complement to the functional and object-oriented cores of the language, not a replacement. littleX itself uses plain obj values and ordinary functions wherever the data at hand is not relational, and a program with no graph in it is none the worse for the paradigm's presence. The inversion earns its keep where the domain has shape: where the entities, their relationships, and computations that range over both are the substance of the problem. The next section argues that this covers more of modern software than one might expect.

7.5 Three Worked Examples

7.5.1 A SOCIAL GRAPH

The social network is the paradigm's home game, and littleX plays it in full. Following a user is one line (`me +>:Follow():+> here`; in the `follow_user` walker) because the relationship is an edge, not a row. Unfollowing deletes the edge it names. The feed, the application's centerpiece query, is the walker in Listing 7.5. From the root, it finds my profile. From my profile, it queues my own tweets and, in a second visit, the tweets of everyone I follow. It gathers each tweet on arrival, then sorts and delivers the lot when the traversal unwinds back to the root.

```
walker load_feed {
  has feed: list[Tweet] = [];

  can run with Root entry {
    mine = [-->[?:Profile]];
    if mine {
      me = mine[0];
      visit [me-->[?:Tweet]];
      visit [me->:Follow:->[?:Profile]-->[?:Tweet]];
    }
  }

  can gather with Tweet entry {
    self.feed.append(here);
  }

  can deliver with Root exit {
    self.feed.sort(key=lambda (t: Tweet) { t.created_at; },
                  reverse=True);
```

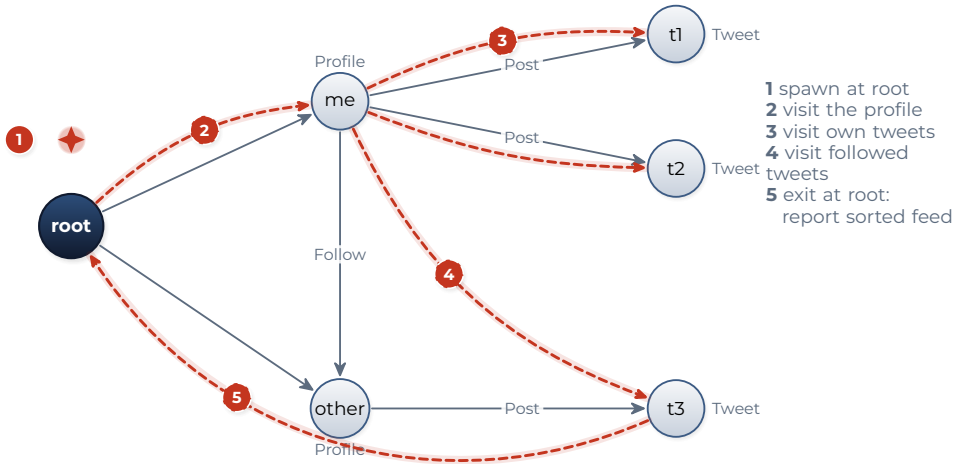



Figure 7.2: The topology of Listing 7.4 and the walk of `load_feed` (Listing 7.5). Entry abilities gather each Tweet on arrival (steps 3 and 4, one hop and two hops from the profile); the Root exit ability delivers the sorted feed as the walker unwinds (step 5). The feed is not a query over tables; it is a path through the data’s shape.

```

    report self.feed;
  }
}

```

Listing 7.5: The littleX feed walker, lightly trimmed: two visit statements define the feed, entry abilities gather it, an exit ability delivers it.

What is notable is what is absent. There is no query language embedded in strings, no object-relational mapping, no schema migration for the join tables the design never needed, and no serialization layer between the “database” and the “application.” The graph is both. When littleX is served, `load_feed` is an authenticated endpoint and each user’s walker runs on that user’s own root, so per-user isolation is not tenancy code but topology. Chapter 14 makes that argument in full. The whole backend of Listings 7.1, 7.2, 7.4 and 7.5 and their siblings is one file of walkers and nodes, which is the shape of the case study we measure in Chapter 15.

7.5.2 A STATE MACHINE AS TOPOLOGY

State machines are usually encoded: an enum and a switch, a transition table, a library’s DSL. Object-spatially, a state machine is not encoded but *drawn*: states are nodes, transitions are typed edges, and the machine’s current state is the walker’s location. The classic turnstile is built in Listing 7.6. The Fire walker carries the input tape as its state. At each State node it consumes one event and follows the matching edge type, and the `else` clauses catch events with no outgoing transition. Rejection is the absence of a path, as Figure 7.3 draws: no error table exists because none is needed.

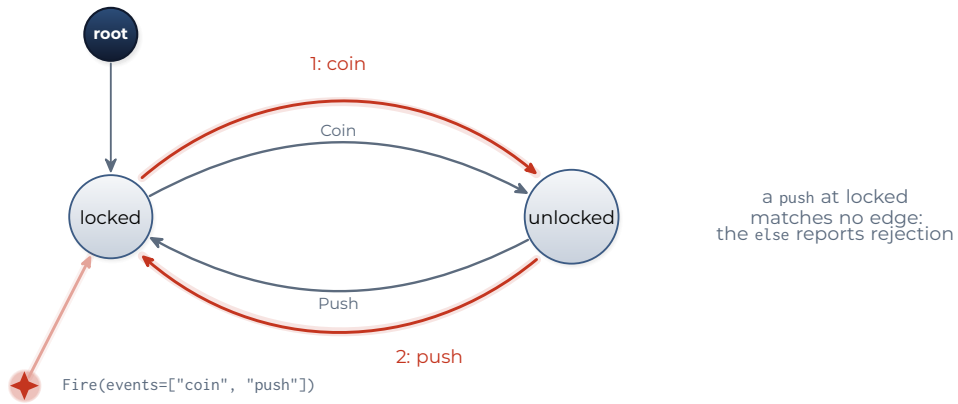


Figure 7.3: A state machine drawn rather than encoded. States are nodes, transitions are edge types, and the machine's current state is the walker's location: the Fire walker consumes one event per arrival and follows the matching edge type. Rejection needs no error table, because it is the absence of a path.

```

node State {
  has name: str = "";
}

edge Coin {}
edge Push {}

walker Fire {
  has events: list[str] = [];

  can step with State entry {
    if not self.events {
      report here.name;
      disengage;
    }
    ev = self.events[0];
    self.events = self.events[1:];
    if ev == "coin" {
      visit [->:Coin:->] else { report "rejected"; }
    } elif ev == "push" {
      visit [->:Push:->] else { report "rejected"; }
    }
  }
}

with entry {
  locked = State(name="locked");
  unlocked = State(name="unlocked");
  root ++> locked;
  locked +>:Coin():> unlocked;
}

```

```

unlocked +>:Push():+> locked;
final = locked spawn Fire(events=["coin", "push"]);
print(final.reports);
}

```

Listing 7.6: A turnstile as topology: states are nodes, transitions are edge types, the current state is the walker’s location.

The gains are the gains of reification. The machine is inspectable: the transition structure is data, walkable by other walkers, so a reachability check or a dead-state audit is itself a small walker, not an analysis bolted onto an encoding. It is extensible at runtime: adding a state is adding a node, with no recompilation of a switch statement. And because the topology is persistent, the pattern scales from a toy acceptor to the long-lived workflows that real systems encode as state machines (order fulfillment, document approval, conversation flow), where the machine must outlive any single process. The same drawing, hung from the root, is the workflow engine.

7.5.3 AN AGENT WHOSE MEMORY IS THE TOPOLOGY IT WALKS

The third example bridges back to Chapter 6. The standard architecture for a memory-bearing AI agent is the fragmented stack of Chapter 1 in miniature: an application, a vector store behind an API, a prompt-assembly layer marshaling text between them. Object-spatially, an agent’s memory is a topology hung from the root, and remembering is walking. The pattern is sketched in Listing 7.7. Episodes of interaction are nodes chained by `Next` edges. The `Recall` walker traverses the chain, accumulating what it finds. The final step delegates composition of the answer to a language model through the meaning-typed function of Chapter 6: a typed by `llm()` declaration whose prompt is synthesized from the signature and its `sem` annotation (Dantanarayana et al. 2025).

```

node Episode {
  has content: str = "";
}

edge Next {}

def compose_reply(question: str, memories: list[str]) -> str by llm();
sem compose_reply = "Answer the question using the remembered episodes as context";

walker Recall {
  has question: str = "",
    memories: list[str] = [];

  can start with Root entry {
    visit [-->[?:Episode]];
  }

  can remember with Episode entry {
    self.memories.append(here.content);
    visit [->:Next:->];
  }
}

```

```
    }  
  
    can answer with Root exit {  
        report compose_reply(self.question, self.memories);  
    }  
}
```

Listing 7.7: An agent whose memory is a persistent chain of episode nodes: recall is traversal, and answer composition is delegated through a meaning-typed function.

The example is deliberately minimal (a linear episodic chain), but the point is structural, not the particular memory discipline. Richer disciplines are richer topologies in the same paradigm: semantic memory as typed edges between concept nodes, salience as edge attributes filtered during traversal, forgetting as edge deletion, and consolidation as a background walker that rewires what an earlier walker laid down. What the agent “knows” has a shape, the shape is first-class and durable, and the agent’s context assembly (the improvised retrieval glue of contemporary agent frameworks) becomes path selection in a language with a semantics for paths. Because the topology hangs from a root, the memory persists across sessions without a database discontinuity, and each user of a served agent gets their own. The two halves of this book meet in a program like this one, and Chapter 15 returns to the pattern at production scale.

Bibliographic Notes

The formal statement of Object-Spatial Programming (its archetypes, its data-spatial semantics, and its relation to existing paradigms) is given by Mars (2025), which this chapter previews informally. The decade of research and production systems from which the paradigm was distilled is chronicled in the account of the Jaseci stack (Mars et al. 2023). The inversion the chapter turns on is as old as its target: Backus (1978) indicted the von Neumann bottleneck and the word-at-a-time style of thought it breeds, though his proposed liberation was functional rather than spatial.

The ancestry compressed into Section 7.4 rewards direct reading alongside Appendix G. The actor model originates with Hewitt et al. (1973) and receives its canonical treatment in Agha (1986). Mobile Ambients (Cardelli and Gordon 2000) give mobility of place a process-calculus foundation. The mobile-agent generation is best entered through White (1997) on Telescript and Lange and Oshima (1998) on Aglets. On the graph side, vertex-centric computation is defined by Pregel (Malewicz et al. 2010), the traversal-language tradition by Gremlin (Rodriguez 2015) and Cypher (Francis et al. 2018), and the broader graph-data lineage is surveyed by Angles and Gutiérrez (2008). The meaning-typed delegation used in the final worked example is developed by Dantanarayana et al. (2025) and in Chapter 6. The chapter’s epigraph is from Perlis’s epigrams on programming (Perlis 1982).

THE OBJECT-SPATIAL CALCULUS

Chapter 7 introduced Object-Spatial Programming through working programs and closed by defining topokinesis: the mobile locus of computation over a topology of data, admitted as a first-class semantic construct. A definition of that kind carries obligations. If walker motion is language semantics rather than a library pattern, it must be possible to state, completely and in about a page, what one step of that semantics is. If the paradigm claims kinship with the process calculi and agent systems of Appendix G, the claim must be adjudicated in their own currency: encodings where the ancestors embed, separations where they do not. And if later chapters are to type motion (Chapter 9) and to implement it in a kernel portable across three substrates (Chapter 13), both need one precise object to be right about. This chapter supplies that object: a core object-spatial calculus with a small-step operational semantics in the structural style of Plotkin (1981).

We have written the chapter to be read, not merely consulted. The calculus is deliberately the smallest system that exhibits the paradigm’s distinctive dynamics (a mutable typed graph, walkers with private state, abilities fired by arrival) and nothing else. Exit abilities and their unwinding discipline, node and edge deletion, edge attributes, and the full proofs are developed in Appendix B. The core here fits in nine reduction rules, and nothing in the chapter depends on machinery the reader cannot hold in mind at once. The system refines the abstract machine sketched in the paradigm’s formal definition (Mars 2025), and the executable kernel described in Chapter 13 is, clause for clause, this semantics made to run.

8.1 Syntax of the Core Calculus

This section fixes the syntax of the core calculus, anchored to a concrete program small enough to trace by hand. Listing 8.1 builds a two-person graph and sends a walker through it, and every construct the calculus models appears at least once.

```
node Person { has name: str = ""; }
edge Knows {}
```

```

walker Greet {
  has seen: list[str] = [];

  can start with Root entry {
    visit [-->];
  }

  can meet with Person entry {
    self.seen.append(here.name);
    report here.name;
    visit [-->:Knows:->];
  }
}

with entry {
  alice = Person(name="alice");
  bob = Person(name="bob");
  root ++> alice;
  alice +>:Knows():+> bob;
  root spawn Greet();
}

```

Listing 8.1: The running example: a two-node graph and a walker that traverses it, accumulating and reporting names.

Throughout, N ranges over node types, E over edge types, and W over walker types. Similarly f ranges over field names, x over variables, c over constants (including booleans), and n, m over *node identifiers*, the addresses of the graph store. A distinguished node type `Root` and identifier r name the root. The syntax of the core is as follows.

values	$v ::= c \mid n \mid \bar{n}$
expressions	$e ::= v \mid x \mid \text{here} \mid \text{here}.f \mid \text{self}.f \mid e \xrightarrow{E} \mid \text{op}(\bar{e})$
statements	$s ::= \text{skip} \mid s_1; s_2 \mid \text{if } e \text{ then } s_1 \text{ else } s_2 \mid \text{self}.f := e \mid \text{here}.f := e$ $\mid \text{let } x = \text{new } N(\bar{f} = \bar{e}) \text{ in } s \mid e_1 \xrightarrow{E} e_2$ $\mid \text{visit } e \mid e \text{ spawn } W(\bar{f} = \bar{e}) \mid \text{report } e \mid \text{disengage}$
walker abilities	$A_W ::= \text{on } N \text{ entry } \{s\}$
node abilities	$A_N ::= \text{on } W \text{ entry } \{s\} \mid \text{on } \star \text{ entry } \{s\}$
declarations	$D ::= \text{node } N \{\bar{f}; \bar{A}_N\} \mid \text{edge } E \mid \text{walker } W \{\bar{f}; \bar{A}_W\}$
programs	$P ::= \bar{D}; s_{\text{main}}$

Three design choices deserve comment. First, the ordinary computational core (arithmetic, comparison, string manipulation, everything Jac inherits from its Python semantics) is compressed into the opaque form $\text{op}(\bar{e})$, a signature of total primitive operations about which the calculus assumes nothing. The object-spatial layer is indifferent to what local computation sits inside an ability, and this indifference is the formal shadow of the supersetting strategy of Chapter 3: the paradigm is a thin

Table 8.1: Surface constructs and their images in the core calculus.

Surface form	Core form	Rule
root spawn Greet()	$e \text{ spawn } W(\overline{f=e})$	SPAWN
can meet with Person entry	$\text{on } N \text{ entry } \{s\}$	ARRIVE
visit [->:Knows:->]	$\text{visit } (\text{here } \xrightarrow{E})$	VISIT
here.name = v	$\text{here.}f := e$	STEP-LOCAL
report here.name	$\text{report } e$	REPORT
disengage	disengage	DISENGAGE
alice = Person(name="alice")	$\text{let } x = \text{new } N(\overline{f=e}) \text{ in } s$	NEW-NODE
alice +>:Knows():+> bob	$e_1 \xrightarrow{E} e_2$	NEW-EDGE

stratum over an arbitrary sequential host. Second, expressions are effect-free. Every effect (on the graph, on the walker's own record, on its itinerary) is a statement, so the rules of Section 8.2 never hide a mutation inside evaluation. Third, the query form $e \xrightarrow{E}$ denotes the ordered outgoing E -neighborhood of a node, matching the notation already used for the statics in Chapter 9. We write $e \xrightarrow{\star}$ for the query that crosses edges of every type (surface [->]), and the surface language's filters and predicates desugar to conditionals over query results. Incoming and undirected forms are symmetric and omitted.

A graph is a store with a distinguished entry point:

$$\mathcal{G} ::= (N, \mathcal{E}, r) \quad N ::= \overline{n \mapsto N\{f=v\}} \quad \mathcal{E} ::= \overline{n \xrightarrow{E} m}$$

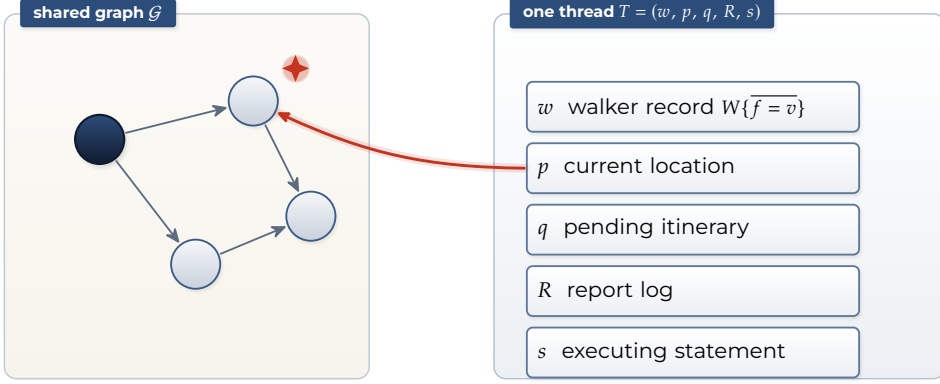
Here N is a finite map from identifiers to typed records and $\mathcal{E}$ is a finite *sequence* of typed directed edges: a sequence rather than a set, because creation order is part of the store, and Section 8.3 leans on it. Edges in the core carry no fields of their own. Attributed edges are a routine extension handled in Appendix B. A program is a set of declarations together with a main statement s_{main} , the calculus image of the surface with entry block. Declarations induce the nominal subtype order $<:$ of Chapter 9, and each walker and node type carries its ability table: for a walker, clauses keyed by node types; for a node, clauses keyed by walker types, with $\star$ for the untyped clause that fires for any visitor. Table 8.1 fixes the correspondence between the surface constructs of Listing 8.1 and the core forms, together with the reduction rule that animates each.

8.2 Operational Semantics of Traversal and Ability Activation

This section defines one step of the dynamics: how a walker moves through the graph, and what fires when it arrives. A walker in flight is a thread with five components: its record instance, its current location, its pending itinerary, its report log, and the statement it is executing (Figure 8.1).

$$w ::= W(\overline{f=v}) \quad T ::= (w, p, q, R, s) \mid \text{done}(w, R) \quad p ::= n \mid \bullet$$

$$\mathcal{W} ::= T \mid \mathcal{W} \parallel \mathcal{W} \quad C ::= \langle \mathcal{W} \mid \mathcal{G} \rangle \quad \mathcal{K} ::= [] \mid \mathcal{K}; s$$



$\mathcal{W} ::= T \parallel \mathcal{W}$: many threads, each private, over the one shared graph. The graph is shared, the threads are not, and this asymmetry is the calculus in miniature.

Figure 8.1: The shape of a configuration $C = \langle \mathcal{W} \mid \mathcal{G} \rangle$. A walker in flight is a five-part thread: its record instance w , its location p pointing into the shared graph, its pending itinerary q , its report log R , and the statement s it is executing. Only $\mathcal{G}$ is shared between threads.

The position p is a node identifier once the walker has arrived somewhere, and the placeholder $\bullet$ between spawn and first arrival. The queue q is a sequence of node identifiers (destinations promised but not yet reached), and R is the sequence of values reported so far. We write ϵ for the empty sequence and silently coerce a single node identifier to the unit sequence where a sequence is expected. A terminated thread collapses to $\text{done}(w, R)$, a quiescent value from which the spawner's environment may read the reports. In the surface language this is the walker object whose reports field the spawner inspects. A configuration $\langle \mathcal{W} \mid \mathcal{G} \rangle$ pairs a soup of threads, composed with the commutative and associative operator $\parallel$, with the one graph they all inhabit: the graph is shared, the threads are private, and this asymmetry is the calculus in miniature. Evaluation contexts $\mathcal{K}$ isolate the leftmost statement in a sequence.

Expressions are evaluated by a partial function $\llbracket e \rrbracket_{w,n,\mathcal{G}}$, defined relative to the walker record w , the current node n , and the graph:

$$\llbracket c \rrbracket = c \quad \llbracket \text{here} \rrbracket = n \quad \llbracket \text{self}.f \rrbracket = w.f \quad \llbracket \text{here}.f \rrbracket = \mathcal{G}(n).f$$

$$\llbracket e \xrightarrow{E} \rrbracket = [m \mid (n' \xrightarrow{E'} m) \in \mathcal{E}, n' = \llbracket e \rrbracket, E' <: E] \text{ in the order of } \mathcal{E}$$

The last clause is where determinism is quietly banked: a query returns its results in edge-creation order, so evaluation is a function, not a relation. In the rules below we drop the subscript. The environment is always that of the configuration on the left of the conclusion.

Ability activation is packaged in a metafunction. When a walker of type W arrives at a node of type N , the statement it executes is the sequential composition of every

matching clause, in an order that gives both parties of the encounter their say:

$$\text{fire}(W, N) = \underbrace{s_1; \dots; s_j}_{\text{walker clauses on } N', N <: N'} ; \underbrace{t_1; \dots; t_k}_{\text{node clauses on } \star} ; \underbrace{u_1; \dots; u_l}_{\text{node clauses on } W', W <: W'}$$

Within each group, clauses appear in declaration order, and matching is subtype-aware on both sides. Node-side bodies are included after a syntactic dualization (their `self` becomes `here` and their `visitor` becomes `self`), so that a single thread, whose `self` is the walker and whose `here` is the node, executes both perspectives without ambiguity. This discharges a promise made in Chapter 7: when traveler and place both declare abilities for the same encounter, the traveler’s typed entries run first, then the place’s untyped entries, then the place’s typed entries.

The dynamics is the least relation $\longrightarrow$ closed under the rules below and under the structural rule `PAR`, which lets any thread step while the rest of the soup stands by:

$$\text{PAR} \frac{\langle \mathcal{W}_1 \mid \mathcal{G} \rangle \longrightarrow \langle \mathcal{W}'_1 \mid \mathcal{G}' \rangle}{\langle \mathcal{W}_1 \parallel \mathcal{W}_2 \mid \mathcal{G} \rangle \longrightarrow \langle \mathcal{W}'_1 \parallel \mathcal{W}_2 \mid \mathcal{G}' \rangle}$$

The nine rules of the core follow. Housekeeping steps ($\mathcal{K}[\text{skip}; s] \longrightarrow \mathcal{K}[s]$, conditional selection, and the walker-field write `self.f := e`, which updates `w` rather than `G`) are routine and relegated to Appendix B. We display the graph-facing representative of the local family.

$$\begin{array}{c}
\text{SPAWN} \\
\frac{\llbracket e \rrbracket = m \quad m \in \text{dom}(\mathcal{N}) \quad \llbracket \bar{e} \rrbracket = \bar{v} \quad w' = W' \{ \overline{f = v} \}}{\langle (w, p, q, R, \mathcal{K}[e \text{ spawn } W' \{ \overline{f = v} \}]) \mid \mathcal{G} \rangle \longrightarrow \langle (w, p, q, R, \mathcal{K}[\text{skip}]) \parallel (w', \bullet, m, \epsilon, \text{skip}) \mid \mathcal{G} \rangle} \\
\\
\text{ARRIVE} \\
\frac{w = W \{ \dots \} \quad \mathcal{G}(m) = N \{ \dots \} \quad \text{fire}(W, N) = s}{\langle (w, p, m \cdot q, R, \text{skip}) \mid \mathcal{G} \rangle \longrightarrow \langle (w, m, q, R, s) \mid \mathcal{G} \rangle} \\
\\
\text{VISIT} \\
\frac{\llbracket e \rrbracket = \bar{m}}{\langle (w, n, q, R, \mathcal{K}[\text{visit } e]) \mid \mathcal{G} \rangle \longrightarrow \langle (w, n, q \cdot \bar{m}, R, \mathcal{K}[\text{skip}]) \mid \mathcal{G} \rangle} \\
\\
\text{STEP-LOCAL} \\
\frac{\llbracket e \rrbracket = v \quad \mathcal{G}' = \mathcal{G}[n.f \mapsto v]}{\langle (w, n, q, R, \mathcal{K}[\text{here}.f := e]) \mid \mathcal{G} \rangle \longrightarrow \langle (w, n, q, R, \mathcal{K}[\text{skip}]) \mid \mathcal{G}' \rangle} \\
\\
\text{REPORT} \\
\frac{\llbracket e \rrbracket = v}{\langle (w, n, q, R, \mathcal{K}[\text{report } e]) \mid \mathcal{G} \rangle \longrightarrow \langle (w, n, q, R \cdot v, \mathcal{K}[\text{skip}]) \mid \mathcal{G} \rangle} \\
\\
\text{DISENGAGE} \\
\frac{}{\langle (w, n, q, R, \mathcal{K}[\text{disengage}]) \mid \mathcal{G} \rangle \longrightarrow \langle \text{done}(w, R) \mid \mathcal{G} \rangle} \\
\\
\text{NEW-NODE} \\
\frac{\llbracket \bar{e} \rrbracket = \bar{v} \quad n' \notin \text{dom}(\mathcal{N}) \quad \mathcal{G}' = \mathcal{G}[n' \mapsto N \{ \overline{f = v} \}]}{\langle (w, n, q, R, \mathcal{K}[\text{let } x = \text{new } N(\overline{f = v}) \text{ in } s]) \mid \mathcal{G} \rangle \longrightarrow \langle (w, n, q, R, \mathcal{K}[s[n'/x]]) \mid \mathcal{G}' \rangle} \\
\\
\text{NEW-EDGE} \\
\frac{\llbracket e_1 \rrbracket = n_1 \quad \llbracket e_2 \rrbracket = n_2 \quad n_1, n_2 \in \text{dom}(\mathcal{N}) \quad \mathcal{G}' = \mathcal{G} \cdot (n_1 \xrightarrow{E} n_2)}{\langle (w, n, q, R, \mathcal{K}[e_1 \xrightarrow{E} e_2]) \mid \mathcal{G} \rangle \longrightarrow \langle (w, n, q, R, \mathcal{K}[\text{skip}]) \mid \mathcal{G}' \rangle} \\
\\
\text{FINISH} \\
\frac{}{\langle (w, p, \epsilon, R, \text{skip}) \mid \mathcal{G} \rangle \longrightarrow \langle \text{done}(w, R) \mid \mathcal{G} \rangle}
\end{array}$$

We write $\mathcal{G}[n.f \mapsto v]$ for the graph with the f field of node n updated, $\mathcal{G}[n' \mapsto N \{ \overline{f = v} \}]$ for the graph extended with a fresh node, and $\mathcal{G} \cdot (n_1 \xrightarrow{E} n_2)$ for the graph with a new edge appended to $\mathcal{E}$. Fields omitted from a construction or spawn take their declared defaults.

The rules repay a slow read. **SPAWN** creates a thread, not a walk: the new walker is instantiated with its arguments, aimed at a single-element queue, and set down *nowhere*. Its position is $\bullet$ and its statement is exhausted, so the only rule that can touch it is **ARRIVE**. **ARRIVE** is the paradigm's characteristic step and the only rule that

moves a walker: when the current statement is spent and the queue is nonempty, the walker relocates to the queue's head and its next statement is manufactured by fire from the ability tables. This is dispatch by arrival, resolved jointly on the walker's type and the node's. If no clause matches, fire yields skip and the walker passes through without incident. `VISIT` does not move anything. It extends the queue, and the distance between `VISIT` and `ARRIVE` (promise versus fulfillment) is what makes the traversal order a programmable discipline, not a fixed evaluation strategy (Section 8.3). `REPORT` appends to a log that only termination makes visible. `DISENGAGE` discards both the remaining statement and the remaining queue, which is why the surface construct halts not only the traversal but the rest of the dispatch chain at the current node. `FINISH` retires a walker whose statement and queue are both exhausted. The two mutation rules, `NEW-NODE` and `NEW-EDGE`, are the only rules that grow the graph, and `STEP-LOCAL` is the only displayed rule that changes it in place.

A program $P = \overline{D}; s_{\text{main}}$ starts in the configuration $C_0 = \langle (w_0, r, \epsilon, \epsilon, s_{\text{main}}) \mid \mathcal{G}_0 \rangle$, where $\mathcal{G}_0$ contains only the root node and w_0 is a distinguished inert walker with no abilities: the main block is a walker that never travels. For Listing 8.1, the main thread performs two `NEW-NODE` steps, one `NEW-EDGE` for `++>` and one for the typed connect, then `SPAWN` aims a fresh Greet thread at r . That thread `ARRIVES` at the root, and fire hands it the body of start. `VISIT` enqueues the root's out-neighborhood, which is the node for `alice`. Arrival there fires `meet`: a local append, a `REPORT` of "alice", and a `VISIT` across `Knows` edges enqueues `bob`. Arrival at `bob` reports again, its `Knows` query returns the empty sequence, and `FINISH` retires the walker with $R = ["alice", "bob"]$. Every step of the trace is forced, a fact Section 8.3 promotes to a proposition.

Two surface conveniences are worth desugaring explicitly, since they recur in every example of Chapter 7. The guarded form `visit ... else` evaluates its query first and runs the alternative exactly when the result is the empty sequence: `visit e else s` $\triangleq$ `let  $x = e$  in if  $x = \epsilon$  then  $s$  else visit  $x$` (with `let` over plain expressions the obvious abbreviation). And a direct visit to a known node, `visit fresh`, is the base case of `VISIT` with a singleton sequence. What the core deliberately omits, beyond deletion and edge state, is the exit phase: in the full language, exit abilities are deferred until everything a node's entry queued has been visited, and they unwind in last-in, first-out order. Formally this adds a stack of pending exits to the thread and a companion to `ARRIVE` and `FINISH` that pushes and pops it. Appendix B gives the extended system, and no result in this chapter changes under the extension.

8.3 Walker Scheduling, Traversal Order, and Determinism

This section fixes the order in which a walker visits and settles how much of that order the calculus determines. The itinerary q is a queue, and the default discipline is first-in, first-out: `VISIT` appends, `ARRIVE` takes the head. Since a node's entry ability typically enqueues that node's successors, the default traversal of a tree is breadth-first. The surface language generalizes the append: `visit :i` splices its destinations into position i of the queue, with out-of-range indices falling back to the tail, so insertion at the front yields depth-first traversal and intermediate indices yield priority schemes. The calculus accommodates this with a family `visiti` whose rule differs from `VISIT` only in replacing $q \cdot \overline{m}$ by the splice of $\overline{m}$ into q at i . Determinism is indifferent to the choice,

because the splice is still a function. Within a single arrival, ability order is fixed by fire: walker clauses, then untyped node clauses, then typed node clauses, declaration order within each group. Across arrivals, entries run in queue order, and, in the full system, exits unwind in reverse arrival order once the queue drains, a call-stack discipline laid over the graph.

The point of all this bookkeeping is a property the paradigm’s practicality depends on: a walker, run alone, is as deterministic as a procedure call.

Proposition 8.1 (Single-walker determinism). *(i) Let C be a configuration containing exactly one active thread, whose statement contains no spawn form. If $C \rightarrow C_1$ and $C \rightarrow C_2$, then C_1 and C_2 coincide up to renaming of the fresh identifiers chosen by NEW-NODE. Consequently C has exactly one maximal reduction sequence up to such renaming, and its arrival order, report sequence, and final graph are uniquely determined. (ii) Under the stack scheduling discipline—among active threads, always reduce the most recently created—the same conclusions hold for every configuration, spawns included.*

Proof sketch. For (i): every statement other than skip decomposes uniquely as $\mathcal{K}[s_0]$ with s_0 not a sequence (induction on the statement), and for each redex s_0 at most one rule applies; when the statement is skip, exactly one of ARRIVE and FINISH applies, by emptiness of the queue. Each rule’s conclusion is determined by its premises, and every premise is computed by a function of the thread and the graph: $\llbracket \cdot \rrbracket$ is a partial function because $\mathcal{N}$ is a map and $\mathcal{E}$ is ordered; fire is a function because declaration order is total; queue operations are functions. The only latitude is the choice of n' in NEW-NODE, quotiented by the renaming. For (ii): the stack discipline makes the schedulable thread unique at every step, and the argument of (i) applies to that thread; a full treatment, including confluence of steps by threads touching disjoint parts of $\mathcal{G}$, is given in Appendix B. $\square$

Clause (ii) is not a theoretical flourish. It is the discipline Jac ships. A surface spawn is synchronous (the spawning walker resumes only when the spawned walker is quiescent), which is the stack discipline itself, and the production kernel maintains a context-local walk stack that implements it (Chapter 13). The claim reported there, that given a fixed graph state the whole schedule is deterministic, is clause (ii) verbatim: the queue is ordered, incidence lists preserve creation order, ability tables preserve declaration order, and these are exactly the three functionalities the proof consumes.

Interleaving across walkers is another matter, and the calculus does not paper over it: when the scheduler is unconstrained (concurrent requests in a served deployment, walkers declared asynchronous), PAR permits any interleaving, and configurations with genuinely shared state are nondeterministic. Two mitigations, both external to the calculus, restore predictability in production. Steps by threads whose reachable subgraphs are disjoint commute, and the scale machinery of Chapter 14 arranges that disjointness by giving each user a private root, so most concurrent walkers never contend. Where they do contend, the runtime makes the walker run its transactional quantum (each run’s effects are committed atomically or replayed, per Chapter 13), so that observable interleavings are serializable. The calculus thus factors the problem the way the implementation does: determinism within a walker is a theorem. Isolation between walkers is an engineering contract layered above the semantics.

8.4 Encodings and Separations: OSP Against Actors, Ambients, and Vertex-Centric Models

Appendix G placed the paradigm’s ancestors historically. The calculus can now place them formally. The results in this section are stated with proof sketches, and Appendix B carries the full constructions.

Actor systems come first, because they embed. In the actor model (Agha 1986; Hewitt et al. 1973), a configuration is a set of stationary agents, each with an address, a state, and a behavior, plus a bag of undelivered messages. A step delivers some message to its addressee, whose behavior runs, sends further messages, creates actors, and designates a replacement behavior.

Proposition 8.2 (Actor systems embed). *There is a compositional translation from a minimal actor calculus—create, send, become—into the object-spatial calculus, mapping each actor to a node, each undelivered message to an in-flight walker, and each behavior to a family of node abilities, such that an actor configuration and its translation are weakly bisimilar: each delivery step corresponds to an ARRIVE of the message walker at the addressee node, bracketed by administrative steps, and conversely.*

Proof sketch. An actor with address a becomes a node n_a whose fields hold the actor’s state; its behavior becomes node abilities on *Msg* entry, dualized as in Section 8.2, whose bodies read the payload from the visiting walker’s fields, update here’s fields for *become*, execute *NEW-NODE* for *create*, and *SPAWN* a fresh message walker aimed directly at the target node for *send*. An undelivered message is a spawned thread still at $\bullet$ with the addressee in its queue; the unconstrained *PAR* scheduling of such threads models the actor model’s delivery nondeterminism exactly. The correspondence and its administrative-step bookkeeping are checked in Appendix B. $\square$

The encoding is instructive twice over. It confirms that OSP restricted to one-hop walkers *is* an actor system (the walker plays the message, the node plays the actor), and it makes the separation legible in the image: the translation never creates an edge. The actor world, rendered object-spatially, is an edgeless graph, because actors hold each other only as addresses squirreled into private state. There is no first-class, queryable, persistent connectivity for a computation to consult. What OSP adds over actors is what the encoding leaves unused: the typed topology, the multi-hop itinerary that interrogates it (visit (here $\xrightarrow{E}$) has no actor counterpart short of a distributed protocol), and durability of the medium itself (Chapter 10).

Mobile Ambients sit on the other side of a genuine separation. An ambient is a named boundary containing processes and other ambients. The containment tree is the spatial structure, and the characteristic reductions (*in*, *out*, *open*) move an ambient, with its entire contents, through the tree (Cardelli and Gordon 2000). Mobility and topology change are one primitive. The object-spatial calculus keeps them apart, and provably so.

Lemma 8.3 (Motion and mutation are orthogonal). *No step of the calculus both changes the graph and moves a walker: for every $C \rightarrow C'$, either $\mathcal{G}' = \mathcal{G}$, or $\mathcal{G}' \neq \mathcal{G}$ and every thread of C persists in C' with its location and queue unchanged.*

Proof. By inspection: ARRIVE, VISIT, SPAWN, REPORT, DISENGAGE, FINISH, and the local steps on walker fields preserve $\mathcal{G}$; STEP-LOCAL on a node field, NEW-NODE, and NEW-EDGE alter $\mathcal{G}$ but touch no location or queue. $\square$

Proposition 8.4 (Ambient mobility does not embed as motion). *There is no translation of the ambient calculus into the object-spatial calculus that (i) represents the containment structure of ambients as a function of the graph component $\mathcal{G}$ and (ii) maps each reduction instance of the in and out capabilities to a sequence of graph-preserving steps of the translated configuration.*

Proof sketch. An in or out step changes containment. By (i), the translated containment is determined by $\mathcal{G}$; by (ii), its image is a sequence of steps with $\mathcal{G}' = \mathcal{G}$, under which every function of $\mathcal{G}$ is constant. So containment both changes and cannot—a contradiction. Lemma 8.3 supplies the invariance; the residual work, carried out in Appendix B, is making hypothesis (i) precise and showing it is forced whenever ambients (which contain other ambients, hence are places rather than travelers) are represented spatially at all. $\square$

The proposition should be read for what it does and does not say. Ambient behavior is of course *computable* in OSP: reify the containment tree as nodes and edges, and implement in and out as edge deletion and creation. But then mobility has become data manipulation (an ambient moves the way a row updates) rather than motion in the semantics' own sense. The two calculi disagree about what moves: ambients move pieces of the spatial medium through itself. Walkers move through a medium that only explicit mutation reshapes. We find the object-spatial resolution truer to the systems this book is about, where the topology is durable shared state (Chapter 10) and computation, not the warehouse, is what travels. The π -calculus (Milner et al. 1992) triangulates the comparison usefully: its processes are stationary and its mobility is the mobility of *links* (scope extrusion lets connectivity evolve), so NEW-EDGE is the object-spatial echo of link mobility, with the difference that an edge is a persistent, typed, attributable value, not an ephemeral capability, while ARRIVE has no π counterpart at all.

Remark 8.5 (Vertex-centric processing as a scheduling discipline). The bulk-synchronous vertex-centric model of Pregel (Malewicz et al. 2010) fits inside the actor image of Proposition 8.2 plus a scheduler: vertex programs are node abilities, messages along edges are one-hop courier walkers, and a superstep is the discipline that arrives every courier spawned in round k before any spawned in round $k+1$. Vertex-centric computation is thus a restriction of the calculus—a particular, barrier-shaped schedule—not an extension of it. What the framework denies its programmer, and the calculus restores, is the free itinerary: a walker whose next hop depends on what it has found, and whose life is not bounded by anyone's driver loop.

8.5 Expressiveness: What Topokinesis Adds, and What It Does Not

Encodings compare the calculus with its neighbors. Expressiveness compares it with its own subsets. The sharp tool here is Felleisen (1991): a construct genuinely adds expressive power over a language when it is not *macro-expressible*, that is, when

no local, compositional translation rewrites each occurrence of the construct, in place, into the unextended language while preserving the observable behavior of all programs. Constructs eliminable only by restructuring the whole program mark genuine distinctions of expressive power. Constructs eliminable by a macro are notation. The question for this book is on which side of that line topokinesis falls.

Fix as the baseline the *sessile fragment* L_0 of the calculus: graphs, the mutation statements, local computation, and a main statement executed at the root, with no walker declarations, no abilities, and no visit, spawn, report, or disengage. L_0 is, quite deliberately, a formal caricature of the conventional arrangement: a heap language with a graph library, computation stationary at one site. Take as observables of a terminated run its report sequences and the root-reachable portion of the final graph, the latter because persistence by reachability (Chapter 10) makes root-reachability the right notion of what a run leaves behind.

Proposition 8.6 (Traversal is not macro-expressible in the sessile fragment). *There is no macro translation of the topokinetic constructs—ability declarations, visit, spawn, disengage—into L_0 that preserves the observables of all programs.*

Proof sketch. Two nonlocalities defeat any candidate macro. First, dispatch: the behavior of visit e depends, through ARRIVE and fire, on the ability tables of the entire program, including declarations arbitrarily far from the occurrence—but a macro image of visit e must be assembled from e and the occurrence’s context alone, so distinct programs agreeing at the occurrence but differing in a distant ability are translated to programs that cannot both be correct. Second, extent: the itinerary q and walker state w persist across ability boundaries, so an L_0 image must thread an explicit representation of walker state through every translated context—a global, defunctionalizing transformation of exactly the kind the macro criterion excludes. The context-based distinguishing argument is carried out in Appendix B. $\square$

Claim 8.7 (Persistence by reachability is not macro-expressible over explicit saving). *Let L_{save} be the variant in which durability is conferred by an explicit `save( $n$ )` primitive rather than by root-reachability. Then reachability persistence is not macro-expressible in L_{save} , while the converse embedding is a one-line macro: $\text{save}(n) \triangleq r \xrightarrow{\text{Saved}} n$ for a reserved edge type.*

Proof sketch. Whether a node created at one program point survives is decided by edges created and deleted at arbitrary other points; a macro image of new or of connect must place its save calls using only local information, and contexts that later link or unlink the node from the root distinguish any fixed placement. The converse macro is immediate, and the asymmetry is the formal content of Chapter 10’s claim that reachability subsumes, and is strictly stronger as a language design than, the save-call convention. Details in Appendix B. $\square$

So topokinesis adds expressive power in Felleisen’s exact sense. It is equally important to record what it does not add, because the restraint is designed. It does not add computational power: a whole-program translation into the sessile fragment plainly exists (reify threads, queues, and ability tables and interpret the rules), and the production runtime is that translation made concrete, a single kernel whose walk scopes, slot tables, and visit queues are the data structures of this chapter’s semantics

(Chapter 13). Felleisen’s distinction between macro and global translation is thus not philosophical bookkeeping here. It names the difference between a paradigm and its own compiler. It does not add communication primitives: walkers never rendezvous, share no channels, and interact only generatively, through what they build and find in the graph. This is the coordination style of Linda’s tuple spaces (Gelernter 1985), chosen because it keeps Proposition 8.1 true and the transactional quantum well defined. And it does not add a theory of distribution: nothing in the rules mentions machines, latency, or partial failure. That silence is deliberate: scale invariance (Chapter 14) is precisely the claim that deployment transformations preserve *this* semantics, and the places where the claim meets its lawful limits are catalogued there, not hidden in the calculus.

Nine rules, one lemma-sized invariant, and two nonlocalities: the formal footprint of topokinesis is small, and the smallness is itself the thesis. A paradigm that fits in a chapter can be checked by a type system (Chapter 9), carried intact across three substrates by one kernel (Chapter 13), and reasoned about when deployments multiply (Chapter 14). Part IV is the record of holding the implementation to that contract.

Bibliographic Notes

The chapter’s method is Plotkin’s structural operational semantics (Plotkin 1981), whose small-step discipline the rules of Section 8.2 follow. The safety properties for the typed calculus, stated in Chapter 9 and proved in Appendix B, use the syntactic technique of Wright and Felleisen (1994). The expressiveness framework of Section 8.5 (macro expressibility and its separation from whole-program translatability) is due to Felleisen (1991). The formal definition of Object-Spatial Programming that the calculus refines, including its account of walkers, abilities, and data-spatial execution, is given by Mars (2025).

The comparative results of Section 8.4 engage the systems surveyed in Appendix G at close range. The actor model originates with Hewitt et al. (1973) and receives its semantic consolidation in Agha (1986). Mobile Ambients are defined by Cardelli and Gordon (2000), whose containment-tree reductions ground the separation of Proposition 8.4. The π -calculus’s link mobility, against which NEW-EDGE is triangulated, is from Milner et al. (1992). Readers who know the join calculus (Fournet and Gonthier 1996) will recognize in abilities a kindred idea (reaction rules localized at a site, fired by what arrives there), though join patterns synchronize messages at a stationary definition rather than dispatching a traveler. Vertex-centric processing enters through Pregel (Malewicz et al. 2010), and the generative, medium-borne coordination style invoked in Section 8.5 descends from Linda (Gelernter 1985).

TYPES AND TOPOLOGY

Chapter 7 introduced Object-Spatial Programming by example, and Chapter 8 fixed its dynamics: configurations of walkers over a graph, traversal as small-step motion, abilities activated by arrival. This chapter supplies the statics. At issue is what a type system should say about a program whose subjects are not values alone but places, relations, and motion. Types in a topokinetic language, we argue, do three jobs that conventional type systems distribute across a language, a database schema, and a pile of validation code.

The first job is classical: archetypes are nominal record types, and their has fields are checked the way fields are checked anywhere (Section 9.1). The second job is distinctive. Because edges are first-class typed relations, the type system speaks about the shape of the graph itself, meaning which relations may hold between which kinds of node and what a traversal expression yields, so that a class of topological errors becomes a compile-time concern (Section 9.2). The third job is imposed by synechism. A type system embedded in a continuous semantic medium inherits the boundaries of that medium. Below Jac sits the dynamically typed Python substrate, which forces a gradual discipline and a carefully chosen soundness posture (Section 9.3). Beside it sits the stochastic substrate of Chapter 6, where the semantic annotations that drive prompt synthesis are best understood as a refinement of the type system, not an accessory to it (Section 9.4).

9.1 Typing Nodes, Edges, and Walkers

In this section, we type the archetypes themselves. Every archetype declaration in Jac introduces a nominal type together with a *kind*: `obj` for plain objects, `node` for locations, `edge` for relations, and `walker` for mobile loci of computation. The data model of an archetype is its set of has fields, each carrying a mandatory type annotation and an optional default. The compiler generates the constructor from these declarations, so the record structure stated in the source is the record structure checked at every

construction site. Listing 9.1 shows the three topological kinds as they appear in the littleX social application.

```
edge Follow {}

node Profile {
  has username: str = "",
    bio: str = "",
    created_at: str = "";
}

walker load_feed {
  has search_query: str = "",
    feed: list[Tweet] = [],
    reports: list[list[Tweet]] = [];

  can run with Root entry {
    mine = [-->[?:Profile]];
    if mine {
      me = mine[0];
      visit [me-->[?:Tweet]];
      visit [me->:Follow:->[?:Profile]-->[?:Tweet]];
    }
  }
  can gather with Tweet entry {
    self.feed.append(here);
  }
}
```

Listing 9.1: Typed archetypes from the littleX social graph: an edge, a node, and a walker whose fields form its record type.

The walker’s fields illustrate how record typing does more than describe memory. The field `search_query` is an input parameter. When the walker is spawned, or invoked as an endpoint in a served deployment (Chapter 14), the constructor arguments are checked against the declared types, so the walker’s record type doubles as the schema of its request payload. The field `feed` is typed accumulator state carried across the traversal. And `reports` types the walker’s report channel: each report statement is checked against the declared element type, so the values a walker sends back to its spawner form a typed contract. A walker that omits the declaration falls back to `list[any]`, a gradual default whose consequences we take up in Section 9.3.

Kinds are part of the static classification, not a comment on intent. Two archetypes with identical fields but different kinds are different types with different affordances: only nodes may be endpoints of edges, only edges may relate, only walkers may be spawned and moved, and `obj` values have no location at all. littleX uses this last kind deliberately. Its report bundles `ProfileBundle` and `ChannelBundle` are `obj` declarations, data without place, the typed carriers in which a walker hands a node to the client together with the edge-derived context (follower lists, member counts) the client needs beside it. Misusing a kind is a compile-time error. For example, the endpoint clause

of Section 9.2 is grammatical only on an edge, and attaching it to a node or walker is rejected outright.

The record discipline is stricter than the Python substrate it compiles to, and deliberately so. All fields of an archetype must be declared up front. Adding attributes to an instance dynamically is treated as an anti-pattern slated for outright rejection, and fields whose values are computed from other fields are marked `postinit` and assigned in a `postinit` hook rather than sprouted ad hoc. The rationale is substrate transparency (Chapter 2): the same record must lower to a Python dataclass, a JavaScript object, and a native struct (Chapter 12), and only a closed, declared shape lowers to all three. Here the type system is not merely catching mistakes. It is the contract that makes one surface expressible on every substrate.

Finally, archetype inheritance generates the subtype order. A declaration `node User(Entity)` makes `User` a subtype of `Entity` with the inherited fields prepended to its record. `littleX`'s walker `follow_user(find_profile)` inherits both the fields and the abilities of its lookup base, adding only the ability that acts on arrival. Subtyping interacts with dispatch: an ability declared with `N` entry activates for arrivals whose dynamic type is `N` or any subtype, an event clause may name a union of types (with `Node1 | Node2` entry, under which here is typed as the union), and typed context blocks let one ability branch on subtypes of its declared arrival type. Dispatch by type is thus not an idiom in Jac but the core control-flow mechanism of the paradigm, which is why the next section treats the typing of traversal itself.

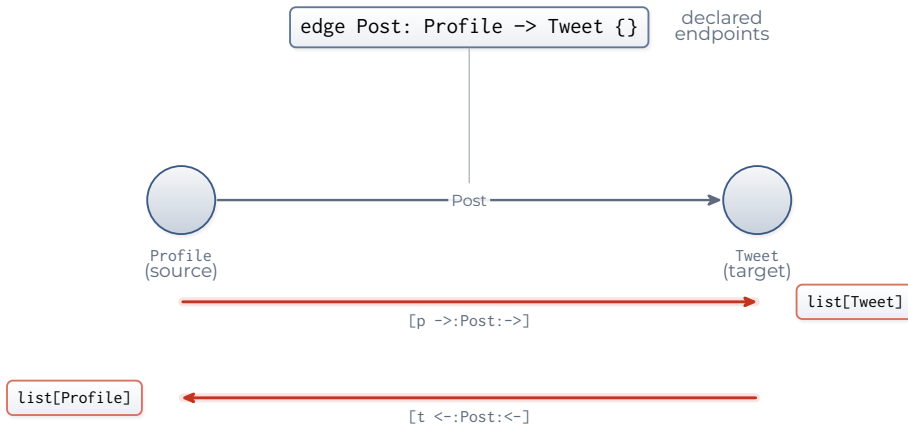
9.2 Edges as Typed Relations

In this section, we lift typing from the node to the relation between nodes. In an object-oriented language a relationship between two objects is encoded as a reference field, and the type system sees only the field. In Jac the relationship is a value of an edge type: it can carry attributes (edge `Friend { has since: int; }`), it can be queried, filtered, and deleted as itself, and, the concern here, it participates in typing. An edge type is a relation type, and the graph is a structure whose well-formedness the type system can constrain.

Traversal expressions are where relation typing pays off. The unqualified neighborhood `[->]` places no constraint on what it reaches, so its element type is any. Precision is recovered in two ways, both visible in Listing 9.1. A type-test filter `[?:Profile]` narrows the element type to `Profile`, discarding non-matching nodes. An edge qualification `->:Follow:->` restricts the traversal to the declared relation, and may add attribute predicates on the edge itself, as in `[->:Friend:since > 2020:->]`. The two compose across hops: `littleX`'s feed walker visits `[me->:Follow:->[?:Profile]->[?:Tweet]]`, a typed path expression reading “along `Follow` to profiles, then to their tweets.” The prefix form `[edge ->]` selects the edge values themselves rather than the nodes they reach, typed accordingly.

Filters recover precision at the use site. Declared endpoints move it to the declaration. An edge type may state the node types it relates, using the traversal arrow so the declaration reads like the navigation it enables:

```
node Profile { has name: str = ""; }
node Tweet { has content: str = ""; }
```



the result type follows the direction of motion

endpoints are bounds, so subtypes are admitted at either end; an undeclared edge stays any-to-any, tightened one relation at a time

Figure 9.1: Declared edge endpoints make traversal a typed operation. One declaration types both directions: the outgoing traversal narrows to the declared target type and the incoming traversal to the source type, so member access on the result resolves statically. Endpoints are bounds rather than exact types, and undeclared edges keep their permissive connectivity, preserving the gradual path.

```

edge Follow: Profile --> Profile {}      # Profile to Profile
edge Post: Profile --> Tweet {}         # Profile to Tweet

with entry {
  me = Profile(name="me");
  following = [me ->:Follow:->];         # inferred list[Profile]
  followers = [me <-:Follow:<-];         # inferred list[Profile]
  my_tweets = [me ->:Post:->];          # inferred list[Tweet]
  if following {
    print(following[0].name);           # resolves statically
  }
}

```

Listing 9.2: Declared edge endpoints make traversal a typed operation whose result type follows the direction of motion.

Three design points deserve note. First, the result type follows the direction of motion: an outgoing traversal narrows to the declared target type, an incoming traversal to the source type. Second, an endpoint is a bound, not an exact type: Follow accepts Profile or any subtype at either end, so heterogeneous graphs built over a node hierarchy remain well typed, and a trailing [?:Sub] filter can still narrow further. Third, endpoints are opt-in and gradual: an undeclared edge Link {} keeps its permissive any-to-any connectivity and list[any] traversals, so a program may tighten

its topology one relation at a time. Figure 9.1 draws the rule from the declaration’s point of view: one declared relation, both directions typed.

This is what we mean by topological constraints as types. The schema of the graph (which relations may hold between which kinds of place) is stated in the language’s static semantics and enforced by `jac check` before the program runs. The contrast with the graph-data tradition is instructive: property-graph systems treat schema as optional metadata validated by the engine, when it is validated at all (Angles and Gutiérrez 2008), and traversal languages such as Gremlin and Cypher check their queries against that schema only at the database boundary (Francis et al. 2018; Rodriguez 2015). In Jac there is no boundary at which a separately maintained schema could live. The archetype declarations are the schema, and the persistent topology of Chapter 10 is an instance of them. The payoff is double. Statically resolved traversal types catch schema violations where they are written, and they make field access on traversal results resolvable at compile time, a precondition for lowering object-spatial code to the native substrate, where dynamic attribute lookup does not exist.

We now make a fragment of this precise, in the notation of Chapter 8. Let Σ be the archetype signature of a program, assigning each declared name its kind and record. For an edge type E with declared endpoints, $\Sigma(E) = (N_s, N_t)$ gives its source and target bounds, and for an undeclared edge type we set $\Sigma(E) = (\text{any}, \text{any})$, recovering the gradual behavior of Section 9.3. The relation $<:$ is the nominal subtype order generated by archetype inheritance. We write $a \xrightarrow{E} b$ for the connect operation (surface $a \text{ } +>:E():+> b$) and $a \xrightarrow{E}$ for the outgoing E -neighborhood of a (surface $[a \text{ } ->:E:->]$). Incoming forms are symmetric. The judgment $\Gamma \vdash_W s \text{ ok}$ types a statement in the body of walker type W .

$$\frac{\text{T-CONNECT} \quad \Gamma \vdash a : N_1 \quad \Gamma \vdash b : N_2 \quad \Sigma(E) = (N_s, N_t) \quad N_1 <: N_s \quad N_2 <: N_t}{\Gamma \vdash a \xrightarrow{E} b : E}$$

$$\frac{\text{T-VISIT} \quad \Gamma \vdash a : N \quad \Sigma(E) = (N_s, N_t) \quad N <: N_s}{\Gamma \vdash_W \text{visit}(a \xrightarrow{E}) \text{ ok}}$$

$$\frac{\text{T-ABILITY} \quad \Gamma, \text{self} : W, \text{here} : N, \text{root} : \text{Root} \vdash s \text{ ok}}{\Gamma \vdash_W \text{can } f \text{ with } N \text{ entry } \{s\} \text{ ok}}$$

T-CONNECT says an edge of type E may be created only between nodes whose static types refine E ’s declared bounds. The expression has type E , the edge being a first-class value. (The surface operator additionally returns its right-hand operand to support chained construction, a convenience the calculus omits.) T-VISIT is the rule the paradigm turns on: a `visit` is well typed only along a declared edge type whose source bound accommodates the walker’s current position, and the destinations it enqueues carry static type N_t . Traversal along an undeclared relation is not rejected. It is typed at `any`, shifting the obligation to filters or to dispatch. T-ABILITY types an

ability body under the assumptions its event clause announces: `self` at the walker's own type, here at the declared arrival type N . The corresponding dynamic rule of Chapter 8 activates the ability at a node n exactly when the dynamic type of n is a subtype of N , so the static assumption on here is justified at every activation. This pairing (activation condition matching arrival typing) is the object-spatial analogue of sound method dispatch, with the receiver replaced by a place.

The rules compose into a preservation property for the topology itself.

Proposition 9.1 (Preservation of topological well-formedness). *If a configuration $\langle \mathcal{W} \mid \mathcal{G} \rangle$, in the sense of Chapter 8, is well typed under Σ and $\langle \mathcal{W} \mid \mathcal{G} \rangle \longrightarrow \langle \mathcal{W}' \mid \mathcal{G}' \rangle$, then $\langle \mathcal{W}' \mid \mathcal{G}' \rangle$ is well typed under Σ ; in particular, every edge of type E in $\mathcal{G}'$ connects nodes whose dynamic types refine the declared bounds $\Sigma(E)$.*

The full statement, together with the progress property and proofs for the core calculus, appears in Appendix B. Here we record only its shape. Because the graph persists beyond any one program run (Chapter 10), the invariant is more consequential than its analogue in a heap-allocated language: a topology certified well formed today is the topology some other walker traverses next month.

9.3 Gradual Typing Against the Python Substrate

In this section, we confront the substrate beneath the topology. The type system just described does not float free. It is bolted to a dynamically typed substrate. Jac's supersetting strategy (Chapter 3) means Python code, Python values, and Python libraries flow into Jac programs as first-class citizens, and the PyPI ecosystem arrives in every state of annotation: fully stubbed, partially hinted, or bare. A fully static type system would break semantic containment (some imported values have no static type), while a fully dynamic one would forfeit the topological guarantees of the previous section. The resolution is the one the gradual typing literature developed: a distinguished dynamic type through which typed and untyped code interoperate (Siek and Taha 2006). In Jac the dynamic type is spelled `any`. Declaration sites (has fields, parameters, return types) carry mandatory annotations, while local bindings may be inferred. A value of type `any` can hold anything, and reading from it yields `any`.

Substrate transparency then forces a second decision that the academic treatments leave open. Sound gradual typing, in the lineage of Siek and Taha (2006), inserts runtime casts or wrappers at every boundary between typed and untyped code, so that a violated annotation is detected the moment it is violated. However, Jac compiles to Python bytecode and shares Python's object model (Chapter 12): a Jac list *is* a Python list, and a wrapped or monitored value would be a marshaling layer by another name, the artifact the language exists to eliminate. Jac therefore adopts erasure, the posture TypeScript chose for the same reason at the JavaScript boundary (Bierman et al. 2014): static types compile away, and no checks are inserted between Jac and Python code. The same posture recurs at the memory-discipline boundary, where the ownership annotations of Chapter 5 are likewise erased rather than reified, and for the same reason. What distinguishes Jac's discipline is that it compensates statically. Where PEP 484 lets `Any` propagate silently into any annotated position, Jac enforces

a strict flow rule inside `.jac` source: a value of type `any` must not silently widen into a destination with a declared non-`any` type. The check fires at every declared sink (annotated assignments, `has` initializers, arguments against declared parameters, returns against declared return types) and recurses element-wise into containers, so `list[any]` is barred from `list[Task]` just as `any` is barred from `Task`. Two kinds of destination stay permissive, captured by the following rule alongside ordinary subsumption:

$$\frac{\text{T-Flow-Any} \quad \Gamma \vdash e : \text{any} \quad T \in \{\text{any}, \text{object}\} \text{ or } T \text{ is a type variable}}{\Gamma \vdash (x : T = e) \text{ ok}}$$

Unannotated locals also remain permissive: a binding without an annotation accepts `any` and becomes `any` itself, so exploratory code pays no ceremony. What the rule forbids is the silent laundering of dynamism into positions the programmer declared static. Listing 9.3 shows the discipline at a real boundary.

```
import json;

def parse(text: str) -> any {
    return json.loads(text);
}

obj Task { has title: str = ""; }

with entry {
    raw = parse('{"title": "ship"}'); # inferred local: raw is any
    # a declared sink (t: Task = raw;) is rejected by the flow rule
    if isinstance(raw, dict) {
        title = raw.get("title", "");
        if isinstance(title, str) {
            t = Task(title=title); # typed flow after narrowing
        }
    }
    d = raw as dict; # unchecked, erased re-typing
}
```

Listing 9.3: The gradual boundary: an untyped value from the Python substrate enters as the dynamic type and must be narrowed, retyped at the source, or cast before it flows into a declared type.

Three repairs clear a strict-any diagnostic, in descending order of preference. Type the source: a `.pyi` stub beside a Python utility, a return annotation on a `def`, or a typed `has` reports declaration on a walker makes the boundary strong and leaves downstream code untouched. Inside `.py` files PEP 484 semantics are preserved, so stubs are ordinary Python practice. Narrow at the boundary: accept `any` explicitly and refine with `isinstance` before flowing into typed sinks. Or cast at the use site: value as `T` retypes the value for the checker while doing nothing at runtime. The cast is deliberately unchecked and erased. It exists for the case where the programmer

knows more than the checker, and it is the documented escape hatch, not a hidden one.

This yields a soundness posture we can state exactly, in three tiers. For the fully annotated fragment that avoids any, we claim type safety in the classical sense (progress and preservation, established by the syntactic method of Wright and Felleisen (1994)) for the core calculus, with statements and proofs in Appendix B. The production checker approximates that idealization in the manner of every industrial type system. For gradually typed programs, the aspiration is the gradual guarantee articulated by Siek et al. (2015): adding or removing annotations should change what the checker rejects, never what a passing program computes, a property erasure gives almost by construction. What Jac does not provide is blame: a violated annotation at an unchecked boundary surfaces as an ordinary Python-level error later, not as a typed cast failure at the boundary, and an unsound as makes this possible even between Jac declarations. This is TypeScript’s bargain (Bierman et al. 2014), accepted knowingly, with the strict flow rule narrowing the surface over which it can be struck. One structural mitigation deserves emphasis: ability dispatch is a genuine runtime type test by construction. A walker arriving at a node activates only the abilities whose declared arrival types its dynamic type refines, whatever the annotations elsewhere in the program claim. In the topological fragment, then, the failure mode of a type mismatch is a non-event, not a corruption. The caveat cuts the other way: a non-event can hide a bug, which is the diagnostic gap that declared endpoints (Section 9.2) close by turning missed dispatch into a static error.

The gradual posture is also where a skeptical reader will press hardest on the jurisdiction proposition of Chapter 2, so we close the section by auditing that claim in its own terms. The proposition promised reach: every crossing expressed in the language lies within the checker’s sight. This section has measured the depth of that reach. Across the dissolved tier and persistence crossings the guarantees are whole: walker payloads, report channels, boundary types, and traversal results are declared once and checked at every use, so the cross-substrate refactorings that a conventional stack delegates to textual search are compile-time errors here. Against the Python substrate the reach remains but the depth is gradual: an unannotated import enters as any, the flow rule stops it at every declared sink, and the three repairs above are how a program tightens its own guarantees, one declaration at a time. What is never on offer is the conventional stack’s third state: the crossing no checker examines at all. In a synechic program every crossing is at least a gradual boundary under one checker’s jurisdiction, refinable to a static one by annotation. In the fragmented stack, the crossings between jurisdictions are refinable by nothing, because no checker stands over them to be strengthened.

9.4 Meaning Types Within the Type System

Chapter 6 introduced the delegation construct by `llm()` and the `sem` annotation, and defined meaning types as the semantic annotations from which prompts are synthesized. We close this chapter by placing that machinery where we believe it belongs: inside the type system, as a refinement of the signatures the checker already maintains.


```

enum Category { WORK, PERSONAL, SHOPPING, HEALTH, FITNESS, OTHER }

def categorize(title: str) -> Category by llm();
sem categorize = "Categorize a task based on its title";

obj Ingredient {
  has name: str;
  has cost: float;
}
sem Ingredient.cost = "Estimated cost in USD";

def plan_shopping(recipe: str) -> list[Ingredient] by llm();
sem plan_shopping = "Generate a shopping list for the given recipe";
sem plan_shopping.recipe = "A description of the meal to prepare";

```

Listing 9.4: Meaning types in the day planner: semantic annotations attach to declared names, and the annotated signature is the specification from which the prompt is synthesized.

Notice what `sem` attaches to: not strings in expressions but declared names (a function, a parameter, a field of an archetype). The unit of delegation in Listing 9.4 is a typed signature, and everything the runtime sends to the model is derived from it: the function's name and `sem`, each parameter's name, type, and `sem`, and the return type together with the annotated structure of the archetypes it mentions, recursively (`list[Ingredient]` carries `Ingredient.cost`'s annotation with it). This is the sense in which typed prompts are signatures (Dantanarayana et al. 2025): the prompt is a compiler artifact synthesized from the type environment, as a marshaling stub is synthesized from an interface declaration, and it is versioned, checked, and refactored with the code because it *is* the code.

Read type-theoretically, a meaning type refines a simple type without changing its representation. The functions `categorize` and a hypothetical `summarize_title` might share the simple type of string to enum, yet they denote different functions, and the `sem` annotation is what distinguishes their meaning types. The checker treats them identically while the prompt synthesizer does not. Refinement runs in the other direction too: the declared return type acts as an output contract at the stochastic boundary. The model's raw output is coerced and validated against the declared type (a `Category` must be one of six variants, a `list[Ingredient]` must parse into records with a `str` name and `float` cost), and failure is a typed error at the delegation site, with the retry and validation machinery of Chapter 6 layered above it in the spirit of design by contract (Meyer 1992). The asymmetry with Section 9.3 is deliberate and, we think, principled: Jac inserts no dynamic checks at the deterministic Python boundary, where checks would be marshaling, and always checks the stochastic boundary, where the substrate offers no invariants of its own. Each substrate gets the discipline its failure model demands.

The synthesis of this chapter's three threads is visible in a single deployment. An archetype's fields type its records. Declared endpoints type the relations among them. And `sem` annotations give the same declarations meaning a model can act on. A walker carrying a `by llm()` ability thus traverses a topology that is simultaneously schema, data, and specification. Nothing separates the type system that checks the

graph from the one that explains it to a language model. In a synechic language there is only one type system, facing every substrate at once.

Bibliographic Notes

Gradual typing enters the literature with Siek and Taha (2006), who isolated the dynamic type and the consistency relation that let typed and untyped code interoperate without the coercion pitfalls of earlier optional typing. Siek et al. (2015) later distilled the criteria, most prominently the gradual guarantee, against which gradual systems are judged, and Section 9.3 states Jac’s posture in their vocabulary. The erasure-based, deliberately unsound end of the design space is best documented by Bierman et al. (2014), whose account of TypeScript’s compromises at the JavaScript boundary parallels Jac’s at the Python boundary. The syntactic method by which we establish safety for the fully annotated core is due to Wright and Felleisen (1994), and Appendix B carries the details.

On the topological side, the formal treatment of Object-Spatial Programming that this chapter’s rules refine appears in Mars (2025). The contrast drawn in Section 9.2 with engine-enforced schema rests on the graph-database literature: Angles and Gutiérrez (2008) survey the model space, and Rodriguez (2015) and Francis et al. (2018) document the traversal and query languages whose typing stops at the database boundary.

For Section 9.4, the meaning-typed abstraction, its prompt-synthesis semantics, and its empirical evaluation are developed in Dantanarayana et al. (2025). The reading of declared return types as enforced output contracts descends from the design-by-contract tradition of Meyer (1992).

PERSISTENCE BY REACHABILITY

The preceding chapters of this part developed topology as a semantic medium: computation expressed as walkers moving through a graph of nodes and edges, with types disciplining both the shapes and the journeys. This chapter extends that medium along a dimension the paradigm has so far left implicit: time. Most data of consequence outlives the process that creates it, and for half a century the standard response has been to delegate long-lived memory to a separate system with its own language, its own type system, and its own notion of identity. The database is where the fragmentation anatomized in Chapter 1 is most thoroughly institutionalized: no discontinuity in a conventional stack is defended by more tooling, more convention, and more resignation than the one between the program and its store.

Jac dissolves this discontinuity with the same construct that carries the rest of the paradigm. The rule is stated in one sentence: *whatever is reachable from the root node persists*. There is no connection to open, no session to manage, no mapping layer between classes and tables, and no save call in the ordinary path of a program. Durability is a property a datum acquires by where it stands in the topology, not an operation the program performs upon it. This chapter develops that rule and its consequences. We first show the rule at work and measure what it removes (Section 10.1), then place it in the lineage of orthogonal persistence, whose program it completes in a restricted form that, we will argue, is precisely what makes it work (Section 10.2). The second half confronts the two obligations any persistence story must discharge in full: consistency when many walkers write the same living graph (Section 10.3), and schema evolution when the graph must change shape without ever shutting down (Section 10.4).

10.1 The Root Node and the Dissolution of the Database Discontinuity

Codd (1970) separated data from program deliberately, and for good reason: in a large shared data bank, applications should not depend on how data is physically arranged,

and a relational schema with a declarative query language delivers that independence. The relational model succeeded so completely that its founding separation hardened into an unexamined premise of application architecture: the program lives in one semantic regime and its durable data in another. For the application programmer, the premise manifests as a taxonomy of glue that Chapter 1 has already cataloged. There are two languages (the host language and SQL), two type systems (classes and columns, reconciled by hand or by an object-relational mapping layer), two identity regimes (object references in memory, primary keys in the store), and two failure vocabularies (exceptions and rollbacks). The object-relational mapper, the standard industrial mitigation (Fowler 2002), does not remove the discontinuity. It staffs it, generating at runtime the marshaling code the programmer would otherwise write. Graph databases relocate the data model without relocating the boundary: systems in the property-graph tradition (Angles and Gutiérrez 2008) and their traversal languages (Francis et al. 2018; Rodríguez 2015) put a topology on the far side of the same connection, so the program still assembles queries as strings or builder chains and still receives results as records to be rehydrated.

The object-spatial alternative begins with a distinguished node. Every Jac program has a root: the anchor of its graph, available in any execution context. In a served deployment every authenticated user has one of their own, a multiplicity whose consequences Chapter 14 develops. The persistence rule attaches to it directly: a node or edge connected to root, directly or transitively, is durable. One that is not, is not. Listing 10.1 is the entire demonstration, drawn from the language’s persistence reference.

```
node Person {
  has name: str;
  has age: int = 0;
}

walker create {
  can s with Root entry {
    here ++> Person(name="alice", age=30);
    here ++> Person(name="bob", age=25);
  }
}

walker dump {
  can r with Root entry { visit [-->]; }
  can p with Person entry { print(f"{here.name}:{here.age}"); }
}
```

Listing 10.1: Nodes attached to the root persist across runs with no persistence code.

Running the create walker attaches two Person nodes to the root. The process then exits. A later invocation of dump (a different process, minutes or months afterward) traverses [-->] and finds alice and bob where they were left. Nothing in the source distinguishes the first run from the second. Locally the runtime writes to an embedded SQLite store under the project’s .jac directory. Configured for scale, the same program persists to a document database across a fleet of pods. The storage backend is a

runtime concern, examined in Chapter 13, and swapping it changes no program text. The developer-facing model is the sentence with which we began.

Identity crosses the dissolved boundary by the same economy. In a conventional stack a single conceptual entity carries three identities: a reference in program memory, a primary key in the database, and a serialized identifier in the JSON that crosses the network. Jac collapses these into one. Every archetype instance has a stable identifier available as `jid(node)`, and the inverse `jobj(id)` resolves an identifier back to the live object. The littleX social application (a compact production-shaped program distributed with the language and examined quantitatively in Chapter 15) uses `jid` as its only identity currency: a node reported by a walker arrives in the browser still carrying its identity, so client code writes `jid(tweet)` on a value it received over the wire, where a REST API would expose a database key. Client code then hands those strings back to lookup walkers such as the one in Listing 10.2.

```
walker find_profile {
  has target_id: str = "";

  can run with Root entry {
    if self.target_id {
      target = jobj(self.target_id);
      if isinstance(target, Profile) {
        visit [target];
      }
    }
  }
}
```

Listing 10.2: A littleX lookup walker resolves a persisted node by its identifier and visits it.

The resolution in Listing 10.2 is not a query. It is a typed lookup into the same semantic medium the walker already inhabits, followed by an ordinary visit. What a conventional application would express as a repository method, a parameterized SQL statement, a row-to-object hydration, and a cache check is here one built-in call and a type test.

We should be precise about what has and has not disappeared. There is still a store on disk. Bytes are still serialized to it. An operator can still inspect it with the `jac db` tooling of Chapter 4. What has dissolved is the *discontinuity*: the program-visible boundary at which expression must change form to cross. The database has moved from the far side of the program to the far side of the runtime, joining the garbage collector and the instruction scheduler among the machinery a language keeps beneath its semantics. This is substrate transparency, in the sense of Chapter 2, applied to the storage substrate.

10.2 Persistence as a Topological Property, Not an I/O Operation

The claim that a language, rather than a library, should own persistence is not new. Atkinson et al. (1983) introduced PS-algol on this conviction, and the research program it launched, orthogonal persistence, was consolidated a decade later by Atkinson and

Morrison (1995) into three principles. *Persistence independence*: the form of a program is independent of the longevity of the data it manipulates, so the same code operates on transient and durable values alike. *Data type orthogonality*: persistence is available to all data types, not to a privileged subset of “database types.” *Persistence identification*: the mechanism that selects which data persists is orthogonal to the program’s expression. It is not an explicit save call, not a special declaration threaded through the code. The canonical identification rule in that literature was reachability from designated persistent roots, and the reader will recognize it: Jac’s rule is that rule, adopted whole. In the terms of this lineage, the language realizes all three principles, with reachability from root as the identification mechanism, walkers that cannot observe whether the nodes they traverse were created this run or years ago (independence), and archetypes whose fields persist whatever their types (orthogonality).

Orthogonal persistence nonetheless remained a research program while the object-relational mapper conquered industry. The sober reading of the record is that the orthogonal systems claimed too much surface. Persisting *the entire language heap* (closures, stacks, threads, handles to operating-system resources) made the persistent set expensive to identify and to checkpoint, and semantically awkward at the edges, where a resurrected file descriptor means nothing. The engineering never became cheap enough to displace a relational orthodoxy that was, meanwhile, delivering data independence and transactions at scale. Jac’s refinement is a restriction that the paradigm makes principled. Object-spatial programming (Chapter 7) already partitions a program into data with *place* and computation with *motion*: nodes and edges form the topology, walkers traverse it, and plain objects travel as values alongside both. Persistence attaches to place. The persistent set is therefore not the heap but the topology: a well-defined, typed, runtime-tracked subgraph whose membership the reachability rule decides exactly. Each archetype instance is backed by an *anchor* that the runtime tracks. When an anchor is reachable from a root, the runtime writes it to the configured backend at each commit and at process exit. Ordinary objects, closures, and walker-local state remain transient, as they should: they are the moving parts, not the remembered world. The moving parts have a lifetime economy of their own, and Chapter 11 gives it the same treatment this chapter gives durability: as a property of place.

The section title’s second clause deserves its own paragraph. In the I/O conception, persistence is an *event*: the program does something, at a moment, to make data durable, and forgetting to do it is a bug. In the topological conception, persistence is a *predicate*: a datum is durable exactly while it stands in a reachable position, and the predicate’s value changes only when the topology changes. Attaching a node with `++>` confers durability. Deleting it with `del`, or severing the last path that connects it to a root, revokes durability. No statement in between maintains it. The analogy with garbage collection is exact and, we think, illuminating. A tracing collector defines *liveness* by reachability from roots and reclaims everything else. The persistence engine defines *durability* by reachability from root and lets everything else evaporate with the process. One rule serves both temporal directions (Figure 10.1): reachability decides what survives the past (collection) and what survives into the future (persistence). A language built on topology gets this symmetry without new concepts, because the reachability structure that walkers navigate is the same structure the persistence

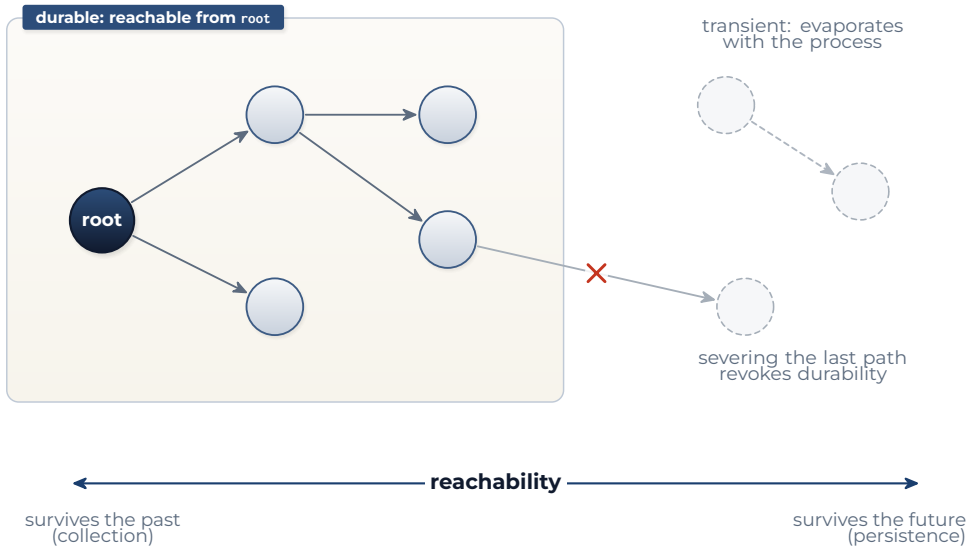


Figure 10.1: Persistence by reachability. One rule serves both temporal directions: whatever is reachable from root is durable at each commit, and whatever is not evaporates with the process, exactly as a tracing collector defines liveness. Attaching a node confers durability; severing its last path revokes it; no statement in between maintains it.

engine consults.

The rule also composes cleanly with multiplicity. Because identification is reachability from a root, and because a served deployment gives each user a root, the persistent world factors into per-user subgraphs with no tenancy code: isolation falls out of the identification rule. Chapter 14 develops this composition as one instance of scale invariance. Here we note only that it is the persistence rule, not an access-control layer, doing the structural work.

10.3 Consistency and Transactions over a Living Graph

A persistent topology that serves live traffic is written concurrently, and a persistence story that stops at durability is not a story. The classical frame remains the right one: the transaction, as consolidated by Gray and Reuter (1993), is a unit of work that succeeds or fails atomically, in isolation from its rivals, with durable effect. Jac draws the transactional boundary at a natural semantic joint: the walker. In a served deployment each spawned walker executes as one unit of work. Its graph writes are staged during traversal and flushed at commit in referential-integrity order: child nodes first, then connecting edges, then the edge-list update on the already-persistent parent. A crash mid-flush can therefore strand at worst an unreachable orphan, never a dangling reference visible to traversal. On the embedded SQLite backend the whole unit executes in a single transaction and rolls back on failure. On the distributed document backend, where cross-document rollback does not exist, the

runtime prechecks versions before staging any write, and the narrow race that survives leaves only a half-linked edge that traversal ignores and an offline integrity pass reclaims. The guarantees are backend-shaped, and we state them as such. The machinery is detailed in Chapter 13.

Isolation over a shared graph is where design philosophies diverge, and Jac’s choice is optimistic concurrency at the granularity of the node. Every node carries a version. When a walker’s traversal *reads* a node’s out-edges (the bracketed expression in Listing 10.3), the runtime snapshots that version. At commit, any edge-list change to a node the walker read is applied with a compare-and-swap against the snapshot. The canonical race this closes is *find-or-create*: two concurrent requests against the same root both observe no profile, and both attach one.

```
walker ensure_profile {
  can go with Root entry {
    found = [-->[?:UserProfile]];
    if found {
      report found[0];
    } else {
      new = here ++> UserProfile(tier="free");
      on_commit(lambda () { grant_signup_bonus(new); });
      report new;
    }
  }
}
```

Listing 10.3: Find-or-create is safe under concurrency, and the deferred side effect fires exactly once.

Under contention, the first committer wins its compare-and-swap. The second conflicts before it can write a duplicate. What happens next distinguishes the system’s temperament: the losing request does not surface an error. The runtime discards its uncommitted work and *replays the walker from the start*. The replay re-reads the graph, now containing the winner’s node, takes the find branch, and returns normally. Two racing creations converge on one node, and both clients observe success. Replay is also why Listing 10.3 defers its external effect: a body that charged a card would charge it once per attempt, so the ambient `on_commit` builtin registers work that runs only after the unit commits and is discarded on abort: exactly-once external effect against at-least-once execution. The policy is configurable. A service that prefers to handle conflicts itself can select immediate typed failure over replay.

The granularity is chosen with an eye to a result worth citing in full. The CALM theorem of Hellerstein and Alvaro (2020) establishes that a program has a consistent, coordination-free distributed implementation exactly when it is monotone: growing the input can only grow, never retract, the output. Jac’s conflict rule charges coordination only where CALM says it is owed. A walker that blindly appends (attaching a log entry without first reading the parent’s edges) takes no version dependency, so concurrent appends to the same node merge without serializing: monotone growth, coordination-free. It is the walker that *checks for absence* before creating, a nonmonotone test whose truth a concurrent writer can retract, that snapshots a version and pays the conflict-and-replay cost, and only on the node it

actually checked. The alignment is not cosmetic: it means the common write patterns of a social graph (posting, liking, logging) proceed lock-free, while the patterns that genuinely require agreement (uniqueness, membership) get it.

These guarantees have limits, and we mark them in the spirit of the lawful boundaries of Chapter 2. Atomicity and convergence are per-walker and per-node. The runtime does not provide multi-node serializable transactions, and an invariant spanning several independently versioned nodes is the program's responsibility to maintain, by routing the writes that guard it through a common node whose version arbitrates. Version granularity is per node, so two different find-or-creates contending on the same parent can each suffer a spurious replay: harmless, since replay re-confirms, but a cost. And distribution does not repeal the impossibility results: under partition, availability and consistency still trade against each other (Brewer 2012; Gilbert and Lynch 2002), and the replicated caches of the scaled runtime deliver the bounded staleness that Vogels (2009) taught production systems to name rather than deny. The single system image of Chapter 14 is a semantic continuity claim, not a physics claim. What the persistence layer promises is that the boundaries which must exist surface as typed, configurable behaviors (replay budgets, conflict responses), never as silent corruption.

10.4 Schema Evolution: Migrating Topologies That Never Shut Down

Schema evolution is the classic objection to language-owned persistence, and it was a genuine weakness of the orthogonal systems: when the store is an extension of the program, editing the program threatens data the way editing a schema threatens a database, except without the migration tooling that decades of relational practice accreted. The objection lands harder still against a topology that never shuts down. A migration script presumes a maintenance window and a single authoritative schema version. A served graph under rolling deployment has neither, since old-version and new-version processes read the same store concurrently. Jac's persistence layer confronts this with a contract stated bluntly in its reference documentation: *edits never delete persisted data*. Around that contract sits a three-tier design: automatic tolerance for mechanical drift, declared rules where intent is required, and quarantine where neither suffices.

Detection comes first. Every archetype class carries a schema fingerprint (a hash of its module, name, and sorted field-and-type signature), and every persisted row is stamped with the fingerprint current at save time. On load, a matching fingerprint takes the fast path. A mismatch routes the row through drift handling. The programmer writes no versioning code and maintains no version integers: the fingerprint is derived, and the drift rules below are matched against the *shape* of the stored row, not against a declared version history, which keeps repair robust when development, staging, and production have each seen different intermediate schemas.

The automatic tier absorbs the mechanical majority of changes. A field added with a default is filled with the default. A changed type passes through a coercion table (strings parse to numbers, booleans, ISO timestamps, and UUIDs, with unions tried variant by variant), and a coercion that fails keeps the raw stored value and

loads the row anyway, preferring a wrong-typed value the program can inspect over data held hostage. A removed field's value is not dropped: it moves to the *attic*, a sub-document that rides along with the row across subsequent saves, so nothing a past version of the program persisted is destroyed by a present version's forgetting. Changes that carry intent, however, are not guessed at. The runtime deliberately refuses to infer that a removed field and an added field of the same type constitute a rename. The programmer declares it, and the declaration lives where the language's separation of declaration and implementation (Chapter 3) says history should live: in the implementation file, leaving the archetype's declaration showing only the present shape, as Listing 10.4 shows.

```
# app.jac (v1) -- as first deployed
node Person {
  has username: str = "",
  bio: str = "";
}

# app.jac (v2) -- the declaration shows only the present
node Person {
  has name: str = "";

  static def __jac_schema__ -> None;
}

# impl/app.impl.jac -- the ledger of the past
impl Person.__jac_schema__ -> None {
  schema_alias("name", stored="username"); # rename: old value flows in
  schema_drop("bio");                      # removal: value kept in the attic
}
```

Listing 10.4: A field rename and a field removal declared as schema history, with the declaration showing only the present.

Loading a v1 row under v2 code renames `username` to `name` in place, preserves the deleted `bio` in the attic, and touches quarantine not at all. Four declaration builders cover the intent space: a class rename (`schema_was`, or equivalently an `archetype_alias` decorator, stackable across a history of renames), a field rename (`schema_alias`), a field removal (`schema_drop`), and an arbitrary idempotent document transform (`schema_upgrade`) as the escape hatch for reshaping that no table can anticipate (splitting a comma-separated string into a list, say). The rules are validated against the live declarations when the class registers, so a contradictory declaration fails the application at startup, before it can corrupt rows mid-traffic. Rolling deployment is handled by dual-writing renamed fields: saves emit both the new and old names so old-version readers keep finding the field they know, and a conflict between the two (an old process wrote the old name against an already-upgraded row) resolves deterministically, the new name winning and the losing value preserved in the attic.

The third tier is the backstop. A row that genuinely cannot load (its class renamed with no alias registered, its payload corrupt, its reconstruction raising) is moved to a quarantine sidecar carrying the full original payload, the error, and a machine-readable reason, never silently deleted. The operator inspects and rescues through the

toolchain: list and show quarantined rows, add a rescue alias without a code deploy, re-attempt recovery, and scan for referential-integrity violations, all as subcommands of `jac db` (Chapter 4). On the scaled backend the loop closes further: after a deploy registers its classes and drift rules, the runtime automatically retries the quarantined rows that deploy plausibly fixed, so shipping the correcting alias is itself the migration.

The runtime does not automate everything, and the limits are documented and worth restating as design commitments, not admissions. Repair is lazy (applied on read, plus the startup retry), so a cold row keeps its old shape until something touches it. There is no background rewrite of the store. Coercion does not recurse into container element types. The upgrade callback covers what the table does not. Attic contents and dual-written shadows accumulate until an explicit cleanup phase that is, at this writing, future work, as is build-time linting of undeclared breaking changes against a schema lockfile. The division of labor is nonetheless coherent: the runtime automates what is mechanically decidable, requires declaration where a change encodes intent no machine can recover, and quarantines, loudly and losslessly, where even declaration fails. A topology that never shuts down cannot promise that evolution is free. It can promise that evolution never costs data, and that the price of intent is a two-line ledger entry in an implementation file rather than a maintenance window.

Bibliographic Notes

The database discontinuity this chapter dissolves was created, in its modern form, by Codd (1970), whose relational model made separation of program and data a virtue of the first order. The graph-data counterpoint, a topological data model that nonetheless kept the program at connection's length, is surveyed by Angles and Gutiérrez (2008) and embodied in the traversal and query languages of Rodríguez (2015) and Francis et al. (2018), which Appendix G reads as topokinesis trapped behind a wire protocol. The intellectual foundation of the chapter is the orthogonal-persistence program: Atkinson et al. (1983) is the founding artifact, and Atkinson and Morrison (1995) the definitive consolidation of its three principles and its reachability-based identification rule, of which Jac's root-reachability is a direct, deliberately restricted realization.

On consistency, Gray and Reuter (1993) remains the canonical treatment of the transaction abstraction that Jac draws at the walker boundary. The distributed limits marked in Section 10.3 rest on the impossibility and trade-off literature: Gilbert and Lynch (2002) for the proof, Brewer (2012) for the mature reading, and Vogels (2009) for the production vocabulary of eventual consistency. The CALM theorem of Hellerstein and Alvaro (2020) supplies the principled dividing line, between monotone and nonmonotone writes, along which the runtime's lock-free appends and conflict-replayed find-or-creates are split. The persistence semantics, concurrency behavior, and schema-evolution machinery described here are documented in the language's persistence reference, and their implementation is the subject of Chapter 13.

TRANSCIENCE BY OWNERSHIP: THE REGION IS THE SUBGRAPH

Everything must go somewhere. There is no “away” to which things can be thrown.

Barry Commoner, *The Closing Circle*, 1971

Chapter 10 gave the topology its rule for the direction of the future: whatever is reachable from the root persists. That rule governs the remembered world. Most topology a running program builds, however, is not meant to be remembered. The parse tree after the query is planned, the search frontier after the path is found, the candidate subgraph a request assembles, scores, and answers from: these are born together, meaningful together, and discarded together, and Chapter 10 called them by name when it drew its partition, the moving parts rather than the remembered world. What actually happens to the moving parts is the gap this chapter closes. On the managed substrate a discarded subgraph dies one object at a time, through a cycle collector whenever the subgraph is cyclic, and object-spatial subgraphs are routinely cyclic. On the native substrate the runtime makes a starker choice: topology is pinned immortal at construction, a deliberate design that treats the graph as persistent state rather than collectible data, so a discarded subgraph does not die slowly. It does not die at all. Measured on a request-shaped workload, the leak is 65 KB per request, without bound, 177 MB after two thousand requests. The paradigm that makes the transient subgraph *visible*, as the thing a walker builds and abandons, gave the memory manager no way to act on what it could see.

This chapter completes the pairing its title announces. Persistence attaches durability to place: a datum endures because of where it stands. Transience, we now argue, should attach to place in exactly the same way: a subgraph should be reclaimable as a unit because its members stand together, and the machinery that owns that unit should be an ordinary value of the language. We first locate the design

in the region tradition and state what the paradigm contributes to it (Section 11.1). We then present the construct: first-class region handles, the open, and the growth rule that lets traversal itself decide placement (Section 11.2). A second membrane closes the boundary between the region and the managed world (Section 11.3), and the same accounting that closes it yields constant-time transfer of whole subgraphs across concurrency boundaries (Section 11.4). The chapter ends with the measurements (Section 11.5).

11.1 The Unit of Death

In this section, we place the design among its ancestors. Memory managers reclaim death one object at a time. Reference counting frees each object as its count reaches zero and is defeated outright by cycles; tracing handles cycles but re-walks the live world to find the dead. Graph workloads violate the underlying assumption, because their data dies in cohorts: the whole scratch subgraph becomes garbage at one program point, together. Systems practice has answered with arenas and custom allocators for decades, effective and unsafe in equal measure (Berger et al. 2002), still measurably effective on modern hardware (van Kempen and Berger 2026), and high-performance graph systems allocate their frontiers this way as a matter of course (Shun and Blelloch 2013). The principled alternative is the region tradition: tie an allocation’s lifetime to something the compiler can see. Tofte and Talpin tied it to lexical scope and inferred effects (Tofte and Talpin 1997), and the MLKit retrospective records with unusual candor how lifetimes that follow scope rather than data forced programmers to restructure programs around the inference (Tofte et al. 2004). Cyclone tied it to type annotations threaded through every pointer (Grossman et al. 2002), and Gay and Aiken to explicit region values with counted references (Gay and Aiken 1998). What no system in this lineage possessed is a way to tie lifetime to the shape of the data itself.

Object-spatial programming supplies exactly that missing handle. Chapter 7 established the subgraph as the paradigm’s unit of meaning: the thing a program builds, traverses, and discards is not a scatter of objects but a connected piece of topology. The insight this chapter builds on is that the same subgraph is two units at once: 1) the program’s unit of meaning, and 2) the correct unit of reclamation, a cohort born together and dying together. When the language already expresses that a cluster of nodes and edges belongs together, the memory manager no longer needs to rediscover cohort death by counting or tracing. The region is the subgraph.

TOPOLOGY-ALIGNED REGIONS

Topology-aligned regions are reclamation as a property of place: a first-class, ownable handle owns an arena; nodes, edges, and objects constructed under its open allocate into it; allocations made by a walker’s abilities anchor to the region of the node being visited, so traversal grows the region it stands in; and the handle’s death reclaims the entire subgraph at once, destructors first, then one bulk free.

11.2 Handles, Opens, and the Growth Rule

The construct has three parts, and the first two will look familiar from Chapter 5, because they are built from its machinery. A Region is an ordinary value of the ownership surface: a binding `r: own Region = Region()` owns it, the handle drops at its liveness-placed death like any owned value, and its drop is the reclamation event. The statement `in r { ... }` *opens* the region, and every node, edge, and object constructed in the block’s dynamic extent allocates into `r`’s arena. Handles have dynamic extent rather than lexical scope: a helper may create a region and return the handle, a callee may grow it through a borrowed `&Region` parameter, and the eventual owner drops it, so the region boundary follows the subgraph rather than the block structure. On the native substrate the arena is a bump allocator over chained chunks; objects with drop abilities append to a destructor log that runs last-in, first-out at teardown, and then the chunks are returned in one motion. Listing 11.1 shows the shape this gives a request handler.

```
walker Score {
  has total: int = 0;
  can s with Stop entry {
    if not here.seen {
      here.seen = True;
      self.total += here.cost;
      visit [-->];
    }
  }
}

def handle(request: Query) -> int {
  r: own Region = Region();
  best = 0;
  in r {
    entry_stop = build_candidates(&r, request); # helper grows r
    best = (entry_stop spawn Score()).total;    # scalar out
  }
  return best; # r drops: destructor log, one bulk free
}
```

Listing 11.1: A request-shaped workload under a region. The subgraph is built and traversed inside the open; one scalar leaves; the handle’s death reclaims all two hundred nodes, their edges, and their satellite allocations at once.

The third part is the one no prior region system could express, and it is where topokinesis pays. A walker traversing a region-resident subgraph executes abilities, and those abilities allocate, so the design must decide where the allocations land. Threading a region parameter through every walker signature would make regions viral, and defaulting to the managed heap would scatter a logically region-resident subgraph across two worlds. The design instead states a *growth rule*: an allocation inside an ability anchors to the region of the node being visited. Anchored to a managed node, it allocates managed. Anchored to a region node, it allocates in that region. The same walker code grows a region when standing in one and

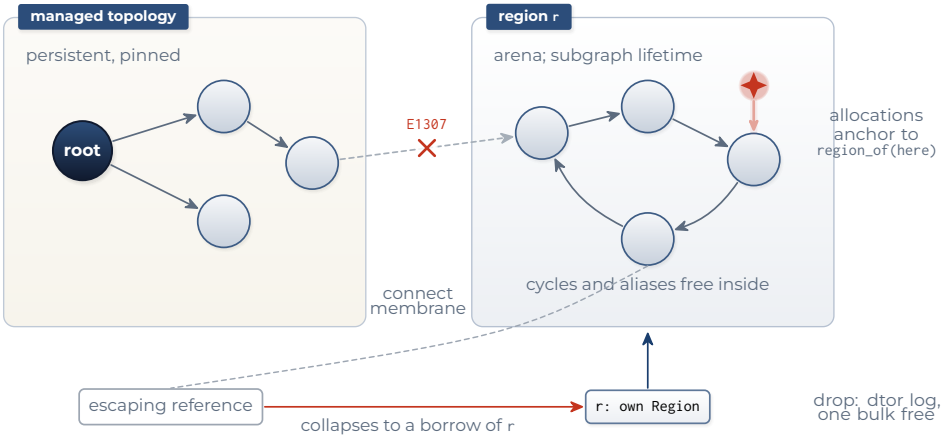


Figure 11.1: Topology-aligned regions, anatomized. The persistent graph stays on the managed side while the region’s arena holds one subgraph, free to alias and cycle internally. A walker standing in the region grows it, because ability allocations anchor to the region of the visited node. The connect membrane statically rejects the one edge that would wire the two extents together, and a reference that leaves the region collapses into a borrow of the handle, so the arena cannot be freed while it lives.

builds ordinary topology when standing on the persistent graph, with no annotation distinguishing the cases, because none is needed: in this paradigm the data’s location *is* the program counter. Asking where the computation stands is a constant-time runtime question, one header test and one load, and it costs one branch per ability-context allocation, a branch the evaluation prices at parity (Section 11.5). Note what the rule preserves inside the open: no ownership discipline at all. Aliasing is free and cycles are free, because reclamation is wholesale and needs no per-object account. The checker’s entire job moves to the boundary, which is the next section’s subject, and Figure 11.1 assembles the whole design in one view: the free interior, the growing walker, and the boundary about to be closed.

11.3 The Second Membrane

Chapter 5 introduced a membrane in the object-capability sense, a boundary that mediates every value passing between two memory regimes (Miller 2006). Regions require a membrane of their own, because a region system beside a managed heap has exactly one failure mode that matters: a reference that survives the bulk free. Everything in this section exists to make that reference unrepresentable in checked code, and the mechanisms will be recognizable, because they are the ownership surface’s machinery pointed at topology.

The first mechanism polices edges. Topology is the graph paradigm’s aliasing: an edge is a persistent, traversable reference, and all edges pass through the connect operators. The *connect membrane* is stated there: an edge may not wire a region-resident node to managed topology, in either direction, and the checker rejects the attempt statically (error[E1307]). Region-internal edges are unrestricted, preserving the free

interior. The rule is what protects traversal itself: a walker on the persistent graph can never step onto topology that might be bulk-freed mid-walk, because no edge leads there. The second mechanism polices ordinary dataflow, and it reuses Chapter 5 directly. A region-rooted value that flows to a binding that cannot outlive the handle is registered as a shared borrow *of the handle*, a move the checker calls *borrow-collapse*, and the existing borrow law takes over: the handle cannot drop, move, or be sent while borrows live. Flows that could outlive the handle are escapes and are rejected, with the analysis default-deny, because a bulk free converts any blind spot into a use-after-free. Scalars and strings copy out freely, and a function with exactly one `&Region` parameter may return region-rooted values, which collapse against that parameter at every call site: an annotation-free sliver of region polymorphism that covers the builder-helper idiom without Cyclone’s effect machinery. The third mechanism makes the two worlds representationally interchangeable. Arena objects carry the same header as managed objects, but the count field holds the sentinel the managed runtime already tests, so every retain and release on a region object is a taken-branch no-op, and managed code holds region references, as collapsed borrows, through completely ordinary code paths. The reader has met this sentinel before: it is the same mark the native runtime uses to pin persistent topology immortal, now marking topology whose death belongs to a handle instead.

Principle 11.1 (Extent soundness). No edge connects objects of different extents; every external reference into a region is either rejected or registered as a borrow of its handle; a handle with registered borrows can neither drop nor move nor send; and managed reference traffic neither extends nor truncates any region object’s lifetime.

The four clauses of principle 11.1 compose into the safety argument. When a handle drops, the third clause says no borrows are registered; the second then says no external references exist except through edges, which the first rules out; and the fourth says the managed runtime holds no lifetime stake. The bulk free therefore reclaims objects to which no checked path retains access. Each clause is enforced by a component the language already owns, the connect operators, the escape analysis, the borrow law, and the header layout, which is the sense in which the safety is by construction rather than by an added verification layer.

11.4 Sendable Subgraphs

Chapter 5 fixed the license for crossing a concurrency boundary: a value may cross only if it is statically race-free, an immutable value no one can write or an owned value moved so that no one else retains it. In this section, we extend that license from values to worlds. A handle with no registered borrows has, by principle 11.1, no live external references at all: the region is isolated in precisely the sense concurrent-ownership systems labor to establish. Isolation makes transfer trivial. Moving the own `Region` handle across a `flow/wait` boundary moves exclusive access to the entire live subgraph: a pointer hand-off, constant-time, no copying, no serialization, no synchronization, because the type system has already proved single ownership. Sending a handle with live borrows is rejected by the ordinary sendability rule, and sending an individual region-rooted value is an escape. The subgraph crosses as a unit or not at all.

The contrast with existing options for moving graph-shaped work measures the claim. Actor systems deep-copy or serialize, paying in the size of the subgraph on both sides of the boundary. Shared-heap tasking hands over reachability and lets the collector arbitrate ownership, admitting the races the type system cannot see, and ORCA makes the collector itself a distributed protocol (Clebsch et al. 2017). Verona reaches the same constant-time endpoint from the opposite direction, a clean-slate language whose reference-capability lattice is the whole type system and in which every allocation is region-placed by construction (Arvidsson et al. 2023), a lineage now being carried into Python itself (Stoldt et al. 2025). The design of this chapter reaches it opt-in: regions are gradual against the managed topology of Chapter 10, and their boundary reuses a borrow law the language already had, adding one rule rather than a capability stratum. One repair accompanied the feature and belongs in the record: walker instances, previously pinned immortal by the native runtime as an implementation convenience, became ordinary managed values with working drop abilities, so spawn-heavy services reclaim their walkers. On the native substrate, region residents are now the only discarded topology that is reclaimed at all, which is the asymmetry the chapter opened with, resolved.

11.5 The Measured Alignment

The claims above are quantitative in consequence, and the full protocol, apparatus, and tables live with the rest of the book’s measurements in Chapter 16 (Section 16.3.1). We report the headlines here in the discipline of that chapter: every number is a median of ten invocations, the baseline is the same source with the handle and open lines mechanically deleted, and no cell counts unless every variant of its configuration produces byte-identical output under the differential oracle, the same instrument Chapter 5 runs over the dial. On a request-shaped build-traverse-discard service, two hundred nodes per request over five hundred requests, regions run 5–6× faster than the erased baseline at 7.8× lower peak footprint with p99 tail latency 13–20× tighter, and fire all one hundred thousand drop hooks where the baseline fires none. The footprint gap compounds with service lifetime, 65 KB leaked per request against a plateau. A million-object subgraph changes hands across a concurrency boundary in 1.0 μ s where deep copy costs 789 ms, five orders of magnitude, and the gap widens with size. And the growth rule’s placement branch measures at 24 against 23 ns per allocation, so code that never opens a region pays nothing it can notice. One further result of the measurement campaign is recorded in Chapter 13: building the benchmark suite exposed the native topology store itself as quadratic and forced its redesign, the adjacency index with region-scoped retraction described there. The suite was not a garnish on the design. It was a design tool.

Part III closes with the topology’s two lifetime rules in hand, and Figure 11.2 draws them as what they are: one design read at two timescales. Reachability from the root decides what survives into the future, and ownership of the handle decides what is reclaimed from the present; durability and transience are both properties of *place*, decided by where a datum stands rather than by operations performed upon it. The machinery that discharges both rules on real substrates, the compiler that lowers the checks and the runtime that keeps the arenas, the anchors, and the stores, is Part IV’s

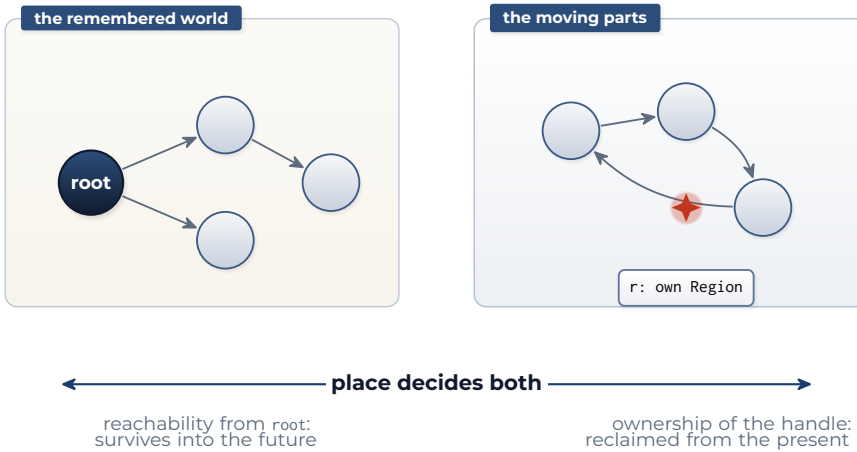


Figure 11.2: The topology’s two lifetime rules, one design at two timescales. Durability attaches to place by reachability from the root; transience attaches to place by ownership of a handle; and no operation in between maintains either. The figure mirrors Figure 10.1, whose rule it completes.

subject.

Bibliographic Notes

The region tradition this chapter extends begins with Tofte and Talpin (1997), whose retrospective (Tofte et al. 2004) is the field’s plainest account of what scope-following lifetimes cost in practice; Cyclone (Grossman et al. 2002) and the explicit regions of Gay and Aiken (1998) made regions safe at the price of annotation machinery this design’s growth rule and single-parameter collapse exist to avoid. The systems practice the design legitimizes, arena allocation as the default idiom of high-performance graph code, is documented by Berger et al. (2002), reappraised on modern hardware by van Kempen and Berger (2026), and embodied in frameworks such as Ligra (Shun and Blleloch 2013). On the concurrency side, Verona’s reference capabilities (Arvidsson et al. 2023) are the nearest relative, reaching constant-time transfer of isolated clusters from a whole-language capability discipline where this chapter reaches it gradually, from borrow accounting; Stoldt et al. (2025) carries that lineage into a dynamic language, and ORCA (Clebsch et al. 2017) represents the alternative of co-designing the collector with the actor discipline. Per-object reclamation at its best remains Perceus (Reinking et al. 2021) with the cycle collection of Bacon and Rajan (2001); regions are orthogonal to both, changing the unit of death rather than optimizing its count. The membrane construct is the object-capability boundary of Miller (2006), met first in Chapter 5 and reused here with topology as its cargo.

PART IV



THE MACHINERY

THE POLYPILER

Parts II and III argued that continuity and motion can be properties of a language itself, not achievements of an integration team. Part IV is the engineering record. It accounts for how one production compiler and runtime discharge those claims, and we begin with the compiler because it is where the synechic thesis becomes mechanical. The requirement is concrete: one surface syntax must enter three substrates as a first-class citizen. Somewhere, then, a single front end must feed three code generators from a single intermediate representation, and the partitioning of a program across tiers must be computed rather than authored. This chapter describes that machinery as it exists in the Jac source tree. It names the actual passes and data structures and keeps to the level of pipeline stages and responsibilities.

One disclosure shapes everything that follows: the compiler is self-hosted. The parser, the unified syntax tree, the analysis passes, the program driver, and the code generators are written in Jac. The compiler is therefore both the subject of this chapter and its largest exhibit: a program of roughly one hundred and ninety thousand lines of Jac, across more than three hundred source files, alongside roughly three and a half thousand residual lines of Python (a bootstrap seed and an import hook), a self-hosting fraction above ninety-eight percent, that exercises the language's declaration/implementation split, its typing machinery, and its Python interoperability on every build of itself. We return to what this certifies in Section 12.3. We note it now so that the reader recognizes the code excerpts below for what they are: not pseudocode illustrating the compiler, but the compiler.

12.1 Compilation, Transpilation, Polypilation

In this section, we name the class of artifact this chapter describes, because the classes the field already has do not fit it. A compiler's target is an instruction set: it lowers one source language to the code of one execution substrate, and its obligation ends when the instructions are correct. A transpiler's target is another source language at the same level of abstraction: it maps text to text and leaves the receiving ecosystem's toolchain

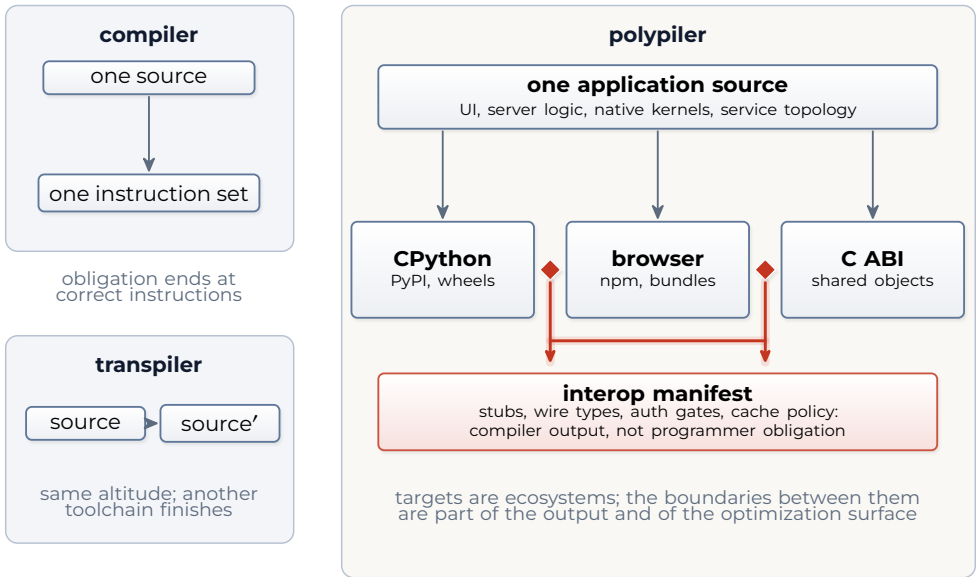


Figure 12.1: Three classes of compiler. A compiler lowers one source to one instruction set, and a transpiler maps source to source at the same altitude. A polypiler takes one application source, spanning every tier, as its unit of compilation, targets ecosystems rather than instruction sets, and reifies the boundaries between its targets into a manifest from which the boundary artifacts are generated and against which they are optimized.

to finish the job. The machinery of this chapter satisfies neither description. Its unit of compilation is the whole polyglot application, the interface, the server logic, the native kernels, and the service topology in one tree. Its targets are ecosystems rather than instruction sets: a lowering is not done when the bytecode or the JavaScript or the object code is correct, but when the artifact stands inside CPython, the browser, or the C ABI as a first-class citizen, importing that ecosystem’s libraries and publishing into its package formats. And its optimization surface includes the boundaries between those targets, because one compiler sees both sides of every crossing. We call a compiler of this class a *polypiler*, and its process *polypilation* (Figure 12.1).

POLYPILER

A *polypiler* is a compiler whose unit of compilation is the whole polyglot application, whose targets are ecosystems rather than instruction sets, and whose optimization surface includes the boundaries between them. *Polypilation* names the process. Etymology: Greek *polys*, many, formed on the pattern of *compiler* and *transpiler*.

The definition becomes operational as four requirements, and the reader will recognize each as a commitment argued earlier in the book. 1) One intermediate representation must carry the heterogeneous targets, or no analysis can see a whole

program: this chapter’s unified syntax tree. 2) The lowerings must be held to one observable semantics, or the shared representation is a fiction: the primitive-semantics discipline of Chapter 3, enforced here by differential testing across the three tails. 3) Ecosystem membership must run in both directions: the language consumes PyPI, npm, and C libraries without wrappers (Chapter 3) and publishes wheels, npm packages, and C-ABI shared objects (Chapter 4); a language that satisfies the first two requirements but not this one is an island, and adoption is structurally impeded. 4) The boundary itself must be reified: what crosses each boundary (signatures, types, effects, access levels) must be computed into a concrete compiler artifact that code generation and optimization consume, the interop manifest of Section 12.4.

The first two requirements are classic retargetable-compilation concerns pushed to an unusual breadth. The third is an economics claim, and it is where the historical record turned. The fourth is the novelty that carries the class, because it converts glue *generation* into glue *optimization*. Once the crossing is an object the compiler owns, the compiler can emit the stubs, wire types, and authentication gates a team otherwise writes by hand, and it can also analyze what crosses, prove what does not need to, and rewrite the program on both sides of a network hop at once. For example, the manifest records an effect signature for every endpoint, and from those signatures the client runtime derives a cache-invalidation policy that no one authored; Chapter 16 measures it. The walker-endpoint projection of Chapter 14 is the same fact spent in another direction.

The historical record splits these requirements into two camps that never met. The multitier languages (Links, Hop, Ur/Web, Eliom, ScalaLoci) demonstrated tier-splitting and RPC inference over two decades of research (Chlipala 2015; Cooper et al. 2006; Radanne et al. 2016; Serrano et al. 2006; Weisenburger et al. 2018), and none achieved durable adoption. The industrial systems nearest the idea were single-source-language designs since retired (the Google Web Toolkit, Microsoft’s Volta) or frameworks that place JavaScript on both tiers (Google 2006; Manolescu et al. 2008; Meteor Software 2012). We argue the missing requirement was never the splitting. It was ecosystem membership: a language that cannot import numpy and React, and cannot publish a wheel or an npm package, asks a team to leave its libraries and toolchains behind, a cost few teams accept. Conversely, the industrial multiplatform toolchains that did achieve membership (Kotlin Multiplatform, Dart, the GraalVM family) never attempted the split: no compiler in those stacks sees one program spanning the boundary (Google 2025; JetBrains 2025; Würthinger et al. 2013). At the time of writing, the membership camp is reaching for the split. React Server Components retrofit tier splitting onto JavaScript from the framework side: string directives (“use client”) mark the boundary, the bundler computes the partition, and the rule that only serializable values cross is enforced by convention and lint tooling rather than by a type system with jurisdiction over both sides (React Team 2020). An ecosystem with full membership is thus rebuilding the multitier capability piecewise, inside its toolchain, without a language to hold it, which we read as evidence for the diagnosis above. The polypiler is the conjunction of the two camps’ capabilities, and the four-tier application of Chapter 15 and the experiments of Chapter 16 are the evidence that the conjunction is realizable and pays.

The proposal contained in the name is a redefinition of *target*. A code generator’s

obligation ends not at correct instructions but at first-class ecosystem membership: imports, package formats, type declarations, and the crossings to every other target the program spans. Our expectation is that the polypiler becomes a recognized class of compiler, as the transpiler and the just-in-time compiler did, and that the interop manifest, or something like it, becomes as standard a compiler artifact as debug information: a machine-readable account of a program's distributed shape that tools, runtimes, and other compilers consume. The relation to Part II's theory is a division of labor between property and mechanism. Substrate transparency (Chapter 2) states what must be true of the surface; polypilation is how a compiler makes it true.

12.2 Pipeline Architecture

Two objects anchor the architecture. `JacProgram` (in `jac0core/program.jac`) is the unit of compilation state. It owns a program module whose *hub* maps every resolved source path to its parsed module. Alongside the hub it holds accumulated errors and warnings, the meaning-typed intermediate records, the lazily constructed type evaluator, and the caches that make recompilation incremental. `JacCompiler` (in `jac0core/compiler.jac`) is the orchestrator: it parses source into the hub, selects a schedule of passes appropriate to the request, and runs the schedule. Every pass is a subclass of a common `Transform` abstraction that maps a module to a module. `run_schedule` does nothing more elaborate than apply each transform in order, checking a cancellation token between passes so that interactive tooling can abandon stale work.

The pipeline's first stage is parsing. The front end is a hand-written recursive-descent parser (`jac0core/parser/parser.jac` and its implementation annex, together roughly seven thousand lines of Jac) over a dedicated lexer. The parser practices error recovery (a `synchronize` routine realigns at statement boundaries after a malformed construct), so that editors and language servers still receive a usable tree for ill-formed input. The front end accepts more than Jac: when `parse_str` is handed a `.py` or `.pyi` file, it parses it with CPython's own `ast` module and runs `PyastBuildPass` to lift the Python tree into the same node vocabulary that Jac source produces. Immediately after parsing, `JacAnnexPass` discovers and weaves in a module's annex files (the `.impl.jac` implementation bodies and `.test.jac` tests that the declaration/implementation split of Chapter 3 separates on disk), so that every later pass sees one complete module.

The product of the front end is the *unified AST*, called the `uni tree` in the sources. Its base node type `UniNode` is refined by mixins for scoping, symbol carrying, documentation, and control-flow participation. Its node set spans the full Python statement and expression space alongside the constructs Python lacks: archetypes, abilities, visit and disengage statements, typed context blocks, and the client, server, and native section markers whose role Section 12.4 takes up. One tree represents programs in both languages. The word "unified" is meant literally.

Analysis is organized as schedules of passes chosen by demand. The schedule the driver returns for full intermediate-representation generation appears in Listing 12.1, abridged from the fallback and lazy import scaffolding that surrounds it in `compiler.jac`.

```
def get_ir_gen_sched -> list[type[Transform[uni.Module, uni.Module]]] {  
    return [  

```

```

    ASTValidationPass,
    SymTabBuildPass,
    DeclImplMatchPass,
    SemanticAnalysisPass,
    SemDefMatchPass,
    CFGBuildPass,
    MTIRGenPass,
    JsxIntrinsicGuardPass,
    CodespacePullPass
  ];
}

```

Listing 12.1: The analysis schedule returned by the pipeline driver, abridged from the self-hosted module compiler.jac.

The early passes establish structure: `ASTValidationPass` checks tree invariants, `SymTabBuildPass` constructs the scope and symbol tables, `DeclImplMatchPass` binds declarations to the implementation bodies the annex pass wove in, and `SemanticAnalysisPass` enforces contextual legality. The later passes enrich: `SemDefMatchPass` attaches the semantic annotations that meaning types rest on, `CFGBuildPass` builds control-flow graphs, `MTIRGenPass` generates the meaning-typed intermediate records from which prompts are synthesized at runtime (Dantanarayana et al. 2025) (the compiler half of the mechanism described in Chapter 6), and `CodespacePullPass` propagates inferred codespace assignments along the import and reference graph, the analysis behind the marker-free placement of Section 3.2. Deeper analysis is engaged only when needed. The type evaluator is a demand-driven inference engine that reads vendored typeshed stubs for the Python standard library. It runs when the user requests checking, and the driver forces it whenever a module contains native or client code, since those lowerings require types. A further schedule backs the explicit `jac check` command: `StaticAnalysisPass`, `AccessCheckPass`, `OwnershipCheckPass` (Chapter 5), the lint passes, and `NativeCapabilityCheckPass`, which verifies that code destined for a restricted substrate uses only what that substrate provides.

Emission is the final schedule, and its ordering is the multi-target story in miniature: `EsastGenPass` (JavaScript), then `NaIRGenPass` and `NativeCompilePass` (native code via LLVM), then `PyastGenPass`, `PyJacAstLinkPass`, and `PyBytecodeGenPass` (Python bytecode). There are not three compilers but one pipeline with three tails (Figure 12.2), each an ordinary pass over the same tree. Each module carries a `CodeGenTarget` record (the gen slot) that accumulates every artifact the tails produce: the Python AST and marshaled bytecode, the JavaScript and extracted CSS, the LLVM IR and native objects, and the interop manifests that describe crossings between them. The stages are summarized in Table 12.1.

Two disciplines run through the whole pipeline. First, error gating: parse errors and blocking semantic errors suppress code generation entirely, and a module whose native compilation fails has its partial native artifacts scrubbed so no downstream consumer sees an inconsistent state. Second, caching: compilation products are serialized into a sectioned container format called JIR. Its numbered sections hold bytecode, meaning-typed IR, LLVM IR, the interop manifest, native objects, a symbol

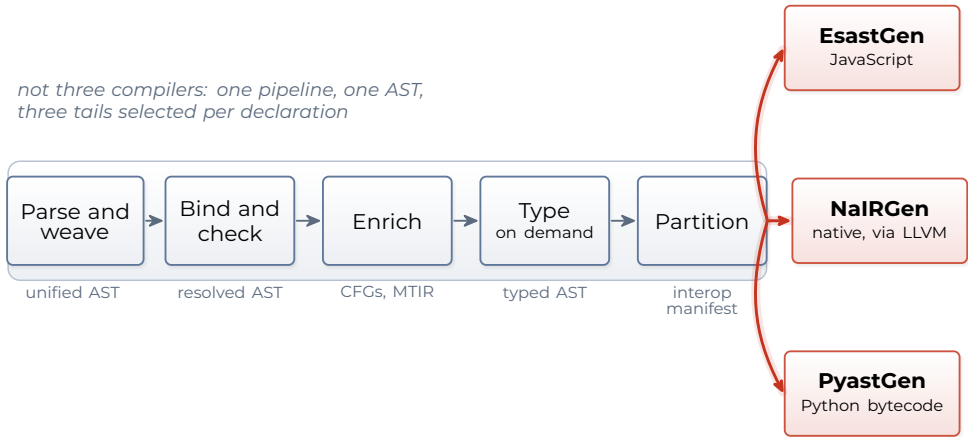


Figure 12.2: The compilation pipeline. One hand-written front end feeds the unified AST through binding, enrichment, on-demand typing, and codespace partitioning; only at emission does the pipeline fork into its three tails. Stage products are named beneath the trunk; Table 12.1 lists the principal passes.

Table 12.1: Stages of the compilation pipeline and their principal passes.

Stage	Representative passes	Product
Parse and weave	recursive-descent parser, JacAnnexPass	unified AST
Bind and check	SymTabBuildPass, DeclImplMatchPass	resolved AST
Enrich	CFGBuildPass, MTIRGenPass	CFGs, meaning-typed IR
Type (on demand)	TypeInferencePass, StaticAnalysisPass	typed AST
Partition	BoundaryAnalysisPass	interop manifest
Emit	EsastGenPass, NaIRGenPass, PyastGenPass	JS, native code, bytecode

index, and debug source. Its header hashes the source content, the compiler version, and the Python version, so that staleness is detected structurally. The same container is how the shipped binary carries the compiler itself, precompiled, inside the payload laid out in Chapter 4.

12.3 Supersetting Python at the Grammar and AST Level

In Chapter 3 we presented semantic containment of Python as a design strategy. In this section we describe its mechanical form. The containment is not source-level. Jac delimits blocks with braces and terminates statements with semicolons. It is grammatical and structural instead: every Python statement form, expression form, comprehension, decorator, and pattern-match construct has a counterpart production in Jac’s grammar and a counterpart node in the unitree, and the Jac-specific constructs extend that node set. The clearest witness is the code generator that lowers the unitree back to Python: its operator table (TOKEN_AST_MAP in PyastGenPass) is a near-bijection between Jac token kinds and nodes of CPython’s ast module. Where the mapping is the identity, Jac is Python. Where it is not, Jac has added something Python cannot say.

Containment also runs in the other direction, and this is the deeper property: Python source is admitted into the pipeline as a peer. As Section 12.2 noted, the front end lifts `.py` and `.pyi` files into the same unified AST via `PyastBuildPass`, and inline Python blocks embed in Jac modules as `PyInlineCode` nodes. Past the front end, the pipeline is language-blind: a Python module imported by a Jac program receives its symbol tables, its type evaluation, and its place in the module hub from the same passes that serve Jac modules. Supersetting is thus not an interoperation feature bolted to the side of the compiler. It is the absence of any point in the pipeline at which the two languages are distinguishable.

The self-hosting fact belongs in this section because it is the strongest form of the containment claim. The `jac0core` package (parser, unitree, passes, program driver) is Jac source, organized in the same interface-plus-annex idiom the language offers its users: `program.jac` declares the shape of `JacProgram`, and `impl/program.impl.jac` carries the bodies. The circularity is broken by `jac0.py`, a single-file bootstrap transpiler that reads the disciplined Jac subset used in `jac0core` and emits equivalent Python in memory at import time. Installed distributions never take that path, because the compiler ships precompiled to JIR inside the binary's payload. Self-hosting certifies several things at once. It certifies expressiveness: the language comfortably carries a hundred-and-ninety-thousand-line system comprising a recursive-descent parser, a demand-driven type evaluator, and three code generators. It certifies the declaration/implementation split at scale: the compiler's interface files read as specifications, and the excerpts in this chapter are shipped source. And it applies a permanent selection pressure: every language regression breaks the compiler's own build before it reaches a user, because the first large program any change must survive is the compiler.

Note, finally, that self-hosting names a frontier rather than a fraction. The ninety-eight percent measures the compiler alone. The artifact that carries the compiler still includes external engines in binary form (the CPython interpreter, the Bun runtime, LLVM itself), and Chapter 4 identifies these as the current position of the self-hosting frontier: components retained for feature completeness, with replacement in Jac as the stated design goal. The compiler is the first engine to have crossed, and it demonstrates the two properties every later crossing inherits: the replacement falls under the language's own verifier, and it adds itself to the selection pressure described above, since a language change must now survive it before reaching a user. The linkers have crossed as well (Section 12.7). The memory discipline the remaining crossings require is the subject of Chapter 5.

12.4 Program Partitioning: Deciding What Runs Where

A synechic program spans substrates, so some authority must decide which code runs on which one. In Jac that authority is a per-declaration placement the compiler computes, from the programmer's explicit markers where they appear and from inference where they do not. Everything downstream of the placement (the crossing, the marshaling, the endpoints) is computed by the compiler. The language recognizes three *codespaces*: server, the Python tier; client, the browser tier; and native, the machine-code tier. Explicit placement takes two lightweight forms. A whole file is placed

by suffix, under a longest-suffix rule maintained in a single registry (`ext_registry`): `.cl.jac` marks a client module, `.sv.jac` a server module, and `.na.jac` a native module. Within any file, `cl`, `sv`, and `na` blocks place individual sections, which the parser records as `ClientBlock`, `ServerBlock`, and `NativeBlock` nodes. Markerless code is placed by inference. JSX expressions and `npm` imports seed the client codespace, C-ABI foreign declarations seed the native one, and archetypes, served endpoints, and Python-ecosystem imports pin a module to the server, with `CodespacePullPass` propagating the seeds along the import and reference graph. A markerless module that none of these constraints claims compiles native by default, and a module whose inferred native lowering fails is recompiled for the server, so the preference is total: it never rejects a program (a `jac.toml` knob, `default_codespace`, restores the server default per project). The front end normalizes every form into one. Placement is stamped as a `CodeContext` onto each top-level element, so that by the first analysis pass it is a per-declaration attribute of the tree, and each module's decided codespace is recorded in its compiled image. During emission, the module code generator selects statements by context, which is how a single module contributes to several artifacts at once. Figure 12.3 shows the placement machinery at work over six top-level declarations: the seeds carry their seeding construct, the arc is `CodespacePullPass` pulling a markerless helper to the client along the reference that names it, and the dashed arrow is the server fallback that keeps the native preference total.

The littleX application of Chapter 15 shows the authoring surface. Its frontend is a client module that imports its server (the walkers and view types of `social_graph.jac`) with an `sv import`, alongside ordinary `npm` packages (Listing 12.2).

```
import from "@jac/runtime" { jacLogin, jacIsLoggedIn }
import from "lucide-react" { Feather, Sun, Moon, Pencil }
sv import from .social_graph {
  setup_profile,
  follow_user,
  create_tweet,
  load_feed
}
sv import from .social_graph { Tweet, ProfileBundle }
```

Listing 12.2: Client-side imports in the littleX frontend, abridged from `frontend.cl.jac`. The `sv`-qualified imports name server functions and the boundary types they return.

Client code calls `load_feed` as an ordinary asynchronous function. Nothing at the call site betrays that the callee runs in another process on another machine. The knowledge of what crosses lives in the compiler's `BoundaryAnalysisPass`, which walks the program after analysis and computes an *interop manifest* for each module. This is the fourth requirement of Section 12.1 discharged: the boundary is no longer a convention held between two codebases but an object the compiler owns. For every callable invoked from a codespace other than its own, the pass records an `InteropBinding`: the callable's name, source context, the set of calling contexts, the route between them, and the typed signature. For every archetype whose values cross a boundary (the `Tweet` node and the `ProfileBundle` report type of the listing) it records a `BoundaryTypeInfo` naming the fields, so both sides of the wire agree on shape. It also records access levels, so a server function not marked `public` never becomes reachable

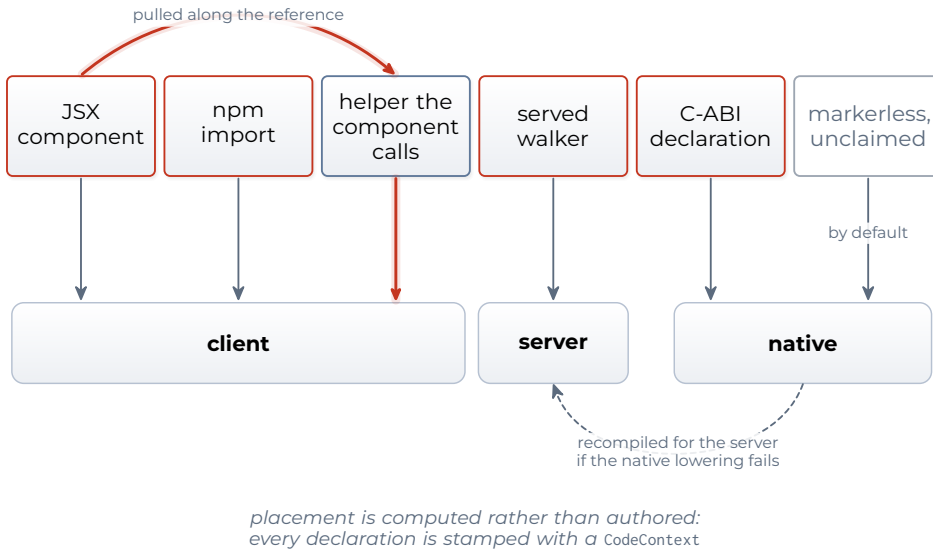


Figure 12.3: Codespace placement over six top-level declarations. The shaded declarations are seeds, named by their seeding construct: JSX and npm imports seed the client, a served walker pins the server, and a C-ABI declaration seeds the native codespace. `CodespacePullPass` pulls the markerless helper into the client codespace along the reference that names it (the arc above the row). A declaration that no constraint claims compiles native by default, and the dashed arrow records why the preference is total: a module whose native lowering fails is recompiled for the server, so inference never rejects a program. Explicit placement, whether a file suffix or a `cl`, `sv`, or `na` block, normalizes into the same per-declaration stamp.

from the client. The driver cooperates by chasing import edges into client modules before emission, ensuring manifests exist for every module a crossing touches.

The emitters then synthesize what conventional stacks make programmers write. On the client side, `EsastGenPass` lowers a call to an `sv`-imported function into an asynchronous stub that invokes the runtime’s `__jacCallFunction`, awaits the typed result, and rehydrates boundary types from their wire form via generated `__from_wire` constructors. On the server side, the same manifest drives the projection of those functions and walkers as endpoints when the program is served (Chapter 14). Native crossings are mediated by the same manifest through C-level marshaling rather than HTTP. The boundary between tiers is real, and the compiler does not pretend otherwise. What distinguishes the synechic design is that the scaffolding at that boundary is compiler output rather than programmer input, which is the substrate-transparency property of Chapter 2 stated operationally. The deeper consequence is the jurisdiction proposition of Chapter 2 made machinery: because stub, endpoint, and wire shape all derive from one recorded declaration, caller and callee are checked against the same typed signature by the one compiler that sees both sides. Representation still changes at the wire, but it changes within the checker’s jurisdiction. The crossing, which in the conventional stack is a discontinuity by both clauses of Chapter 1’s definition, here

fails to be one by either.

12.5 Lowering to Python Bytecode

The default lowering targets the managed substrate: CPython bytecode, in the lineage of virtual-machine code descending from p-code through the JVM (Lindholm and Yellin 1999). The lowering is deliberately thin. `PyastGenPass` constructs standard ast trees, mapping Python-congruent constructs almost one-to-one and lowering Jac-specific constructs (archetype declarations, abilities, visit statements, the spatial operators) into calls against the runtime library that Chapter 13 describes. `PyJacAstLinkPass` then back-links every generated Python node to the Jac node it came from. Diagnostics, debuggers, and runtime tracebacks can therefore speak in terms of Jac source positions. Finally `PyBytecodeGenPass` hands each module's tree to CPython's own compiler, marshals the resulting code object into the module's gen record, and the JIR cache persists it so an unchanged module never recompiles.

Loading is the inverse path: a meta-path importer registered with CPython resolves `.jac` modules, retrieves bytecode from the program or its caches, and executes it as a normal Python module. The consequence is the first-class citizenship claimed in Chapter 3: a Jac archetype is a Python class at runtime, a Jac module is a Python module, and a PyPI package imports into Jac source with no binding layer because there is nothing to bind. Both sides are already the same substrate. The interpreter in question is the private CPython carried inside the launcher's payload (Chapter 4), so this lowering is also the one on which the compiler runs itself.

12.6 Lowering to JavaScript and WebAssembly

The client tail begins with `EsastGenPass`, which lowers the unitree into an ESTree-shaped JavaScript syntax tree (defined in `estree.jac`) and finishes with an unparser that prints it as modern JavaScript. JSX is native to Jac's grammar, so a component is simply a function returning a `JsxElement`. The pass analyzes each component's has fields and its dependency-listed entry abilities to recover reactive intent (state and effects), and maps them onto a pluggable framework backend. React, Preact, and Solid backends are registered, and the selection is a compilation parameter, not an authoring dialect: the littleX frontend of Listing 12.2 contains no framework-specific code. A scoped-styles mechanism extracts per-component CSS into the module's gen record alongside the JavaScript.

Two further pieces keep the client substrate semantically continuous with the others. A generated runtime prelude (`jac_runtime_js`) supplies Python-congruent behavior for the built-in types (string, list, and dictionary methods, integer semantics, iteration protocols), so that a function moved between codespaces does not change meaning. The type evaluator also extends across the ecosystem boundary: just as it reads vendored typeshed stubs to type PyPI imports, it locates and parses TypeScript declaration files to type npm imports, so the `lucide-react` import in the listing is checked with the same machinery as an import of a Python module.

WebAssembly (Haas et al. 2017) enters as the client's native floor. Native-codespace code appearing in a web project is lowered through the same LLVM path described

next, retargeted to wasm32: the build merges vendored wasm32 libc bitcode into the module, the compiler's own wasm linker produces the final binary, and a host ABI layer defines the boundary across which JavaScript and the wasm instance exchange values. No external toolchain (no Emscripten, no system linker) participates. A browser-run, natively compiled program is an ordinary build product of the same pipeline.

12.7 Native Lowering through LLVM and Static Linkage

The third tail leaves managed substrates entirely. NaIRGenPass lowers the typed unitree to LLVM IR (Lattner and Adve 2004). Because machine code cannot defer typing to a runtime, the driver forces the type evaluator's inference over any module containing native code before this pass runs. By default the generated IR carries an automatic reference-counting object model. A backing cycle collector is emitted alongside it, and its sweep stays off until a runtime switch turns it on, so default native execution is plain counting. Each heap object bears a header with an allocation identifier, a type tag, the reference count, and a destructor slot. The IR also carries a Python-congruent subset of the standard library, from math, time, os, and random through json, datetime, hashlib, and TLS sockets, backed by native implementations, so the native codespace remains the same language rather than a C-like dialect wearing Jac syntax. Memory strategy, however, is an emit-time property, not a semantic one. The same pass can be directed to emit counting without the collector, or, for modules whose bindings carry the ownership discipline of Chapter 5, no memory-management runtime at all, and program meaning is identical across the modes. Notably, the object-spatial constructs survive the descent: when native code references nodes, edges, or walkers, the code generator links in a native OSP kernel, so topological semantics are not a privilege of the managed substrate.

The gradient of emission modes is the machinery behind the continuity claim of Chapter 5. There the managed-versus-systems divide was dissolved as semantics: the ownership dial, the membrane, and the guarantees that make incremental adoption trustworthy. Here the dial acquires its emit-time mechanics. Unannotated code is fully managed, and even for it the core analysis proves elisions of counting traffic that would be wasteful to emit. A module may mark some bindings and not others. A region reclaims a whole extent at once while the collector stands behind everything around it. The emission modes then relax the runtime in steps until a fully covered module carries none at all. Every step keeps the same expression and the same meaning under one verifier's jurisdiction, and every step is reversible. The type system of Chapter 9 made static assurance adoptable by degrees, and the ownership surface does the same for memory discipline. The pairing is deliberate.

The strongest of those modes is made checkable, not aspirational, by the ownership surface of Chapter 5. Borrows erase entirely at code generation. A *region* opened with `in handle { }` is a collective allocation extent, reclaimed wholesale at the handle's death: on this substrate, a bump-allocated arena torn down with one walk of its destructor log and a single free. The architectural point is where the analysis lives. A core pass verifies the discipline once, over the same unified tree every backend consumes, and stamps its conclusions as facts alongside the types. The native backend

spends those facts, never re-deriving them. A module held to the full discipline compiles headerless: allocation is a bare `malloc` at construction, and release is a direct destructor call placed at last use. The build can *assert structurally* that no counting or collector machinery survived into the artifact, turning “no garbage collector in the binary” from a slogan into a checked property. The same facts do lawful-boundary duty at the thread crossing (Chapter 2). A value crosses the parallel constructs only as a deeply immutable value, as a moved owner, or as a scalar, so a data race is a checked error at the point of the crossing, and a moved region handle carries its whole population to another thread without a copy. The narrowings the regime imposes are likewise declared where they originate, not discovered in production. An unhandled raise in an enforced module aborts, since unwinding is a service of the managed runtime the module has renounced. Region opens are not yet accepted inside such modules, an engineering gap, not a designed limit.

`NativeCompilePass` then drives code generation through LLVM 22, statically linked into the toolchain as a single library behind a stable binding ABI (the “`jacllvm`” component of the binary anatomy), so no system compiler is consulted. For inline `na` blocks inside a managed application, the pass compiles and loads the result in-process and wires the crossings recorded in the interop manifest. For standalone `.na.jac` programs, it produces objects for a self-contained executable.

The final step is the one conventional toolchains outsource. The compiler contains its own linkers, one per object format: `ElfLinker`, `MachOLinker`, and `PeLinker`, alongside the `Wasmlinker` of the previous section. Each can emit executables and shared objects directly, including fully static executables against the vendored `libc` and startup archives, with a `musl` runtime carried for Linux. No `ld`, no `lld`, no installed C toolchain of any kind stands between `Jac` source and a zero-dependency binary, which is what allows the toolchain chapter (Chapter 4) to claim the linker as an internalized tier. Interoperation with the C ecosystem is symmetric: native code imports C libraries directly, and public symbols can be exported as a C-ABI shared library, making `Jac` a producer as well as a consumer of the native world’s lingua franca.

Stepping back, the three tails discharge the “one syntax, three lowerings” commitment of Chapter 3 in a specific architectural sense: each lowering is an ordinary pass over the same unified AST, consuming the same symbol tables, the same types, and the same interop manifests. Nothing in the architecture is specific to the number three. A fourth substrate would be a fourth tail (a code generator and a linker backend), not a fourth compiler, and that observation, developed further in Chapter 18, is why we regard the pipeline as evidence for a language class, not a single artifact.

12.8 The Design Space of Polypilation

The machinery of this chapter is one point in a design space, and this section maps the space, because the class defined in Section 12.1 is larger than its first member. A conventional stack runs at least four compilation engines whose pipelines and intermediate representations never meet: the host language’s compiler, the client toolchain’s transpiler, the bundler, and the native compiler. Unifying them into one pipeline forces a set of design decisions that single-target compilers never face. We

enumerate six, state the decision this compiler made at each, and note the alternatives a successor could defensibly take.

1) *Node-set design*. A shared representation chooses among the union of the substrates' construct spaces, their intersection, or a small core with per-target extensions. The unitree takes the union: every Python statement and expression form has a node, the Jac-specific constructs extend the set, and Python source is lifted into the same tree as a peer (Section 12.3). The union costs node count and buys language-blindness in every later pass. An intersection design costs expressiveness at the surface. A core-plus-extensions design costs every pass a dispatch on which extension it is looking at.

2) *Fact placement*. Every analysis runs either once over the shared tree or once per code generator. The rule here is that core passes stamp facts (symbols, types, control flow, ownership conclusions, placement) and the three tails spend them without re-derivation (Section 12.7). The alternative, per-tail re-analysis, is what the conventional stack does by construction: the type checker, the bundler, and the minifier each re-parse and re-resolve the same program, and no two of them agree on what they saw.

3) *Typing discipline per target*. The substrates disagree about when types must exist: bytecode defers to a runtime, machine code cannot. The resolution here is demand-driven inference, forced whenever a module contains native or client code (Section 12.2), with one evaluator reading typeshed stubs and TypeScript declaration files so that both foreign ecosystems are typed by the same machinery. The alternative, a mandatory whole-program type phase, simplifies the pipeline and forfeits the gradual adoption path of Chapter 9.

4) *The semantic contract*. A shared representation is a fiction unless the lowerings are held to one observable semantics. The contract here is the primitive-semantics interface of Chapter 3 (the fixed meanings of the built-in types, operators, and builtins), implemented by a runtime prelude on the JavaScript tail and a native standard-library subset on the LLVM tail, and enforced by differential testing that runs the same programs through all three tails and asserts identical output. Note that the differential suite does for the semantic contract what the interop manifest does for the boundary: it converts a convention into a checked artifact. Whether such a conformance suite can standardize across implementations, as debug-information formats did, is recorded among the open problems of Chapter 18.

5) *Boundary reification*. The crossings between targets are either conventions held between artifacts or objects the compiler owns. This compiler computes an interop manifest per module (Section 12.4): typed bindings, boundary type shapes, access levels, and routes. The manifest is what converts glue generation into glue optimization, and it is the artifact we expect to outlive the implementation.

6) *The delivery unit*. A compiler that owns the whole program graph decides what the unit of delivery is, and the bundler dissolves into that decision. Each duty a bundler performs is a compiler duty under another name: tree-shaking is dead-code elimination over the module graph, code-splitting is the partitioning of Section 12.4, the chunk graph is the module hub, and the hydration boundary is a calling convention the manifest records. Bundlers exist as separate engines because the unit of compilation (the module) diverged from the unit of delivery (the chunk) and no

Table 12.2: The design space of polypilation: six decisions every polypiler must make, the assignment this compiler chose, and the principal alternatives a successor could defensibly take.

Decision	This compiler	Alternatives
Node-set design	union of the substrates' construct spaces (the unitree)	intersection; small core with per-target extensions
Fact placement	core passes stamp facts once; the three tails spend them	per-tail re-analysis, as the conventional stack does
Typing per target	demand-driven inference, forced by native or client code	mandatory whole-program type phase
Semantic contract	primitive-semantics interface, enforced by differential testing	per-tail conventions, drift discovered downstream
Boundary reification	interop manifest computed per module	conventions suspended between artifacts
Delivery unit	the compiler owns the chunk graph; the bundler dissolves	external bundler engines with authored configuration

compiler saw both. Note that this dissolves the authored artifact, not the mechanism: bundling still executes inside the toolchain, with no bundler configuration authored in the project, and the delivery policies a bundler encodes (chunk granularity against cache behavior) move into the toolchain with it.

Table 12.2 collects the six decisions and their roads not taken in one place. The partitioning decision also generalizes beyond this chapter. The toolchain makes the same decision at four granularities: the codespace places a declaration on a substrate, the client and server markers place code on a tier, an `sv import` places a module in a service of its own (Chapter 14), and the project kind places the whole program under a delivery treatment (Chapter 4). All four are computed from the same program graph, with explicit markers overriding inference at each granularity, so the deployment topology of an application is an output of compilation rather than an input to it.

These six decisions define the space, and the three-tails architecture of this chapter is one assignment of answers. Chapter 18 treats the assignments a successor could make differently, and its open problems record the two questions the space leaves open: the minimal fact set a shared representation must carry, and whether the representation and its conformance suite can standardize across polypilers.

Bibliographic Notes

The native tail rests on the compilation framework of Lattner and Adve (2004), whose intermediate representation and target abstractions made library-linkable code generation practical. The WebAssembly target follows the design laid out by Haas et al. (2017). The managed lowering sits in the long lineage of portable virtual-machine code, from Pascal-P's p-code (Nori et al. 1981) to the JVM specification of Lindholm and Yellin (1999). Readers comparing architectures should set Jac's arrangement

against Truffle and GraalVM (Würthinger et al. 2013), which invert it: many languages carried on one substrate, where Jac carries one language onto many substrates.

Program partitioning has its scholarly roots in the multitier literature: Links (Cooper et al. 2006) compiled a single program into server and client halves, and Weisenburger et al. (2020) surveys the placement and splitting strategies that family developed. Jac’s boundary analysis generalizes that tradition from the two tiers of a web application to the three codespaces described here, with the native tier and the ecosystem-typing machinery (typeshed stubs and TypeScript declarations read by one type evaluator) extending past the web’s edge. The meaning-typed records generated in the analysis schedule are the compile-time half of the mechanism introduced by Dantanarayana et al. (2025) and developed in Chapter 6. The compiler sources themselves are the primary reference for this chapter: the self-hosted core under `jaclang/jac0core` and the analysis and emission passes under `jaclang/compiler` in the artifact described in Appendix E.

THE RUNTIME CORE

The compiler of Chapter 12 ends its work holding artifacts: Python bytecode, JavaScript, native object code. Nothing in those artifacts yet honors the promises the language makes at its surface. Persistence by reachability (Chapter 10) is a promise about what survives process exit. Walker traversal (Chapters 7 and 8) is a promise about how computation moves at run time. Scale invariance (Chapter 14) is a promise that none of this changes when the deployment does. Static translation cannot discharge promises about dynamic behavior. The runtime core is the machinery that does, and this chapter is its engineering record.

Two design commitments organize everything that follows. The first is narrowness: the runtime presents itself to compiled code as a single facade of operations, and every object-spatial construct in a Jac program (every `spawn`, `visit`, `edge connection`, and `report`) lowers to a call on that facade. Because the contract between program and substrate is this thin, the substrate behind it can be substituted. A set of mediation hooks lets a distributed deployment replace the in-process defaults for context creation, storage, serving, and inter-service transport without the program observing any difference. The second commitment is self-hosting: the runtime core is written in Jac. The directory `jaclang/jac0core` declares the runtime interface in `runtime.jac` and implements it in `impl/runtime.impl.jac`, using the language's own declaration and implementation split (Chapter 3). The traversal kernel, graph memory, and persistence engine examined below are all Jac source. Self-hosting is not an ornament. It forces the machinery that provides the semantic medium to live inside that medium, and, as we will see with the traversal kernel, it lets a single source file serve as the engine on all three substrates.

This chapter proceeds in four parts, matching the runtime's architecture: the facade and its hook points (Section 13.1), the in-memory representation of topology (Section 13.2), the engine that moves walkers through it (Section 13.3), and the engine that makes the results durable (Section 13.4).

13.1 The Runtime as Substrate Mediator

The runtime’s public face is a single class, `JacRuntimeInterface`, composed by inheritance from roughly fifteen capability groups: `JacContext` (execution contexts and threading), `JacPersistence` (commit, save, destroy, object resolution), `JacAccessValidation` (permission checks), `JacGraph` (edge queries, connection, disconnection, path resolution), `JacWalker` (spawn, visit, disengage), `JacArchetype` (archetype construction and ability registration), `JacModule` (importing and reloading Jac modules), `JacIntrospection`, `JacLLM` (the meaning-typed machinery of Chapter 6, mediated by the same facade (Dantanarayana et al. 2025)), `JacServe` (API serving and client bundles), and a handful of smaller groups for testing, builtins, and configuration. A subclass, `JacRuntime`, adds the process-wide state: the compiler and program handles, a thread pool, the default execution context, and the table of loaded modules.

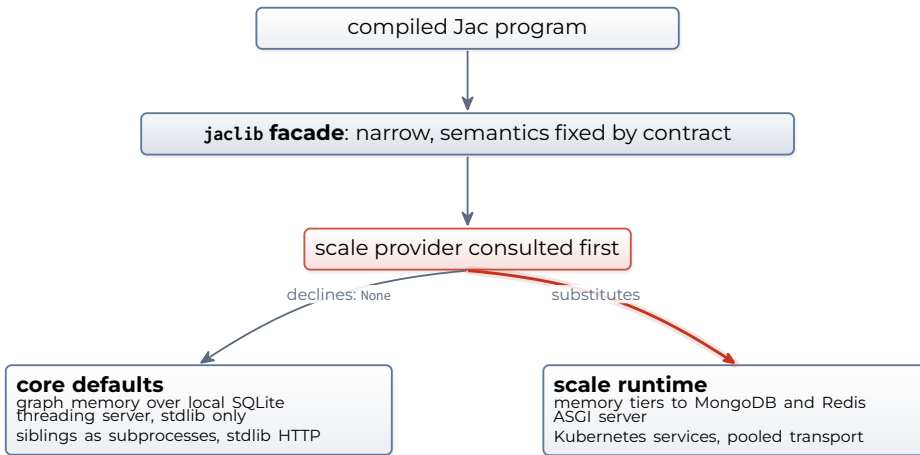
This facade is the entire application binary interface between compiled Jac and its substrate. The Python code generator emits calls against a small binding module, `jaclib`, whose exported names (`connect`, `spawn`, `visit`, `root`, `refs`, `destroy`, and their siblings) are direct aliases of facade statics. When a program writes `a ++> b`, the emitted code calls `connect`. When it spawns a walker onto a node, the emitted code calls `spawn`. Nothing else crosses the boundary. The narrowness is deliberate: an interface small enough to enumerate is an interface small enough to substitute.

Substitution is arranged through mediation hooks, not a general event bus. The pattern is uniform and worth stating in full, because the correctness of scale invariance rests on it. The core ships with a built-in scale plugin. A lazily evaluated, cached lookup imports `JacScalePlugin` from `jaclang.scale` if present. Every facade operation whose mechanism varies with deployment consults the provider first. The provider either returns a substitute or *declines* by returning `None`, in which case the core default runs. Listing 13.1 shows the pattern verbatim for API server selection.

```
impl JacServe.get_api_server_class -> type {
  _sp = _scale_provider();
  if _sp is not None {
    _scaled = _sp.get_api_server_class();
    if _scaled is not None {
      return _scaled;
    }
  }
  import from jaclang.runtimelib.server { JacAPIServer }
  return JacAPIServer;
}
```

Listing 13.1: The decline-to-default mediation pattern, taken from the runtime implementation. The scale provider is consulted first and may decline, in which case the zero-dependency core default is used.

The mediated operations span the runtime’s contact points with the outside world. Execution-context creation defaults to an in-process context whose graph memory sits over a local SQLite file. The scale plugin substitutes a context whose memory tiers extend to MongoDB and Redis. The API server defaults to a threading server built entirely on the Python standard library. The plugin substitutes an ASGI server when



a hook substitutes mechanism, never semantics

Figure 13.1: The mediation architecture. Compiled programs speak only to the narrow facade; the always-registered scale provider is consulted first and declines outside a genuine scale runtime, so `jac run` keeps the core defaults byte for byte. Either column discharges the same contract: what an operation means never moves.

its dependencies are installed. User management, blob storage, the service registry, and service-to-service calls follow the same shape: the core resolves sibling services as local subprocesses reached over stdlib HTTP, while the plugin resolves them as Kubernetes services reached through a pooled transport.

Because the scale plugin is always registered, the decline contract carries semantic weight: each hook returns `None` unless the process is genuinely a scale runtime (an environment marker set by the serving entry point, or a configured distributed backend), so a plain `jac run` keeps the core defaults byte for byte. The governing invariant is that a hook may substitute *mechanism*, never *semantics* (Figure 13.1). What a facade operation means is fixed by its contract. The hooks choose only how that meaning is discharged on the machine at hand. This is the runtime half of substrate transparency (Chapter 2), and it is what entitles Chapter 14 to claim that deployment-scale transformation preserves program meaning.

Beyond the scale provider, the facade exposes a small set of registration points for the same purpose at finer grain. `set_persistent_memory_provider` lets a plugin supply the durable storage tier that graph memory will use (Section 13.4). `set_shared_root_resolver` lets a deployment designate a graph root shared across users. `on_commit` registers effects deferred until a unit of work commits. A subclass hook on the archetype base class lets the serializer automatically register every user-defined archetype for durable encoding. The historical contrast is instructive: the functions that middleware systems placed outside the language, in brokers and

object services the programmer configured by hand (Bernstein 1996), appear here as operations of the language’s own runtime, hookable but semantically fixed. The architecture descends from the Jaseci runtime stack, where an early form of this mediation was called the diffuse execution environment (Mars et al. 2023).

The same mechanism-not-semantics discipline governs value memory. The language’s ownership surface (Chapter 5) fixes, as semantics, the points at which a value dies. An owned value’s death is placed by liveness rather than block structure. A region open reclaims its whole population when its handle dies (Chapter 11). An archetype’s drop ability runs exactly once at that death. How death is discharged is substrate mechanism. On the managed substrate, reclamation remains the host collector’s affair: the runtime instruments region-allocated archetypes so their drop hooks fire at the region’s close, while the memory itself stays garbage-collected. On the native substrate the same open is a bump-allocated arena freed in one motion. Under the enforced ownership regime the destructor call is a statically placed instruction with no collector behind it at all. One contract, three discharges, and a program observes the same drops at the same points on each. That is the transparency invariant of Chapter 2 applied at the granularity of a single allocation.

13.2 Graph Memory

Every archetype instance in a running program leads a double life. Its data plane is an ordinary object of the host substrate: in the managed lowering, a Python dataclass with the fields the programmer declared. Its control plane is an *anchor*: the runtime’s ledger entry for the instance, created lazily the first time the runtime touches the object and cached on it thereafter. The anchor family is small enough to reproduce (Listing 13.2).

```
obj Anchor {
    has archetype: Archetype,
    id: UUID = field(default_factory=uuid4),
    root: (UUID | None) = None,
    access: Permission = field(default_factory=Permission),
    persistent: bool = False,
    hash: int = 0;
}

obj NodeAnchor(Anchor) {
    has archetype: NodeArchetype,
    edges: list[EdgeAnchor],
    topology_index_data: (bytes | None) = None,
    version: int = 0;
}

obj EdgeAnchor(Anchor) {
    has archetype: EdgeArchetype,
    source: NodeAnchor,
    target: NodeAnchor,
    is_undirected: bool;
```

```
}

```

Listing 13.2: The anchor family from the runtime core, condensed to its state. Anchors are the runtime’s ledger entries for archetype instances. The archetype itself remains an ordinary host object.

The base anchor carries identity (a UUID), ownership (the id of the user root that owns the object), an access-control record, a persistence flag, and a content hash recording the last durably written state. Node anchors add the incidence list (edges are first-class anchors with source and target, so the topology is stored symmetrically on both endpoints) plus a version counter for optimistic concurrency and an optional encoded topology index, both of which earn their keep below. Walker anchors record traversal state: the path walked, the pending destinations, and the ignore list.

Identity deserves emphasis because everything distributed in Chapter 14 leans on it. The UUID is the sole identity of a graph object. Anchor equality and hashing are defined over it and nothing else. Programs meet it through the ambient builtin `jid`, which returns the hex form of an object’s anchor id, and its inverse `jobj`, which resolves an id back to an object through graph memory, subject to a read-access check. The identifier encodes no location: no host, no shard, no file. The same `jid` resolves against a laptop’s SQLite store and against a replicated document store, which is what allows topology to move between them without programs noticing.

Identity also enables laziness. When an anchor is deserialized, the anchors it references arrive as *stubs*: id-only shells whose payload has not been fetched. Attribute access on a stub triggers population from graph memory, and a central resolution routine mediates the awkward cases: a stub that cannot be populated is quarantined and pruned, so a broken reference degrades to absence instead of crashing a traversal. Laziness in turn makes locality a first-order concern, and the runtime addresses it in three layers.

The first layer is tiered caching. Each execution context owns a `TieredMemory`: an L1 identity map (a dictionary from id to anchor, guaranteeing one live anchor object per id per context), an optional L2 cache, and an L3 durable store. Reads promote upward through the tiers (Listing 13.3). Writes land in L1 and are reconciled with L3 only at commit, as Section 13.4 details.

```
impl TieredMemory.get(id: UUID) -> (Anchor | None) {
  if (anchor := self.__mem__.get(id)) {
    return anchor;
  }
  if self.l2 and (anchor := self.l2.get(id)) {
    self.__mem__[anchor.id] = anchor;
    return anchor;
  }
  if self.l3 and (anchor := self.l3.get(id)) {
    self.__mem__[anchor.id] = anchor;
    if self.l2 {
      self.l2.put(anchor);
    }
    return anchor;
  }
}
```

```

    return None;
}

```

Listing 13.3: The tiered read path of graph memory. A miss falls through the identity map and cache to the durable store, and the result is promoted on the way back up.

The second layer is traversal-aware prefetching. Before an edge query runs, the runtime sweeps the query’s origin frontier, collects every unpopulated edge stub, and fetches them in one batched round trip. A second batch fetches the unpopulated node stubs at the far ends of those edges. Pointer-chasing over a remote store is the classic pathology of graph workloads: one round trip per hop per object. Batching the frontier bounds it to two round trips per traversal step regardless of fanout.

The third layer caches the *shape* of the graph separately from its state. A root’s node anchor may carry a compact topology index: a type table, a node table, and adjacency triples with per-type buckets, encoded to bytes and maintained incrementally by callbacks on edge creation, node save, and destruction. The index answers typed multi-hop reachability questions (resolve the ids satisfying a chain such as “outgoing edges of type *F* to nodes of type *P*, then onward to nodes of type *Q*”) entirely over identifiers, so the query planner can decide what a traversal needs before materializing a single archetype. Topology, being touched by every traversal, is worth indexing even when state is too large or too cold to cache.

One further bookkeeping habit of graph memory threads through to persistence: whenever a traversal reads a node’s incidence list, the execution context records that node’s current version in a read-set. Nothing is done with the record at read time. It becomes the basis of conflict detection at commit.

13.3 The Walker Execution Engine

The engine that moves walkers is deliberately the smallest component of the runtime: a single self-contained source file, `osp_kernel.jac`, with no imports and two data structures (Listing 13.4). Everything substrate-specific is factored out through a four-function record, and that factoring, not any single algorithmic idea, is the kernel’s design.

```

obj WalkScope {
  has wlk: any,
  path: list,
  next: list,
  ignores: list,
  reports: list,
  disengaged: bool = False,
  next_head: int = 0;
}

obj OspRuntime {
  has tag_of: Callable[[any], int],
  is_a: Callable[[int, int], bool],
  node_desc: Callable[[int], NodeDesc],
  on_complete: Callable[[WalkScope, bool], None];
}

```

```
| }
```

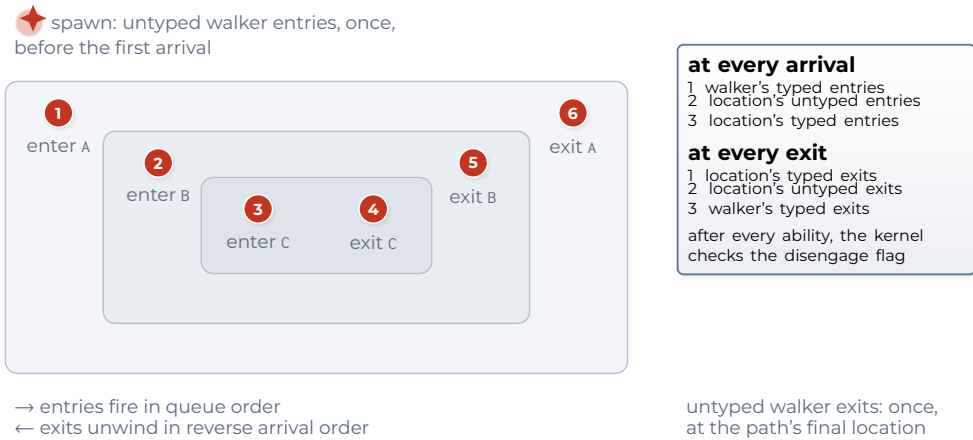
Listing 13.4: The kernel’s two data structures. A walk scope is the per-walker execution state. The runtime record is the kernel’s only window onto its substrate.

A WalkScope is the live state of one walker run: the walker itself, the path of locations already entered, the pending visit queue (next with its cursor `next_head`), the ignore list, accumulated reports, and the disengage flag. Spawning a walker seeds the queue with the spawn target (spawning onto an edge enqueues the edge and then its target node, so edge abilities fire before the arrival at the node) and pushes a fresh scope onto a walk stack. Scheduling is then queue discipline: locations are dequeued from the head, which the kernel implements as an advancing cursor rather than a shifting list, and `visit` appends new destinations at the tail (or splices at a caller-supplied insertion index when a program wants breadth-first or priority behavior). Members of the ignore list are filtered at both insertion and dequeue, with the filter skipped entirely in the common case of an empty ignore list.

Ability dispatch at each arrival proceeds in three passes, in an order that gives both parties of the encounter their say. The first pass runs the walker’s typed entry abilities whose trigger matches the location’s type, and trigger matching is subtype-aware. The second pass runs the location’s untyped entry abilities, which fire for any visitor. The third pass runs the location’s typed entry abilities matching the walker’s type. Symmetrically on exit, the location’s typed exits run, then its untyped exits, then the walker’s typed exits. Two untyped walker events bracket the whole run: the walker’s untyped entry abilities fire once at spawn, before the first arrival, and its untyped exit abilities fire once at the final location of the path. After every individual ability the kernel consults the disengage flag, so disengage halts not just the traversal but the remainder of the dispatch chain at the current location.

The interleaving of entries and exits has a structure worth naming. The kernel processes the queue with an explicit phase-machine stack rather than recursion: when a location’s entry passes complete, the kernel drains the queue *within* that location’s pending exit, so each later arrival nests inside the exits of every earlier one. Entry abilities therefore fire in queue order, while exit abilities unwind in reverse arrival order once the queue is empty: a call-stack discipline laid over the graph. Figure 13.2 draws this schedule for a three-location walk, with the per-arrival dispatch order beside it. Note that the numbered entries descend left to right in queue order, and each later arrival sits nested inside the pending exit of every earlier one. Given a fixed graph state, the whole schedule is deterministic: the queue is ordered, incidence lists preserve insertion order, and ability tables preserve declaration order. The formal small-step account of this schedule is the subject of Chapter 8. The kernel is that semantics made executable.

The ability tables themselves are prepared outside the kernel. When an archetype class is constructed, the runtime collects its declared abilities into per-class entry and exit lists. The managed-substrate binding then flattens these into slot tables keyed by integer type tags, with `tag_of` mapping any object to its tag and `is_a` implementing subtype triggering via the host’s class hierarchy. The binding’s `on_complete` copies the scope’s path into the walker’s anchor, attaches accumulated reports to the walker (folding them into the enclosing scope when one walker spawns another, or streaming



a call-stack discipline laid over the graph

Figure 13.2: The kernel's schedule for a walk that visits A, then B, then C. Entries (steps 1 to 3) fire in queue order, and each later arrival nests inside the pending exits of every earlier one, so exits (steps 4 to 6) unwind in reverse arrival order once the queue is empty. The right panel gives the dispatch order at every individual location; because the disengage flag is checked after every ability, disengage halts the remainder of the chain wherever it fires. Given a fixed graph state, the entire schedule is deterministic.

them to the request's report queue in a server context), and, for a successful top-level run, commits graph memory.

That last clause fixes the runtime's unit of work: *the walker run is the transactional quantum*. Everything a walker writes between spawn and completion is reconciled with durable storage as one unit by the persistence engine of Section 13.4.

The kernel's portability claim is not rhetorical. The native compilation pipeline copies `osp_kernel.jac` into a native-codespace unit, compiles it through LLVM, and links it into the output binary, with type tags assigned at compile time by a stable string hash computed during the compiler's OSP analysis pass. The web pipeline copies the same file into a client-codespace unit and compiles it to JavaScript. The managed substrate imports it directly. One traversal algorithm, one source file, three substrates: the execution engine is itself an exhibit of the substrate transparency it serves (Chapter 2). Concurrency is handled at the binding, not in the kernel. In the server binding the walk stack is context-local, so concurrent requests on threads or async tasks execute walkers in isolation, and an async variant of the kernel awaits coroutine-valued abilities for walkers declared asynchronous. The scheduling model refines the abstract machine sketched in the paradigm's formal definition (Mars 2025).

On the native substrate the kernel is joined by a topology store of its own, since no anchor-bearing host object exists there. Edges live in a per-node adjacency index: hash-indexed row chains over flat integer arrays, giving constant-time connection and degree-proportional edge queries (Figure 13.3). The store earns a mention in the engineering record because its first form did not have those bounds. The original

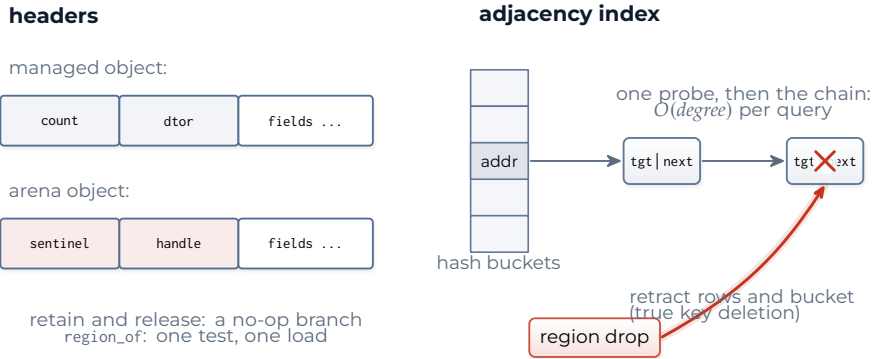


Figure 13.3: The native topology store. On the left, arena objects carry the managed header shape with the count field holding the sentinel the fast paths already test and the dead destructor slot holding the owning handle, which is what makes asking where an object lives a constant-time question. On the right, edges live in per-node row chains reached through one hash probe, and a region’s drop retracts every row and bucket the dead subgraph owned before the arena is freed.

implementation was a global edge table scanned linearly per query, and the region benchmark suite of Chapter 11 exposed it at full size: per-visit cost grew with every edge ever created, and rows outlived the regions that owned them. The redesign makes the store’s death discipline match the language’s. When a region drops, the runtime retracts every row and bucket the dead subgraph owned before the arena’s bulk free, with true key deletion rather than value resets, so a recycled address can never resurrect dead topology and the index’s footprint follows the live graph rather than the graph’s history. The episode is this chapter’s clearest instance of a lesson the evaluation chapters formalize: the differential suite is not merely a validator of the machinery but a design instrument for it.

13.4 The Persistence Engine

The durable tier behind graph memory is organized as a small hierarchy of store contracts: an abstract memory interface (get, put, delete, query, commit, and their async forms), a volatile in-memory implementation, a cache contract, and a persistent contract that adds changeset application, quarantine management, aliasing, and integrity checking. The default persistent store is an embedded SQLite database kept under the project’s `.jac` directory, beside the sources. The storage-provider hook of Section 13.1 substitutes a distributed backend when one is configured, and tiered memory asks the hook before falling back to SQLite. The programming model above the store does not change with the substitution: the same mediation invariant as everywhere else.

What lands in the store is decided topologically, as Chapter 10 developed semantically: marking an object persistent records its owning root and cascades along incident edges, and creating an edge whose either endpoint is persistent persists the edge (and, transitively, the other endpoint). Application code contains no save

calls on the hot path. The commit at walker completion drives everything. This is orthogonal persistence in the sense of the persistent-programming tradition (Atkinson et al. 1983; Atkinson and Morrison 1995), with reachability from a root standing in for the type-based or declaration-based criteria of that lineage.

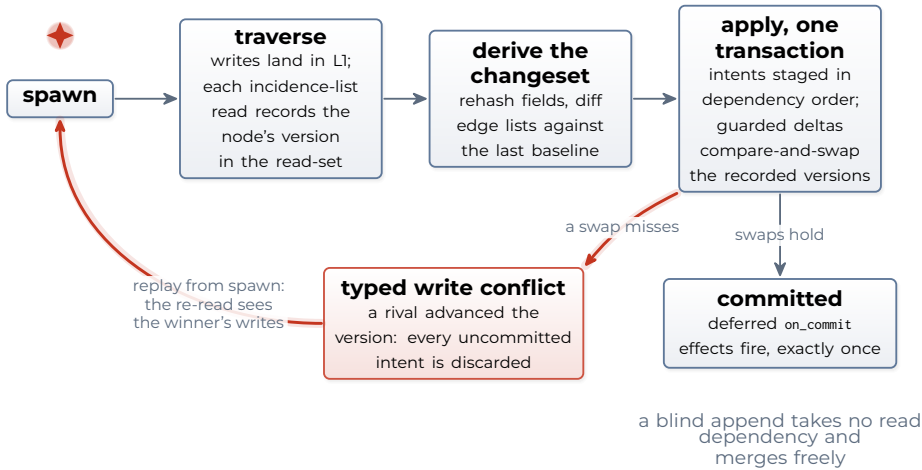
Serialization is schema-aware in both directions. Every archetype class is registered with the serializer automatically at class construction, and carries a fingerprint (a truncated SHA-256 over its module, name, and field names and types) that is stamped onto every row at save time. On load, a fingerprint match takes the fast path. A mismatch triggers best-effort repair: added fields take their defaults, changed types are coerced where a lawful coercion exists, and declared drift rules and class aliases handle renames. A row that cannot be loaded at all is quarantined in a sidecar table, never dropped, and can be inspected and recovered with the toolchain’s database commands. The engine’s contract is blunt: schema edits never silently delete persisted data.

Change tracking is derivational rather than interceptive. There is no write-through layer recording assignments. Instead, each anchor’s fields are hashed at load and after each commit, and at the next commit the engine recomputes the hashes to derive the dirty field set, while incidence-list changes are derived as deltas against the edge ids recorded at the last baseline. The derived changes populate a `ChangeSet` of write intents classified by a six-way operation enum (Listing 13.5), each intent carrying its dependencies: an edge creation depends on its endpoints, a deletion on its incident objects.

```
enum WriteOp {  
    NODE_CREATE = 0,  
    EDGE_CREATE = 1,  
    EDGE_LIST_DELTA = 2,  
    FIELD_UPDATE = 3,  
    EDGE_DELETE = 4,  
    NODE_DELETE = 5  
}
```

Listing 13.5: The write operations of the persistence engine. A commit stages intents in this order, so that created objects exist before the links that cite them.

Application is transactional in the classical sense of a unit of work (Gray and Reuter 1993). On the SQLite path, the entire staged changeset executes inside a single immediate transaction. Intents whose dependencies failed are skipped, never half-applied, and any write conflict rolls the whole transaction back, so a losing unit of work leaves the store byte-for-byte unchanged. Conflicts are detected optimistically at node granularity. Recall that graph memory records a node’s version whenever a traversal reads its incidence list. At commit, an edge-list delta on a node in the request’s read-set is applied with a compare-and-swap against that recorded version. A concurrent committer will have advanced the version, the swap misses, and the engine raises a typed write conflict. The server’s default policy is convergence: abort the loser, discard its uncommitted work, and replay the walker from its spawn. The replay re-reads the graph (now containing the winner’s writes), and the canonical check-then-create pattern takes its find branch, so two racing creations converge on one object without application-level locking. Writes that took no read dependency



the walker run is the transactional quantum

Figure 13.4: The persistence engine’s unit of work. A walker’s writes stay in graph memory until completion; the commit derives what changed by rehashing, stages intents in dependency order, and applies them in one transaction, with every guarded edge-list delta compare-and-swapped against the version recorded when the walker read that node. A losing walker is not an error: its work is discarded and the run replays from spawn, where the re-read finds the winner’s writes and the canonical check-then-create takes its find branch. Deferred effects fire only for the attempt that commits.

(blind appends to a node the walker never queried) carry no version guard and merge freely, so only guarded patterns pay the replay cost. External side effects are kept exactly-once under replay by the `on_commit` deferral: registered callbacks run only after a successful commit and are discarded on abort, so they fire once, for the attempt that wins. Figure 13.4 traces the unit of work end to end. The path to follow is the losing one: a compare-and-swap misses, the engine discards every uncommitted intent, and the replay arrow returns the walker to its spawn, where the re-read finds the winner’s writes.

Access control threads through the same machinery. Permissions live on anchors (a general level plus per-root grants) and are evaluated during traversal reads, connection, destruction, and commit collection, so an object owned by one user root is invisible to another’s walkers unless granted, and a write intent is recorded only when the acting root holds the required level. Multi-user isolation is thereby a property of graph memory itself, not of an application layer. Chapter 14 builds root multiplicity on this foundation. The engine as a whole (reachability-driven writes, derivational change tracking, staged transactional application, optimistic convergence) is the matured form of the persistence design first deployed in the Jaseci runtime stack (Mars et al. 2023).

Bibliographic Notes

The runtime core descends from the Jaseci runtime stack, whose account by Mars et al. (2023) describes the first production form of the ideas recorded here: a runtime that owns persistence, multi-user isolation, and scale-out as language machinery rather than application scaffolding. The formal definition of Object-Spatial Programming by Mars (2025) specifies the traversal and ability-dispatch semantics that the kernel of this chapter implements. Chapter 8 restates that semantics as a small-step calculus.

The persistence engine sits in a long lineage. Orthogonal persistence, the doctrine that an object's longevity should be independent of its type and of explicit input/output code, was articulated for PS-algol by Atkinson et al. (1983) and surveyed comprehensively by Atkinson and Morrison (1995). Jac's contribution is to make the persistence criterion topological, a matter of reachability from a root. The unit-of-work discipline, staged application, and optimistic concurrency of Section 13.4 are applications of the transaction-processing canon of Gray and Reuter (1993). Finally, the mediation architecture of Section 13.1 is best read against the middleware tradition surveyed by Bernstein (1996): the broker services that tradition placed between programs and their environment appear here inside the language runtime, as hookable operations with fixed semantics. That is the difference, in this book's terms, between gluing substrates together and mediating them from within.

SCALE INVARIANCE

The preceding two chapters described how a Jac program is compiled and how its topology lives in memory and in storage. This chapter takes up the claim that those mechanisms exist to serve: that a program written for one user at one terminal is, without textual change, the same program that serves a population of users across a fleet of machines. That sameness is continuity carried into the dimension of deployment scale. In the conventional stack this transformation is a rewrite. The script that proved the idea is abandoned for a service. That service acquires a web framework, an authentication layer, a tenancy discriminator threaded through every query, a serialization schema, a sharding scheme, and a directory of deployment manifests, each one a stratum of the glue catalogued in Chapter 1. The working hypothesis of this chapter is that none of these strata encode application meaning. They encode the *shape of the deployment*, and the shape of the deployment is a runtime concern, as garbage collection is a runtime concern.

We proceed from definition to machinery to limits. Section 14.1 makes the invariance claim precise. Sections 14.2 to 14.4 show how the runtime absorbs, in turn, the script-to-service boundary, the one-user-to-many boundary, and the one-machine-to-many boundary, grounding each in littleX, the social application that serves as this book’s running artifact. Section 14.5 then states plainly what the abstraction does not and cannot hide, connecting back to the lawful-boundaries principle of Chapter 2, and Section 14.6 closes with the absorption of the deployment layer itself.

14.1 Definition: Invariance under Deployment-Scale Transformation

The abstract of this book promises a runtime that “presents a single system image across processes, machines, and users.” The phrase *single system image* is borrowed deliberately from the cluster operating-systems literature, where it names the illusion that a collection of machines is one machine (Buyya et al. 2001). The OS-layer efforts of that tradition, surveyed in Appendix F, unified processes and file systems beneath

an unmodified system-call interface. Jac relocates the same ambition to the language layer: the unit over which the illusion is maintained is not a process table but a program’s persistent topology, and the interface held invariant is not read and write but the semantics of walkers, nodes, and edges. We give the property its canonical statement.

SCALE INVARIANCE

A language exhibits *scale invariance* when program semantics are invariant under deployment-scale transformation: one user to N users, one machine to M machines, transient to persistent.

The statement compresses three independent axes, and it is worth making the compressed term precise.

Definition 14.1 (Deployment-scale transformation). A *deployment configuration* of a program P is a triple $\delta = (u, m, \ell)$, where $u \geq 1$ is the number of principals whose interactions P serves, $m \geq 1$ is the number of machines across which the runtime executes P , and $\ell \in \{\text{transient}, \text{persistent}\}$ states whether the topology reachable from the program’s roots outlives the process that created it. A *deployment-scale transformation* is any change of configuration $\delta \rightarrow \delta'$. A language is scale invariant when, for every program and every such transformation, (i) the program text is unchanged, and (ii) each principal’s interaction with the program—the walkers spawned and the report values received—is observationally equivalent across the two configurations, up to the lawful observables of Chapter 2: latency, availability under partition, and the interleaving of concurrent principals.

Two features of this definition deserve emphasis. First, the invariance is *per principal*. We do not claim that the global execution of a thousand-user deployment is equivalent to any single-user execution. That claim would be false the moment two users’ walkers interleave. We claim that each user’s view through their own interactions is the view a single user of a single-machine transient run would have obtained, given the same reachable data. Second, the invariance is semantic, not operational. A configuration with more machines answers with different latencies and fails in different ways than a configuration with one. Clause (ii) explicitly excludes these observables from the equivalence, and Section 14.5 is devoted to why excluding them is rigor, not evasion.

The third axis of the definition, transient to persistent, was the subject of Chapter 10: reachability from root is the persistence criterion, and no program text distinguishes a run whose graph dies with the process from a run whose graph survives it. This chapter completes the other two axes, as summarized in Table 14.1.

14.2 From Script to Service: Walkers as Projected Interfaces

Consider the feed walker of littleX, reproduced in Listing 14.1 from the repository source. It is an ordinary object-spatial program in the sense of Chapter 7: it enters at the caller’s root, walks to the caller’s profile, fans out over the caller’s own tweets and

Table 14.1: The three axes of deployment-scale transformation and the machinery that absorbs each.

Axis	Transformation	Absorbing machinery
Users	one to N	per-user roots (Section 14.3)
Machines	one to M	distribution runtime (Section 14.4)
Lifetime	transient to persistent	persistence by reachability (Chapter 10)

those of followed profiles, accumulates typed views, and reports the collection once on exit.

```
walker load_feed {
  has search_query: str = "",
    feed: list[Tweet] = [],
    reports: list[list[Tweet]] = [];

  can run with Root entry {
    mine = [-->[?:Profile]];
    if mine {
      me = mine[0];
      visit [me-->[?:Tweet]];
      visit [me->:Follow:->[?:Profile]-->[?:Tweet]];
    }
  }

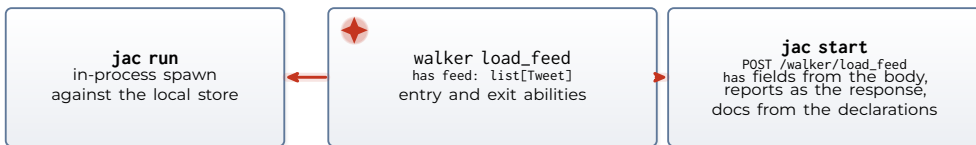
  can gather with Tweet entry {
    self.feed.append(here);
  }

  can deliver with Root exit {
    report self.feed;
  }
}
```

Listing 14.1: The littleX feed walker, abridged from the repository source. The declared report channel types the walker’s response contract.

Nothing in this text is about HTTP. Yet the declaration is already a complete interface description: the `has` fields name and type the parameters a caller may supply, and the `reports` channel names and types what the caller receives. The runtime exploits this completeness. Under `jac run`, spawning the walker is an in-process operation against the local store. Under `jac start`, the same program becomes a server, and every walker is *projected* as an API endpoint: `load_feed` is served at `POST /walker/load_feed`, its `has` fields drawn from the request body, its reported values returned as the response, with interface documentation generated from the same declarations. The transformation from script to service is a change of invocation, not of source (Figure 14.1).

```
jac run --entry load_feed social_graph.jac # script: in-process spawn
jac start main.jac                       # service: walkers become endpoints
```



script to service is a change of invocation, not of source

Figure 14.1: The walker as projected interface. One declaration executes in-process under `jac run` and is served as an authenticated endpoint under `jac start`, its request, response, and documentation all derived from the same fields and abilities. There is no parallel artifact to keep faithful.

```
curl -X POST http://localhost:8000/walker/load_feed \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{"search_query": ""}'
```

Listing 14.2: One program, two configurations. The walker executes in-process as a script and is projected as an endpoint as a service.

A foreign client may speak this protocol directly, as the `curl` invocation in Listing 14.2 shows. But the more consequential projection is the one the compiler performs for the program's own web tier. The littleX browser code spawns the walker with the same expression the script uses:

```
impl app.loadData -> None {
  feedResult = root spawn load_feed(search_query="");
  tweets = feedResult.reports[0] if feedResult.reports else [];
}
```

Listing 14.3: The littleX client loading the feed. The `spawn` expression in browser code is compiled to an HTTP invocation of the projected endpoint.

On the server substrate this expression is a local `spawn`. On the browser substrate the compiler lowers it to a call against `/walker/load_feed`, serializing arguments outbound and rehydrating the typed `Tweet` nodes inbound, each still carrying the identity examined in Chapter 10. The developer writes neither route, nor controller, nor serializer, nor client stub. The count of hand-maintained interface artifacts between littleX's frontend and backend is zero. The projection is also secure by default: an unannotated walker requires an authenticated caller, and only the explicit `:pub` modifier exposes an endpoint anonymously, a default that matters because projection is automatic.

It is instructive to compare this with remote procedure call, the mechanism that first promised invocation transparency (Birrell and Nelson 1984). RPC made a remote call look like a local one at the call site, but the price was a parallel artifact: an interface definition maintained beside the program, from which stubs were generated and against which versions drifted. Here there is no parallel artifact, because the walker declaration *is* the interface. The projection is derived from the same text that

defines local behavior. What is projected is not a procedure but a mobile computation: the endpoint does not name code to execute at a fixed site, it names a walker to release at the caller’s root. The consequences of that inversion unfold in the next two sections. The Jaseci runtime that preceded the present system demonstrated this walker-as-endpoint projection in production (Mars et al. 2023). The current runtime inherits and generalizes it.

14.3 User Multiplicity as Root Multiplicity

The second axis of Definition 14.1 is the population axis: one user to N . In the conventional stack this axis is the most invasive of the three, because multi-tenancy is implemented *inside* application code: a user identifier column on every table, a filter clause on every query, a middleware layer whose omission on any one code path is a data leak. The tenancy discriminator is glue in the strict sense of Chapter 1 (text that encodes deployment shape, not application meaning), and it is uniquely dangerous glue, because its failures are silent and its audit surface is every line that touches data.

Jac absorbs the axis with a single semantic decision: *root* is not a global constant but an indexical. Every principal registered with a served deployment receives a root node of their own, and every authenticated request binds *root* to the root of the caller. The littleX walker in Listing 14.4 illustrates what follows from this. The traversal `[->[?:Profile]]` starting at *root* denotes “the caller’s profile”: for every caller, whichever caller it is.

```
walker create_tweet {
  has content: str;

  can run with Root entry {
    visit [->[?:Profile]];
  }

  can make with Profile entry {
    new = here +>:Post():> Tweet(
      content=self.content, author_username=here.username,
      created_at=_now()
    );
    grant(new, level=WritePerm);
    report new;
  }
}
```

Listing 14.4: The littleX tweet creation walker from the repository source. Root-relative traversal makes the same text mean the calling user’s data for every user.

Isolation, under this decision, is not enforced by tenancy code. It is the default geometry (Figure 14.2). Distinct roots anchor disjoint reachable subgraphs, and a walker spawned at one user’s root cannot wander into another user’s data because no edge leads there. The dangerous property of the conventional stack (that every query is one missing filter away from a leak) inverts: in Jac, every traversal is scoped by construction, and it is *sharing* that requires explicit text. The grant call in Listing 14.4

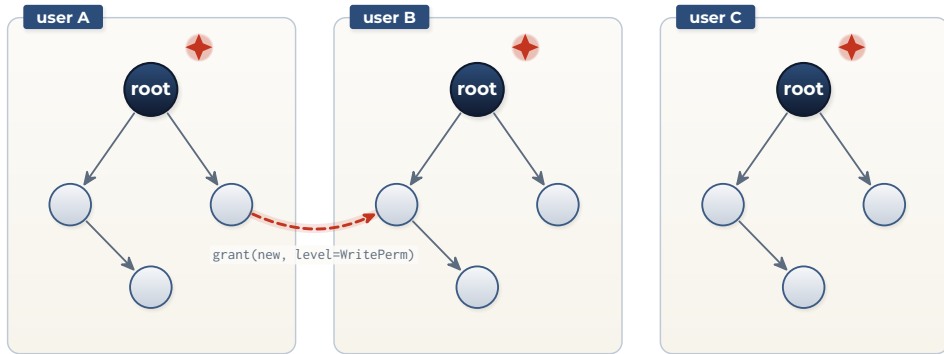


Figure 14.2: User multiplicity as root multiplicity. Each user's data is the subgraph reachable from that user's root, and each request's walker is spawned on the root of its authenticated principal. Isolation is the default geometry rather than a filter to remember; sharing is the explicit grant that opens one node across the boundary.

is that text: it opens the new tweet to other principals at a stated permission level, so that followers' feed walkers may read what they can now reach. Finer grants target specific principals' roots, and deliberate cross-population operations announce themselves in the code: littleX's `get_all_profiles` is marked `walker:pub` and fans out with `allroots()`, and the deployment's public commons is addressed as `root.shared`. Access control thereby reduces to reachability plus permission arcs on the topology itself: a policy surface one can read off the graph rather than reconstruct from scattered conditionals.

The invariance claim survives the degenerate case. Outside a server there is one principal, whose root is the local root, and the commons collapses onto it: `jid(root.shared)` equals `jid(root)` in a script run. The program text that serves N users is the text that ran as a script, with the population axis absorbed entirely by the binding of `root`.

14.4 Distribution: Sharding the Topology, Moving the Walker

The third axis is machines: one to M . Object-spatial semantics admits two dual motions for crossing it, and the runtime employs both.

The first motion brings data to the walker. A served deployment runs as a set of replicas that hold no authoritative state. The topology of record lives in a shared persistent backend, fronted by the memory hierarchy of Chapter 13: a request-scoped in-process cache above a shared cache tier above the store, with cross-replica invalidation keeping re-reads coherent. A walker spawned by a request executes on whichever replica received it, and the stretch of topology it traverses materializes beneath it on demand. In this mode m is invisible to the program by construction:

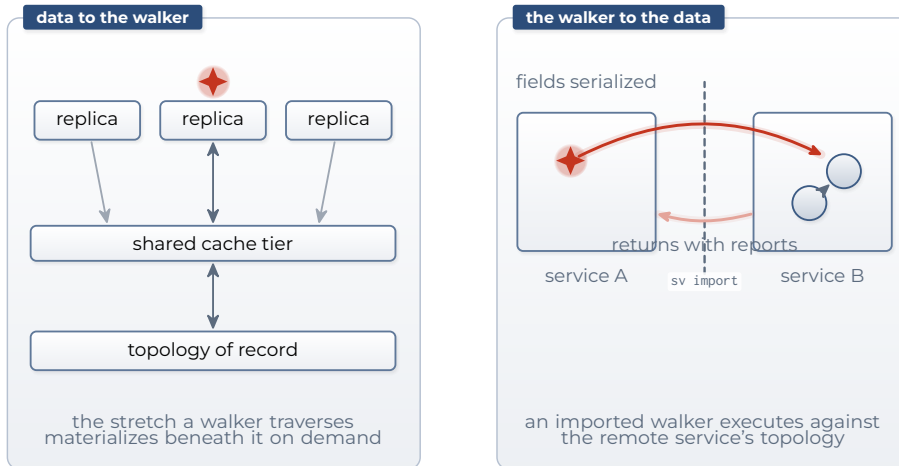


Figure 14.3: The two dual motions of distribution. On the left, replicas hold no authoritative state: a walker runs where the request lands, and the topology it touches materializes beneath it through the cache tiers, so no program point can observe how many machines exist. On the right, a service boundary declared by one `sv import` sends the walker to the data: its fields are serialized, it executes against the remote topology, and it returns with its state and reports rehydrated, indistinguishable at the call site from a local spawn.

no program point names a machine, so no program point can depend on how many there are.

The second motion sends the walker to the data. The service split itself is one statement. A server module that imports another server module's callables with `sv import` has declared a service boundary, and the runtime materializes that boundary at whatever scale the deployment runs. Under `jac start`, the imported module is spawned automatically as its own local process and reached over HTTP, so the boundary is real even on a laptop: two processes, two process identifiers, one program. Under `jac start -scale`, the same import becomes a separately deployed and separately scaled service, its routes discovered from the import and its identity handled at the gateway (the runtime machinery on both rungs is the mediation table of Chapter 13). This is the microservice architecture of the conventional stack, derived from an import statement rather than maintained as an estate of route declarations and hand-written HTTP clients. Note that the service boundary is computed by the same machinery as the codespace boundary of Chapter 12: the import is recorded as a fact on the program graph, and everything downstream of it (routes, transport, identity) is derived rather than authored. Invoking an imported walker becomes a migration: the walker's fields are serialized, the walker executes on the remote service against that service's topology, and the executed instance returns with its state and reports rehydrated, indistinguishable at the call site from a local spawn. Figure 14.3 sets the two motions side by side.

```
# notify_service.jac (provider)
walker:pub Greet {
```

```
    has name: str;
    can greet with Root entry {
        report f"hello, {self.name}";
    }
}

# dispatcher_service.jac (consumer)
sv import from notify_service { Greet }

walker:pub TriggerGreet {
    has who: str;
    can run with Root entry {
        rg = Greet(name=self.who); # executes on the provider
        report rg.reports[0];
    }
}
```

Listing 14.5: Walker migration across services, from the runtime reference. Instantiating the imported walker serializes its state, executes it on the provider machine, and rehydrates the executed instance.

Note that the consumer's call is a bare construction, `Greet(name=self.who)`, not a spawn. The imported walker has no local graph to walk. At a service boundary, construction is the migration, and the value that comes back is already the executed instance.

This is the topokinetic motion of Chapter 7 lifted across the machine boundary: the locus of computation moves, and the topology stays put. Sharding follows the grain the language already established. Because walkers are overwhelmingly root-local (every littleX walker begins at the caller's root and traverses its reachable subgraph), the per-user root is the natural unit of partition, and a placement policy that pins each root's subgraph to a shard and routes spawns accordingly involves the program text not at all. We state the present engineering position plainly: the replicas-over-shared-store motion is fully realized in the shipping runtime, walker migration is realized at service granularity through the explicit import above, and automatic placement of roots onto shards is a runtime policy layer, an evolution of the same machinery, not a change to the language.

Two ancestral systems locate this design in the tradition. Orleans introduced the *virtual actor*: an actor that exists logically at all times, is activated on whatever machine the runtime chooses, and whose placement is wholly the runtime's concern (Bykov et al. 2011). Jac nodes are virtual in this sense (a node is a persistent identity that materializes in memory where and when a traversal requires it), but they are passive where Orleans grains are active: computation does not reside in the node awaiting messages, it arrives as a walker. Erlang contributes the complementary half. Its processes are location-transparent, share-nothing, and cheap enough to model each concern as its own mobile unit of state and control (Armstrong 2003). A walker resembles an Erlang process in carrying identity and state across machines, but it moves *through* a shared persistent topology rather than around an ether of process identifiers. Both systems argued, from different directions, that distribution belongs

to the runtime's placement policy, not to program structure. Scale invariance is that argument carried to its conclusion for topological programs.

14.5 The Physical Limits: Latency, Failure, and Cost

A claim of this shape has been made before, and its most famous critics were right. Waldo and colleagues argued that unified local and remote objects rest on a confusion: latency, memory access, partial failure, and concurrency make remote interaction *semantically* different from local interaction, and systems that paper over the difference (the RPC-descended middlewares of Appendix F) collapse when the difference asserts itself (Waldo et al. 1994). Any language claiming scale invariance owes an account of why it is not the same mistake. Chapter 2 named the obligation the lawful-boundaries principle: continuity of expression must not be purchased by denying boundaries that impossibility results guarantee. We give the account limit by limit.

Latency. A visit that crosses a shard costs orders of magnitude more than one served from the in-process cache, and no runtime can pretend they cost the same. Scale invariance does not claim it does. Definition 14.1 excludes latency from the equivalence. What the language does is keep the cost out of the program's *meaning* while leaving it visible to the program's *operator*: the traversal that was correct on one machine is correct on twenty, and making it fast is a placement problem, not a rewrite.

Partial failure and ordering. A multi-machine deployment partitions, and the CAP results bound what any system may promise when it does: not consistency, availability, and partition tolerance at once (Brewer 2012; Gilbert and Lynch 2002). Nor does any global ordering of events exist across machines except as communication constructs one (Lamport 1978), which is why the definition's equivalence is per principal, not a claim of global serializability. Within these bounds the runtime makes a specific, documented choice. Writes are guarded by per-node optimistic concurrency: a conflicting commit is detected, the losing request's work is discarded atomically, and the request replays against the winner's state. Two racing find-or-create walkers converge on one node, and external side effects deferred through `on_commit` fire exactly once, for the attempt that wins. A deployment that prefers to surface conflicts instead configures the runtime to return a typed conflict error. The boundary is thus lawful in the sense of Chapter 2: the program text does not change, but the operational contract (replay latency, or a typed failure a client must handle) is stated and observable. It is chosen, never pretended away. The runtime's own distinction between writes that need this coordination and writes that do not echoes the CALM principle, which characterizes when distributed consistency is achievable without coordination (Hellerstein and Alvaro 2020): blind appends to the topology commute and proceed lock-free, while check-then-create, which is not monotone, pays the compare-and-swap. Where an application relaxes consistency further, it inherits the well-understood obligations of eventual consistency (Vogels 2009).

Cost. Finally, the transformation from one machine to M is not free. It is paid in machines. Scale invariance moves the expense of scaling from engineering time (the rewrite, the tenancy audit, the interface drift) to runtime resources and operator attention. This is a trade, and we present it as one. The single system image hides

the shape of the deployment from the program. It does not, and should not, hide the physics or the bill from the people running it.

14.6 Deployment Absorbed

One layer of the inventory of Chapter 1 remains: the deployment layer itself, the container images and orchestration manifests and infrastructure-as-code that describe to a cluster what the program already knows about itself. In the terms of Chapter 4, this layer is the operations half of the hidden tier: a substrate boundary crossed by hand.

The runtime absorbs it the same way it absorbed the build. The command `jac start -scale` derives the deployment from the program and its manifest: it packs the project into a sealed artifact with precompiled modules, generates the container image and the orchestration resources, provisions the storage and cache tiers of Section 14.4, and attaches autoscaling and monitoring policies declared in `jac.toml`. Scaling policy, ingress, and per-service replica counts are configuration on the project manifest, not artifacts authored beside the program. Beneath this surface stands ordinary, battle-tested orchestration, the lineage running from Borg to Kubernetes (Burns et al. 2016). The point is not to replace that machinery but to demote it: from a tier the programmer authors to an implementation detail a language operation drives.

The serverless movement anticipated half of this absorption. Its promise was that programmers ship functions while the platform owns provisioning and scale-out. Its acknowledged limits were statelessness and the glue required to marry ephemeral functions to external storage (Jonas et al. 2019). A projected walker endpoint is recognizably a cloud function (invoked on demand, scaled by policy, billed by use), but it is a cloud function whose state problem is solved in the language: the persistent topology of Chapter 10 is the stateful substrate whose absence the serverless critique identified. With deployment absorbed, the definitional claim of Section 14.1 becomes an observable fact about a command line. The program that ran as `jac run` serves as `jac start` and deploys as `jac start -scale`, and the diff between those configurations (one user or many, one machine or many, transient or persistent) is empty.

Bibliographic Notes

The term *single system image* and its OS-layer history are surveyed by Buyya et al. (2001). Appendix F treats that lineage, from distributed operating systems through process-migration clusters, as part of the general history of continuity attempts. The critique that disciplines this chapter is Waldo et al. (1994), best read against the mechanism it criticizes, remote procedure call as consolidated by Birrell and Nelson (1984). The impossibility results bounding Section 14.5 are the CAP conjecture as proved by Gilbert and Lynch (2002) and reframed by its originator in Brewer (2012), with Lamport (1978) supplying the deeper point that ordering itself is a construct of communication. Hellerstein and Alvaro (2020) characterizes the class of programs for which coordination-free consistency is possible, and Vogels (2009) states the practitioner’s contract for the eventually consistent regime.

On the distribution designs compared in Section 14.4, the virtual actor model is introduced by Bykov et al. (2011), and the Erlang philosophy of location-transparent, share-nothing processes is developed in Armstrong (2003). Both reward reading as runtime-placement arguments rather than as API designs. The orchestration substrate beneath Section 14.6 is recounted from the inside by Burns et al. (2016), and the serverless promise and its statelessness limits are assessed in the Berkeley view of Jonas et al. (2019). The walker-as-endpoint projection has a production ancestor in the Jaseci runtime stack, described by Mars et al. (2023), whose scale-out experience motivated much of the machinery this chapter records. The case that the whole stack beneath the function, state included, belongs to the runtime rather than to glue beside it was argued for this lineage in Mars (2022), and this chapter is that position matured into mechanism.

PART **V**



EVALUATION AND PRAXIS

CASE STUDIES

A book that claims a language removes things faces an awkward evidentiary problem. Presence is easy to demonstrate: one prints a listing. Absence is not. The argument of Parts II through IV is an argument about absence. It claims that a synechic surface eliminates the marshaling layers, API scaffolding, and build orchestration cataloged in Chapter 1, and that the topokinetic paradigm eliminates the database discontinuity and the tenancy code that conventionally surround persistent, multi-user state. The only way to exhibit an absence is to take a complete, working artifact, inventory everything it contains, and let the reader observe what is not there. That is the method of this chapter.

Our evidentiary discipline follows from the disclosure rules that govern this part of the book. Every file count and line count reported below is measured directly from the Jac repository at the revision this book describes, using nothing more elaborate than `wc`. Line counts are physical lines including comments and docstrings, which if anything overstates the size of the artifacts. Where a claim would require controlled experiments (performance comparisons, defect rates, developer productivity), we defer to Chapter 16 and say so. Brooks (1987) distinguished the essential complexity of a problem from the accidental complexity of the machinery we build around it. The case studies that follow are, in effect, four measurements of how much accidental complexity remains when the discontinuities are removed. We examine a full-stack social application, a conversational AI product pattern, a native game delivered simultaneously as machine code and WebAssembly, and, in a more reflective register, the qualitative record of a decade of Jaseci-lineage deployments.

15.1 A Full-Stack Social Application: One Codebase, Zero Glue, Measured

The littleX application is a Twitter-style social network maintained in the Jac repository as its flagship example: user accounts with authentication, profiles with editable

Table 15.1: Composition of the littleX application, measured from the repository. Line counts are physical lines including comments and docstrings.

Layer	Files	Lines
Backend: graph schema, report types, and walkers	1	466
Entry point	1	9
Application shell (declarations and implementations)	2	621
Feature components	11	1,556
Reusable UI primitives	19	1,346
Client utility library	1	19
Tests (server-side and full-stack)	2	415
Total	37	4,432

biographies, a follow graph, tweets with likes and threaded comments, channels with membership and channel-scoped posts, hashtag trending, search, and a responsive browser interface with light and dark themes. It is not a toy in the pedagogical sense (it is a complete product with a test suite), but it is small enough to inventory exhaustively, which is what makes it useful as evidence.

The entire application comprises 37 Jac source files totaling 4,432 physical lines, of which 415 are tests. Table 15.1 gives the decomposition. Beyond the Jac sources, the project contains exactly three other artifacts: a stylesheet, an SVG logo, and a single `jac.toml` manifest that declares the project’s Python and npm dependencies in one place and configures how the application is served.

The architecture divides into three layers, all in one semantic medium. The backend is a single file, `social_graph.jac`, whose 466 lines contain everything a conventional stack would spread across a database schema, an ORM layer, a service layer, and a REST controller tier: three node archetypes (`Profile`, `Tweet`, `Channel`), four edge archetypes (`Follow`, `Post`, `Member`, `ChannelPost`), three plain-object report types, and twenty walkers. Seventeen of the walkers constitute the application’s entire API surface. The remaining three are shared traversal bases from which action walkers inherit their lookup behavior. The frontend consists of a 318-line declaration file, a 303-line implementation file holding the handler bodies (the declaration/implementation split of Chapter 3), eleven feature components, and nineteen UI primitives. The primitives are ports of a conventional component library, expressed in Jac and scaffolded through the toolchain’s component integration declared in `jac.toml`. The entry point that binds the whole application together is nine lines.

The contract between the tiers is not a protocol document or a generated client SDK. It is the graph’s own types. Walkers report `Tweet` and `Profile` nodes directly, and a reported node crosses the wire still carrying its identity, so browser code can ask `jid(tweet)` of a value it received and hand the string back to a lookup walker (Chapter 10). Where the client needs edge-derived context beside a node (follower lists, member counts, the viewer’s membership), the walker reports one of two typed bundles: plain `obj` archetypes that carry the node itself rather than a copy of its fields. Listing 15.1 shows both bundles.

```
| obj ProfileBundle {
```

```

    has profile: Profile,
        followers: list[Profile],
        following: list[Profile],
        tweets: list[Tweet];
}

obj ChannelBundle {
    has channel: Channel,
        member_count: int,
        is_member: bool,
        posts: list[Tweet] = [];
}

```

Listing 15.1: The two typed report bundles of the littleX backend: each carries graph nodes together with the edge-derived context the client needs beside them.

The walkers themselves are the API. Listing 15.2 shows the walker that posts a tweet: it spawns at the calling user’s root, moves to their profile, attaches a new Tweet node through a typed Post edge, grants read permissions on the new node, and reports the node itself. There is no route registration, no request parsing, no serializer: the runtime projects each walker as an authenticated endpoint automatically, in the manner developed in Chapter 14, and generates the interface documentation for the running service as a byproduct.

```

walker create_tweet {
    has content: str;

    can run with Root entry {
        visit [-->[?:Profile]];
    }

    can make with Profile entry {
        new = here +>:Post():> Tweet(
            content=self.content, author_username=here.username, created_at=_now()
        );
        grant(new, level=WritePerm);
        report new;
    }
}

```

Listing 15.2: The walker that creates a tweet, verbatim from the littleX backend.

On the other side of the conventional divide, the frontend imports the backend’s walkers and archetypes directly: `sv import from .social_graph` names the server-side module, and the compiler partitions the program across substrates as described in Chapter 12. Invoking a walker from the browser is written as it would be written on the server. Listing 15.3 shows the handler that runs when the user posts: the same `root spawn` expression that spawns a walker locally here lowers to an authenticated request against the projected endpoint, and the reports that come back are values of the imported types, not JSON to be re-validated by hand.

```

impl app.postTweet -> None {

```

```

    if not composerText.strip() {
        return;
    }
    text = composerText;
    composerText = "";
    root spawn create_tweet(content=text);
    feedResult = root spawn load_feed(search_query="");
    tweets = feedResult.reports[0] if feedResult.reports else [];
    profileResult = root spawn get_profile();
    myProfile = profileResult.reports[0] if profileResult.reports else myProfile;
    trendResult = root spawn get_trending();
    trendingTags = trendResult.reports[0] if trendResult.reports else [];
}

```

Listing 15.3: The client-side post handler, verbatim from the littleX frontend implementation file.

Multi-user semantics are equally in-language. Each authenticated user has a private root, so a walker such as `load_feed` traverses only its caller's stretch of the topology by default. The explore and trending walkers deliberately widen their traversal across all user roots with `allroots()`, deduplicating nodes reachable by more than one path. Cross-user visibility of individual nodes is granted explicitly (`grant(new, level=WritePerm)`) at the moment of creation. Authentication itself is four imported functions from the runtime library: `jacSignup`, `jacLogin`, `jacLogout`, and `jacIsLoggedIn`. The isolation and sharing behavior is not asserted but tested. The full-stack test compiles to JavaScript, boots the backend module on an ephemeral port, signs up two users through the real client flow, and verifies that a tweet posted by one user appears in the other's feed only after a follow edge is created. The server-side suite exercises the same walkers over real HTTP semantics against isolated per-test graphs.

The measurement that matters most, however, is the count of what is absent. Working through the inventory of Chapter 1 against this codebase, littleX contains:

- no database schema, ORM models, or migration scripts: persistence is reachability from root (Chapter 10);
- no route table, controller classes, or hand-written REST scaffolding: the API surface is projected from walker declarations (Chapter 14);
- no serializers, data-transfer-object mappers, or client SDK: graph nodes and their report bundles cross the tier boundary as themselves, identity included;
- no separate frontend build configuration: no `package.json`, no bundler config file, no lockfile authored by hand; the npm dependencies and build settings live in the same `jac.toml` stanza that names the Python dependencies (Chapter 4);
- no authentication service, session middleware, or token plumbing beyond the four imported runtime functions; and
- no interface specification maintained by hand: the served application generates one from the walker signatures.

A complete accounting names what does remain. The stylesheet is ordinary CSS. The `jac.toml` manifest, at 65 lines, declares two Python dependencies and some twenty npm packages. The ecosystems are inherited, not hidden, and the manifest is where that inheritance is stated. And the nineteen UI primitives, while written in Jac, are ports of an existing component library rather than original code. None of these is glue in the sense of Chapter 1: no artifact in the project exists to move data across a boundary, restate a contract in a second notation, or orchestrate one toolchain from another. That is the zero being claimed, and it is a zero one can check by exhaustive enumeration of 37 files.

The multitier literature has long measured its systems by the boilerplate they eliminate (Weisenburger et al. 2020). The measurement here extends that tradition downward into the build system and outward into persistence and multi-tenancy, where tierless languages historically stopped (Chapter 2). Chapter 16 performs the corresponding comparative measurement against a conventional stack implementing the same feature set.

15.2 A Conversational AI Product in Production

The second case study concerns the pattern at the heart of the conversational AI products that have been built on the Jaseci lineage since its inception (Mars et al. 2023): a language-model-backed agent whose durable state is a persistent graph and whose behaviors are walkers over that graph. The artifact we study is the day-planner example maintained in the repository, which distills that production pattern to its minimum: an assistant that categorizes tasks and turns a natural-language meal description into a structured shopping list, with every AI interaction expressed as meaning-typed delegation (Chapter 6) and every piece of state a node reachable from the user's root.

The repository carries the same product three times, as a deliberate architectural comparison. The basic variant is a single 283-line file containing the entire application (server functions, AI delegation, and the reactive browser interface), with all users sharing one graph. The auth variant changes the endpoint declarations from `def:pub` to `def:priv`, and by that change alone every authenticated user receives an isolated root. The business logic is untouched, which is scale invariance (Chapter 14) exhibited as a diff. The walkers variant re-expresses the backend in object-spatial style: the module carrying the entire server, together with the entry point that mounts the client, is 159 lines.

The meaning-typed core is identical across the three variants, and it is where the discontinuity this case study measures is crossed: the boundary between deterministic code and a stochastic model. Listing 15.4 shows it in full.

```
obj Ingredient {
  has name: str,
    quantity: float,
    unit: Unit,
    cost: float,
    carby: bool;
}
```

```

sem Ingredient.cost = "Estimated cost in USD";
sem Ingredient.carby = "True if this ingredient is high in carbohydrates";

def categorize(title: str) -> Category by llm();
sem categorize = "Categorize a task based on its title";

def generate_shopping_list(meal_description: str) -> list[Ingredient] by llm();
sem generate_shopping_list = "Generate a shopping list of ingredients needed for a
    described meal";

```

Listing 15.4: The complete AI integration of the day planner: typed output shapes, semantic annotations, and two delegated functions.

Sixteen lines constitute the application’s entire relationship with the language model. There is no prompt template, no response parser, no JSON-repair loop, no retry scaffold in the application code: the prompt is synthesized from the function signature, the enum and object types, and the `sem` annotations, and the result arrives as a typed value: a `Category`, or a `list[Ingredient]` whose elements carry a boolean `carby` and a float `cost`. The same `obj Ingredient` that shapes the model’s output is the type the client renders, so the LLM boundary and the tier boundary are crossed by one declaration. This is the mechanism of Chapter 6 doing product work rather than benchmark work. Its quantitative evaluation against prompt-engineering baselines is reported by Dantanarayana et al. (2025) and taken up in Chapter 16.

In the walker variant, delegation happens inside traversal, which is the shape production conversational systems take: the agent’s memory is the topology it walks (Chapter 7). Listing 15.5 shows the task-creation walker: it arrives at the caller’s root, consults the model to categorize the incoming title, and attaches the resulting node to the persistent plan graph in the same breath.

```

walker:priv AddTask {
  has title: str,
    reports: list[Task] = [];

  can create with Root entry {
    category = str(categorize(self.title)).split(".")[1].lower();
    new_task = here ++> Task(title=self.title, category=category);
    report new_task;
  }
}

```

Listing 15.5: Meaning-typed delegation inside a walker over the persistent plan graph, verbatim from the walker variant of the day planner.

The multi-step pattern appears in the shopping-list walker, which queues a traversal over the existing `ShoppingItem` nodes, calls the delegated generator, attaches the fresh items, and lets its with `ShoppingItem` entry ability retire the old ones as a side effect of the visit: generation and graph maintenance interleaved in one agent rather than coordinated across a service boundary.

A word on framing, per the evidentiary discipline stated at the outset. The day planner is a distillation resident in the repository, not itself a commercial deployment,

and we do not report proprietary metrics from the production systems whose pattern it distills. The structural claim is the one that can be verified from the artifact: a conversational AI product of this shape consists, in its entirety, of typed output declarations, semantic annotations, and walkers over a persistent per-user graph. The step from shared prototype to isolated multi-user service is a visibility annotation, not an architecture. The longitudinal observations in Section 15.5 speak, qualitatively, to how this pattern has fared under sustained production use.

15.3 A Native-Compiled Game Delivered as WebAssembly

The third case study leaves the managed substrate entirely. The repository’s cube-shooter example is a first-person 3D game (perspective camera, mouse look, movement, firing, floating targets, a live frame-rate counter) written in 627 lines of Jac and compiled to a standalone native executable that links directly against a precompiled release of the raylib graphics library. There is no Python runtime in the delivered binary, no C source in the project, and, more tellingly, no C compiler or system linker anywhere in the build: the native pathway emits object code and writes the ELF or Mach-O executable itself, recording the library dependency and a relative runpath so the binary finds its sibling library wherever it is launched.

The foreign-function surface is the C-import block sketched (abridged) in Listing 15.6. The library is named by logical name (no path, no extension), and the backend resolves the platform-correct filename from the target triple, so the one source builds unmodified for Linux, macOS, and Windows. Every declaration in the block becomes an extern symbol resolved from the shared library. C structs passed by value can be declared as obj archetypes in the same block and lowered to the platform ABI.

```
import from raylib {
  def InitWindow(width: i32, height: i32, title: str) -> None;
  def rlVertex3f(x: f32, y: f32, z: f32) -> None;
  def rlColor4ub(r: u8, g: u8, b: u8, a: u8) -> None;
  def rlFrustum(
    left: f64, right: f64, bottom: f64, top: f64, znear: f64, zfar: f64
  ) -> None;
}
```

Listing 15.6: Binding a precompiled C library by declaration, abridged from the shooter source.

The example ships with a deliberate control: a 579-line Zig port of the same game, binding the same entry points of the same precompiled library through Zig’s `extern fn`, rendering through the same scalar immediate-mode API, and producing pixel-identical output. The Zig twin exists as the conventional-toolchain baseline (same library, ordinary linker), and the example’s benchmark harness runs both builds head to head. The sample run recorded in the repository, on a software-rendered development machine, shows the two builds tracking each other within roughly two percent of average frame rate. We treat that as an existence proof of parity on one machine, not a result, and defer controlled measurement to Chapter 16. The methodological point stands independent of the numbers: the native pathway is not

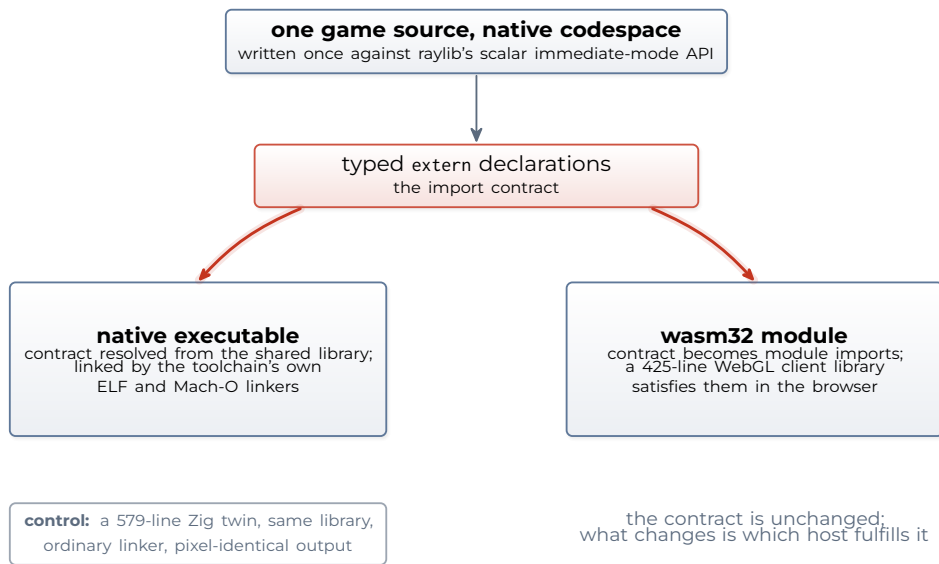


Figure 15.1: One game, two deliveries. The same source in a native codespace compiles to a native executable and, under the `wasm32` target of the same pure-Jac linker, to a browser module whose imports a reusable WebGL library satisfies. The Zig twin is the conventional-toolchain baseline against which parity is checked.

competing against an interpreter but against a systems language on its own terms, with the same dependency and identical per-frame work.

The same game is then delivered to the browser, and this is where the case study earns its title. The web variant places the identical game source, written once against raylib's scalar immediate-mode surface, in a native codespace within a full-stack module, compiled to WebAssembly (Haas et al. 2017) by the same pure-Jac linker targeting `wasm32`. The extern declarations that the native build resolves from a shared library become, under the WebAssembly target, the module's imports. A 425-line reusable client library satisfies them with a WebGL emulation of the same scalar API and drives the game's frame function from the browser's animation loop, while the client page that mounts the canvas and renders the heads-up display is an ordinary component in the same module. One command builds and serves the whole: the compiled game module, the client bundle, and the page. The import contract is unchanged between native and browser delivery. What changes is which host fulfills it (Figure 15.1).

The count of absences is again the measurement. There are no C headers and no binding generator. The FFI is a block of typed declarations. There are no per-platform build scripts for the native side and no separate compile-to-web toolchain for the browser side. Both artifacts fall out of the same compiler driven by a target flag (Chapter 12), and the toolchain that does this fits in the single self-contained binary of Chapter 4. A game, a systems-language baseline, and a browser deployment coexist in a directory whose only build artifact descriptions are a shell script for downloading the third-party library and a `jac.toml`.

Table 15.2: Source inventory of the showcase application. One language, one build; mixed modules carry client and native code in a single file.

Subsystem	Modules	Lines
Client interface (showcase and embedded app)	64	7,138
Client handler implementations	2	389
Object-spatial backend (two graph modules)	2	737
Server services (analytics, source viewer)	4	1,091
Native game (one native module, twin entries)	3	1,889
Benchmark and capture automation (in Jac)	2	649
Total	77	11,893

The repository’s wider example suite extends the same continuity to other delivery targets, and we note it here only briefly. A chess engine of 220 declaration lines and 782 implementation lines exercises the declaration/implementation split on purely algorithmic code. A 664-line notes application compiles to a desktop program through an embedded-browser target. The mobile UI examples re-express littleX itself against a portable component vocabulary that compiles both to the web and, through React Native, to native mobile applications from one source tree. Each is a point on the same claim: the delivery substrate is a compilation decision, not an architecture.

15.4 A Four-Tier Showcase and Its Production Twin

The fourth study composes everything the first three exhibit separately, and it is the one for which the conventional comparison has actually been built. The artifact is an interactive showcase application of 11,893 lines of Jac across 77 modules and no other implementation language: a public tour site whose sections are live demonstrations (a guestbook persisted to the object-spatial graph, a microservice call, a streaming endpoint, and the in-browser game of Section 15.3), with a complete embedded microblogging application (authentication, feed, follows, channels, profiles) whose seventeen walkers run in the same server and persist to the same graph. Table 15.2 gives the inventory. All counts in this section were measured from the repository revision archived with the artifact guide (Appendix E).

The composition facts are checkable by inspection. One toolchain invocation compiles the tree into CPython endpoints, the browser bundle, a WebAssembly binary, and a native executable; no per-tier build system and no external toolchain participates. The application makes 50 cross-boundary calls (45 walker spawns, 4 function-stub awaits, one streamed endpoint) against 23 distinct remote callables, carrying nine view-model types across the wire, and every crossing is checked by the one compiler that sees both sides. The microservice section is one `sv import`: the analytics module is declared a service boundary by its import (Chapter 14), spawns as its own process on first call, and the page displays the two process identifiers as proof. The streamed endpoint crosses both hops, the analytics generator yielding through the gateway to the browser without buffering. The interop manifest computed for the whole application (Chapter 12) holds 43 bindings, effect signatures on all 29 endpoints, and authentication requirements on 19 of them. Three foreign ecosystems arrive

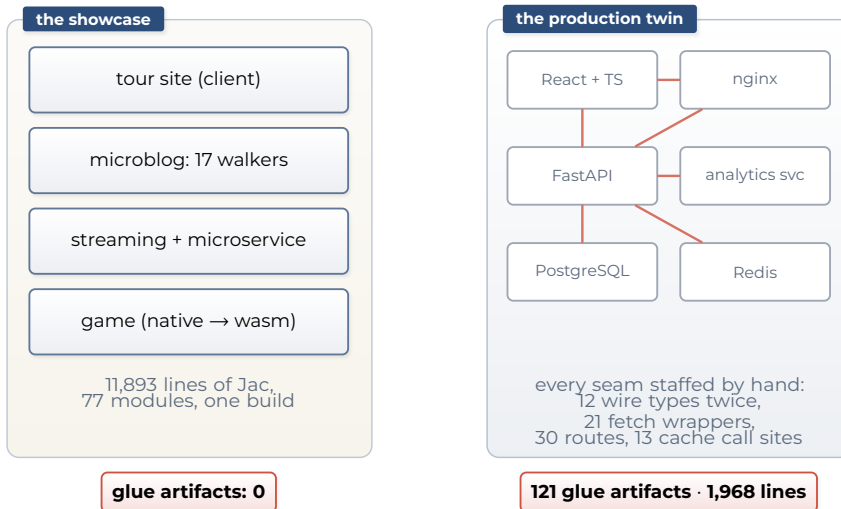


Figure 15.2: The showcase and its independently written production twin at 100 percent endpoint parity. The four tiers on the left are one language and one build with nothing between them; the twin’s boxes are conventional best practice, and every marked seam is hand-maintained glue: 121 artifacts, 1,968 lines, and 107 pairs of facts kept true in two places by discipline alone.

in one module set: Pygments from PyPI on the server, React, Radix, and Tailwind from npm on the client, and raylib bound by typed foreign declarations in the native module. And the absences are countable: the repository contains zero TypeScript, Python, YAML, or interface-description files, zero route registrations, zero serializers, and exactly two fetch calls, one consuming the event stream and one loading a static asset. The system also measures itself: the benchmark harness that times the native game against its Zig twin is a Jac program, and the source-viewing section serves the application’s own modules, lexed by a Jac tokenizer.

The comparison object is not an enumeration this time. The showcase has an independently written, production-grade twin, public as `this_is_not_jac` beside the showcase’s own repository: a React and TypeScript frontend, a FastAPI backend over PostgreSQL with Redis and nginx, a separate analytics microservice, and the same game hand-written in C and compiled to WebAssembly. The twin reaches 100 percent endpoint parity, implementing all 23 remote callables and the streamed endpoint. An exhaustive inventory of every artifact in it that exists only to carry data across a boundary finds 121 glue artifacts totaling 1,968 hand-written lines: 12 wire types defined twice (24 definition sites), 21 fetch wrappers, 30 route declarations, and 13 cache call sites, plus 632 further lines of deployment description. Each of those artifacts is a fact stated in two places that the team must keep true by hand, and the corresponding counts in the Jac column are zero by construction. Figure 15.2 sets the pair side by side, seam for seam. Chapter 16 deconstructs this pair into controlled experiments, one boundary at a time.

15.5 Longitudinal Notes from a Decade of Deployments

The four studies above are synchronic: artifacts frozen at a revision, inventoried in full. The final study is diachronic and necessarily looser. The Jaseci lineage has been in continuous research and production use for roughly a decade: across academic deployments, commercial conversational AI products built by startup teams on the earlier Jaseci stack (Mars et al. 2023), and the successive rewrites of the language and runtime that led to the system this book describes (a history given in Appendix D). What follows are practitioner observations from that span. They are qualitative and, inevitably, drawn from the vantage of those who built the system. We report them as experience, not as controlled findings, and Chapter 16 quantifies what can be quantified.

The first and most instructive observation is about what survived the rewrites. The language surface changed radically across the decade: from an interpreted domain-specific language, to a superset of Python compiled onto the managed runtime, to the self-hosted, multi-target compiler of Chapter 12. The persistence engine, service machinery, and type system were each rebuilt more than once. What persisted essentially unchanged was the paradigm: nodes, edges, walkers, abilities keyed on arrival, reporting as the return channel, and persistence by reachability from a per-user root. Applications written against the earliest stack remain recognizable, walker for walker, as the idioms of Section 15.1. When every layer of an implementation is replaced and the programs keep their shape, the value evidently lives in the abstraction rather than in any implementation of it. This is the strongest internal evidence we possess that object-spatial semantics, later formalized in Mars (2025), captures something stable about the applications people actually build.

The second observation concerns a class of defects that teams stopped having. In conventionally layered products, a persistent fraction of defects lives at the discontinuities: a serializer disagreeing with a schema, a client model drifting from an endpoint, an environment variable diverging between build configurations (Chapter 1). In deployments on this stack, that class is structurally absent for internal boundaries: there is no second statement of the contract to drift. The defects did not vanish. Logic errors, traversal mistakes, and misjudged permission grants took their place. But boundary drift re-entered only at the edges the medium does not absorb: third-party APIs, payment providers, and external data feeds, where teams write ordinary integration code and the usual discipline reapplies. Synechism, in production, relocates the discontinuities to the system's true exterior. It does not abolish exteriority.

Third, the physical limits of Chapter 2 surfaced where the theory says they must. Latency and partial failure do not disappear because invocation is uniform across tiers, and early users occasionally wrote chatty cross-tier loops that a visible network boundary would have discouraged. The critique of Waldo et al. (1994) held. The mitigation was observability and idiom, not denial (Chapter 17 returns to this). Operationally, the dominant recurring cost of long-lived deployments was schema evolution over living graphs: topologies that never shut down must be migrated in place, and the machinery of Chapter 10 earned its complexity there. And the single-artifact toolchain, whatever its virtues of reproducibility, concentrates trust. Teams adopting it asked, reasonably, what it costs to leave, which is the lock-in

objection we take up directly in Chapter 17.

Fourth, the human record. Teams on this stack were consistently small relative to the surface area they shipped. The pattern is visible in miniature in Table 15.1, where one file is the backend of a social network. Developers arriving from Python were productive quickly, as supersetting intends (Chapter 3). The genuine learning curve was never syntax but topology: learning to phrase a feature as a traversal rather than a query followed by a loop (Chapter 7). That curve is real, it is the price of admission to the paradigm, and a decade of watching people climb it is a large part of why this book spends four chapters teaching it.

None of these observations substitutes for measurement, and we do not offer them as such. They are the field notes that motivated the measurements: the next chapter takes the claims that survived a decade of contact with production and subjects them to counting.

Bibliographic Notes

The evidentiary stance of this chapter (inventorying artifacts and measuring eliminated boilerplate) has its roots in the multitier programming literature, whose systems from Links onward were routinely argued through worked web applications and the tier code they removed. Weisenburger et al. (2020) surveys that tradition and its evaluation habits. Brooks (1987) supplies the older frame: if glue is accidental complexity, case studies are how one estimates the accident's share. The cautionary counterweight is Waldo et al. (1994), whose argument that distribution cannot be pretended away reappears in our longitudinal notes as lived experience rather than position.

The systems studied here connect to their primary sources as follows. The Jaseci stack and the production applications of its first generation are described by Mars et al. (2023). The object-spatial paradigm exercised by littleX and the day planner is formalized by Mars (2025). The meaning-typed delegation at the core of the conversational pattern is developed and evaluated by Dantanarayana et al. (2025), whose measurements complement the structural inventory given here. WebAssembly, the delivery substrate of the third study, is presented by Haas et al. (2017). All artifacts examined in this chapter live in the public repository and are exercised by its test suite, and Appendix E describes how to reproduce every count reported above.

QUANTITATIVE EVALUATION

The preceding chapters argue. This chapter measures. The claims of Parts II and III are qualitative in statement but quantitative in consequence, and each consequence can be checked. If Jac is synechic, then applications written in it should exhibit measurably fewer boundary artifacts than their conventional equivalents. If the toolchain absorbs the hidden tier of Chapter 4, the configuration surface should shrink to something countable on one hand. If scale invariance holds (Chapter 14), the cost of crossing from one machine to many should appear in performance curves rather than in program text. A treatise that declined to check these consequences would leave its central argument resting on anecdote.

We write this chapter under a discipline the reader should hold us to. Evaluations of a language conducted by that language's designers carry a structural conflict of interest, and no earnestness of tone dissolves it. Our response is methodological rather than rhetorical. We report only numbers we have measured. We identify exactly what was measured and how. We label every comparison object that is a description of standard practice rather than a measured implementation. And we mark every result that awaits its apparatus with an explicit placeholder, *[to be measured]*, rather than an estimate. The structural measurements of Section 16.1 are taken directly from the repository that accompanies this book and can be reproduced with a shell one-liner. The boundary experiments of Section 16.2 are measured from the four-tier showcase of Section 15.4 and the independently written twin built to shadow it. The performance and human-subject results of Sections 16.3 and 16.4 are reported here as protocols, fixed in advance so that the eventual numbers can be judged against a design that predates them. Section 16.5 then turns the skeptic's instruments on our own methodology.

Our primary unit of analysis is littleX, the full-stack social application introduced as a case study in Chapter 15. It is small enough to inventory exhaustively and complete enough to exercise every discontinuity this book claims to dissolve: client and server, persistence, authentication, multi-user isolation, and deployment. Section 16.2 adds

Table 16.1: Source inventory of littleX by tier. Counts are raw line counts including comments and blank lines, taken at the repository revision archived with the artifact guide.

Tier	Files	Lines
Graph and service tier	2	475
Client application tier	13	2,177
Presentational component kit (scaffolded)	20	1,365
Tests	2	415
Jac source, total	37	4,432
Configuration (one TOML manifest)	1	65
Stylesheet (one CSS file)	1	173

a second measured object, the showcase pair of Section 15.4, whose twin makes controlled boundary comparisons possible.

16.1 Glue Elimination: Lines, Artifacts, and Dependency Surface

Chapter 2 proposes a cost model for glue: the burden a stack imposes is measured not by the volume of application logic but by the boundary-crossing overhead layered around it. In this section, we operationalize that model with four counts, each defined so that a third party can recompute it: 1) the number of distinct general-purpose languages an application’s authors must write, 2) the number of auxiliary dialects (configuration, query, markup, build) they must additionally maintain, 3) the number of *glue artifacts*, meaning files whose sole purpose is to ferry, re-describe, or re-validate data across a substrate boundary rather than to express application behavior, and 4) the number of representation shifts a single datum undergoes along the application’s canonical write path. The first three are static properties of a repository. The fourth is read off a trace of one end-to-end operation.

16.1.1 THE MEASURED SIDE: AN INVENTORY OF LITTLEX

Table 16.1 gives the inventory. All counts are raw line counts (`wc -l`) with comments and blank lines included. Any convention for discounting comments introduces a judgment call we prefer not to exercise on our own behalf, and the raw count errs against us. The graph and service tier (the entire backend) is two files: `social_graph.jac`, which defines three node archetypes, four edge archetypes, three typed report objects, and twenty walkers, and a nine-line entry module. The client application tier comprises the frontend declarations, their implementations, and eleven feature components. The presentational component kit is scaffolded by the toolchain’s component generator and then owned by the project, as a conventional React project vendors its component library. We report it in its own row because it is generated code an author edits rarely, and we include it in the total because it lives in the tree and counts against us if anything. The entire application (backend, frontend, tests, styling, and configuration) is forty files, of which exactly one is configuration.

That single manifest carries the whole dependency surface: twenty-five third-party packages declared across two foreign ecosystems (three from PyPI, twenty-two from npm, development tooling included on both sides), resolved by one install operation, as Chapter 4 describes. The conventional equivalent of this one file is a Python manifest, a JavaScript manifest, their two lockfiles, and two separately operated package managers.

The glue-artifact count for littleX is zero, and it is worth being precise about what that claim means, because it is the measurement this section rests on. Consider the application’s canonical write path: a user posts a tweet. The client-side handler executes `root spawn create_tweet(content=text)`, and the walker it names is the following, quoted in full from the graph tier.

```
walker create_tweet {
  has content: str;

  can run with Root entry {
    visit [--[?:Profile]];
  }

  can make with Profile entry {
    new = here +>:Post():+> Tweet(
      content=self.content, author_username=here.username, created_at=_now()
    );
    grant(new, level=WritePerm);
    report new;
  }
}
```

Listing 16.1: The complete server-side artifact behind the post-a-tweet operation in littleX, from the social_graph module.

This walker is simultaneously the business logic, the API endpoint, the persistence operation, and the authorization boundary. There is no route declaration anywhere in the repository, because the serving runtime projects walkers as endpoints (Chapter 14). There is no client binding layer, because the frontend imports the walkers and the archetypes they report directly from the server module with a single server-import declaration and invokes them as values. There is no serializer, validator, or schema mirror, because Tweet is defined once and is the same type on both sides of the wire. There is no ORM model or migration chain, because the Tweet node persists by reachability from the user’s root (Chapter 10). And there is no authentication middleware in the application source, because session establishment is a handful of runtime calls on the client and root isolation on the server (Chapter 14). The twenty walkers of `social_graph.jac` constitute the entire service surface of the application. Trace the representation shifts along the write path as authored in program text (Figure 16.1): the tweet is a Jac value in the click handler and a Jac node in the graph, and no artifact in the repository describes it in any intermediate form. The physical boundary still exists: an HTTP request is made, bytes are serialized. That crossing is synthesized by the compiler (Chapter 12), not authored, and the cost model of Chapter 2 deliberately counts authored expression, since that is what substrate transparency is a claim about.

the authored forms of the conventional write path, absent from the repository:

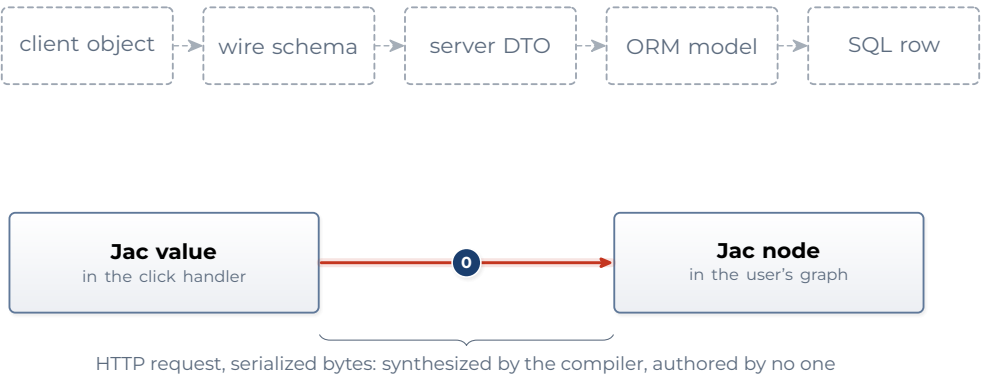


Figure 16.1: The write path of a tweet in littleX, as authored. The tweet is a Jac value in the click handler and a Jac node in the graph, with zero authored representation shifts between them; the physical crossing exists but is the compiler’s. The conventional stack authors the five forms shown dashed, none of which appear in the littleX repository.

16.1.2 THE COMPARISON SIDE: THE CANONICAL COMPOSITION

The natural comparison object is the stack a competent team would conventionally assemble for the same application: a React and TypeScript frontend built by Vite, a FastAPI backend with Pydantic request and response schemas, SQLAlchemy models over PostgreSQL with an Alembic migration chain, JWT-based authentication middleware, and Docker Compose with a reverse proxy for deployment. We have not built this implementation, and we decline to fabricate line counts for it. A straw competitor written by us, half-heartedly, would be a worse scientific object than no competitor at all. What we can do instead is enumerate the composition *structurally*, because its artifact classes are fixed by the architecture rather than by any particular team’s diligence, and they are documented at length in the multitier literature’s account of what tierless languages exist to remove (Cooper et al. 2006; Weisenburger et al. 2020). Table 16.2 places the two columns side by side, with the right column explicitly labeled for what it is: the canonical composition, enumerated, not a measured competitor.

The most telling row is schemas per domain entity. In the canonical composition, a tweet exists as a SQLAlchemy model, a pair of Pydantic schemas, a TypeScript interface, and a migration’s DDL: four descriptions of one concept whose mutual drift is a standing defect class, the schema-coherence problem that Chapter 1 places at the center of the glue taxonomy. Along the canonical write path, the datum authored by the developer changes representation at each of these stations, in both directions. In littleX the corresponding count is one definition and zero authored shifts. This is the boundary-crossing metric of Chapter 2 instantiated, and it is the sense in which we say the measurement of glue elimination is already complete. The number of glue artifacts in the repository is not small, it is zero, and zero is a number one measures

Table 16.2: Artifact and dialect counts. The littleX column is measured from the repository; the right column enumerates the canonical React, FastAPI, and PostgreSQL composition structurally and is a description of standard practice, not a measured implementation.

Dimension	littleX (measured)	Canonical stack (enumerated)
General-purpose languages	1	3 (TypeScript, Python, SQL)
Auxiliary dialects	2 (TOML, CSS)	7+ (JSON, YAML, INI, HTML, CSS, Dockerfile, dotenv)
Package manifests	1	2, plus 2 lockfiles
Schemas per domain entity	1	3–4 (ORM, wire, client, DDL)
Route and controller artifacts	0	1 router module per resource
Client binding artifacts	0	fetch layer or generated client
ORM models and migrations	0	models plus migration chain
Authentication plumbing	0 authored	middleware, hashing, tokens
Deployment artifacts	0	Dockerfiles, compose, proxy

by exhaustive inspection rather than by benchmark harness.

What these counts do not show is equally important to state. They do not show that littleX took less time to write than its conventional equivalent (that is a claim for Section 16.4), nor that it runs faster (Section 16.3), nor that a team maintaining it for five years would thank us (a question the longitudinal record of Chapter 15 bears on but does not settle). They show that a particular, structurally defined category of software (the category this book has argued from its first chapter onward is the dominant accidental cost of the field (Brooks 1987)) is absent from this codebase, not minimized.

16.1.3 THE COMPARISON MEASURED: A PRODUCTION TWIN

The paragraph above declines to build a straw competitor for littleX, and the refusal stands. For the four-tier showcase of Section 15.4, however, the comparison exists on equal terms: an independently written, production-grade twin (React and TypeScript, FastAPI, PostgreSQL, Redis, nginx, a separate analytics microservice) at 100 percent endpoint parity. There the count is a measurement rather than an enumeration: 121 glue artifacts totaling 1,968 hand-written lines, among them 12 wire types defined twice, 21 fetch wrappers, 30 route declarations, and 13 cache call sites, and in total 107 pairs of facts that must be kept true in two places by hand. The corresponding counts in the Jac application are zero, for the same structural reason as littleX’s. The enumerated column of Table 16.2 therefore has a measured sibling, and the two agree on every artifact class the table names.

16.2 Boundary Experiments on the Showcase Pair

In this section, we deconstruct the showcase pair of Section 15.4 into controlled experiments, each extracting one boundary or mechanism from the application and measuring it against the twin. The methodological point is the extraction itself: because the workloads are cut from a working application rather than written for the

evaluation, they inherit the application’s shapes (real view models, real traversals, real component trees) rather than the evaluator’s expectations. Every number below was measured from the pair at the revision archived with the artifact guide, and none is synthetic.

The first experiment asks whether interface drift is still expressible. We generated 47 paired boundary mutations across the twin (a renamed field, a changed type, a dropped route, and their kin), each applied to one side of a boundary only. At the boundary itself, the conventional checkers catch none of them: `tsc` and `mypy` jointly report 0 of 47, because each half of the twin remains internally consistent while the pair is jointly wrong. The same mutations applied to the Jac application are flagged at compile time as hard errors in 33 of 47 cases, and as errors or warnings in 37 of 47. The misses are dominated by mutations no type system can see: fields that are dead in practice, and value changes that preserve the type.

The second experiment measures the boundary as an optimization surface, the conversion Chapter 12 claims for the reified boundary. The `interop` manifest records an effect signature for every endpoint, what it reads and what it writes, and from those signatures the client runtime derives a cache-invalidation policy that no one authored. Replaying a recorded interaction trace, the derived policy eliminates 34 percent of round trips with zero read-your-writes staleness, where a time-to-live policy tuned to the same trace admits 33 stale reads. Against an oracle’s zero unnecessary invalidations, the derived policy performs 20, and the blunt alternative of invalidating everything on every write performs 30. The residual staleness the policy admits is confined to the cross-user class that any bounded policy shares.

The third experiment prices the generated stub, and we report it against ourselves with the decomposition stated. Through the full serve path, an authenticated graph read answers at a median of 35.1 milliseconds, against an in-process floor of 7.9 milliseconds for the same function: the gap is the runtime’s serve path, not the marshaling the stub performs. The twin’s tuned path (`nginx`, `gunicorn`, `Redis`) answers cache hits in 1.29 milliseconds, a number a mature stack earns and ours does not yet match. What the twin cannot do is move a service without rewriting it. In Jac, resolving a service import in-process rather than across processes is a configuration change, and the same analytics call costs 0.0003 milliseconds in-process and 3.13 across processes, against the twin’s hand-written service hop at 8.3.

The last experiments measure the native tail and the build itself. Identical sources lowered natively and to CPython produce matching outputs and separate by 53× on a matrix kernel, 22× on a recursion benchmark, and 25× on a physics step, with the native binary at 4,448 bytes, statically linked. The self-hosted compiler rebuilds its own 94 modules from a cold cache in 60.8 seconds and from a warm one in 0.94, a 65-fold cache effect, and the full application builds in roughly 19 seconds, dominated by the client bundling step, which the toolchain drives internally with no bundler configuration authored in the project (Section 12.8). These are the polypilation costs a team pays for the absences counted above, and we report them so the trade is priced rather than implied.

16.3 Performance: Traversal, Lowering, Service, and Scale

The performance question for a synechic language is not whether it wins races. It is the *price of continuity*: what constant factors, if any, does the single semantic medium cost relative to writing each tier natively in its host substrate, and how do those factors behave under load and under scale-out. A language that eliminated glue by imposing a large multiple on every operation would have relocated its costs, not removed them, and Chapter 17 takes up the ceiling arguments in earnest. In this section, we define the suite that measures that price. The suite comprises seven families, summarized in Table 16.3. The harness, workloads, pinned versions, and hardware specification are distributed with the artifact guide (Appendix E). One family, the topology-lifetime measurements of Section 16.3.1, has completed its runs and is reported below with final numbers; every other result cell in this section is a placeholder pending the finalized runs.

The topology family isolates the cost of the object-spatial constructs themselves: what a visit costs beyond a method call, what ability dispatch on arrival costs beyond a conditional, and how traversal throughput scales with topology shape and size. The baseline must be double-ended. Against hand-written host-language object graphs, the interpreted lowering will pay a constant per-visit factor for dispatch and anchor bookkeeping, and the suite exists to pin that factor down. Against persistent stores queried through traversal languages (Francis et al. 2018; Rodriguez 2015), the comparison inverts: there the baseline pays a query-boundary cost on every interaction that a walker, living in the same semantic medium as its data, does not. We also include a deliberately adversarial workload (whole-graph analytics of the kind that bulk-synchronous systems are engineered for (Malewicz et al. 2010)) because a walker is the wrong tool for it, and a suite that omitted the workloads its subject should lose would be an advertisement (see Section 16.5).

The lowering family measures the cost of substrate transparency directly. One kernel set (numeric loops, string manipulation, data-structure churn) is written once in Jac and lowered to each of the three substrates of Chapter 12. The baselines are idiomatic implementations authored natively for each substrate, the native ones compiled through the same LLVM generation that underlies the toolchain (Lattner and Adve 2004). The metric is the ratio of lowered to native-authored runtime, per substrate, with the WebAssembly path measured under the browser engine as a variant of the native family (Haas et al. 2017). The hypothesis under test is that the ratio is close to unity. Since the lowerings emit ordinary bytecode, ordinary JavaScript, and ordinary machine code rather than interposing an interpretation layer, deviation from unity localizes to identifiable runtime services (anchor tracking, gradual-type checks) rather than to a diffuse tax. Where the measured ratio departs from unity we will report the profile, not the excuse. The memory-continuum family extends the same question along the ownership dial of Chapter 5: kernels drawn from the repository’s differential suite run under each emission regime, with identical output asserted across all of them, so one family simultaneously tests the claim that memory strategy is not semantics and measures what each step of the continuum buys.

Table 16.3: The performance benchmark suite: families, baselines, and primary metrics. Results are pending and will be reported against the protocol fixed here.

Family	Baseline	Primary metric	Result
Topology microbenchmarks: node and edge creation, neighborhood filtering, walker traversal over chain, star, random, and scale-free graphs	Hand-written Python object graphs; a property-graph store queried via Gremlin and Cypher	Operations per second; per-visit overhead	<i>[to be measured]</i>
Substrate lowering overheads: a fixed kernel set lowered to bytecode, JavaScript, and native code	Idiomatic Python, JavaScript, and C implementations of the same kernels	Runtime ratio, lowered versus native-authored	<i>[to be measured]</i>
Memory continuum: allocation-heavy kernels (trees, symbolic differentiation, an interpreter loop) compiled under the collector-backed, counting-only, and zero-RC regimes	The same kernels under the collector-backed default, with output identity asserted across regimes	Runtime and peak-memory ratio per regime	<i>[to be measured]</i>
Topology lifetime: build-traverse-discard services, walker-spawn reclamation, and subgraph transfer under regions	The same source with region annotations mechanically erased	Time, peak RSS, tail latency, drop counts, per-transfer cost	measured (Section 16.3.1)
Persistence overhead: identical traversals over transient and persistent roots	Transient execution of the same program	Slowdown factor; write amplification	<i>[to be measured]</i>
Service projection: littleX walker endpoints under synthetic load	A FastAPI, SQLAlchemy, and PostgreSQL reimplementation of the same API	Requests per second; p50 and p99 latency	<i>[to be measured]</i>
Scale-out: replica sweep of the deployed service	The same program, single node	Throughput versus replica count	<i>[to be measured]</i>

Table 16.4: The topology-lifetime family. Bare is the mechanically erased baseline under reference counting; medians of ten invocations, digest identity green in every cell. The bare footprint rows grow without bound while the region rows plateau.

Metric	Bare	Region	Ratio
Build-discard time (acyclic)	117.6 ms	19.2 ms	6.1×
Build-discard time (cyclic)	101.8 ms	20.7 ms	4.9×
Peak RSS, 500 requests	48.1 MB	6.1 MB	7.8×
Peak RSS, 2,000 requests	177 MB	19.5 MB	9.1×
p99 request latency (acyclic)	1.36 ms	100 μ s	13.6×
p99 request latency (cyclic)	2.31 ms	111 μ s	20.8×
Drop hooks fired, of 100,000	0	100,000	—
Walker spawns reclaimed, of 20,000	0	20,000	—
Growth-rule branch, per allocation	23 ns	24 ns	0.96×

Table 16.5: Cost to transfer one subgraph across a concurrency boundary, in nanoseconds per transfer. The region move stays within 26 ns to 1.0 μ s across four orders of magnitude of subgraph size while both alternatives grow with the subgraph.

Objects	Move	Deep copy	Serialize
10 ²	26	31,478	61,385
10 ³	41	199,417	381,007
10 ⁴	123	1,967,105	3,928,931
10 ⁵	388	30,887,548	51,098,661
10 ⁶	1,020	788,736,844	628,595,263

16.3.1 THE TOPOLOGY-LIFETIME FAMILY, MEASURED

The family measuring Chapter 11 has completed its runs, and we report it in full. The protocol is an intra-language ablation with a mechanical control: each region kernel’s baseline is the same source with the handle and open lines deleted by an erasure tool, so the diff isolates the region machinery and nothing else. Four kernels cover the chapter’s claims: a build-traverse-discard service (two hundred nodes per request, five hundred requests, in acyclic and cyclic configurations), a walker-spawn service whose twenty thousand walkers each carry a drop-managed resource, a subgraph-transfer kernel comparing a region move against deep copy and serialization from 10² to 10⁶ objects, and a microbenchmark of the growth rule’s placement branch over two million allocations. Every cell is the median of ten invocations on one machine (a 64-core x86-64 host running Linux), and no cell counts unless every variant of its configuration produces a byte-identical output digest: the differential identity oracle of Chapter 5, extended over the backend, collector-mode, and transfer-strategy matrix. Two disclosures bound the claims. The measurement kernel reports peak memory at 1 MiB granularity, and all reported cells sit well above that floor. And the baselines are intra-language: cross-language ports (a Rust arena idiom, say) are excluded because their faithfulness cannot be established on equal terms, so the family prices the region machinery against the language’s own managed floor, not against other languages.

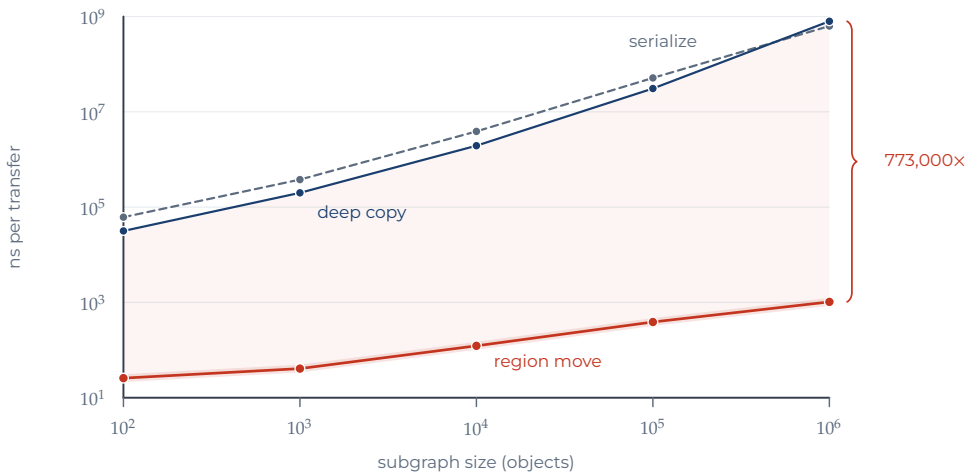


Figure 16.2: The transfer sweep of Table 16.5, drawn. Both axes are logarithmic. Deep copy and serialization grow with the subgraph across four orders of magnitude of size; the region move stays within 26 ns to 1.0 μ s, because only the handle crosses the boundary, and the gap at a million objects is a factor of 773,000.



Figure 16.3: Footprint against service lifetime for the build-traverse-discard service. The erased baseline leaks 65 KB per request without bound, because discarded managed topology is pinned immortal; the region variant’s footprint plateaus, reaching only 21.5 MB even at twice the plotted range. The gap is not a constant factor but a divergence.

Table 16.4 carries the service-shaped results, Table 16.5 the transfer sweep, and Figures 16.2 and 16.3 draw the two shapes the tables can only tabulate: the flat move under the growing alternatives, and the leak diverging from the plateau. Three readings deserve emphasis. First, the baseline rows are a measurement of the native substrate’s deliberate design, not a strawman: managed topology is pinned immortal (Chapter 11), so the bare service leaks 65 KB per request forever, and the comparison is reclamation against leak-until-exit, which is what the paradigm offered before regions. Second, the collector-mode choice moves every number by under 13 percent, confirming that collector strategy is irrelevant when the topology itself is never collected. Third, the transfer table’s move column drifts from 26 to 1,020 ns across a 10,000× size range not because any per-object work occurs but because of cache pressure on the hand-off path after building ever-larger subgraphs; both alternatives grow with the subgraph, to a 773,000× gap at a million objects. The family also repaid its construction cost in kind: its full-size runs exposed the native topology store’s original quadratic scan and drove the redesign recorded in Chapter 13, which is the protocol-as-instrument lesson of Section 16.4 arriving early.

The persistence and service families measure the cost of the dissolved discontinuities of Chapters 10 and 14. Persistence by reachability trades an explicit I/O layer for runtime bookkeeping on graph mutation. The suite measures the slowdown of identical traversals over transient versus persistent roots and the write amplification per mutated anchor, against the background of the orthogonal-persistence tradition whose engineering lessons the design inherits (Atkinson and Morrison 1995). Service projection is measured end to end: the littleX walker surface under synthetic load, against a FastAPI reimplementations that we intend to publish as part of the artifact so that its quality can be audited and improved by parties with no stake in our conclusion. The scale-out family sweeps replica counts under the orchestrated deployment path (Chapter 14), which provisions onto Kubernetes-managed infrastructure of the lineage described by Burns et al. (2016). The quantities of interest are throughput scaling and the latency cost of crossing from the single-machine to the distributed configuration, the boundary that, as Waldo et al. (1994) insist and Chapter 14 concedes, no transparency can lawfully abolish. The expected shape is sublinear-but-monotone throughput scaling with a measurable isolation overhead per user root. The measured curve is *[to be measured]*.

16.4 Developer Studies: Time to First Application and Defect Rates

The claims most central to this book’s economics (that removing discontinuities removes the dominant accidental cost of application construction) are, in the end, claims about developers, and no repository inventory can settle them. In this section, we report the design of three instruments, two human-subject and one mechanical, and we are plain about their status: at the time of writing they are designed, not executed. Publishing the protocol before the results is a deliberate constraint on ourselves.

The first instrument measures time to first application. The design is between-subjects: participants with matched web-development experience, drawn from professional developers and upper-level students, are randomly assigned to Jac or

to the canonical stack of Section 16.1, given equivalent curated documentation, and asked to build a specified multi-user micro-application: accounts, posts, follows, a feed, persistence across restarts, and a deployed endpoint, a deliberate subset of littleX. Success is defined by a fixed acceptance suite run against the deployed application, so that "done" is not self-reported. The response variable is wall-clock time to acceptance, right-censored at a fixed budget and analyzed with standard survival methods. The toolchain is instrumented on both arms to log phase transitions (first successful run, first persisted datum, first passing cross-tier call, first deployed endpoint) because the distribution of time across phases is more diagnostic than the total: the synechic hypothesis predicts specifically that the boundary phases shrink.

The second instrument measures defect rates, in two settings. In the controlled setting, the acceptance suite's failure log is coded by site of origin using the glue taxonomy of Chapter 1: marshaling errors, drift between parallel schema definitions, authentication and session faults, and build or deployment misconfiguration, as against in-tier logic errors. The hypothesis is falsifiable and specific: the discontinuity-origin defect classes shrink on the Jac arm, while in-tier logic defect rates are statistically indistinguishable. The language claims to remove discontinuities, not to improve its users' reasoning. In the field setting, issue trackers of production Jac applications are mined and defect classes compared against matched open-source applications on conventional stacks. This arm is observational and correspondingly weaker, and we will report it as such.

The third instrument measures the reach of verification directly, and it needs no human subjects. The design operationalizes the refactoring argument of Chapter 1: in each arm's completed application, a scripted change is applied to the central record (rename one field, change one field's type, delete one field), and the arm's own toolchain is then run to exhaustion. The response variables are counts: how many of the sites that must change are flagged by a compiler or checker, how many are reachable only by textual search, and how many stale sites remain silent after the arm's tooling reports clean. The synechic prediction is specific and falsifiable: on the Jac arm the flagged count equals the must-change count (the rename of the once-declared record surfaces every stale use, across tiers, as a build error), while on the conventional arm the checked fraction is bounded by each tool's jurisdiction and the remainder lands in the coherence obligations κ that the metric of Chapter 2 counts. This instrument is the cheapest of the three to run and the most direct, because it measures neither speed nor skill but jurisdiction itself: the property, argued in Chapter 19, on which the economics of machine authorship turn. Its cells too are *[to be measured]*.

Until these studies run, the evidentiary base is thinner: a decade of production deployments whose longitudinal character Chapter 15 recounts, the published experience of the Jaseci stack's earlier generation in building scale-out products with small teams (Mars et al. 2023), and, for the meaning-typed slice of the language, the controlled comparisons of meaning-typed programs against prompt-engineering frameworks reported by Dantanarayana et al. (2025) in the study that accompanies Chapter 6. Every cell of the developer-study results is, at present, *[to be measured]*, and we would rather print that phrase than a number we cannot defend.

16.5 Threats to Validity

Selection. littleX was written by people fluent in Jac and sympathetic to its idioms, and it was selected as an evaluation object in part because a social graph is a natural fit for topological expression. An application chosen adversarially (a numerical pipeline, a compiler, a spreadsheet) would exercise the object-spatial constructs less and flatter the language less. The canonical-stack column of Table 16.2 is likewise our selection: we chose the composition we believe is most widely taught and deployed, but a team using an end-to-end TypeScript stack with a schema-sharing layer would present a smaller (though still nonzero) glue surface, and our enumeration does not represent them.

Construct. Line counts and artifact counts are proxies for a cognitive quantity, the burden of maintaining boundary machinery, and the mapping from text to burden is not linear. Expressiveness and its costs have resisted formal quantification for as long as the question has been posed seriously (Felleisen 1991), and counting files cannot capture, for instance, the difficulty of debugging a dissolved boundary when it misbehaves, a cost Chapter 17 treats as real. Raw `wc -l` counts include comments and scaffolded code. Both conventions were chosen to err against our thesis, but they remain conventions.

Author-designed-benchmark bias. We name this explicitly because it is the deepest threat in the chapter: the performance suite of Section 16.3 was designed by the same people who designed the language it evaluates, and benchmark design is a high-dimensional space in which flattering choices can hide even from their makers. Traversal-shaped workloads are where walkers shine. Our mitigations are the inclusion of workloads chosen because the language should lose them (bulk analytics in the style of Malewicz et al. (2010), relational aggregation of the kind the relational model was built for (Codd 1970)), publication of the complete harness and of the competing FastAPI implementation for third-party audit and extension, and the protocol-before-results ordering of Section 16.4. Mitigation is not elimination. Independent replication is, and Appendix E exists to lower its cost.

External validity. One application, in one domain, written by experts, generalizes poorly by construction. The developer studies draw from populations (students, professionals recruited for a study) that may not resemble the teams that matter. And the canonical stack is a moving target that will have shifted by the time these words are typeset. Two asymmetries deserve explicit note because they cut in opposite directions. The ecosystem-maturity confound biases against Jac: participants on the conventional arm benefit from decades of accumulated documentation, tooling polish, and searchable folklore that no young language possesses, so a time-to-first-application victory for Jac would understate the language-design effect while a loss would overstate its absence. Novelty and experimenter effects bias the other way: participants aware they are using the studied artifact, taught from materials its authors curated, may perform unrepresentatively well. The design controls what it can (equivalent curated documentation on both arms, acceptance defined mechanically), and the residue is reported here.

None of these threats, we believe, touches the one measurement this chapter reports as final: the artifact inventory of Section 16.1 is an exhaustive count of a public

repository, and its central value is zero. The reader is invited to run `wc` and disagree.

Bibliographic Notes

The evaluation questions of this chapter inherit their framing from the multitier programming literature. The survey by Weisenburger et al. (2020) catalogs both the glue artifacts that tierless languages exist to remove and the persistent difficulty of evaluating that removal quantitatively, and Cooper et al. (2006) is the classic statement of the tierless goal against which Section 16.1's counts can be read. The distinction between essential and accidental effort that underlies our construct choices is owed to Brooks (1987), and the formal elusiveness of expressiveness claims to Felleisen (1991). On the performance side, the traversal baselines are drawn from the graph-processing and graph-query traditions (Malewicz et al. (2010) for bulk-synchronous analytics, Rodriguez (2015) and Francis et al. (2018) for traversal and query languages over property graphs), while the lowering methodology leans on the compilation infrastructure literature (Haas et al. 2017; Lattner and Adve 2004) and the persistence family on the orthogonal-persistence tradition (Atkinson and Morrison 1995). The scale-out protocol is shaped by the orchestration lineage recounted in Burns et al. (2016) and by the enduring caution of Waldo et al. (1994) that distribution's boundaries are physical before they are linguistic.

The evidentiary base specific to the Jac lineage is drawn from three works. Mars et al. (2023) reports production experience with the stack's earlier generation. Mars (2025) states the object-spatial model this chapter's microbenchmarks measure. And Dantanarayana et al. (2025) provides the controlled evaluation of meaning-typed programming that stands as the template for the kind of study Section 16.4 commits the rest of the language to. The artifact guide in Appendix E specifies the pinned revisions, harnesses, and hardware under which every placeholder in this chapter is to be discharged.

OBJECTIONS AND LIMITS

There are two ways of constructing a software design: One way is to make it so simple that there are obviously no deficiencies, and the other way is to make it so complicated that there are no obvious deficiencies.

C. A. R. Hoare, 1981

A book that has argued for continuity through every chapter before this one owes its reader the strongest available case against it. Hoare's dichotomy names the danger. The fragmented stack of Chapter 1 is, at the scale of its components, the first kind of design: each component is small enough to have obviously no deficiencies, and the complication hides in the glue between them. A synechic language makes the opposite wager. It gathers compiler, runtime, persistence, client framework, and deployer into one medium, and a fair critic will ask whether we have built the second kind of design: a system so encompassing that its deficiencies are no longer obvious. This chapter is our attempt to make them obvious.

We proceed by steelmanning. Each objection is stated in the strongest form we know how to give it, as an informed critic would argue it, before we respond. Where the response is complete, we say so. Where it is partial, we concede the remainder. The discipline is the one Brooks (1987) taught the field: to separate what a technology essentially cannot do from what it accidentally does not yet do, and to claim credit for neither. Some of the objections below name accidents we expect engineering to retire. At least two name essential trade-offs that any language in this class must carry permanently. The reader deciding whether to build on Jac deserves to know which are which.

17.1 The Lock-In Objection: What It Costs to Leave

The lock-in objection holds that a synechic language concentrates into a single adoption decision the risk that the fragmented stack keeps diversified across replaceable vendors.

Its strongest form begins from an observation this book has treated as a cost and reframes it as an asset: the discontinuities of the conventional stack are also its exits. An application assembled from a frontend framework, a web server, a relational database, and a queue is coupled to each of these through narrow, commodity interfaces (HTTP, SQL, POSIX) behind which stand multiple interchangeable implementations. Each component can be replaced without abandoning the others. The discontinuities function as an insurance market: the price of glue, paid continuously, buys the option to leave any single vendor at bounded cost. A synechic language cancels the policy. Adopting Jac is one correlated bet on a language, a compiler, a runtime, a persistence model, and a deployment story at once. If the language stagnates or its steward fails, the whole application is stranded together, not one tier of it. The multitier literature has long identified this concentration of risk as a barrier to adoption (Weisenburger et al. 2020), and the industry’s memory of integrated platforms (fourth-generation languages, image-based environments in the Smalltalk tradition (Kay 1993)) is a memory of systems that were pleasant to enter and expensive to exit. Hiring compounds the bet: a team can recruit against React and Postgres from a deep labor pool, but it cannot yet recruit against walkers.

Our answer has two parts, and then a concession. The first part is that substrate transparency cuts in both directions. A language whose programs entered a proprietary virtual machine would deserve the objection in full, but Jac compiles *into* its host substrates rather than beside them: the managed lowering is ordinary Python bytecode, the web lowering is ordinary JavaScript, and the native lowering is ordinary object code (Chapter 12). There is no foreign runtime format inside which the program’s value is trapped. The artifacts a Jac program leaves behind are artifacts of the largest ecosystems in the industry.

The second part is that the toolchain ships a maintained exit. The source projection of the build command, `jac build -as source`, materializes, from a full-stack project, the conventional application that the language had been suppressing: a backend of plain Python served by FastAPI, with one route per server walker gated by each walker’s declared visibility; a frontend of compiled JavaScript with an ordinary `package.json` and Vite configuration; a `requirements.txt` that names stock PyPI packages and pins nothing Jac-specific; and build scaffolding (a Makefile, a README, environment files), so that the exit is an operable project. No `.jac` file survives in the output. The runtime library the compiled modules import is vendored into the output as ordinary Python source, stripped of the compiler and command-line machinery, and becomes the departing team’s property. The repository treats this exit as a contract: the language’s test suite asserts, among other properties, that every emitted Python file parses, that the ejected tree contains zero Jac sources, and that the dependency manifest carries no pin on the language’s own packages. The shape of the result appears in Listing 17.1.

```
ejected/
  backend/
    main.py      FastAPI app: a route per server walker,
                  gated by declared visibility
    server.py    the compiled module, ordinary Python
    requirements.txt stock PyPI packages, no Jac-specific pins
    jaclang/     vendored runtime library (no compiler, no CLI)
  frontend/
```

package.json	the project's npm dependencies
vite.config.js	dev-server proxy wiring to the backend
src/	client pages compiled to JavaScript
Makefile	install, dev, and build targets
README.md	

Listing 17.1: The ejected output for a small full-stack project: a conventional two-tier application with no Jac sources remaining.

Ejection thus converts the lock-in objection from a claim about captivity into a claim about price, and the price is a revealing one: what a departing team receives is the fragmented stack itself (two projects, two dependency manifests, a proxy configuration, an authentication layer made visible) that the language had absorbed. What made the conventional stack's crossings exits was never the discontinuity itself but the narrow, commodity boundary at each one, and ejection materializes those boundaries on demand. The cost of leaving a synechic language is to resume paying for the discontinuities everyone else pays for continuously.

The concession is the asymmetry of topological code. Ejection removes the toolchain and the surface syntax. It does not remove the paradigm. The ejected walkers still execute as walkers on the vendored runtime. Persistence is still by reachability. The program's shape remains object-spatial. A team that wants not merely to leave the toolchain but to leave OSP (to arrive at idiomatic Python over an ORM and a hand-written router) faces a genuine rewrite, because a program organized around traversal does not mechanically translate into a program organized around queries. This is paradigm adhesion rather than format captivity, and it is the same species of cost incurred by leaving Erlang's supervision trees (Armstrong 2003) or any other paradigm-bearing language. What is distinctive here is breadth: because the paradigm reaches through more of the stack, adhesion reaches through more of the stack. We state the gradient plainly. Leaving the toolchain is cheap and mechanized. Leaving the ecosystem was never necessary. Leaving the paradigm is a rewrite, and the more deeply a program commits to topological expression, the more its ejected form is a runtime the team now maintains rather than a codebase they would have written.

17.2 Debugging Dissolved Boundaries

Boundaries are where engineers attach their instruments. An HTTP boundary yields a request log, a replayable payload, and a place to interpose a proxy. A SQL boundary yields a query log and a plan explainer. A process boundary yields the operating system's entire observability apparatus. The misery of the fragmented stack is also its glass: every boundary the programmer pays for is a vantage point with decades of accumulated tooling. The objection follows directly. When the language dissolves the boundaries, it dissolves the vantage points, and it does so while adding machinery the programmer never wrote: the API projection is synthesized from walker declarations (Chapter 14), the persistence layer is implied by reachability (Chapter 10), the partitioning is the runtime's decision. When one of these misbehaves, there is no artifact of the programmer's own authorship to read. Worse, a single logical operation now spans substrates that do not share a call stack: a click handler

in a compiled client page, an ability running on the server, a delegated call into a language model. The critic asks, reasonably, what a stack trace even means in such a system, and observes that a programmer in the conventional stack can always drop down to the boundary and read the wire, while a programmer beneath a dissolved boundary has nowhere lower to stand.

The first part of our answer is a distinction the book's vocabulary makes exact: what an instrument needs is a boundary (a defined cut at which behavior can be observed), not a discontinuity. The conventional stack observes at its discontinuities not because they are good vantage points but because they are the only places where the system's behavior is forced through a narrow, textual representation that a tool can tap. The tap shows the engineer a serialized payload (the very representation the discontinuity demanded), not what the application was doing. A synechic runtime can instead draw observation boundaries along the program's own semantics: walker spawn, node arrival, ability activation, report. A traversal trace (which walker visited which nodes, which abilities fired, in what order, with what reports) is an account of an execution in the program's own vocabulary, and for the class of applications this book targets it is a strictly more legible object than an interleaved pile of HTTP lines and SQL text.

The second part of our answer is concrete and present in the implementation. On the managed lowering, the compiler stamps every emitted Python AST node with the source coordinates of the originating Jac construct, so a runtime exception surfaces with the .jac file and line of the program the author wrote, not a location in generated code. Across the distributed cut, the scale runtime speaks the standard observability grammar: it propagates W3C trace-context identifiers through the request path and opens server-side spans, so a client action and the walker work it triggers are correlated in the same way the rest of the industry correlates its microservices, with the difference that the correlation is the runtime's obligation, not each application's.

The third part is the concession, and it is substantial. For the debugging claim to hold in full, the tooling floor must include: source-coordinate fidelity on all three lowerings, not only the managed one; traversal-level tracing as a first-class, always-available runtime service, not a deployment feature; in-place inspection of the persistent topology, so that an engineer can query the live graph as easily as they once ran `psql`; and cross-substrate correlation surfaced in the everyday developer workflow, not only in production telemetry. Parts of this list are shipped fact and parts are engineering debt, and we decline to blur the two. More permanently, dissolving boundaries raises the floor of obligatory tooling forever: a conventional stack outsources much of its observability to the native tools of its discontinuities, while a synechic runtime must carry its own instruments for as long as it lives, because its programmers have nowhere else to stand. We accept that tax as a condition of the model, and we note where the bill falls due: the day a production incident forces an engineer beneath the medium is the day the continuity claim is genuinely tested, and no chapter of theory substitutes for the runtime meeting them there with adequate glass.

17.3 The Leaky-Abstraction Objection, Steelmanned

The objection has a folk form and a sharp form. The folk form is Spolsky (2002): all non-trivial abstractions leak. An abstraction saves the time of working but not the time of learning, because on the day it fails the programmer must understand the machinery it hid, and the taller the abstraction, the further the fall. The sharp form is Waldo et al. (1994): latency, memory access, partial failure, and concurrency make distributed computing *essentially* different from local computing, not incidentally different. Four decades of systems that unified the two in expression, from the founding promise of RPC (Birrell and Nelson 1984) through its object-oriented apotheosis in CORBA (Vinoski 1997), produced programs whose failure modes their authors could not reason about. On this reading, a synechic language is the most ambitious act of pretending yet attempted: it declares continuous not only the network but the database, the build system, the browser, and, with the meaning-typed constructs of Chapter 6, a stochastic model. The law of leaky abstractions predicts leaks in proportion.

Our answer turns on a distinction the objection's own history supports: uniformity of *expression* is not uniformity of *behavior*. The systems Waldo indicted promised the second (a remote call indistinguishable from a local call in cost and failure mode), and they deserved the indictment. Substrate transparency, as defined in Chapter 2, promises only the first, and pairs it with the lawful-boundaries principle developed there: the physical differences between substrates are not hidden but surfaced *as semantics*. Latency appears as the language's concurrency and asynchrony constructs rather than as a stall inside an innocent-looking call. The client-server cut is written into the program text, in module roles and in the declared visibility of walkers, so production is never the first to reveal it. Partial failure arrives as values and exceptions the program is obliged to handle. Delegation to a language model is wrapped in the typed contracts, validation, and retry semantics of Chapter 6, because the stochastic substrate is the leakiest of all. It is worth recording that this is not a rebuttal of Waldo's paper but a compliance with it: the authors' conclusion was that the difference between local and remote must be made visible in the interface. A synechic language agrees, and relocates the visibility from the plumbing, where it manifests as marshaling code and configuration, into the language, where it manifests as meaning.

Two concessions are owed. The first is the base rate. Distribution transparency has failed repeatedly and expensively, the practitioners who lived through CORBA are rational to price that history into any new claimant, and a language cannot argue its way out of a reference class. It can only measure its way out, which is the burden Chapter 15 and Chapter 16 take up. The second concession is that Spolsky's deeper point survives our distinction intact. When the medium does leak (a lowering emits pathological code, a traversal saturates a shard, a model returns confident nonsense that passes a weak contract), the machinery the programmer must then understand is large: a multi-target compiler, three substrates, a distributed runtime. The total system beneath a Jac program exceeds the total system beneath a single-tier script, and leaks tax the total. The mitigations are architectural: a small semantic core that fits in a calculus (Chapter 8), and a legible lowering, for which ejection doubles as an inspection tool, since a skeptical engineer can read the ordinary Python their program would become. But the summary of this section is a trade, knowingly

taken: synechism reduces the everyday frequency of boundary errors by removing hand-written boundaries, in exchange for concentrating rarer failures in a deeper stack.

17.4 Performance Ceilings and the Sufficiently Smart Runtime

The objection descends from Wirth (1995): software grows fatter faster than hardware grows faster, and layered convenience is the mechanism. A language that internalizes the toolchain, the runtime, the persistence engine, and the deployer is convenience raised to a system, paid for out of the user's cycle budget. Its modern edge is the "sufficiently smart compiler" tradition of skepticism, transposed to the runtime. Scale invariance (Chapter 14) promises that the runtime will make placement, partitioning, and caching decisions that a distributed-systems engineer would otherwise make by hand. The skeptic replies that the engineer's decisions encode knowledge the runtime cannot infer: which consistency can be slackened, where the data has gravity, which failure the business can absorb, what the cost envelope is. A hand-partitioned system writes that knowledge into the boundaries it draws. A single system image must be conservative wherever it is ignorant, and conservatism has a price. The objection therefore has two prongs: a constant-factor tax at the median (graph indirection, ability dispatch, persistence bookkeeping) and a ceiling at the tail, where performance is the product and the single system image loses to designs whose every boundary was placed by a profile.

We answer the two prongs differently. To the median, our claim is economic rather than absolute. The conventional stack is not free at runtime either: it serializes and deserializes at every boundary it forces the programmer to write, and it pays those marshaling costs on every request, tuned by no one. The overwhelming majority of applications live their entire lives in the regime where discontinuity costs dominate and hand-partitioning never pays for its own engineering: the same observation that underwrites the economic case for serverless platforms (Jonas et al. 2019). For that regime, the relevant comparison is not Jac against a hand-tuned system but Jac against the untuned fragmented stack the team would actually have built, and the constant factors on each side must be measured. Chapter 16 reports the measurements and the methodology. To the tail, we concede the ceiling almost entirely. When a workload's economics justify a dedicated team per boundary, hand-partitioned designs will beat a single system image, and the largest systems in the world are hand-partitioned for reasons that are not inertia. What the language offers at the boundary between the regimes is a gradient, not a cliff: the native lowering exists so that a hot kernel can leave the managed substrate for LLVM-compiled code without leaving the program, and the deployment machinery accepts progressively more explicit direction as a team learns where its profile disagrees with the runtime's defaults. A team that outgrows the medium everywhere at once was never this book's audience. A team that outgrows it in one function should not have to eject to say so.

One further concession is owed on the skeptic's strongest ground. The automatic-distribution component of scale invariance (sharding a live topology, migrating walkers across machines) is the youngest machinery described in this book, and it is the rightful target of the sufficiently-smart-runtime critique. Our position is that the

semantics are fixed and the policies will improve beneath them, and we are aware that this is the shape of promise the field has learned to discount. The correct response to that discount is longitudinal measurement, not rhetoric, and we have tried to structure the evaluation of Chapter 16 so that the promise is checkable.

17.5 When Not to Use Jac

A design argument earns trust by naming its own complement. The preceding sections answered objections. This one concedes territory outright. There remain classes of systems for which Jac is the wrong choice because the mismatch is essential rather than accidental, and the list, drawn strictly, is shorter than convention would predict.

- *Entrenched polyglot infrastructure.* The continuity dividend accrues to whole applications. An organization whose discontinuities already work, whose teams mirror those discontinuities, and whose services span the JVM, Go, and a decade of operational tooling will find that introducing Jac as one more component *adds* a discontinuity instead of removing one: the partial-continuity trap of Appendix F, entered from the opposite direction. The language’s containment of Python (Chapter 3) gives Python-centric teams a genuine incremental path, but for a heterogeneous estate the advice is to adopt at the granularity of a whole product or not at all.
- *Programs that will never grow.* A one-file script, run by one person, holding no state, gains almost nothing from scale invariance. The cost of writing it in Jac is near zero. So is the payoff.

Three categories that convention would place on this list do not belong on it, and none of the omissions is casual. The first is hard-real-time and safety-critical control. The reflex is to exclude any language with a garbage collector, and the reflex is right about the managed substrate and wrong about the language. A control kernel written as an enforced native module compiles headerless (Chapter 5): no collector, no reference counting, every allocation and release placed at a statically proven point, destruction deterministic, and the absence of memory-management machinery checked in the emitted artifact rather than claimed. That is the memory discipline a hard deadline demands, obtained without leaving the language. What stands between such a kernel and a certified deployment is qualification tooling (worst-case-execution-time analysis, certified builds), and that is an ecosystem’s maturity, not a property of the design.

The second is the tiny embedded target. The runtime of Chapter 13 assumes server-class resources, but the native tail does not carry it. A headerless module lowers through the embedded LLVM to a dependency-free static executable measured in kilobytes (the kernel binary of Section 16.2 is 4,448 bytes), with no collector and no runtime support beyond what the program itself uses. The single system image ends where the devices are too small to be citizens of it; the language does not.

The third is the workload with no topology. Nothing in the language conscripts a program into Object-Spatial Programming. The containment of Chapter 3 means a dense numerical kernel, a stream transformation, or a stateless batch pipeline is

written in the traditional idioms, and written in one language that still supplies the toolchain of Chapter 4, the type discipline of Chapter 9, the native tail of Chapter 12, and the ownership dial where the profile demands it. Where there is no graph, walkers are simply not used, and the rest of the language does not leave with them.

The pattern across the list is the book's thesis read in the negative. Jac pays where the cost of a system is dominated by its boundaries, and pays again where its data has persistent shape. It does not pay where the existing discontinuities are working organizational assets, and it cannot pay where the program is too small to have boundaries at all. We believe the first description covers a large and growing fraction of the software actually being written (Chapter 18 argues that the fraction is the interesting design space), but it is a fraction, and this chapter is where we say so in print.

Bibliographic Notes

The chapter's epigraph is from Hoare's Turing Award lecture (Hoare 1981), whose account of ambition and failure in language design remains the standard against which a language book should test its own claims. Brooks (1987) supplies the essence-versus-accident discipline our concessions try to follow. The leaky-abstraction objection is stated in its folk form by Spolsky (2002) and in its sharp form by Waldo et al. (1994), which we regard as the most important critique any transparency claim must answer. The promise it indicts begins with Birrell and Nelson (1984), and its canonical industrial casualty is chronicled by Vinoski (1997). Readers weighing the lock-in objection will find the adoption barriers of unified programming models surveyed by Weisenburger et al. (2020), a history of the pleasures and adhesions of image-based environments in Kay (1993), and an instructive parallel for paradigm adhesion in Armstrong's account of Erlang (Armstrong 2003), a language whose exit costs are inseparable from its virtues. The performance critique descends from Wirth (1995). The economic answer we give it, that median workloads are dominated by boundary costs rather than tuning opportunities, parallels the argument made for serverless platforms by Jonas et al. (2019). The evidentiary burden this chapter repeatedly defers is taken up in Chapter 15 and Chapter 16.

PART VI



THE LANGUAGE CLASSES

A TAXONOMY OF SYNECHIC AND TOPOKINETIC LANGUAGES

To this point the book has argued from one artifact. Parts II and III defined two classes of language. A language belongs to the synechic class when it presents one continuous semantic medium across substrates (Chapter 2). A language belongs to the topokinetic class when it makes the locus of computation a first-class mobile construct over a persistent topology (Chapter 7). Parts IV and V showed that one production language is a member of both classes, and reported what happened when it was. But a class demonstrated by a single member is, so far, only a fact about that member. It becomes a fact about language design under three conditions: 1) each class's defining property is stated in a form that other languages can satisfy or fail, 2) the machinery membership requires is separated from the accidents of the first member, and 3) the questions the first member could not answer are written down for those who come after. That generalization is this chapter's work.

In this chapter we proceed in four steps. First we treat the two properties as axes of a design space and place the systems of the historical record, and Jac itself, at defensible coordinates on the resulting plane. Second we extract from the engineering record of Parts II through IV the machinery that proved essential, and advance it as membership conditions for the two classes. Third we separate what in Jac's design is essential to the classes from what is contingent history, as design principles for successor languages. Fourth we state the open problems (formal, systems, and human) that one artifact and one book cannot close. A taxonomy is also an invitation: both classes are larger than their one common member, and this chapter is addressed to whoever builds the next member of either.

18.1 The Two Axes as a Design Space

In this section, we read the two properties as axes and locate systems along them. Chapter 2 and Chapter 7 gave the definitions as properties a language has or lacks.

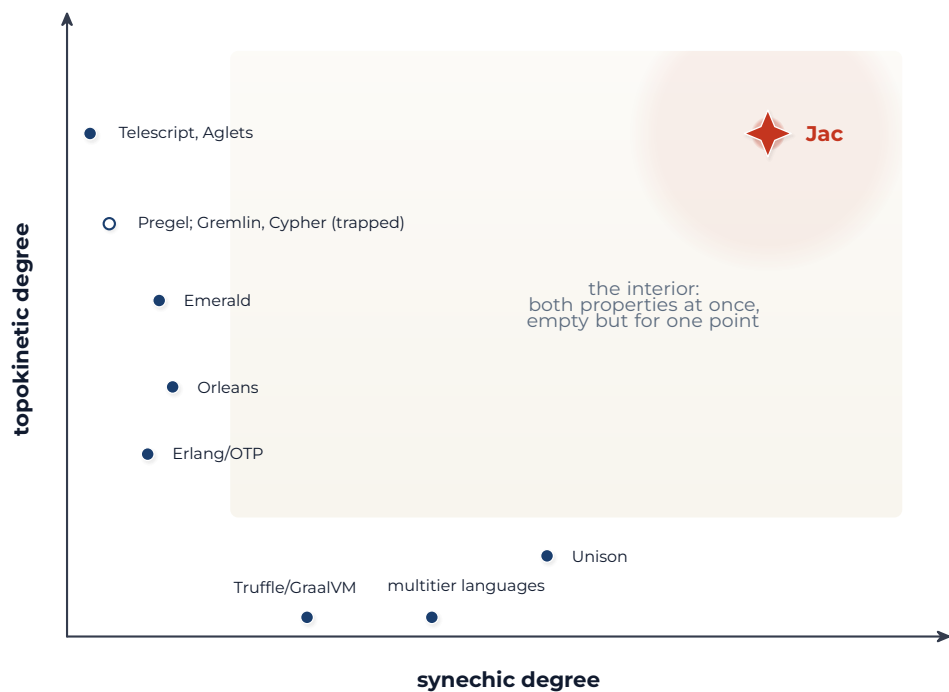


Figure 18.1: The design space read as a plane, with the coordinates of Table 18.1. Every prior system sits on a margin: real synechic degree with no motion along the bottom, real motion with no medium up the side, and the trapped traversal systems confined behind a component boundary. Degrees are qualitative; the argument for each placement is in the text.

For taxonomy we read them instead as axes along which a system sits at a degree. The synechic axis measures how much of the substrate territory a single semantic medium spans: the tiers of an application, the foreign ecosystems it draws on, the toolchain that builds and ships it, the deployment infrastructure it runs on, the memory disciplines it renders as one continuum (Chapter 5), and, after Chapter 6, the stochastic executors it delegates to. This measure is not a matter of taste. The glue cost model of Chapter 2 makes it quantitative, as the boundary-crossing burden that remains at each program point. The topokinetic axis measures the extent to which the locus of computation is a semantic construct. It runs from none, where loci are fixed and data streams to them, through *placeable* loci, which a runtime may situate anywhere but which stand still once situated, to fully mobile loci that move through a durable topology of data under program control, with computation triggered by arrival.

The first fact about the plane is that its axes are independent, and the historical record proves it, because the record’s systems occupy the margins. The multitier languages sit at a real synechic degree and a topokinetic degree of zero. Telescript sits at a real topokinetic degree and a synechic degree of zero. No prior system we know of sits in the interior (Figure 18.1). The remainder of this section walks the margins, system by system, and Table 18.1 records the coordinates.

The multitier family (Links, Hop, Ur/Web, Eliom, ScalaLoci) is the record's most direct approach along the synechic axis, and Appendix F has already told its story as history (Chlipala 2015; Cooper et al. 2006; Radanne et al. 2016; Serrano et al. 2006; Weisenburger et al. 2018). Read as coordinates, the family achieved partial synechism of a precise shape: one semantic medium across the tiers of exactly one application form, the web program, with the foreign ecosystems, the toolchain, and the deployment infrastructure left outside the medium's edge. Its topokinetic degree is zero, and instructively so: the multitier compiler does move code, but once, at compile time, to fixed sites. Each tier is a stationary locus to which data is thereafter delivered. Weisenburger et al. (2020) survey the family, and nothing in the survey's design space includes motion of computation at run time as a semantic category.

GraalVM occupies the synechic axis in the opposite orientation. A synechic language is one surface spanning many substrates. The Truffle framework is the inverse: one substrate hosting many surfaces (Würthinger et al. 2013). Its unification is real, but it happens below the programmer, whose medium remains plural. A polyglot program is still written in several languages, with the substrate smoothing the crossings. We placed this strategy in Appendix F as inversion. In the plane's terms it is genuine synechic degree accrued in a direction that cannot reach the surface where programs mean things, and its topokinetic degree is zero. Read as compiler classes rather than coordinates, the horizontal margin holds the two camps of Chapter 12: research multitier languages that split tiers without ecosystem membership, and industrial multiplatform toolchains (the retired Google Web Toolkit and Volta, the current Kotlin Multiplatform and Dart) that achieved membership without the split (Google 2006, 2025; JetBrains 2025; Manolescu et al. 2008). The polypiler is the conjunction, and no prior system on the margin attempted both.

Unison earns a coordinate no other system holds (Unison Computing 2024). Its continuity is of code identity. Every definition is content-addressed, named by a hash of its structure, so that the codebase becomes an immutable medium in which a definition is the same object on every machine and at every time. Its toolchain, the codebase manager and the absence of builds in the conventional sense, is partially internalized in a way the multitier family never attempted. This is real synechism along a segment of the axis (code identity and toolchain) that Jac itself addresses by different means (Chapter 4). Unison also lets computations be serialized and moved between machines, which gives it a nonzero but incidental topokinetic degree. Relocation is a deployment capability, not a semantics of traversal, and there is no persistent data topology for a locus to move through.

The actor lineage supplies the plane's most instructive near misses, because it is often mistaken for topokinesis. Erlang/OTP made processes location-transparent (a process identifier addresses its process identically whatever node it lives on) and built one of the most reliable distributed platforms in industrial history on that uniformity (Armstrong 2003). But an Erlang process, once spawned, is a fixed site, and everything that moves is a message. This is the von Neumann convention (data streaming to stationary compute) executed superbly at datacenter scale, which is why we score the locus as placeable rather than mobile. Orleans sharpened the placement half: virtual actors are activated wherever the runtime chooses, deactivated, reactivated elsewhere, their state persisted meanwhile (Bykov et al. 2011). Yet the locus is placed

by the runtime rather than moved by the program, data still streams to it, and there is no topology of data as a semantic domain. On the synechic axis both systems score low for a shared reason: each lives inside one ecosystem's border (Erlang on its own virtual machine with foreign code across port and native-interface discontinuities, Orleans as a framework within its host platform), and neither spans tiers, ecosystems, or toolchain.

Emerald is the lineage's genuine mover (Black et al. 2007). Objects relocate between machines under language control, through a construct for moving an object and parameter-passing modes that move arguments with a call, making Emerald the earliest system in our record for which the position of a computation is program-expressible. But motion in Emerald is placement for performance: the object is moved so that subsequent invocations are local, not so that computation traverses a structure. There is no persistent topology, no notion of arrival triggering behavior, and the whole system presumed a homogeneous cluster of one language. Telescript went further along the axis than any predecessor. Travel is a primitive: an agent executes an instruction and resumes on another machine, and agents move among places, a topology the language names (White 1997). Yet the topology is of hosts, not of the program's data. Agents are transient, dying with their journeys. And the whole apparatus lived as middleware above its host systems, confined to a proprietary network whose failure took the idea down with it, as Appendix G recounted. The Aglets system re-embodied the idea as a Java library (Lange and Oshima 1998): motion demoted from semantics to framework, the form in which it faded.

The final margin is the one Appendix G called trapped topokinesis. Pregel asks the programmer to think like a vertex (to write computation from the position of the data, the inversion this book defends) and then executes that inversion as a batch framework behind a system boundary (Malewicz et al. 2010). Gremlin is more striking still: its traversal machine is built of traversers that genuinely move through a graph, a mobile locus in everything but standing (Rodriguez 2015). But the graph lives in a database reached across a driver discontinuity, the traversal language is a guest embedded in host programs, and nothing of the motion is a construct of the language the application is written in. Cypher completes the trajectory: the traversal is declarative, and even the motion is compiled away (Francis et al. 2018). In every case the topokinetic content is real and the class membership is absent: the paradigm is trapped inside a component, and the component sits across the kind of boundary whose cost Part I counted.

Two readings of Table 18.1 matter. The first is that the interior of the plane is empty except for one point. Every prior system hugs a margin, and each is held there by the mechanisms Appendix F named as the partial-continuity trap: boundary displacement, ecosystem exclusion, layer mismatch, scope closure. The margins are not evidence that the interior is uninhabitable. They are evidence that no prior effort aimed at both axes at once. The second reading is a discipline for ourselves. We place Jac farthest along both axes of any system we know, and we do not place it at the axes' ends. The synechic axis terminates, if it terminates, at a medium that admits no visible boundary at all, and Chapter 2 argued that no language should want to reach it: latency, partial failure, and cost must surface, and in Jac they do, as visible properties (Chapter 14). The topokinetic axis extends past current practice into

Table 18.1: A placement of prior systems and Jac on the two axes. Degrees are qualitative; the arguments for each coordinate are given in the text.

System	Synechic degree	Topokinetic degree
Multitier languages (Links, Hop, Ur/Web, Eliom, ScalaLoci)	Partial—one medium across the tiers of a web program; ecosystems and toolchain outside	None—fixed sites; code moves once, at compile time
Polyglot runtimes (Truffle/GraalVM)	Partial, inverted—one substrate below many surfaces; the programmer’s medium stays plural	None
Unison	Partial, orthogonal—content-addressed continuity of code identity; toolchain partly internalized	Incidental—computations relocatable for deployment; no data topology
Erlang/OTP	Low—one language on its own virtual machine; foreign code across port discontinuities	Placeable—location-transparent processes, stationary once spawned
Orleans	Low—a framework inside one host ecosystem	Placeable—virtual actors activated anywhere, state persisted; data streams to the locus
Emerald	Low—one language over a homogeneous cluster	Partial—objects move under program control, as placement; no persistent topology, no arrival semantics
Mobile agents (Telescript, Aglets)	None—middleware above hosts, on closed networks	Substantial—travel as a primitive over places; topology of hosts, not data; agents transient
Vertex-centric and traversal systems (Pregel; Gremlin, Cypher)	None—frameworks and databases across driver discontinuities	Trapped—mobile or vertex-situated computation confined inside a component
Jac	By construction—tiers, ecosystems, toolchain, deployment, memory discipline, and the stochastic substrate; physical boundaries surface lawfully	By construction—first-class mobile loci over a reachability-persistent topology

regimes (planet-scale traversal, fully automatic walker placement) where Chapter 17 conceded open ground. The coordinates “by construction” in the table’s last row mean that the properties hold by the machinery of Parts II through IV, not that either axis has been exhausted.

18.2 Necessary and Sufficient Machinery for Membership

If the classes are to outlive their first common member, membership in each must be decidable by inspection of a language’s design, not by resemblance to Jac. This section extracts from Parts II through IV the machinery that proved essential and states it as six conditions. We advance the necessity of each in the engineer’s sense rather than the logician’s: for each condition we exhibit the discontinuity that reopens without it,

with a system from Section 18.1 as the witness that the reopening is not hypothetical. Sufficiency we claim constructively, and we return to what a formal version would require in Section 18.4.

Principle 18.1 (Containment surface). A synechic language semantically contains an incumbent language of a living ecosystem, so that the ecosystem’s libraries, idioms, and installed knowledge are native expressions of the new medium rather than foreign calls across its edge.

Without containment, the medium’s edge excludes the working world’s software, and every practical program reintroduces glue at that edge. This is the mechanism that confined the multitier family to research use despite demonstrated semantic superiority. Jac discharges the condition by supersetting Python (Chapter 3). The condition itself names only the containment, not the host.

Principle 18.2 (Multi-substrate lowering). The single surface compiles into each substrate it claims—entering each host runtime as a first-class citizen of that runtime—rather than bridging to substrates through interface layers.

Entering and bridging differ in where the discontinuity lives. A bridge is glue relocated below the language, and it resurfaces as the foreign-function ceremonies of Chapter 1. The Java Native Interface is the canonical witness. Entering means the emitted artifact is indistinguishable, to the substrate, from artifacts born there: Python bytecode among bytecode, JavaScript among JavaScript, machine code among machine code (Chapter 12). The substrate’s own ecosystem is then inherited under principle 18.1 rather than annexed in GraalVM’s direction.

Principle 18.3 (Toolchain internalization). Building, dependency resolution across all inherited ecosystems, serving, and deploying are operations of the language artifact itself, not external practice.

The argument of Chapter 4 was that the build system is a substrate boundary in disguise. A language continuous at the semantic layer but assembled and shipped by discontinuous means has hidden a tier, not dissolved it. Every family in Appendix F teaches this condition by omission, and Unison is the partial exception that shows internalization is independently achievable.

Principle 18.4 (First-class mobile locus). The unit of computation is a value of the language whose position in a topology changes under program control, with computation triggered by arrival.

This is the topokinetic axis stated as machinery, and each clause excludes a near miss. Position under *program* control excludes the placeable loci of Erlang and Orleans, where the runtime situates a stationary site. Arrival-triggered computation (the ability dispatch of Chapter 7, formalized in Chapter 8) excludes Emerald, whose objects move without any semantics of what happens when they get there. And position *in a topology of data* excludes Telescript, whose agents traveled a topology of hosts.

Principle 18.5 (Reachability persistence). The topology persists by reachability from a root, outliving any process, so that motion has somewhere durable to happen.

Without durable topology, topokinesis degenerates into an in-memory traversal library. With topology durable only in a separate system, it is trapped behind that system's discontinuity, and Pregel and Gremlin are the witnesses, holding real topological computation inside components that programs must reach across a boundary. The condition is developed in Chapter 10, and its lineage runs through orthogonal persistence (Atkinson and Morrison 1995): persistence as a property of reachability, not an act of input/output.

Principle 18.6 (Single system image). The runtime presents one continuous image across processes, machines, and users, so that the semantics of the first five conditions are invariant under deployment-scale transformation.

This condition binds the axes together. It completes synechism upward (deployment is territory the medium spans, not practice outside it), and it gives topokinesis its distributed meaning, since a locus that cannot move past a process boundary is mobile only in miniature. The single-system-image tradition built the image below the language, where programs could not see it (Buyya et al. 2001). Emerald built motion inside a cluster whose administrative perimeter it could not cross. The language-layer version is stated as scale invariance in Chapter 14, with its machinery in Chapter 13.

The six conditions partition by class: principles 18.1 to 18.3 are the conditions of the synechic class, principles 18.4 and 18.5 of the topokinetic class, and principle 18.6 belongs to their intersection. A language satisfying the first three and not the next two is a member of the synechic class alone: a coherent and worthwhile design, occupying the plane's horizontal margin at a greater degree than the multitier family reached. The converse design, a member of the topokinetic class alone, is also coherent but historically fragile: it is Telescript's coordinate, and Appendix G argued that mobility without a continuous medium beneath it kept being built as middleware and dying as middleware. On sufficiency our claim is the constructive one: Parts IV and V exhibit a system satisfying all six conditions and exhibiting both properties end to end, in production (Mars et al. 2023). Whether the conditions are sufficient as a theorem (whether every language satisfying the six must exhibit substrate transparency and first-class topokinesis under the formal definitions) requires the formal apparatus we do not yet have, and it heads the open problems of Section 18.4.

18.3 Design Principles for Successors

In this section, we tell successors what to copy and what to leave behind. A successor language should copy the conditions and feel free about everything else, and the distinction deserves to be drawn explicitly, because the first member of a class is always in danger of being mistaken for its definition. Several features of Jac are contingent. Python is the contained incumbent, but any living ecosystem serves principle 18.1, and a successor might contain TypeScript, or a JVM language, with different but analyzable consequences for which ecosystems arrive free. The particular trio of substrates (managed bytecode, the web, native code) is likewise contingent. The condition is coverage of the territory the language's applications inhabit, and a successor aimed at embedded systems would choose differently. The vocabulary of nodes, edges, walkers, and abilities is Jac's spelling of principle 18.4 (Mars 2025).

The topokinetic class requires the locus, the topology, and the arrival semantics, not the words. The components embedded in the toolchain artifact (Chapter 4) are implementation choices behind principle 18.3, which names an invariant (the artifact is self-contained), not an inventory. The constitution of those components is likewise contingent: the self-hosting frontier of Chapter 4 records which are written in Jac and which are carried engines, and Section 12.8 enumerates the six polypilation decisions a successor would answer on its own terms. Even the delegation construct of Chapter 6 is contingent in its surface and, we would argue, no longer contingent in its presence: the stochastic substrate is a substrate, and a language claiming the synechic property in the era of large language models must span it by some typed means (Dantanarayana et al. 2025).

What is essential travels less as a feature list than as a set of commitments, and we state them as advice. Inherit, do not annex. The value of a synechic medium is proportional to what already lives inside it, and every ecosystem placed across the edge returns as glue (Appendix F). Enter substrates, do not bridge to them, for a bridge is a discontinuity wearing the language's colors. Internalize the toolchain from the first design, because retrofitting it later means refactoring the language's own boundary: we know of no case of a language successfully absorbing its build system after an ecosystem had grown around external tooling. Make persistence a property of reachability, not an application programming interface, or the database discontinuity of Chapter 1 re-forms around whatever interface is chosen. Gradualize every discipline the medium internalizes: types, memory, placement, and artifact shape each ship in Jac as an inferred default with a per-declaration override and a reversible path (Chapters 4, 5, 9 and 12), and the pattern is what lets a program adopt each discipline by degrees instead of by rewrite. Absorb the language-shaped components of the toolchain first (parsers, checkers, code generators, linkers) and carry the hardened engines while the memory discipline matures, keeping the carried set visible as a list, the self-hosting frontier of Chapter 4, rather than treating it as a foundation. And keep faith with the Waldo critique (Waldo et al. 1994): continuity of expression, never identity of behavior. Latency, partial failure, and cost surface as lawful, visible properties of the program, because a class that forgets this inherits the RPC lineage's production failures along with its ambition (Chapter 2).

Two harder-won principles come from living with the design rather than deriving it. The first is the *containment bind*: principle 18.1 weds the language to its host's evolution, and the wedding has no divorce. A superset surface must track every release of the incumbent forever (the transposed form of the porting burden that Appendix F charged against Truffle), so a successor should choose a host whose evolution is deliberate, and should confine the surface's dependence on host semantics to a well-marked kernel, so that tracking is a maintenance task rather than a redesign. The second is an ordering constraint: continuity is not incrementally adoptable by the language itself. A medium that launches spanning two substrates and adds the third later will find that its community has already built the bridge, and the bridge has calcified into the kind of structure Part I catalogs. Better a shallow entry into the whole claimed territory than a deep entry into part of it. Successors should also inherit the book's evaluative discipline: the classes' claims are quantitative, and the glue cost model of Chapter 2 and the measurement designs of Chapter 16 apply to

any member, which is how the classes avoid becoming a genre of rhetoric.

18.4 Open Problems: Formal, Systems, and Human

In this section, we set down the open problems, in three families. The formal problems are the deepest. The first is full abstraction for substrate transparency. Chapter 2 defines the property in terms of program expression, and the strong form of the claim is semantic: the lowerings of Chapter 12 preserve and reflect observational equivalence, so that two program fragments equivalent in the surface language remain equivalent, and distinguishable only if surface-distinguishable, in every substrate and in every context each substrate supplies. This is a compiler-correctness program for a multi-target language, harder than single-target verified compilation because the contexts include each substrate's foreign code, and no such result exists for any language of this shape. The second formal front is the metatheory of the object-spatial calculus. Chapter 8 and Appendix B give the operational semantics and the soundness development in the syntactic style (Wright and Felleisen 1994), but behavioral equivalence for walkers (what bisimulation should mean when the observables include position and topology) remains unformulated. The expressiveness separations sketched against actors and ambients (Cardelli and Gordon 2000) deserve the rigor of Felleisen's framework (Felleisen 1991): whether topokinetic constructs are macro-expressible in actor calculi, or whether eliminating them requires global restructuring, is the formal content of the claim that OSP is a paradigm rather than a library. Third, the gradual typing of Chapter 9 meets multi-substrate lowering in unexplored territory: whether the gradual guarantee (Siek et al. 2015) survives three lowerings simultaneously is open. Fourth, the stochastic substrate has no soundness story at all: what a type system may promise when an executor is a distribution over outputs (Chapter 6) is a question the gradual-typing and refinement-types communities have barely opened.

The systems problems concentrate where the two axes meet at scale. Distributed walker scheduling is the central one. When the topology is sharded across machines (Chapter 14), every traversal step poses an online choice: move the walker to the data, move or replicate the data toward the walker, or batch and defer. The choice wants a cost model of the kind the mobility literature began (Black et al. 2007) but never completed, informed by measured migration costs, locality statistics of real topologies, and the failure semantics the single system image must lawfully expose. Second, the transactional story of Chapter 10 must meet long traversals over a live, multi-user graph. Which fragments of the object-spatial model are coordination-free in the sense that recent distributed theory makes precise (Hellerstein and Alvaro 2020) is, we believe, the most promising route to traversal at planetary scale, and it is unmapped. Third, reachability persistence at scale needs a garbage-collection theory: what unreachability means, and who may decide it, when roots are per-user and the graph is shared. Fourth, the observability problem of Chapter 17 generalizes to the synechic class: debugging a medium without discontinuities requires tooling that reintroduces boundaries as views, as instruments, without reintroducing them as semantics, and no such tooling tradition exists yet. Fifth, the polypiler of Chapter 12 poses a standardization problem: whether the interop manifest can standardize

across implementations, as debug-information formats did across compilers, so that tools, runtimes, and other compilers consume one machine-readable account of a program's distributed shape regardless of which polypiler produced it. The question extends to the representation itself: Section 12.8 enumerates six decisions any polypiler must answer, and the minimal fact set a shared representation must carry (symbols, types, placement, boundaries), together with a shared differential conformance suite standing in for full abstraction until the formal result exists, is unmapped. Sixth, the self-hosting frontier of Chapter 4 poses the artifact-floor question: how much of a language's own substrate (interpreter, code generators, codecs) can be carried in the language itself, and what remains at the floor as a permanent import from the host world.

The human problems may govern whether the classes have a future at all. The first is pedagogy. Topological thinking was taught by example in Chapter 7, and Chapter 15 suggests practitioners acquire it, but the field has no learning progression for path-thinking as it has for structured control flow or recursion (no equivalent of the loop invariant for the traversal), and the developer-study designs of Chapter 16 are only a beginning. The second is the sociology of growth. Steele (1999) argued that a language cannot be delivered large. It must be grown, and grown by its users. A synechic language sits awkwardly against this wisdom, because it internalizes territory (toolchain, deployment, substrate entry) that language communities have always grown as separate projects, and its value lies in that territory's continuity. Three questions remain open: how a community grows a medium without fracturing it, how the containment bind is serviced across decades and maintainers, and how the synechic class avoids a future in which each member is a silo whose continuity stops at the next member's border. Synechic languages mutually discontinuous would rebuild the fragmented stack one level up. These are questions of institutions as much as of design, and the plain statement is that we have arguments where we would prefer evidence. What Part V measured is one language and its deployments. What the classes require is a generation of members. The book closes, in Chapter 19, by arguing that the wager is worth making. This chapter's contribution is the map on which successors can mark where they landed.

Bibliographic Notes

The systems placed in this chapter's design space are documented in the sources their histories cited. For the synechic margin, the multitier family is covered in the primary papers (Chlipala 2015; Cooper et al. 2006; Radanne et al. 2016; Serrano et al. 2006; Weisenburger et al. 2018) and in the survey by Weisenburger et al. (2020), the polyglot inversion in Würthinger et al. (2013), and content-addressed continuity in the Unison documentation (Unison Computing 2024). For the topokinetic margin, Armstrong (2003) remains the definitive statement of the Erlang philosophy and Bykov et al. (2011) of virtual actors. Black et al. (2007) is both a language history and an unusually candid account of what object mobility did and did not buy. White (1997) and Lange and Oshima (1998) document the mobile-agent episode from inside, and the trapped-topokinesis systems are presented in Malewicz et al. (2010), Rodriguez (2015), and Francis et al. (2018). Readers comparing this chapter's coordinates against

the fuller histories should consult Appendix F and Appendix G, whose evidence the placements compress.

The membership conditions draw their lineages from three sources. Atkinson and Morrison (1995) treats persistence as a property rather than an operation, Buyya et al. (2001) covers single system images and the layers at which they can be built, and Waldo et al. (1994) states the disclosure constraint any member of the synechic class must satisfy. The Jac-specific discharges are documented in Mars (2025), Dantanarayana et al. (2025), and Mars et al. (2023). For the open problems, Felleisen (1991) defines the expressiveness framework in which the paradigm claim should be settled, Wright and Felleisen (1994) the soundness style the calculus development follows, Siek et al. (2015) the criteria a gradual system must preserve, Cardelli and Gordon (2000) the mobility calculus against which separations should be proved, and Hellerstein and Alvaro (2020) the coordination-freeness theory we expect to matter for traversal at scale. On growth, Steele (1999) is the essential meditation, and the reader who finishes this book should read it next: it poses, for languages in general, the question this chapter poses for these classes in particular, whether a designed thing can become a grown thing without ceasing to be itself.

CODA: CONTINUITY AS A DESIGN ETHIC

Synechism is that tendency of philosophical thought which insists upon the idea of continuity as of prime importance in philosophy.

C. S. Peirce, 1902

In this book, we have made the case for two classes of programming language and built the one language that is a member of both. The first class is the synechic, defined by continuity, the subject of this closing chapter: a medium can be unbroken across the substrates convention treats as separate worlds. The second is the topokinetic, defined by motion: the locus of computation can travel through a persistent world rather than stand still while data is delivered to it. We opened with an inventory, and we close with the ethic that inventory demands. Chapter 1 analyzed one unremarkable application and found that the greater part of what its team maintains exists not to compute anything but to carry meaning across boundaries the problem never asked for. The chapters between have done the technical work of the response. Part II defined the property that dissolves such boundaries and the discipline that marks what must remain (Chapter 2). Part III gave the mobile locus of computation a paradigm and a semantics (Chapters 7 and 8). Part IV recorded the machinery that realizes both in a single production language. Part V measured the result and steelmanned the case against it (Chapters 16 and 17). And Chapter 18 generalized from the artifact to the classes.

What remains is not argument but orientation: an account of the word at the center of the title. This coda makes three passes at that account. The first pass sets out the doctrine, and the debt the field takes on by naming itself from a nineteenth-century metaphysician. The second pass sets out the economics that make the debt urgent now. The third pass returns to the application where the book began, re-described in the medium the book has defined.

19.1 Peirce's Synechism and the Metaphysics of the Unbroken

The word at the root of this book's title entered philosophy in 1892, in a *Monist* essay called "The Law of Mind," where Charles Sanders Peirce named the doctrine around which he was organizing his metaphysics, coining the term from the Greek *synecheia*: continuity, literally a holding-together (Peirce 1892). The essay's immediate subject is mind: ideas, Peirce argued, are not discrete counters shuffled by association but continuously spreading affections, each shading into its neighbors, the apparent atomism of thought an artifact of coarse description. The doctrine he drew from the argument was general, and the definition he later supplied for a philosophical dictionary (our epigraph) states it at full generality. The continuous is prior. The discrete is derived. A theory that rests on a brute discontinuity has not explained anything. It has marked the spot where its explanation gave out.

Peirce was careful about the doctrine's logical status, and his care is what makes the word borrowable. Synechism, in his hands, is not a dogma about what the universe must contain. It is a regulative principle of inquiry: a rule about what an explanation is permitted to look like. Confronted with an apparent gap, the synechist does not deny the appearance. He declines to accept it as final, and treats the discontinuity as a symptom of incomplete analysis until the world forces the opposite conclusion. Peirce's fallibilism, his refusal to grant any piece of knowledge absolute finality, is the same commitment turned on inquiry itself: knowledge shading into uncertainty with no line between them.

A treatise on a programming language has no business quoting a metaphysician unless the borrowed word does work that no nearer word could do. We state that work plainly. Peirce addressed his principle to nature, and in nature it runs a genuine risk: the gap under investigation may, in the end, be real. Whether the world is at bottom continuous is a question on which the physics of the century after him has had complicated things to say. The artificial world runs no such risk, because it is made of descriptions and of nothing else. In nature a discontinuity may be *discovered*. In software every discontinuity was *authored*. The discontinuities of Chapter 1 are exactly this: not facts about computation but defaults of description, carried out of a 1945 machine report and hardened by seventy years of habit into something the field mistakes for terrain. Even the hardest partition among languages reads this way: the divide between collected and ownership-disciplined languages, deep enough to name language families, becomes in Chapter 5 a continuum a module walks by degrees inside one medium. Peirce could only counsel inquirers to prefer the continuous hypothesis. The language designer can enact it. In philosophy, synechism is a wager about the world. In language design it is a specification, one that Chapter 2 writes down as substrate transparency and Part IV shows can be met. That is why we say the borrowing is earned, not decorative. The doctrine that continuity is prior, transposed into a medium that consists entirely of choices, becomes the observation that every discontinuity is a cost somebody chose, and the imperative that follows from noticing.

From the doctrine, then, an ethic. We call it the *synechistic* ethic: the form the regulative principle takes when the world under description is artificial. It has two clauses. First: treat every discontinuity in a programming medium as a defect of description until it is proven a fact of the world. Second: when it is proven a fact of

the world (latency, partial failure, cost, the stochastic contract of a model), surface it in the semantics at the point where it is incurred, rather than conceal it. Both clauses bind. The first forbids the resignation that treats the fragmented stack as maturity, the settled view that twelve notations and a specialist per discontinuity are what serious software looks like. The second forbids the opposite vice: the false continuity that Waldo et al. (1994) indicted, in which a medium pretends the network away and collects the debt at the worst possible moment. A faithful medium is continuous exactly where the world is continuous, and discontinuous exactly where it is not.

19.2 Discontinuity and the Economics of Software in the Era of AI Authorship

In this section, we retake the count of the discontinuities for the economy that is arriving. Chapter 1 counted their cost in human attention: onboarding weeks, the wiki page that enumerates how to run the application locally, and the specialist roles whose existence maps one-to-one onto the boundaries of the stack. That ledger assumed that people write the code. The assumption is expiring as this book goes to press. Large language models, established as capable executors of natural-language task specifications (Brown et al. 2020), now author a substantial and growing share of the world's new code.

It is tempting to conclude that the economics of glue have thereby been repealed. A machine can emit the four copies of the record in seconds, so the copies look free, and the tempting move is to let the fragmented stack stand and staff it with generators. The inference fails three times over.

First, consider what abundant authorship amplifies. Glue is the most mechanically derivable text in software: it restates decisions already made, in notations fixed by convention. And it dominates the corpus on which the models were trained, because it dominates the repositories of the glue era. A generator of code is therefore, before it is anything else, a generator of glue. When machines write most of the code, glue is most of what they write.

Second, the cost of glue was never the cost of writing it. The analysis of Chapter 1 located the expense at verification: discontinuities are precisely the program points where no verifier has jurisdiction, where coherence between representations is maintained by convention and drift is discovered in production by whoever is on call. Falling generation cost against fixed verification cost does not make the stack cheaper. It moves the binding constraint. A machine that emits ten times the glue emits ten times the unverifiable surface, deposited where the tools cannot see. Among human authors the four-copy record was a discipline problem. Among machine authors it is the same problem executed faster, and a fluent generator drifts more plausibly than a tired one. Cheap authorship of unverifiable text is not productivity. It is liability issued at volume.

Third, the loop closes on itself. Models learn the fragmented stack from the fragmented stack: every generated serializer, manifest, and binding returns to the corpus as evidence, for the next model, that this is what software is. A field that wants a different equilibrium must change what there is to learn, and the only instrument that changes what there is to learn is the medium in which programs are written.

The deeper account of the stakes was available fifty years before the models arrived. Dijkstra (1972) located the central difficulty of programming not in the machine but in the strictly limited size of the programmer's skull, and cast the whole discipline as the art of keeping small what must be held in mind at once. The context window is the machine's skull. It is larger than ours, and it is bounded, and in the fragmented stack it fills with the same obligation that fills the human head: every invariant that convention requires and no tool records (the casing translation in a forgotten serializer configuration, the wire shape two type systems honor on faith) must be reconstructed and held, in mind or in context, for the duration of every change. A synechic medium changes that term of the equation for both kinds of author at once. When the invariants live in the semantics, the knowledge required for a change contracts to the change. When the checker has whole-program jurisdiction, "held in mind" becomes "enforced by tool". And the scarce resource, skull or context, is spent on the problem rather than the plumbing.

There is precedent for thinking of a language this way. Kay (1993) recounts how Smalltalk was composed of a single substance (objects all the way down, each a recursion on the notion of the computer itself) so that the system would be one medium rather than a stack of translations. The ambition was a medium one thinks in, not a tool one operates. Smalltalk's continuity ended at the boundary of its image. The synechic program is that ambition carried past the image boundary: across foreign ecosystems, across tiers, across the toolchain, and, through the meaning-typed constructs of Chapter 6 (Dantanarayana et al. 2025), across the boundary of determinism itself. The model is then no longer only an author outside the program but a substrate within it. The medium becomes the ground both kinds of author share: one notation, one type discipline, one truth about the program that human and machine can both check.

The economic conclusion is compact. When authorship was scarce, languages competed on expressiveness: on how much intention a scarce hour of writing could carry. Now that authorship is abundant, the scarce factor is jurisdiction, the reach of the instruments that can examine a proposed change and say no. Semantics and the tools that enforce them are the only such instruments, and the language is where their reach is decided. A synechic language is, on this reading, the maximal extension of jurisdiction: the decision to leave no program point where the only reviewer is hope. That is the case for continuity in the era of machine authorship, and it is a case about governance, not convenience.

19.3 What Programming Looks Like When the Discontinuities Are Gone

In this section, we return to the application where the book began. The example of Chapter 1 was deliberately ordinary: a task-management product with an AI assistant, assembled according to best practice into twelve notations, five package ecosystems, half a dozen build artifacts, and eight representational boundaries between a user's click and its consequence, its central record maintained in four copies containing one decision and three echoes. We can now say, in the vocabulary this book has spent

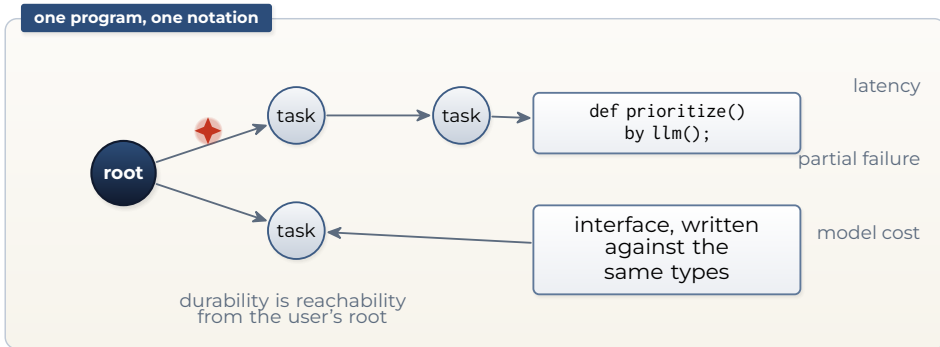


Figure 19.1: The application of Chapter 1, re-described. The record is declared once as a node type, tasks persist by reachability from the user's root, the assistant is a function whose executor is by `llm()`, and the interface is written against the types it renders. Of Figure 1.1's eight authored boundaries, none survive as conventions. The three ticks at the edge are their entire lawful residue: latency and partial failure from the wire crossings, model cost from the stochastic one, and nothing at all from the other five (Table 19.1).

sixteen chapters earning, what that application is when it is described rather than assembled.

It is one program, in one notation, in a medium that is synechic and topokinetic (Figure 19.1). The record is declared once, as a node type. Every representation the substrates require (the stored shape, the wire shape, the client shape) is the compiler's obligation, derived rather than authored (Chapter 3). A user's tasks are the nodes reachable from that user's root, and their durability is a property of the reachability itself, not an I/O regime administered through a mapper (Chapter 10). The assistant is a function whose executor is declared at its definition, by `llm()`, its prompt synthesized from the meaning the program already carries and its reply constrained by the declared return type (Chapter 6). The interface is written against the very types it renders, in the same file if the author likes. The boundary between tiers is a placement, not an architecture. The scaffolding that stood between the halves (route tables, mirrored schemas, the generated client) is a projection the runtime performs from the program's own declarations (Chapter 14). The packages of three foreign ecosystems arrive through ordinary imports: McIlroy's catalog of mass-produced components, at last with couplings that fit (McIlroy 1968). The compiler, package managers, server, and deployer travel inside the single artifact that is the toolchain (Chapter 4). Where a hot path earns systems discipline, the module turns the ownership dial in place instead of leaving for a second language (Chapter 5). Even the cache and the queue, compensations for stationary compute, are demoted from architecture the developer operates to optimizations the runtime may make. And the count of glue artifacts in such a codebase is not small but zero, a number Chapter 16 measures the only way zero can be measured: by exhaustive inspection.

We trace the click again. When the user asks the assistant to prioritize her tasks, the handler awaits a typed call, and the call crosses from browser to server as a call, checked on both sides by the one compiler that sees both. On the far side, computation

Table 19.1: The ledger of Figure 1.1, settled. Five of the eight crossings leave nothing behind; the residue of the other three is the set of lawful boundaries of Figure 19.1.

Crossing	Dissolved by	Lawful residue
1 handler to generated client	one compiler checks both sides (Chapter 12)	none
2 client to HTTP request	wire shape derived, not authored (Chapter 12)	latency
3 request to route	same signature on both sides (Chapter 12)	partial failure
4 route to queue job	the walker run is the unit of work (Chapter 13)	none
5 job to prompt assembly	prompts synthesized from meaning types (Chapter 6)	none
6 prompt to model call	reply held to the return type (Chapter 6)	model cost
7 parse to ORM write	persistence by reachability (Chapter 10)	none
8 write to cache invalidation	invalidation derived from effects (Chapter 12)	none

goes to the data rather than the data to the computation: a walker enters at her root and moves along the topology her tasks form, carrying the intent, doing at each node what arrival there demands (Chapter 7). Delegation to the model, where it occurs, is a typed expression whose contract the runtime enforces. Of the eight representational boundaries the original example imposed between click and consequence, none survives as a convention. What remains is what the world insists on: the latency of the wire, the possibility of partial failure, the cost and stochasticity of the model call, each visible in the program at the point where it is incurred, in the semantics rather than in the lore (Chapter 17).

The ledger opened in Chapter 1 can now be settled line by line, and Table 19.1 reads it back. Each numbered crossing of Figure 1.1 is listed against the mechanism that dissolves it and the chapter that records the machinery. Note that five of the eight rows end in *none*: those crossings were pure description, and nothing in the world corresponds to them. The residue of the other three, latency, partial failure, and model cost, is exactly the set of lawful boundaries drawn at the edge of Figure 19.1. The eight authored badges of the opening figure reduce to the three ticks of the closing one, and the reduction is the book’s argument in a single subtraction.

Practiced this way, the discipline is smaller in its accidents and untouched in its essence. The essential difficulties Brooks (1987) identified, the irreducible complexity of the problem and the fashioning of the conceptual structures themselves, are what remains when twelve notations become one. No medium decides what is worth building. But the accidental spend changes shape. The specialist roles that mapped one-to-one onto the discontinuities lose the walls that held them up, and the org chart can stop being a picture of the glue and go back to being a picture of the problem. The second program, the one that built the first, is gone, absorbed into the artifact. And the figure at the center of the practice has changed. Under the two silent assumptions of the founding machine report (Neumann 1945) (computation stationary, the machine the program’s world) the paradigmatic program was a schedule for moving data past a fixed point of compute, the pattern Backus (1978) indicted from inside the machine and Chapter 1 found again at the scale of the data center. When both assumptions are dropped at once, the paradigmatic program is a walker: a locus of computation that travels through a persistent world, written in a medium that does not end at any machine’s edge. That figure is what this book has been about.

We close as Peirce would have us close: fallibly. Jac is an existence proof, not a

terminus. Chapter 18 names the classes precisely so that successors can occupy them, and a synechist is not permitted to believe the inquiry finished, least of all about his own artifact. The same standard applies to the artifact itself. The payload of Chapter 4 still carries external engines: an interpreter, a JavaScript runtime, a code generator. Each is a discontinuity the medium's own verifier cannot yet see, retained for feature completeness, with replacement in Jac as the stated goal and gradual borrow checking as the mechanism. The self-hosting frontier has moved in one direction only, and the ethic's first clause applies to the language's own implementation no less than to the stacks it replaces. But the ethic itself we commend without hedging. For seventy years the field has treated its discontinuities as geography: features of the terrain to be mapped, staffed, and toured. They were never geography. They were descriptions, inherited from a report about a machine with one memory, repeated until they looked like the world. Peirce asked philosophy to regard reality as unbroken until reality itself objects. Software is the one world made entirely of regard. It can be unbroken the moment we describe it so.

Bibliographic Notes

The philosophical spine of this chapter is Peirce (1892), the *Monist* essay in which synechism receives its fullest single statement and its name. The epigraph reproduces the definition Peirce later supplied for Baldwin's *Dictionary of Philosophy and Psychology* in 1902. Readers approaching Peirce from software rather than philosophy may find the regulative reading emphasized here (synechism as a rule of explanation rather than a cosmology) the most durable part of his doctrine, and it is the part this book has transposed: Chapter 2 turns the maxim into a definition, and Chapter 16 into a measurement. The disclosure clause of the synechistic ethic descends from Waldo et al. (1994), whose catalog of the essential differences between local and distributed computing this book has treated throughout as a specification of the discontinuities a language must surface rather than dissolve.

The second section's economics rest on an old observation and a new capability. The old observation is Dijkstra (1972), which locates the cost of programming in the limited size of the programmer's skull. The new capability is documented in Brown et al. (2020), after which the skull in question is as often a context window. Kay (1993) is the design memoir in which the ideal of the language as a uniform medium is most fully articulated, and the natural companion to this book's attempt to carry that ideal past the image boundary. Dantanarayana et al. (2025) supplies the mechanism by which the stochastic author is absorbed into the medium as a substrate. The inventory re-taken in the final section is that of Chapter 1, whose framing sources (Neumann (1945), Backus (1978), Brooks (1987), and McIlroy (1968)) bound the arc this coda closes. The engineering record of the language that made this book possible is Mars et al. (2023) and Mars (2025), and the evidence that its claims survive contact with production occupies Chapters 15 and 16.

IRRATIONAL NOT TO

Jac is the language I've always dreamt of,
and it has now realized its full form for me.
I will use it forever,
and for everything,
it would be irrational not to.

From this point on,
I will continue to work on Jac for everyone else,
because I love Jac
and I want it to live forever,

I won't,

So it would be irrational not to.

For me, it's irrational not to use Jac,
and it's irrational not to work on Jac.
And. . .
I don't talk to irrational people.

THE JAC LANGUAGE REFERENCE

This appendix is a curated reference for the Jac surface syntax used throughout the book. It is not a grammar dump: the full formal grammar ships with the language's toolchain and evolves with it, and reproducing it here would date the book without informing the reader. What a treatise needs instead is a compact, accurate account of the constructs its arguments depend on, enough that a reader can parse every listing in the preceding chapters, write a working program, and know where the corners of the language lie. Every snippet in this appendix is drawn from, or verified against, the language's own reference documentation and example applications; where the current implementation has a known gap, we say so.

The organization follows the shape of the language rather than the shape of a grammar. Appendix [A.1](#) covers lexical structure and how modules are laid out on disk. Appendix [A.2](#) covers the declaration forms: the archetypes, fields, functions, abilities, and the declaration/implementation split. Appendix [A.3](#) covers the object-spatial constructs that Chapter 7 introduced informally and Appendix [B](#) treats formally. Appendix [A.4](#) summarizes the statement and expression repertoire in tabular form, and Appendix [A.5](#) sketches the client tier and Python interoperation. Design rationale is deliberately absent here; for that, the reader should return to Chapter 3.

A.1 Lexical Structure and Module Layout

A Jac source file is UTF-8 text. Statements terminate with semicolons and blocks are delimited by braces; these are the two most visible departures from Python, whose expression and statement vocabulary Jac otherwise contains. There is one deliberate exception to brace delimiting: the bodies of case arms inside match statements are indentation-delimited, following the Python construct they inherit. Line comments begin with #, block comments are bracketed by `/*` and `*/`, and docstrings, triple-quoted string literals, precede the declaration they document rather than following its header.

Identifiers follow the usual convention of a letter or underscore followed by letters, digits, or underscores. Keywords are reserved, but any keyword can be used as an

identifier by prefixing it with a backtick. A small set of names (`self`, `Self`, `super`, `root`, `here`, `visitor`, `init`, `postinit`) are built-in references rather than ordinary identifiers and are written without escaping. Listing A.1 shows a complete module: docstring, import, a module-level global declared with `glob`, a function, and an entry block.

```
"""A minimal Jac module."""
import math;

glob GREETING: str = "Hello";

def area(r: float) -> float {
    return math.pi * r ** 2;
}

with entry {
    print(f"{GREETING}: area is {area(2.0):.2f}");
}
```

Listing A.1: A minimal Jac module: docstring, import, global, function, and entry block.

The `with entry` block is the module's executable body: it runs whenever the module loads, replacing Python's convention of bare top-level statements. The variant `with entry: __main__` runs only when the module is executed directly, replacing the `if __name__ == "__main__"` idiom. A module may declare several entry blocks; they execute in order.

Imports come in a handful of forms, gathered in Listing A.2. Plain `import` binds a module, optionally renamed with `as`; the braced form `import from m { a, b }` binds selected names; `import type from` binds a name for type annotations only, lowering to a type-checking guard so that circular type references between modules do not force a runtime import cycle. The `include` statement, by contrast, merges the target module's declarations directly into the current scope. Because the language is substrate-transparent, the same import syntax reaches every ecosystem: a dotted name resolves against Jac and Python modules on the server, against npm packages (written as string specifiers such as `@jac/runtime`) on the client, and against C shared libraries in native code, where the braced body declares the foreign function signatures.

```
import os;
import pandas as pd;
import from sklearn.linear_model { LinearRegression }
import type from billing { Invoice }
include utils;
```

Listing A.2: Import forms: module import, aliasing, selective import, type-only import, and include.

A logical module may span several files sharing a base name, called annexes. The head file `m.jac` holds declarations and entry points; `m.impl.jac` holds implementation bodies for declared-but-unimplemented members; `m.test.jac` holds tests; and the variant annexes `m.sv.jac`, `m.cl.jac`, and `m.na.jac` hold code destined for the server, client, and native substrates respectively. All are discovered and merged automatically

Table A.1: File extensions and their roles in a Jac module.

Extension	Role
.jac	Head module: universal code, declarations, entry points
.sv.jac	Server-variant annex (compiles to Python bytecode)
.cl.jac	Client-variant annex (compiles to JavaScript)
.na.jac	Native-variant annex (compiles via LLVM)
.impl.jac	Implementation annex for declared members
.test.jac	Test annex

Table A.2: Declaration keywords of the Jac language.

Keyword	Declares
obj	An object type with declared fields and methods
node	An object type that occupies a location in the graph
edge	A first-class typed relation between two nodes
walker	A mobile computation with state and abilities
class	A Python-compatible class, for host-specific machinery
enum	An enumeration, optionally over a base type
glob	A module-level variable
has	A typed field inside an archetype
def	A function or method, explicitly invoked
can	An ability, triggered by walker arrival or departure
impl	An implementation body for a declared member
sem	A semantic annotation attached to a declaration

at compile time. Table A.1 summarizes the roles. Within a single file, a braced block (`cl { ... }`, `sv { ... }`, `na { ... }`) places a section, and a single-statement prefix such as `sv` places one statement. Markerless code is placed by inference: `JSX` and `npm` imports mark a module as client code, C-ABI foreign declarations mark it as native, and archetypes, served endpoints, and Python-ecosystem imports pin it to the server. A markerless module that none of these constraints claims compiles native by default and falls back to the server lowering when the native one cannot carry it (Chapter 12). The `sv import` prefix carries more than placement, and it has three readings. In client code it imports the server’s surface, so that walkers and functions invoke as typed remote calls. In server code it declares the imported module a separate service, the microservice boundary of Chapter 14. And importing archetype or object types through it types the values that cross either boundary. Appendix A.5 returns to the client tier in more detail.

A.2 Declarations

Declarations are the backbone of a Jac program, and the language is deliberately declaration-rich: the data model, the interfaces, and the semantic annotations that drive both tooling and model-based delegation all live in declaration syntax rather than in convention. Table A.2 indexes the forms; the rest of the section treats each in turn.

The four archetypes (`obj`, `node`, `edge`, `walker`) share one declaration shape. Fields are introduced with `has`, carry mandatory type annotations, and may take defaults; several fields may share one `has` separated by commas. Fields without defaults become required constructor arguments; the constructor itself is generated, so there is no hand-written `__init__`. A field whose value derives from other fields is marked `postinit` and assigned in a `postinit` method that runs after construction. Class-level state uses `static has`, and a `has` binding followed by an accessor block (getter, setter, deleter) declares a property rather than a stored field. Listing A.3 shows the core of the form.

```
obj Rectangle {
  has width: float,
    height: float;
  has area: float postinit;

  def postinit {
    self.area = self.width * self.height;
  }

  def scale(factor: float) -> None {
    self.width *= factor;
    self.height *= factor;
    self.area = self.width * self.height;
  }
}
```

Listing A.3: An object declaration with fields, deferred initialization, and a method.

Nodes and edges use the identical field syntax; what changes is where instances live and how they connect, which is the subject of Appendix A.3. An edge may declare the node types it connects, written with the traversal arrow after a colon, in which case traversals along it infer the endpoint type without a filter. Listing A.4 gathers the forms. Inheritance for every archetype is written with parenthesized bases, as in `obj Dog(Animal)`, and applies uniformly to nodes, edges, and walkers; `override` marks a deliberate redefinition and `abst` marks an abstract member.

```
node Profile {
  has username: str = "",
    bio: str = "";
}

node Tweet {
  has content: str = "";
}

edge Friend {
  has since: int,
    strength: float = 1.0;
}
```

```
edge Post: Profile --> Tweet {}
```

Listing A.4: Node and edge declarations, including an edge with fields and an edge with typed endpoints.

Functions and methods are declared with `def` and require type annotations on parameters and, when a value is returned, on the result. The parameter grammar matches Python's in full: positional-only markers (`/`), keyword-only markers (`*`), variadic `*args` and `**kwargs`, and defaults. `static def` declares a receiverless method and `class def` a class-bound one. Lambdas exist with annotated, parenthesized parameters and braced bodies, as in `lambda (x: int) { x + 1; }`, where a single expression statement is the implicit return. An optional return annotation follows the parameter list (`lambda (x: int) -> int { x + 1; }`), and a zero-parameter lambda may omit the parentheses entirely (`lambda { refresh(); }`). Decorators apply with the usual `@` prefix.

The second callable form, `can`, declares an *ability*: a body that is never invoked by name but triggered by an encounter between a walker and a location. Its event clause names the counterpart type and the phase: `entry` or `exit`. Inside a walker, `can gather with Item entry` runs when the walker arrives at an `Item` node; inside a node, the clause names the visiting walker's type instead. Omitting the type makes the ability fire for any counterpart. Listing A.5 shows the canonical accumulator walker: fan out from the root, gather at each node, deliver once on exit.

```
node Item {
    has value: int = 0;
}

walker ListItems {
    has items: List[int] = [],
        reports: List[List[int]] = [];

    can start with Root entry {
        visit [-->];
    }

    can gather with Item entry {
        self.items.append(here.value);
        visit [-->];
    }

    can deliver with Root exit {
        report self.items;
    }
}
```

Listing A.5: A walker declaration with state, typed reports, and entry and exit abilities.

Enumerations are declared with `enum`, with optional explicit values and an optional base type after a colon: `enum HttpStatus: int` produces members that compare and compute as integers. Listing A.6 shows both forms.

```
enum Color {
```

```

    RED = "red",
    GREEN = "green",
    BLUE = "blue"
}

enum HttpStatus: int {
    OK = 200,
    NOT_FOUND = 404,
    SERVER_ERROR = 500
}

```

Listing A.6: Enumerations, plain and with a typed base.

Module-level variables use `glob`, and access modifiers attach to any declaration as a colon-suffixed tag: `:pub`, `:protect`, `:priv`. The three tags carry context-dependent meaning. On a member inside an archetype they are classic encapsulation: public, visible to subclasses, private to the declaring type. On a top-level symbol they govern cross-module visibility: `:pub` exports beyond the project, `:protect` restricts to modules under the same `jac.toml` root, and `:priv` restricts to the declaring module. On a served function or walker they additionally decide endpoint authentication, where only `:pub` is open to anonymous callers and everything else, including the unmarked default, requires authentication (see Chapter 14).

```

glob:pub API_VERSION = "1.0";
glob:protect retry_budget = 3;
glob:priv _cache = {};

```

Listing A.7: Global declarations with the three access tags.

Two declaration forms complete the family, and both separate a declaration from something ordinarily fused with it. The `impl` form detaches a body from its signature: an archetype may declare a method as a bare signature and provide the body in an `impl` block, in the same file or in an `.impl.jac` annex. This is the mechanism behind the interface/implementation discipline discussed in Chapter 3.

```

obj Calculator {
    has value: float = 0.0;
    def add(x: float) -> float;
}

impl Calculator.add {
    self.value += x;
    return self.value;
}

```

Listing A.8: The declaration and implementation split.

The `sem` form detaches *meaning* from structure: it attaches a semantic description string to a function, parameter, or field, addressed by dotted path. These annotations are the source material for the compiler's prompt synthesis when a function's implementation is delegated to a language model with by `llm()`, the meaning-typed machinery of Chapter 6 (Dantanarayana et al. 2025). A delegated function is a signa-

ture with no body; its return type and the `sem` annotations on it and its types define the contract the model must satisfy.

```
enum Category { WORK, PERSONAL, SHOPPING, HEALTH, OTHER }

def categorize(title: str) -> Category by llm();
sem categorize = "Categorize a task based on its title";

obj Ingredient {
  has name: str,
    cost: float;
}
sem Ingredient.cost = "Estimated cost in USD";
```

Listing A.9: Semantic annotations and a function implemented by delegation to a language model.

A.3 Object-Spatial Constructs

The constructs in this section realize the topokinetic half of the language (Mars 2025). Chapter 7 develops them by example and Appendix B gives their operational semantics; here we fix the syntax.

Graph construction is expression syntax. The connection operator `++>` draws a directed generic edge from its left operand to its right and evaluates to the right operand, so connections chain: `root ++> a ++> b`. The mirrored form `<++` draws the edge in the other direction, and `<++>` connects both ways. Typed edges are instantiated inline between split arrows: `alice +>:Friend(since=2020):+> bob` creates a `Friend` edge carrying data, and `<+:Colleague(...):+>` is its bidirectional counterpart. Deletion is symmetric: `del n` removes a node, and `a del -> b` removes the edges between two nodes.

A walker begins executing when it is *spawned* at a location: `root spawn create_tweet(content="hi")` constructs the walker like any object and sets it moving at `root`. The `spawn` expression evaluates to a response object whose `reports` list accumulates every value the walker reported during its traversal. Declaring `has reports: list[T]` on the walker types this channel end to end, so report statements are checked against `T` at the source and consumers receive typed values rather than `any`. Listing A.10 shows a complete round trip, condensed from the littleX reference application.

```
walker create_tweet {
  has content: str;

  can run with Root entry {
    visit [-->[?:Profile]];
  }

  can make with Profile entry {
    new = here +>:Post():+> Tweet(content=self.content);
    report new;
```

```

    disengage;
  }
}

with entry {
  result = root spawn create_tweet(content="first post");
  print(result.reports);
}

```

Listing A.10: Spawning a walker and consuming its reports.

Within an ability, four statements govern the walker’s journey. The `visit` statement queues destinations; traversal is breadth-first by default, and an optional insertion index (`visit : 0 : [->]` places new destinations at the front of the queue) yields depth-first and custom orders. A `visit` may carry an `else` block that runs when the target expression produces nothing to visit, which makes find-or-create idioms local, as in Listing A.11. The `report` statement appends a value to the response and continues; `skip` returns from the current ability only, like an early return; and `disengage` terminates the walker outright: no further abilities run, and no queued nodes are visited.

```

walker ensure_profile {
  can run with Root entry {
    visit [->[?:Profile]] else {
      fresh = here ++> Profile();
      visit fresh;
    }
  }
}

```

Listing A.11: A visit statement with an else clause: find the profile or create one and walk into it.

The bracketed expressions appearing as `visit` targets are *edge reference expressions*, and they are ordinary expressions: anywhere a list is expected, an edge reference produces the nodes (or edges) reachable from a context. The plain forms `[->]`, `[<-]`, and `[<->]` follow outgoing, incoming, and both edge directions from the current node. Typed forms constrain the edge, filters constrain edge attributes during traversal or node type and attributes after it, and the `edge` keyword selects the edge objects themselves rather than the nodes they lead to. An explicit anchor before the arrow (`[me ->:Follow:->]`) starts the reference at any node-valued expression, and arrows compose for multi-hop queries. Listing A.12 enumerates the forms against minimal declarations.

```

node Person {
  has age: int = 0;
}

edge Friend {
  has since: int = 2020;
}

```

Table A.3: Special references and the contexts in which they are valid.

Context	self	here	visitor
Walker ability	the walker	current node	—
Node ability	the node	—	visiting walker
Object method	the object	—	—
Module-level code	—	—	—

```

walker Traverser {
  can query with Person entry {
    outgoing = [-->];
    incoming = [<--];
    both     = [<-->];
    typed    = [->:Friend:->];
    recent   = [->:Friend:since > 2020:->];
    people   = [-->][?:Person];
    adults   = [-->][?age >= 18];
    links    = [edge -->];
    hops     = [here ->:Friend:-> ->:Friend:->];
  }
}

```

Listing A.12: Edge reference expressions: direction, edge typing, attribute filters, node filters, edge selection, and multi-hop composition.

Four special references orient code within the topology, and their meaning depends on where the code sits; Table A.3 fixes the contexts. The essential asymmetry is between `here` and `visitor`: an ability declared in a walker sees the current location as `here`, while an ability declared in a node sees the arriving walker as `visitor`. Both sides of an encounter can therefore hold code, each referring naturally to the other. The root reference is available everywhere and names the root node of the current graph, which, under the persistence semantics of Chapter 10, is the current user’s root when the program is served. In a served deployment `root.shared` additionally names the public graph shared across users.

A small library of built-in functions rounds out the spatial vocabulary: `jid` returns an object’s stable identity, `jobj` resolves an identity back to its object, `allroots` enumerates reachable roots, and `grant` and `revoke` adjust permissions on nodes (with levels such as `ReadPerm`, `ConnectPerm`, and `WritePerm`) when data is shared across users. One clarification closes the section: an edge’s `can` abilities fire when the walker’s itinerary includes the edge itself. Visiting an edge reference (`visit [edge ->]`) or spawning onto an edge runs the edge’s abilities and then carries the walker on to the target node, while a plain node visit such as `visit [->]` moves between nodes without waking the edges between them.

A.4 Statements and Expressions

The statement and expression core of the language is Python’s, with brace-and-semicolon lexical form and a set of additions. Rather than narrate constructs the

Table A.4: Operator families of the Jac expression language.

Family	Forms	Notes
Arithmetic	<code>+ - * / // % ** @</code>	Python semantics throughout
Comparison	<code>== != < <= > >=, is, in</code>	Chainable, as in Python
Logical	<code>and or not</code>	Short-circuiting
Bitwise	<code>& ^ ~ « »</code>	Integers; set algebra on sets
Assignment	<code>=, :=, += and kin</code>	Walrus binds in expressions
Ternary	<code>a if c else b</code>	
Cast	<code>v as T</code>	Unchecked static re-typing
Delegation	<code>expr by target</code>	Right-associative; by <code>llm()</code>
Null-safe	<code>?. ?[]</code>	None instead of raising
Pipes	<code> > < ; :> <; :.> <.</code>	Function, atomic, method
Connection	<code>++> <++ <+>; +>:E(...):+></code>	Graph construction
Edge reference	<code>[->] [->:E:->] [?:T] [edge ->]</code>	See Appendix A.3
Spawn	<code>loc spawn W(...)</code>	Yields response with <code>.reports</code>
Concurrency	<code>flow f(); wait fut</code>	Launch and join parallel work
Ownership	<code>own e; &e &mut e</code>	Rebox; borrows, erased at codegen

Table A.5: Statement forms of the Jac language.

Form	Meaning
<code>if c { } elif c { } else { }</code>	Conditional
<code>while c { } else { }</code>	Loop; else on normal completion
<code>forever { }</code>	Unconditional loop; exits by <code>break</code> or <code>return</code>
<code>for x in xs { } else { }</code>	Iterator loop
<code>for i = 0 while i < n with i += 1 { }</code>	C-style loop: <code>init</code> , <code>bound</code> , <code>step</code>
<code>match v { case pat: ... }</code>	Structural patterns; indented bodies
<code>switch v { case k: ... default: }</code>	C-like dispatch with <code>fallthrough</code>
<code>try { } except T as e { } finally { }</code>	Exceptions; else clause allowed
<code>raise E(...) from cause;</code>	Raise, with optional chaining
<code>assert c, "message";</code>	Assertion
<code>with expr as name { }</code>	Context manager
<code>return e; / yield e; / yield from g;</code>	Function returns and generators
<code>report e;</code>	Append to the walker's reports
<code>skip;</code>	Return from the current ability
<code>disengage;</code>	Terminate the walker
<code>visit targets; (optional else { })</code>	Queue traversal destinations
<code>del x; / a del -> b;</code>	Delete a node or object / an edge
<code>in h { }</code>	Open region <code>h</code> for allocation
<code>break; / continue;</code>	Loop control

reader already knows, we tabulate: Table A.4 groups the operator families, and Table A.5 the statement forms.

A few placements in the precedence order deserve note, since they are where Jac's additions meet the inherited core. The `by` operator is right-associative and binds loosely, immediately above the walrus, so an entire expression is delegated rather than a fragment of it. The `as` cast sits immediately below the ternary, so `a + b as T` casts the sum. The pipe operators bind more loosely than arithmetic, so `a + b |> f` pipes

the sum into `f`. At the tight end, `spawn`, the atomic pipes, and the connection operators bind immediately below `call`, `index`, and `attribute access`, which is why chains like `root ++> a ++> b` and here `+:Post():+:> Tweet(...)` parse without parentheses. Comprehensions, generator expressions, f-strings with format specifications, slicing, and unpacking all carry over from Python unchanged; the special filter comprehension `items[?value > 0]` extends indexing with the same attribute-filter syntax used in edge references.

One departure from the Python core deserves its own paragraph, because it is invisible in a table: assignment scoping. There are no `global` or `nonlocal` declarations. A bare assignment in a function body, including the augmented forms such as `+=`, binds to the nearest enclosing binding of the name, whether that is a local of an enclosing function or a module-level variable declared with `glob`, and introduces a new local only when no enclosing binding exists. Shadowing an outer name is therefore always explicit: a typed declaration, as in `x: int = 5;`, creates a fresh local regardless of what encloses it, and must appear before the name's first use in the scope.

Concurrency surfaces twice in the table. The `async` and `await` keywords mark cooperative, event-loop concurrency exactly as in Python, and walkers as well as functions may be `async`. The `flow` and `wait` pair is thread-pool parallelism: `flow f()` launches the call in the background and evaluates to a future, and `wait` joins it.

The ownership surface is a family of binding markers and one statement, all `opt-in`, defined as semantics in Chapter 5 and discharged by the machinery of Chapter 12. A typed declaration may carry a marker (`x: own T = e` for unique ownership with checked moves, `x: imm T = e` for deep, transitive immutability), and parameter positions accept the same prefixes. The move discipline is affine: an owned value that is never consumed simply dies at scope exit, and only use after a move is an error. The borrow forms `&e` and `&mut e` lend shared or exclusive access without consuming the owner and compile to the bare operand on every substrate. The statement `in h { ... }` opens the region named by the handle `h`, an ordinary owned value of the ambient type `Region`, so that allocations within the block belong to it and are reclaimed together at the handle's death; `in Region() { ... }` opens an anonymous region coextensive with the block. Escapes are policed at the region boundary rather than within it: scalars copy out freely, a prefix `own e` reboxes a scalar or string into a fresh copy that lawfully exits, and a helper taking a `&Region` parameter is a legal carrier of region-rooted values. The markers also gate the concurrency constructs above: a value crosses `flow` and `wait` only as a deeply immutable `imm`, as a moved owner (a moved `own Region` carries its whole population, uncopied), or as a scalar, and a live borrow may not cross at all. Finally, an archetype may declare a reserved ability named `drop`, the destructor hook run once at the value's death; like `postinit` it is a name, not a keyword. Listing A.13 shows the surface in one program. Three caveats: region opens are not yet accepted inside modules compiled under the enforced zero-RC regime of Chapter 5; on the managed substrate `drop` fires automatically only for region-allocated values; and the native backend currently places some drops at frame rather than statement granularity, so a program should rely on drops landing between last use and scope exit, not on their exact position within that span.

```

    has payload: str = "";
    def drop { print("released: " + self.payload); }
}

def stamp(p: &mut Packet) { p.payload = "stamped"; }

def send(p: own Packet) -> int { return len(p.payload); }

with entry {
  a: own Packet = Packet(payload="hi");
  stamp(&mut a);           # exclusive borrow; a remains the owner
  print(send(a));          # move: reading a again would be an error
  d: imm Packet = Packet(payload="fixed");
  print(d.payload);        # readable, but deeply immutable
  title = "";
  r: own Region = Region();
  in r {
    b = Packet(payload="arena");
    title = own b.payload; # rebox: a copy that may exit the region
  }
  print(title);            # prints "arena"; r dies after its last use
}

```

Listing A.13: The ownership surface in one program: markers, a borrow, a move, and a region open with a reboxed escape. When the handle `r` dies, the region's population is reclaimed in one motion and its drop hooks run.

A.5 The Client Tier and Python Interoperation

Everything to this point describes the language independent of substrate; this final section covers the two boundaries a full-stack program crosses most often. The design argument that neither crossing is a discontinuity is made in Chapter 3; here we record only the syntax.

Client code lives in a `.cl.jac` annex, in a `cl { ... }` block, or, most often, in a markerless module whose JSX expressions and `npm` imports the compiler infers as client code. Within it, functions returning `JsxElement` are user-interface components, and JSX-style element syntax is part of the expression grammar. A component declares its local state with the same `has` syntax an archetype uses; assigning to a `has` variable re-renders the component. The ability syntax reappears with client semantics: `can with entry` runs when the component mounts, and `can with [dep] entry` re-runs when a dependency changes. A `try` block in a component may carry an awaiting clause naming its loading state: the awaiting body renders while the `try` body's asynchronous work is outstanding, a suspense fallback carried in the grammar rather than in a framework idiom. Listing A.14 shows a component from the day-planner reference application, condensed: it fetches its data on mount by awaiting `get_tasks`, a `def:pub` function declared in the same module's server codespace. The call site is an ordinary typed function call: the compiler synthesizes the HTTP request, serialization, and rehydration of `Task` values. Client walker spawns follow the same pattern: `root`

spawn `load_feed()` from client code invokes the served walker on the caller's root and returns its typed reports. Imports in client code resolve against npm (`import from react { useState }`), and asset imports such as `import "./styles.css"`; participate in the build.

```
import "./styles.css";

def:pub TaskList -> JsxElement {
  has tasks: List[Task] = [];

  async can with entry {
    tasks = await get_tasks();
  }

  return
    <div class="column">
      <h2>Today's Tasks</h2>
      {[<li key={jid(t)}>{t.title}</li> for t in tasks]}
    </div>;
}
```

Listing A.14: A client component: state with `has`, a mount ability, and a typed call into the server codespace. No placement marker appears; the JSX and the asset import place the module on the client.

On the server substrate, Jac compiles to Python bytecode, and interoperation with Python is import syntax rather than a foreign-function interface. A Jac module imports Python packages directly, as in Listing A.15, and the reverse direction is equally plain: once the toolchain is installed, Python's import machinery loads `.jac` files transparently, so from `graph_tools` import `Task` in a `.py` file imports a node type declared in `graph_tools.jac` and constructs instances of it as ordinary Python objects. For code that must remain verbatim Python (legacy logic, performance-tuned routines), a `::py::` block embeds Python source inline in a `.jac` file, and its definitions are visible to the surrounding module. Values crossing from untyped Python arrive typed as `any`, and the gradual-typing discipline of Chapter 9 governs their flow: an `any` value does not silently enter a declared non-`any` destination, and is admitted by typing the source, annotating the destination as `any` and narrowing, or casting explicitly with `as`.

```
import pandas as pd;
import from sklearn.linear_model { LinearRegression }

with entry {
  df = pd.DataFrame({"a": [1, 2, 3], "b": [4, 5, 6]});
  model = LinearRegression();
  print(df.describe());
}
```

Listing A.15: Python interoperation: PyPI packages imported and used directly.

The native tier completes the substrate set (`.na.jac` annexes compile through LLVM, import C libraries by declaring their signatures in an `import from` block, and

exchange calls with the Python codespace), but its surface adds only the fixed-width numeric types and the foreign-signature import form to what this appendix has already covered. For the machinery beneath all three lowerings, the reader should turn to the compiler and runtime chapters of Part IV; for the constructs' semantics, to [Appendix B](#); and for every coined term used along the way, to the glossary in [Appendix C](#).

THE OBJECT-SPATIAL CALCULUS: RULES AND PROOFS

This appendix completes the formal development that Chapter 8 began and Chapter 9 extended with statics. The nine-rule core presented there was chosen for readability; here we give the complete system: the full syntax (Appendix B.1), every reduction rule (Appendix B.2), the typing judgments for nodes, edges, walkers, and ability dispatch (Appendix B.3), and the metatheory (Appendix B.4), which covers type safety in the syntactic style of Wright and Felleisen (1994), the single-walker determinism theorem with a full proof, and the actor encoding Chapter 8 promised. We close with an inventory of what is proved and what is conjectured (Appendix B.5).

Notation is carried over verbatim from Chapter 8: N, E, W range over node, edge, and walker types, n, m over node identifiers, r names the root, $\mathcal{G} = (N, \mathcal{E}, r)$ is the graph, configurations are $\langle \mathcal{W} \mid \mathcal{G} \rangle$, and $\longrightarrow$ is the reduction relation, in the structural style of Plotkin (1981), refining the abstract machine of Mars (2025). New metavariables (ι for edge identifiers, t for thread names, X for exit stacks) are introduced as they arise.

B.1 The Full Syntax

The full calculus extends the core of Chapter 8 in six places: edges acquire identifiers and typed fields; threads acquire names, which are ordinary values, so that a spawner can later collect a walker’s reports; statements acquire a loop form, plain let, deletion, the indexed visit, and the wait form that reads a terminated walker’s report log; abilities acquire exit clauses; and the expression root denotes r , matching the typing

context of Chapter 9. Everything else is as before.

values	$v ::= c \mid n \mid \bar{n} \mid \iota \mid \bar{\iota} \mid t$
expressions	$e ::= v \mid x \mid \text{here} \mid \text{root} \mid \text{here}.f \mid \text{self}.f \mid e.f$ $\mid e \xrightarrow{E} \text{edges}(e, E) \mid \text{op}(\bar{e})$
statements	$s ::= \text{skip} \mid s_1; s_2 \mid \text{if } e \text{ then } s_1 \text{ else } s_2 \mid \text{while } e \text{ do } s \mid \text{let } x = e \text{ in } s$ $\mid \text{self}.f := e \mid \text{here}.f := e \mid e.f := e'$ $\mid \text{let } x = \text{new } N(\bar{f} = e) \text{ in } s \mid e_1 \xrightarrow{E(\bar{f}=e)} e_2 \mid \text{del } e$ $\mid \text{visit } e \mid \text{visit}_i e \mid \text{let } x = e \text{ spawn } W(\bar{f} = e) \text{ in } s$ $\mid \text{let } x = \text{wait } e \text{ in } s \mid \text{report } e \mid \text{disengage}$
walker abilities	$A_W ::= \text{on } N \text{ entry } \{s\} \mid \text{on } N \text{ exit } \{s\}$
node abilities	$A_N ::= \text{on } W \text{ entry } \{s\} \mid \text{on } \star \text{ entry } \{s\}$ $\mid \text{on } W \text{ exit } \{s\} \mid \text{on } \star \text{ exit } \{s\}$
declarations	$D ::= \text{node } N \{\bar{f}; \bar{A}_N\} \mid \text{walker } W \{\bar{f}; \bar{A}_W\}$ $\mid \text{edge } E : N_s \rightarrow N_t \{\bar{f}\}$
programs	$P ::= \bar{D}; s_{\text{main}}$

Field declarations $\bar{f}$ carry types and constant defaults, as in Chapter 9; an edge declaration may omit its endpoint bounds, in which case they default to the permissive top bound. The core forms of Chapter 8 are recovered by erasure: the unindexed statement $e \text{ spawn } W(\bar{f}=e)$ is $\text{let } x = e \text{ spawn } W(\bar{f}=e) \text{ in skip}$ with x unused, and the unattributed connect is the attributed one with an empty field list.

The store gives edges identity and state, keeping creation order as the order of bindings:

$$\mathcal{G} ::= (N, \mathcal{E}, r) \quad N ::= \overline{n \mapsto N\{f = v\}} \quad \mathcal{E} ::= \overline{\iota \mapsto (n \xrightarrow{E\{\bar{f}=v\}} m)}$$

A thread carries one component beyond those of Chapter 8: the exit stack X , the node identifiers at which exit abilities are owed. Threads are named; we suppress the name wherever a rule neither creates nor consults it.

$$T ::= (w, p, q, X, R, s)_t \mid \text{done}(w, R)_t \quad X ::= \bar{n} \quad p ::= n \mid \bullet$$

Surface constructs desugar as in Chapter 8, with three additions. The deletion arrow $\text{del } \rightarrow b$ is del applied to the matching subsequence of $\text{edges}(a, \star)$; the indexed $\text{visit } :0:$ and $\text{visit } :-1:$ are visit_0 and plain visit ; and spawning a walker and reading its reports field is $\text{let } x = e \text{ spawn } W(\dots) \text{ in let } y = \text{wait } x \text{ in } s$. Edge-predicate filters such as $[->:\text{Friend}:\text{since } > 2020:->]$ desugar to conditionals over $\text{edges}(e, E)$ and their fields, as Chapter 8 noted for node filters.

B.2 The Complete Small-Step System

Evaluation extends the partial function $\llbracket e \rrbracket_{w,n,\mathcal{G}}$ of Chapter 8 pointwise: $\llbracket \text{root} \rrbracket = r$; $\llbracket e.f \rrbracket$ is the f field of the edge bound to $\llbracket e \rrbracket$; the node query is as before, read off

the attributed store in creation order; and $\llbracket \text{edges}(e, E) \rrbracket$ is the sequence of matching edge identifiers, in the same order. Evaluation remains a partial function, undefined exactly when it dereferences an identifier absent from the store: the situation deletion creates, and the one blemish on progress that Appendix B.4 accounts for.

Ability activation splits into an entry and an exit metafunction: $\text{fire}_{\text{en}}(W, N)$ is precisely the fire of Chapter 8, computed over entry clauses, and $\text{fire}_{\text{ex}}(W, N)$ is the same composition over exit clauses: walker clauses, then the node's $\star$ clauses, then the node's typed clauses, declaration order within each group, node-side bodies dualized. Both are total, yielding skip when nothing matches.

The relation $\longrightarrow$ is the least relation closed under PAR of Chapter 8 and the rules below, given in four groups. The first is administrative: sequencing, branching, iteration, binding, and the walker-field write, none of which touch the graph or the itinerary.

$$\begin{array}{c}
 \text{SEQ} \\
 \hline
 \langle (w, p, q, X, R, \mathcal{K}[\text{skip}; s]) \mid \mathcal{G} \rangle \longrightarrow \langle (w, p, q, X, R, \mathcal{K}[s]) \mid \mathcal{G} \rangle \\
 \\
 \text{IF-TRUE} \\
 \hline
 \llbracket e \rrbracket = \text{true} \\
 \hline
 \langle (w, p, q, X, R, \mathcal{K}[\text{if } e \text{ then } s_1 \text{ else } s_2]) \mid \mathcal{G} \rangle \longrightarrow \langle (w, p, q, X, R, \mathcal{K}[s_1]) \mid \mathcal{G} \rangle \\
 \\
 \text{IF-FALSE} \\
 \hline
 \llbracket e \rrbracket = \text{false} \\
 \hline
 \langle (w, p, q, X, R, \mathcal{K}[\text{if } e \text{ then } s_1 \text{ else } s_2]) \mid \mathcal{G} \rangle \longrightarrow \langle (w, p, q, X, R, \mathcal{K}[s_2]) \mid \mathcal{G} \rangle \\
 \\
 \text{WHILE} \\
 \hline
 \langle (w, p, q, X, R, \mathcal{K}[\text{while } e \text{ do } s]) \mid \mathcal{G} \rangle \longrightarrow \\
 \langle (w, p, q, X, R, \mathcal{K}[\text{if } e \text{ then } (s; \text{while } e \text{ do } s) \text{ else skip}]) \mid \mathcal{G} \rangle \\
 \\
 \text{LET} \\
 \hline
 \llbracket e \rrbracket = v \\
 \hline
 \langle (w, p, q, X, R, \mathcal{K}[\text{let } x = e \text{ in } s]) \mid \mathcal{G} \rangle \longrightarrow \langle (w, p, q, X, R, \mathcal{K}[s[v/x]]) \mid \mathcal{G} \rangle \\
 \\
 \text{STEP-SELF} \\
 \hline
 \llbracket e \rrbracket = v \quad w' = w[f \mapsto v] \\
 \hline
 \langle (w, p, q, X, R, \mathcal{K}[\text{self}.f := e]) \mid \mathcal{G} \rangle \longrightarrow \langle (w', p, q, X, R, \mathcal{K}[\text{skip}]) \mid \mathcal{G} \rangle
 \end{array}$$

The second group governs motion and the walker life cycle. ARRIVE now pushes the destination onto the exit stack, recording a debt that EXIT repays in last-in, first-out order once the queue drains: the unwinding discipline of Chapter 8, a call-stack shape laid over the graph. The GONE variants discard destinations that deletion has invalidated, so a stale itinerary is skipped rather than stuck. SPAWN binds the fresh thread's name in its continuation, and JOIN is the report-accumulation rule: a wait on a terminated thread reads its report log and nothing else. DISENGAGE discards statement, queue, and exit stack alike: the walk ends immediately and pending exits are forgiven, a reading we flag in Appendix B.5.

$$\begin{array}{c}
 \text{SPAWN} \\
 \frac{\llbracket e \rrbracket = m \quad m \in \text{dom}(\mathcal{N}) \quad \llbracket \bar{e} \rrbracket = \bar{v} \quad w' = W' \{ \overline{f = v} \} \quad t' \text{ fresh}}{\langle (w, p, q, X, R, \mathcal{K}[\text{let } x = e \text{ spawn } W'(\overline{f = v}) \text{ in } s])_t \mid \mathcal{G} \rangle \longrightarrow \langle (w, p, q, X, R, \mathcal{K}[s[t'/x]])_t \parallel (w', \bullet, m, \epsilon, \epsilon, \text{skip})_{t'} \mid \mathcal{G} \rangle} \\
 \\
 \text{ARRIVE} \\
 \frac{w = W\{\dots\} \quad \mathcal{G}(m) = N\{\dots\} \quad \text{fire}_{\text{en}}(W, N) = s}{\langle (w, p, m \cdot q, X, R, \text{skip}) \mid \mathcal{G} \rangle \longrightarrow \langle (w, m, q, m \cdot X, R, s) \mid \mathcal{G} \rangle} \\
 \\
 \text{ARRIVE-GONE} \\
 \frac{m \notin \text{dom}(\mathcal{N})}{\langle (w, p, m \cdot q, X, R, \text{skip}) \mid \mathcal{G} \rangle \longrightarrow \langle (w, p, q, X, R, \text{skip}) \mid \mathcal{G} \rangle} \\
 \\
 \text{VISIT} \\
 \frac{\llbracket e \rrbracket = \bar{m}}{\langle (w, n, q, X, R, \mathcal{K}[\text{visit } e]) \mid \mathcal{G} \rangle \longrightarrow \langle (w, n, q \cdot \bar{m}, X, R, \mathcal{K}[\text{skip}]) \mid \mathcal{G} \rangle} \\
 \\
 \text{VISIT-AT} \\
 \frac{\llbracket e \rrbracket = \bar{m} \quad q' = \text{splice}(q, \bar{m}, i)}{\langle (w, n, q, X, R, \mathcal{K}[\text{visit}_i e]) \mid \mathcal{G} \rangle \longrightarrow \langle (w, n, q', X, R, \mathcal{K}[\text{skip}]) \mid \mathcal{G} \rangle} \\
 \\
 \text{EXIT} \\
 \frac{w = W\{\dots\} \quad \mathcal{G}(m) = N\{\dots\} \quad \text{fire}_{\text{ex}}(W, N) = s}{\langle (w, p, \epsilon, m \cdot X, R, \text{skip}) \mid \mathcal{G} \rangle \longrightarrow \langle (w, m, \epsilon, X, R, s) \mid \mathcal{G} \rangle} \\
 \\
 \text{EXIT-GONE} \\
 \frac{m \notin \text{dom}(\mathcal{N})}{\langle (w, p, \epsilon, m \cdot X, R, \text{skip}) \mid \mathcal{G} \rangle \longrightarrow \langle (w, p, \epsilon, X, R, \text{skip}) \mid \mathcal{G} \rangle} \\
 \\
 \text{FINISH} \\
 \frac{}{\langle (w, p, \epsilon, \epsilon, R, \text{skip}) \mid \mathcal{G} \rangle \longrightarrow \langle \text{done}(w, R) \mid \mathcal{G} \rangle} \\
 \\
 \text{DISENGAGE} \\
 \frac{}{\langle (w, n, q, X, R, \mathcal{K}[\text{disengage}]) \mid \mathcal{G} \rangle \longrightarrow \langle \text{done}(w, R) \mid \mathcal{G} \rangle} \\
 \\
 \text{JOIN} \\
 \frac{\llbracket e \rrbracket = t'}{\langle (w, n, q, X, R, \mathcal{K}[\text{let } x = \text{wait } e \text{ in } s])_t \parallel \text{done}(w', R')_{t'} \mid \mathcal{G} \rangle \longrightarrow \langle (w, n, q, X, R, \mathcal{K}[s[R'/x]])_t \parallel \text{done}(w', R')_{t'} \mid \mathcal{G} \rangle}
 \end{array}$$

Here $\text{splice}(q, \bar{m}, i)$ inserts $\bar{m}$ at position i of q , out-of-range indices falling back to the tail; visit_0 therefore yields depth-first traversal and plain visit breadth-first, as Chapter 8 described.

The third group is graph mutation. **NEW-EDGE** now installs an attributed, identified edge; **STEP-EDGE** updates an edge field in place; the two deletion rules are keyed

on the shape of the evaluated operand (node identifier or edge-identifier sequence) and are never simultaneously applicable. We write $\mathcal{G} \setminus m$ for the store with m and every incident edge removed, and $\mathcal{G} \setminus \bar{\iota}$ for removal of the listed edges; both preserve the creation order of what remains, and the former keeps the store free of dangling endpoints, an invariant the preservation theorem leans on.

$$\frac{\text{NEW-NODE} \quad \llbracket \bar{e} \rrbracket = \bar{v} \quad n' \notin \text{dom}(\mathcal{N}) \quad \mathcal{G}' = \mathcal{G}[n' \mapsto N\{\bar{f} = \bar{v}\}]}{\langle (w, n, q, X, R, \mathcal{K}[\text{let } x = \text{new } N(\bar{f} = \bar{e}) \text{ in } s]) \mid \mathcal{G} \rangle \longrightarrow \langle (w, n, q, X, R, \mathcal{K}[s[n'/x]]) \mid \mathcal{G}' \rangle}$$

NEW-EDGE

$$\frac{n_1, n_2 \in \text{dom}(\mathcal{N}) \quad \llbracket \bar{e} \rrbracket = \bar{v} \quad \iota \notin \text{dom}(\mathcal{E}) \quad \mathcal{G}' = \mathcal{G} \cdot (\iota \mapsto n_1 \xrightarrow{E\{\bar{f} = \bar{v}\}} n_2)}{\langle (w, n, q, X, R, \mathcal{K}[e_1 \xrightarrow{E(\bar{f} = \bar{e})} e_2]) \mid \mathcal{G} \rangle \longrightarrow \langle (w, n, q, X, R, \mathcal{K}[\text{skip}]) \mid \mathcal{G}' \rangle}$$

STEP-LOCAL

$$\frac{\llbracket e \rrbracket = v \quad n \in \text{dom}(\mathcal{N}) \quad \mathcal{G}' = \mathcal{G}[n.f \mapsto v]}{\langle (w, n, q, X, R, \mathcal{K}[\text{here}.f := e]) \mid \mathcal{G} \rangle \longrightarrow \langle (w, n, q, X, R, \mathcal{K}[\text{skip}]) \mid \mathcal{G}' \rangle}$$

STEP-EDGE

$$\frac{\llbracket e_0 \rrbracket = \iota \quad \iota \in \text{dom}(\mathcal{E}) \quad \llbracket e \rrbracket = v \quad \mathcal{G}' = \mathcal{G}[\iota.f \mapsto v]}{\langle (w, n, q, X, R, \mathcal{K}[e_0.f := e]) \mid \mathcal{G} \rangle \longrightarrow \langle (w, n, q, X, R, \mathcal{K}[\text{skip}]) \mid \mathcal{G}' \rangle}$$

DEL-NODE

$$\frac{\llbracket e \rrbracket = m \quad m \in \text{dom}(\mathcal{N}) \quad \mathcal{G}' = \mathcal{G} \setminus m}{\langle (w, n, q, X, R, \mathcal{K}[\text{del } e]) \mid \mathcal{G} \rangle \longrightarrow \langle (w, n, q, X, R, \mathcal{K}[\text{skip}]) \mid \mathcal{G}' \rangle}$$

DEL-EDGE

$$\frac{\llbracket e \rrbracket = \bar{\iota} \quad \bar{\iota} \subseteq \text{dom}(\mathcal{E}) \quad \mathcal{G}' = \mathcal{G} \setminus \bar{\iota}}{\langle (w, n, q, X, R, \mathcal{K}[\text{del } e]) \mid \mathcal{G} \rangle \longrightarrow \langle (w, n, q, X, R, \mathcal{K}[\text{skip}]) \mid \mathcal{G}' \rangle}$$

The fourth group is report accumulation and interleaving, unchanged from Chapter 8 except for the exit-stack component threaded through REPORT.

REPORT

$$\frac{\llbracket e \rrbracket = v}{\langle (w, n, q, X, R, \mathcal{K}[\text{report } e]) \mid \mathcal{G} \rangle \longrightarrow \langle (w, n, q, X, R \cdot v, \mathcal{K}[\text{skip}]) \mid \mathcal{G} \rangle}$$

PAR

$$\frac{\langle \mathcal{W}_1 \mid \mathcal{G} \rangle \longrightarrow \langle \mathcal{W}'_1 \mid \mathcal{G}' \rangle}{\langle \mathcal{W}_1 \parallel \mathcal{W}_2 \mid \mathcal{G} \rangle \longrightarrow \langle \mathcal{W}'_1 \parallel \mathcal{W}_2 \mid \mathcal{G}' \rangle}$$

Remark B.1 (Conservativity over the core). For a program in the core fragment of Chapter 8—no exit clauses, attributed edges, deletion, or wait—the extended system's

runs differ from the core's only in bookkeeping: every `ARRIVE` pushes a stack entry that a later `EXIT` pops with $\text{fire}_{\text{ex}} = \text{skip}$. Arrival order, report sequences, and the final graph are unchanged, so every result of Chapter 8 transfers to the full system, as claimed there.

B.3 The Type System

The statics elaborate the fragment displayed in Chapter 9 into a full set of judgments. Types are

$$\tau ::= b \mid N \mid E \mid W \mid \text{list } \tau \mid \text{thr } W$$

where b ranges over base types, archetype names are types of their kind, and $\text{thr } W$ types the name of a spawned thread running walker type W . The signature Σ collects the declarations: $\Sigma(N)$ is a record $\overline{f : \tau}$ with N 's ability table, $\Sigma(E) = (N_s, N_t, \overline{f : \tau})$ an edge type's endpoint bounds and fields, $\Sigma(W) = (\overline{f : \tau}, \rho_W, \overline{A_W})$ a walker's record, declared report type ρ_W , and abilities; $<$ is the nominal subtype order of Chapter 9. Throughout we work in the fully annotated fragment (any does not occur), the regime for which Chapter 9 claims classical safety.

The judgments are $\Sigma; \Gamma \vdash e : \tau$ for expressions, $\Sigma; \Gamma \vdash_W s \text{ ok}$ for statements in the body of walker type W (the form used in Chapter 9), and $\Sigma \vdash D \text{ ok}$ for declarations, with `self`, `here`, and `root` typed by lookup in Γ . The structural rules (skip, sequencing, conditionals with boolean guards, while, let, subsumption) are standard and omitted. The characteristic expression rules follow.

$$\begin{array}{c}
 \text{T-SELF-FIELD} \quad \frac{\Gamma(\text{self}) = W \quad \Sigma(W)(f) = \tau}{\Sigma; \Gamma \vdash \text{self}.f : \tau} \qquad \text{T-HERE-FIELD} \quad \frac{\Gamma(\text{here}) = N \quad \Sigma(N)(f) = \tau}{\Sigma; \Gamma \vdash \text{here}.f : \tau} \\
 \\
 \text{T-EDGE-FIELD} \quad \frac{\Sigma; \Gamma \vdash e : E \quad \Sigma(E) = (N_s, N_t, \overline{f : \tau}) \quad f_i = f}{\Sigma; \Gamma \vdash e.f : \tau_i} \\
 \\
 \text{T-QUERY} \quad \frac{\Sigma; \Gamma \vdash e : N' \quad \Sigma(E) = (N_s, N_t, _) \quad N' <: N_s}{\Sigma; \Gamma \vdash e \xrightarrow{E} : \text{list } N_t} \\
 \\
 \text{T-EDGES} \quad \frac{\Sigma; \Gamma \vdash e : N' \quad \Sigma(E) = (N_s, N_t, _) \quad N' <: N_s}{\Sigma; \Gamma \vdash \text{edges}(e, E) : \text{list } E}
 \end{array}$$

T-QUERY is the rule displayed in Chapter 9: traversal along a declared relation is typed at the bound the direction of motion selects. The statement rules bind the topological constructs to the signature, with **T-CONNECT** that of Chapter 9 extended to edge fields.

$$\begin{array}{c}
\text{T-ASSIGN-SELF} \\
\frac{\Gamma(\text{self}) = W \quad \Sigma(W)(f) = \tau \quad \Sigma; \Gamma \vdash e : \tau' \quad \tau' <: \tau}{\Sigma; \Gamma \vdash_W \text{self}.f := e \text{ ok}} \\
\\
\text{T-ASSIGN-HERE} \\
\frac{\Gamma(\text{here}) = N \quad \Sigma(N)(f) = \tau \quad \Sigma; \Gamma \vdash e : \tau' \quad \tau' <: \tau}{\Sigma; \Gamma \vdash_W \text{here}.f := e \text{ ok}} \\
\\
\text{T-NEW} \\
\frac{\Sigma(N) = \overline{f : \tau} \quad \Sigma; \Gamma \vdash \bar{e} : \bar{\tau}' \quad \bar{\tau}' <: \bar{\tau} \quad \Sigma; \Gamma, x : N \vdash_W s \text{ ok}}{\Sigma; \Gamma \vdash_W \text{let } x = \text{new } N(\overline{f=e}) \text{ in } s \text{ ok}} \\
\\
\text{T-CONNECT} \\
\frac{\Sigma(E) = (N_s, N_t, \overline{f : \tau}) \quad \Sigma; \Gamma \vdash e_1 : N_1 \quad \Sigma; \Gamma \vdash e_2 : N_2 \quad N_1 <: N_s \quad N_2 <: N_t \quad \Sigma; \Gamma \vdash \bar{e} : \bar{\tau}' \quad \bar{\tau}' <: \bar{\tau}}{\Sigma; \Gamma \vdash_W e_1 \xrightarrow{E(\overline{f=e})} e_2 \text{ ok}} \\
\\
\text{T-VISIT} \\
\frac{\Sigma; \Gamma \vdash e : \text{list } N' \quad N' \text{ a node type}}{\Sigma; \Gamma \vdash_W \text{visit } e \text{ ok}} \\
\\
\text{T-SPAWN} \\
\frac{\Sigma(W') = (\overline{f : \tau}, \rho, _) \quad \Sigma; \Gamma \vdash e : N' \quad \Sigma; \Gamma \vdash \bar{e} : \bar{\tau}' \quad \bar{\tau}' <: \bar{\tau} \quad \Sigma; \Gamma, x : \text{thr } W' \vdash_W s \text{ ok}}{\Sigma; \Gamma \vdash_W \text{let } x = e \text{ spawn } W'(\overline{f=e}) \text{ in } s \text{ ok}} \\
\\
\text{T-WAIT} \quad \text{T-REPORT} \\
\frac{\Sigma; \Gamma \vdash e : \text{thr } W' \quad \Sigma; \Gamma, x : \text{list } \rho_{W'} \vdash_W s \text{ ok}}{\Sigma; \Gamma \vdash_W \text{let } x = \text{wait } e \text{ in } s \text{ ok}} \quad \frac{\Sigma; \Gamma \vdash e : \tau \quad \tau <: \rho_W}{\Sigma; \Gamma \vdash_W \text{report } e \text{ ok}} \\
\\
\text{T-DEL} \\
\frac{\Sigma; \Gamma \vdash e : \tau \quad \tau \in \{N', E', \text{list } E'\}}{\Sigma; \Gamma \vdash_W \text{del } e \text{ ok}}
\end{array}$$

The rules for visit_i , $e.f := e'$, and disengage are the evident analogues; T-VISIT generalizes the composite rule of Chapter 9, recovered by composing with T-QUERY. Ability dispatch is typed on both sides of an encounter: a walker clause under the assumptions its event clause announces (the rule of Chapter 9, with exit clauses typed identically) and a node clause in its dualized form, the form fire_{en} and fire_{ex} splice into a thread. A node's $\star$ clause must be typable for every declared walker type, of which there are finitely many.

$$\frac{\text{T-ABILITY-W} \quad \Sigma; \Gamma, \text{self} : W, \text{here} : N, \text{root} : \text{Root} \vdash_W s \text{ ok}}{\Sigma; \Gamma \vdash_W \text{on } N \text{ entry } \{s\} \text{ ok}}$$

$$\frac{\text{T-ABILITY-N} \quad \Sigma; \Gamma, \text{self} : W, \text{here} : N, \text{root} : \text{Root} \vdash_W \text{dual}(s) \text{ ok}}{\Sigma; \Gamma \vdash_N \text{on } W \text{ entry } \{s\} \text{ ok}}$$

A program is well typed, $\vdash P \text{ ok}$, when every declaration checks under Σ and s_{main} checks in the inert main walker's body with $\text{here} : \text{Root}$.

Safety is stated over configurations, so the judgments must extend to run-time objects. This is the store-typing apparatus of Wright and Felleisen (1994), adapted to a graph.

Definition B.2 (Well-formed store). $\vdash \mathcal{G} : \Sigma$ holds when (i) every binding $n \mapsto N\{\bar{f} = \bar{v}\}$ has exactly the fields $\Sigma(N)$, each value of its declared type (node identifiers typed by their store bindings); and (ii) every edge $\iota \mapsto (n_1 \xrightarrow{E\{\bar{f}=\bar{v}\}} n_2)$ has $n_1, n_2 \in \text{dom}(N)$ with dynamic types refining the bounds $\Sigma(E)$, and correctly typed fields.

Definition B.3 (Well-formed configuration). $\vdash C : \Sigma$ holds when $\vdash \mathcal{G} : \Sigma$, thread names are pairwise distinct, every thread-name value occurring in C names a thread of C of the walker type its thr type declares, and every active thread $(w, p, q, X, R, s)_t$ with $w = W\{\dots\}$ satisfies: the fields of w are typed per $\Sigma(W)$; every element of R has type ρ_W ; q and X contain node identifiers; and there is a node type N such that $\Sigma; \text{self} : W, \text{here} : N, \text{root} : \text{Root} \vdash_W s \text{ ok}$ and, if $p \in \text{dom}(N)$, the dynamic type of $\mathcal{G}(p)$ refines N . For $\text{done}(w, R)_t$ only the record and report conditions apply.

The existential N is the subtlety deletion forces: a thread whose current node has been deleted out from under it (by itself, via del here , or by a concurrent walker) remains well formed (its residual statement is still typed under the arrival-time assumption), but evaluation at here may now be undefined. Preservation is untouched. Progress must name it, and does.

B.4 Metatheory

B.4.1 TYPE SAFETY

Three lemmas carry the safety proofs. The first is standard, and its proof, with weakening, is the usual induction.

Lemma B.4 (Substitution). *If $\Sigma; \Gamma, x : \tau' \vdash_W s \text{ ok}$ and v has type $\tau'' <: \tau'$ in a well-formed store, then $\Sigma; \Gamma \vdash_W s[v/x] \text{ ok}$.*

Lemma B.5 (Evaluation). *Suppose $\vdash \mathcal{G} : \Sigma$, the walker record and current node satisfy the typing context, and $\Sigma; \Gamma \vdash e : \tau$ for closed e . Then $\llbracket e \rrbracket_{w,n,\mathcal{G}}$, if defined, is a value whose dynamic type refines τ ; and it is defined unless evaluation dereferences an identifier absent from the store. In a run of the deletion-free system it is always defined.*

Proof. Induction on e . Constants, `here`, `root`, and `self.f` are immediate; `here.f` and `e.f` are defined iff their subject is live, and then store well-formedness supplies the field at its declared type. Queries and `edges(e, E)` are total once their subject evaluates, since they filter the finite store, and T-QUERY's bound premise with clause (ii) of Definition B.2 types their elements. `op` is a signature of total operations. In the deletion-free system identifiers never leave the store, so no dereference can fail. $\square$

Lemma B.6 (Dispatch). *If every declaration of P is well typed, W is a walker type, and N a node type, then $\Sigma; \text{self} : W, \text{here} : N, \text{root} : \text{Root} \vdash_W \text{fire}_{\text{en}}(W, N) \text{ ok}$, and likewise for fire_{ex} .*

Proof. Each spliced clause is typed by T-ABILITY-W or T-ABILITY-N under assumptions $\text{self} : W', \text{here} : N'$ with $W <: W'$ and $N <: N'$; narrowing the context along $<:$ preserves statement typing (routine induction), and sequential composition of well-typed statements is well typed. $\square$

Theorem B.7 (Preservation). *If $\vdash C : \Sigma$ and $C \longrightarrow C'$, then $\vdash C' : \Sigma$.*

Proof. Case analysis on the rule; PAR is the induction. The administrative group changes only the statement, whose residual typing follows from the redex's (LET via Lemma B.4, WHILE by unfolding). ARRIVE: the new statement is typed by Lemma B.6 with the existential witness N set to the dynamic type of $\mathcal{G}(m)$; EXIT is symmetric. SPAWN and JOIN follow from their typing rules, Lemma B.5, configuration well-formedness (the done thread's log has type list ρ_W), and Lemma B.4. NEW-EDGE is the crux for the topology: T-CONNECT puts the static types of e_1, e_2 below the bounds $\Sigma(E)$, and Lemma B.5 puts the dynamic types of n_1, n_2 below the static types, so the new edge satisfies clause (ii) of Definition B.2. DEL-NODE removes all incident edges with the node, so no dangling endpoint arises, and affected threads remain well formed because the position check is conditioned on liveness. The remaining cases are routine. This establishes, in passing, the topological preservation property stated in Chapter 9: every edge of a reachable store respects its declared bounds. $\square$

Theorem B.8 (Progress). *If $\vdash C : \Sigma$, then one of the following holds: (a) every thread of C is done; (b) $C \longrightarrow C'$ for some C' ; (c) some thread is stuck at a dangling dereference—an evaluation, connect, or spawn whose subject identifier has been deleted; or (d) every active thread is blocked at a wait whose target is not done. In the deletion-free system (c) is impossible; under the stack scheduling discipline of Theorem B.12, (d) is impossible.*

Proof. Suppose some thread is active. If its statement is `skip`: a nonempty queue enables ARRIVE or ARRIVE-GONE according as the head is live; an empty queue and nonempty stack enable EXIT or EXIT-GONE; both empty enable FINISH. Otherwise the statement decomposes as $\mathcal{K}[s_0]$ with s_0 a redex (Lemma B.10 below), and for each redex form some rule matches once its evaluation premises are defined—which Lemma B.5 guarantees unless a dereference dangles, case (c); guards evaluate to booleans by typing, and `del`'s operand shape selects exactly one deletion rule by T-DEL. The one redex with a premise beyond evaluation is `wait`: a live target leaves the thread blocked, and if every active thread is so blocked, that is (d)—which can genuinely occur under unconstrained scheduling, since two walkers that learn each other's names through the graph can wait on each other. Without deletion, identifiers are never

removed, so (c) cannot arise; under stack scheduling, the proof of Theorem B.12(ii) shows a schedulable thread's waits are always enabled. $\square$

Claim B.9 (Soundness). *By Theorems B.7 and B.8, a well-typed, deletion-free, stack-scheduled program never becomes stuck. With deletion, the sole failure mode is the dangling dereference of case (c), which the production runtime surfaces as a checked run-time error rather than undefined behavior.*

B.4.2 DETERMINISM

We now prove the determinism proposition of Chapter 8 in full, restated for the extended system. Call a statement a *redex* if it is either an atomic statement other than `skip`, or a composition `skip; s`.

Lemma B.10 (Unique decomposition). *Every statement other than `skip` admits exactly one decomposition $\mathcal{K}[s_0]$ with s_0 a redex.*

Proof. Induction on the statement. An atomic non-skip statement decomposes only as $[[s_0]]$. For `skip; s2`, the whole composition is the redex under $\mathcal{K} = []$, and no decomposition descends left, since `skip` contains no redex. For $s_1; s_2$ with $s_1 \neq \text{skip}$, induction gives $s_1 = \mathcal{K}_1[s_0]$ uniquely, hence $(\mathcal{K}_1; s_2)[s_0]$, and the top-level alternative is excluded because such a composition is not a redex. Contexts descend only into the left component, so no other decomposition exists. $\square$

Lemma B.11 (Functionality). *For a fixed thread and store, each rule's premises and conclusion are computed by partial functions, except for the choice of fresh identifiers in `NEW-NODE`, `NEW-EDGE`, and `SPAWN`. Moreover, any permutation π of identifiers fresh for C that respects edge-creation order is a bijection on configurations with $C \rightarrow C'$ iff $\pi(C) \rightarrow \pi(C')$.*

Proof. $[[\cdot]]$ is a partial function because $\mathcal{N}$ and $\mathcal{E}$ are finite maps and query results are read in creation order; `fireen`, `fireex`, `splice`, and the queue operations are functions because matching is decidable and declaration order total. For the renaming claim: no rule inspects an identifier beyond equality and store membership, and fresh edges are appended at the tail, so order-respecting permutations commute with every rule. Routine inspection. $\square$

Theorem B.12 (Single-walker determinism). *(i) Let C be well formed with exactly one active thread, whose statement contains no `spawn` form. If $C \rightarrow C_1$ and $C \rightarrow C_2$, then $C_2 = \pi(C_1)$ for a permutation of fresh identifiers; consequently C has exactly one maximal reduction sequence up to renaming, with uniquely determined arrival order, report sequence, and final graph. (ii) Under the stack discipline—among active threads, only the most recently created may step—the same conclusions hold for every well-formed configuration, spawns included.*

Proof. (i) The active thread is the only one that can step (done threads are inert, and `JOIN` steps the waiting thread). If its statement is `skip`, exactly one rule is enabled: with $q = m \cdot q'$, `ARRIVE` and `ARRIVE-GONE` have complementary side conditions on $m \in \text{dom}(\mathcal{N})$; with $q = \epsilon$ and X nonempty, likewise `EXIT` and `EXIT-GONE`; with both empty, only `FINISH`. Otherwise Lemma B.10 yields a unique redex, and at most one

rule matches it: the only forms served by two rules are the conditional and `del`, disjoint by the evaluated guard and operand shape respectively, and a `wait` redex has at most one matching done thread since names are distinct. By Lemma B.11 the conclusion is determined up to fresh identifiers, and any two choices differ by an order-respecting permutation, so $C_2 = \pi(C_1)$. Since π commutes with $\longrightarrow$, the sequences from C_1 and C_2 correspond pointwise; induction along the run gives uniqueness up to renaming, of which arrival order, reports, and the graph-up-to-renaming are invariants.

(ii) Creation order totally orders threads, so the discipline designates exactly one schedulable thread: the creation-maximal active one. The argument of (i) applies verbatim to it; the new case is `SPAWN`, which creates a fresh creation-maximal thread to which control immediately transfers—precisely the synchronous spawn the surface ships, as Chapter 8 observed. When the schedulable thread reaches a `wait`, its target is done: the only active threads are its spawn ancestors, and an ancestor’s name is unknowable to it, since the binder holding that name lies in a continuation that has not yet run and so has never been written anywhere. `JOIN` is thus enabled and deterministic, and induction along the run completes the proof. $\square$

The confluence property that Chapter 8 invoked for concurrent walkers on disjoint subgraphs is a commutation lemma. The *footprint* of a step is the thread (for `JOIN`, both threads) it involves together with the node and edge identifiers its premises evaluate and its conclusion modifies.

Lemma B.13 (Commutation). *If $C \longrightarrow C_1$ and $C \longrightarrow C_2$ by steps of distinct threads with disjoint footprints, then there is C' with $C_1 \longrightarrow C'$ and $C_2 \longrightarrow C'$, choosing fresh identifiers disjointly.*

Proof. By inspection: every rule’s enabledness and effect depend only on, and modify only, its footprint, so disjoint steps remain enabled after one another and their effects compose in either order. Routine. $\square$

B.4.3 THE ACTOR ENCODING

Chapter 8 stated that a minimal actor calculus embeds, with each actor a node, each undelivered message an in-flight walker, and each behavior a family of node abilities. We fix the source calculus, the translation, and the bisimulation.

Definition B.14 (Minimal actor calculus). Fix finite sets of behavior tags β and message tags k , and a table assigning to each pair (β, k) a straight-line handler

$$h ::= \text{end} \mid \text{send } e_a \ k'(\bar{e}); h \mid \text{create } x = \beta'(\bar{e}); h \mid \text{become } \beta(\bar{e}); h$$

whose expressions range over the actor’s state, the message payload, and *op*. A configuration $\langle A; M \rangle$ pairs a finite map from addresses a to pairs $(\beta, \bar{v})$ with a finite multiset of messages $(a, k, \bar{v})$. The single reduction, `DELIVER`, removes a message from M and atomically runs the matching handler, which may add messages, allocate fresh addresses, and replace a ’s behavior and state (Agha 1986; Hewitt et al. 1973).

Definition B.15 (Translation). The image uses one node type `Act` with fields *st* (the state) and *beh* (the behavior tag, a constant), and one walker type `Msgk` per message tag,

whose fields carry the payload. For each k , Act declares one node ability on Msg_k entry whose (dualized) body branches on here.beh and executes the image of the selected handler: send becomes a SPAWN aimed at the target address, addresses being node identifiers; create becomes NEW-NODE ; become becomes the two field writes on here . An actor $a \mapsto (\beta, \bar{v})$ translates to a node n_a ; an undelivered message $(a, k, \bar{v})$ to a thread $(\text{Msg}_k\{\bar{v}\}, \bullet, n_a, \epsilon, \epsilon, \text{skip})$. No edges are created anywhere in the image.

Because a DELIVER is atomic while its image is a sequence of small steps, the correspondence is stated against the *delivery-atomic* transition system: write $C \Rightarrow C'$ when C steps by the ARRIVE of some $\bullet$ -thread and then by that thread's steps alone until it is done. By Lemma B.13, fine-grained interleavings of deliveries at distinct actors are equivalent to delivery-atomic ones, their footprints being disjoint; two deliveries at the same actor are not, and there atomicity must be supplied by the runtime's transactional quantum (Chapter 13), a hypothesis recorded in Appendix B.5.

Proposition B.16 (Actor embedding). *Let $\mathcal{R}$ relate each actor configuration to its translation. Then $\mathcal{R}$ is a bisimulation between DELIVER and $\Rightarrow$: if $\langle A; M \rangle$ DELIVERS to $\langle A'; M' \rangle$ by consuming message μ , its translation $\Rightarrow$ -steps to the translation of $\langle A'; M' \rangle$ by arriving μ 's thread, and conversely every $\Rightarrow$ -step from a translated configuration is the image of exactly one DELIVER .*

Proof. Forward. Let $\mu = (a, k, \bar{v})$. Its thread sits at $\bullet$ with queue n_a ; actors are never deleted, so ARRIVE is enabled and $\text{fire}_{\text{en}}(\text{Msg}_k, \text{Act})$ is the single matching node clause, dualized, whose branch on here.beh selects the image of the handler for $(A(a), k)$. The thread's run is unique up to fresh names by Theorem B.12(i). By induction on the handler, each send image adds one $\bullet$ -thread (one added message), each create image allocates one node (one fresh address, names chosen coherently by Lemma B.11), each become image updates exactly n_a 's two fields. At end the queue is empty (handler images contain no visit), the exit stack holds only n_a , $\text{fire}_{\text{ex}} = \text{skip}$, and EXIT then FINISH retire the thread, landing componentwise on the translation of $\langle A'; M' \rangle$.

Backward. In a translated configuration every active thread is a message thread at $\bullet$ with statement skip , so the only possible first step of a $\Rightarrow$ -step is the ARRIVE of some message thread, determined by a message μ , and the run it initiates is the forward simulation of μ 's DELIVER . The delivery nondeterminism of the actor model is mirrored exactly by the choice PAR leaves open among $\bullet$ -threads. $\square$

As Chapter 8 observed, the force of the proposition lies partly in what the image never uses: the translation constructs an edgeless graph, so everything OSP adds over actors is exactly what the encoding leaves on the table.

B.4.4 SEPARATIONS AND EXPRESSIVENESS

The ambient separation of Chapter 8 required one hypothesis to be made precise: that the translation represents containment spatially.

Definition B.17 (Spatial representation). A translation T of ambient configurations into object-spatial configurations is *spatial* if there is a function cont from graph stores to containment forests over ambient names such that $\text{cont}(\mathcal{G}_{T(P)}) = \text{forest}(P)$ for every

ambient configuration P , where $\text{forest}(P)$ is P 's containment tree (Cardelli and Gordon 2000).

Proposition B.18 (Ambient mobility does not embed as motion). *No spatial translation maps every instance of the in and out capabilities to a sequence of graph-preserving steps.*

Proof. Take $P \rightarrow P'$ an in-step with $\text{forest}(P) \neq \text{forest}(P')$ (a sibling entering a sibling suffices). If each capability instance maps to graph-preserving steps, then $\mathcal{G}_{T(P')} = \mathcal{G}_{T(P)}$, so $\text{forest}(P') = \text{cont}(\mathcal{G}_{T(P')}) = \text{cont}(\mathcal{G}_{T(P)}) = \text{forest}(P)$, a contradiction. The orthogonality lemma of Chapter 8, which extends to the full system by the same inspection, identifies the graph-preserving steps as exactly the motion steps, giving the proposition its intended reading: ambient mobility cannot be rendered as walker motion. $\square$

Remark B.19. Chapter 8 claimed the hypothesis is forced whenever ambients are represented spatially at all. The argument is definitional: an ambient is a place—it has an interior—whereas a thread has a location and no interior, so a translation recovering containment from thread-local state has encoded the forest into private records, which is precisely the mobility-as-data-manipulation reading the proposition is designed to exclude, not a counterexample to it.

Finally, the expressiveness results. Following Felleisen (1991), a *macro translation* eliminates a set of constructs via one fixed syntactic abstraction per construct: each occurrence is rewritten in place to a fixed context filled with the images of its immediate subphrases, phrase categories are preserved (declarations map to declarations), and the translation is homomorphic elsewhere. The sessile fragment L_0 is as in Chapter 8, including while and let; observables are termination and the root-reachable final graph. The key locality fact is immediate from the syntax: field writes name their field statically, so a statement can write only field names occurring textually in it, and an L_0 declaration contributes types and constant defaults but no executable code.

Proposition B.20 (Traversal is not macro-expressible in the sessile fragment). *There is no macro translation of the topokinetic constructs into L_0 preserving the observables of all programs.*

Proof. Suppose T were such a translation; its finitely many contexts mention a finite set F of field names. Pick $f \notin F$ and consider the pair P_f, Q_f : both declare node N_f with field f defaulting to 0 and a walker W_f with one ability on N_f entry, whose body is $\text{here}.f := 1$ in P_f and skip in Q_f ; both mains create an N_f node, connect it to the root, and spawn W_f at it. Their observables differ: $f = 1$ versus $f = 0$ on the reachable node. But the translated mains are identical (identical source text), the translated N_f declarations are identical, and the differing walker declarations translate to L_0 declarations, which contribute no statements—so every executed statement of either translated program comes from one shared image whose field-write alphabet lies in F plus the fields the source main writes, and f is in neither. Both translated runs leave f at its shared default, the translated observables agree, and T fails on one of the pair. The second nonlocality of Chapter 8—walker state and itinerary persisting across ability boundaries—yields an independent argument of the same shape, which we omit. $\square$

The proof is rigorous relative to the stated formalization, in particular the requirements that images be fixed contexts and that declarations remain code-free. Liberalized notions reopen the question. Appendix B.5 records this, along with the status of the companion claim on persistence by reachability.

B.5 What Is Proved and What Is Conjectured

A formal appendix earns its keep by being candid about its own boundaries, so we close with an inventory.

Proved outright, for the full system as defined here: preservation (Theorem B.7); progress in the four-case form of Theorem B.8, with its two strengthenings; soundness (claim B.9); single-walker determinism in both clauses (Theorem B.12); commutation (Lemma B.13); conservativity over the nine-rule core (Remark B.1); the actor embedding (Proposition B.16); and the ambient separation relative to Definition B.17 (Proposition B.18). All are hand proofs; none is mechanized, and mechanization is the obvious next step for the safety and determinism results.

Three results hold under hypotheses the reader should see plainly. The actor embedding is a bisimulation with respect to the delivery-atomic system $\Rightarrow$, not the fully interleaved $\longrightarrow$: deliveries at distinct actors are recovered by commutation, but atomicity at a shared actor is supplied by the runtime's transactional quantum (Chapter 13), an engineering contract layered above the calculus, exactly as Chapter 8 framed it. The safety theorems cover the fully annotated fragment; the gradual surface of Chapter 9 weakens this deliberately, and the gradual guarantee of Siek et al. (2015) is an aspiration there, not a theorem here. And the macro-expressibility separation (Proposition B.20) is proved relative to a specific formalization of macro translation; whether it survives translations permitted to generate names from subphrases, or adequacy weakened to graph encodings, is open.

Two statements we label conjectures, and mean the word. First, that persistence by reachability is not macro-expressible over explicit saving, the claim of Chapter 8: the forward macro is immediate and the locality argument of Proposition B.20 adapts in outline, but we have not carried the adaptation through under a fixed adequacy relation. Second, that the exit discipline formalized here (exits unwind in reverse arrival order once the queue drains) coincides with the production kernel's recursive post-order execution on every schedule: the two provably agree on chains and depth-first itineraries, but on breadth-first schedules over branching topologies the identification is a conformance question audited by testing, not proof. Two smaller design readings share that status: `Disengage` forgiving pending exits, and the `Gone` rules silently skipping deleted destinations. The calculus-to-implementation correspondence as a whole rests on construction and test suite (Chapter 13), not a verified refinement proof; and nothing here formalizes distribution: that silence is by design, and the scale-invariance claims built on it are the subject of Chapter 14.

The reader who wants the paradigm's formal definition in its original register should consult Mars (2025). The calculus of this book refines that account, and the deltas (attributed edges, the exit stack, thread names and wait, the four-case progress theorem) are the price and the yield of pushing it to proofs.

GLOSSARY OF TERMINOLOGY

Every argument that coins terms owes its readers a ledger. This appendix gathers the vocabulary of the book into one alphabetical place: the eight terms we coin, each with its etymology and its canonical definition repeated verbatim from the chapter that introduces it, together with the supporting vocabulary (the constructs of Jac and the technical terms of the argument) defined as the book uses them. An entry here is a reminder, not a replacement. The defining chapter, cross-referenced in every case, carries the formal statement, the motivation, and the caveats, and nothing in this appendix supersedes it. Definitions of language-level terms are checked against the language reference and the littleX application examined in Chapter 15.

Three register rules from the terminology canon bear restating before the list. *Synechism* names the theory of continuity developed in Part II; *synechic* is the property adjective for languages; *synechistic* is reserved for the design ethic of Chapter 19. Likewise *topokinesis* names the phenomenon and *topokinetic* the property adjective. The nine coined terms are marked as such in their entries.

ability The unit of dispatch in Object-Spatial Programming: a block of computation declared with the keyword `can` on a walker, node, or edge, bound to an event of arrival or departure rather than to a caller’s invocation. The declaring clause names the counterpart type and the phase of the encounter: on a walker, `can gather with Tweet` entry runs when the walker arrives at a node of type `Tweet`; on a node, the type in the clause names the arriving walker, so the same encounter is programmable from either side. Entry abilities run at arrival; exit abilities are deferred until everything the walker queued from that node has been visited, unwinding in last-in, first-out order. Introduced in Chapter 7; given operational semantics in Chapter 8.

archetype A kind of declarable type in Jac. Alongside `obj`, the language’s counterpart to a Python class, the language adds three archetypes carrying spatial semantics: `node`, `edge`, and `walker`. All archetypes declare state with `has`, methods with `def`, and abilities with `can`, may inherit within their kind, and participate uniformly

in the declaration/implementation split (see *impl*). Introduced in Chapter 3 and Chapter 7.

boundary crossing An edge in a program's reference structure (a call, a data flow, or an import) whose two endpoints are assigned to different substrates under a placement. The crossing is the unit at which glue accrues in a conventional stack, and the unit over which substrate transparency demands a null cost. Defined formally in Chapter 2, whose boundary-crossing metric renders the burden countable; Chapter 16 applies the count to real codebases.

borrow-collapse The escape discipline at the region boundary: a region-rooted value flowing to a binding that cannot outlive the region's handle is registered as a shared borrow of the handle, so the ownership surface's existing borrow law (no drop, move, or send while borrows live) polices the region boundary without new machinery. Flows that could outlive the handle are rejected as escapes, default-deny. Defined in Chapter 11.

composition thesis The book's third claim, added to its two theses in Chapter 1: the synechic and topokinetic classes of language are independent in definition and compounding in value. Either class can be inhabited alone, and the historical record inhabits each margin separately, but the deepest dissolutions (the disappearance of the database into the language above all) require a member of both classes at once. Chapter 18 gives the claim its design-space form, where the single-system-image condition binds the two classes' membership conditions together.

connect membrane The edge discipline at the region boundary: an edge may not wire a region-resident node to managed topology, in either direction, rejected statically at the connect operators (error[E1307]), while region-internal edges are unrestricted. Because all inter-object topology passes through the connect operators, the rule guarantees that no traversal path leads from the persistent graph into memory that a bulk free may reclaim. The second instance of the membrane construct of Chapter 5, with topology as its cargo. Defined in Chapter 11.

discontinuity A boundary at which the representation of meaning must change and over which no verifier has jurisdiction (Definition 1.1). Both clauses matter: a call between two modules of one language changes no representation and is seen whole by one compiler, so it is a boundary but not a discontinuity; an HTTP crossing between two type systems fails both clauses, and it is at discontinuities that glue accrues and defects pool. The book's first thesis is that the discontinuities of the conventional stack are contingent; the essential boundaries that remain (latency, partial failure, cost) are surfaced as lawful boundaries rather than dissolved. Defined in Chapter 1; given a cost model in Chapter 2.

drop hook The destructor ability of the ownership surface: an archetype may declare a reserved ability named *drop*, run exactly once when a value dies and before its fields are torn down. Death is placed by liveness rather than block structure,

and the hook fires at the same program point under every native memory mode, so a change of memory strategy does not change observable output. Introduced with the ownership surface of Chapter 5; its substrate-uniform discharge is an instance of the mediation invariant of Chapter 13.

edge The archetype declaring a first-class, typed relationship between two nodes. An edge may carry data fields and abilities of its own, so “Alice follows Bob since 2020” is one declaration rather than a foreign key and a join table. Edges are created with the connect operators (`++>` untyped, `++:Follow():>` typed) and traversed with edge references such as `[->:Follow:->]`. Introduced in Chapter 7; edges as typed relations with declared endpoints are a concern of Chapter 9.

glue The marginal expression a boundary crossing demands: the program text required to realize a call, data flow, or import when its endpoints lie on different substrates, less the text the same edge would require were they co-located. Foreign-function bindings, serializers, interface-description files, generated API clients, and object-relational mappings are all glue; none would survive co-location, and that counterfactual is how the definition identifies them. Taxonomized in Chapter 1; defined formally in Chapter 2.

gradual borrow checking (coined term) Memory discipline as a continuum within one language: unannotated code retains fully managed semantics, ownership annotations introduce affine values with moves, borrows, deep immutability, and deterministic destruction, adoptable one declaration at a time, and a closed, checked boundary mediates every value that crosses between the two regimes. The name deliberately parallels gradual typing, from which the design inherits its trustworthiness criteria; the property gradualized is when memory dies, not who may point at whom. Defined in Chapter 5; native mechanics in Chapter 12; runtime discharge in Chapter 13.

growth rule The placement rule of topology-aligned regions: an allocation inside an ability anchors to the region of the node being visited, decided by a constant-time header test on here. A walker traversing a region-resident subgraph therefore grows that region, and the same walker on managed topology allocates managed, with no annotation distinguishing the cases. Defined in Chapter 11.

impl The keyword realizing Jac’s declaration/implementation split. An archetype or function declaration states the interface (fields, signatures, ability headers) and an `impl` block, in the same file or a companion `.impl.jac` file, supplies the bodies, as in `impl Calculator.add { ... }`. The split keeps the declared shape of a program legible at a glance while the mechanics live elsewhere, a separation Chapter 3 counts among the language mechanics of a synechic surface.

jid The unique, stable identifier of an archetype instance (a “Jac identifier”), obtained with the built-in `jid(...)` and resolved back to a live object with `jobj(...)`. Because nodes and edges persist, a `jid` is a durable handle rather than a transient pointer: a node reported by a littleX walker arrives in the browser still carrying its `jid`, the client passes tweet and channel `jids` back across requests, and lookup

walkers resolve them to the node in question before visiting it. Introduced with the persistence model of Chapter 10.

jurisdiction The reach of a verifier: the set of program points and crossings that a compiler, type checker, or other static instrument can examine and reject. The second clause of Definition 1.1 is a jurisdiction claim (glue accrues exactly where no verifier’s jurisdiction extends), and the jurisdiction proposition of Chapter 2 states the converse benefit: in a substrate-transparent language every crossing lies within the checker’s jurisdiction. The reach is gradual against untyped substrates (Chapter 9) and is realized operationally by the compiler’s boundary analysis (Chapter 12); Chapter 19 argues that in the era of machine authorship jurisdiction, not authorship, is the scarce factor in software economics.

lawful boundary An essential boundary that a synechic language must surface as semantics rather than dissolve. Latency, partial failure, monetary cost, and stochasticity are properties of physically and organizationally separated computation, not artifacts of representation (Waldo et al. 1994); the lawful-boundaries principle of Chapter 2 requires that each appear in the language as declared, typed, handleable expression (an awaited cross-tier call, a `by  $\text{U}\text{M}()$` clause) and never arrive solely as undeclared runtime behavior. The contrast class is the discontinuities (dialects, marshaling, toolchains) that substrate transparency eliminates.

meaning types (coined term) Semantic annotations from which prompts are automatically synthesized, making delegation of program logic to large language models (by `U $\text{M}()$`) a typed language feature rather than string engineering. The name records what the annotation carries: not the representation of a value but its meaning, the content from which a prompt can be derived mechanically. In Jac the `sem` statement attaches such an annotation to a function, field, or type, and the runtime synthesizes the model’s instructions from the annotated signature. Defined in Chapter 6, placed within the type system in Chapter 9, and presented formally by Dantanarayana et al. (2025).

membrane The closed boundary between the owned and managed memory regimes, admitting exactly three crossings: sealing (storing an owned value into managed storage moves it, one-way, and consumes the source binding), reboxing (an explicit copy that mints fresh owned storage from a scalar or string), and exceptional control, which aborts at the crossing rather than unwinding into frames without unwind metadata. The name is adapted from the object-capability literature (Miller 2006), where a membrane mediates every reference between two object graphs; here it mediates memory discipline rather than access. Defined in Chapter 5.

node The archetype declaring an object that occupies a location in a topology. A node holds data fields, may declare abilities that fire when walkers of a named type arrive or depart, connects to other nodes through edges, and persists when reachable from a root node. Nodes are the “smart places” of the paradigm: they know when they are visited and can react. Introduced in Chapter 7.

Object-Spatial Programming (OSP) (coined term) The concrete paradigm realizing topokinesis: programs expressed as walkers traversing a persistent topology of nodes and edges, with abilities triggered by arrival. The name is a deliberate foil to object-oriented programming: where *orientation* organizes computation around encapsulated objects that are invoked, *spatiality* organizes it around located objects that are visited. Introduced informally in Chapter 7, formalized in Chapter 8 and Appendix B, and first presented in Mars (2025).

ownership The opt-in discipline by which a binding, rather than a runtime, accounts for a value's life. A binding marked *own* is the value's unique owner, and every transfer (assignment, call, return, field store) is a compiler-checked move (the discipline is affine: dying unconsumed is lawful); *imm* marks deep immutability; the borrow forms *&* and *&mut* lend temporary access under a many-readers-or-one-writer rule and erase at code generation. Unannotated code is untouched and every step of adoption is reversible, so a module may stand anywhere on the continuum from fully collected to fully owned; graph objects stay managed by default, and the topology a walker allocates under a region open is kept apart from the managed graph by the checker, so the discipline never complicates the object-spatial constructs. Verified by a core compiler pass whose conclusions the native lowering spends as facts; developed as gradual borrow checking in Chapter 5, with the native mechanics in Chapter 12.

ownership dial The four-position surface through which gradual borrow checking is adopted, per module: managed (no annotations, the checker tracks nothing), annotated (the discipline enforced on tagged names only), enforced (every heap-typed interface position owned, as hard errors), and headerless (collector-free native code whose freedom from reference counting is machine-asserted over the emitted intermediate representation). The guarantees strengthen monotonically as the dial turns, modules at different positions link into one binary, and every step is reversible. Defined in Chapter 5; emission mechanics in Chapter 12.

polypilation The process a polypiler performs: lowering one polyglot application into several ecosystems under one analysis, with the boundary artifacts among the outputs rather than among the programmer's obligations. Register: the noun names the process; *polypiler* names the class of compiler. Named in Chapter 12.

polypiler (coined term) A compiler whose unit of compilation is the whole polyglot application, whose targets are ecosystems rather than instruction sets, and whose optimization surface includes the boundaries between them. Etymology: Greek *polys*, many, formed on the pattern of *compiler* and *transpiler*. The requirement it adds to classic retargetable compilation is the reified boundary: the interop manifest that converts glue generation into glue optimization. Substrate transparency states the property; polypilation is the mechanism that discharges it. Defined in Chapter 12; evaluated in Chapter 15 and Chapter 16.

region An ownable allocation extent of the ownership surface. The statement in `h { ... }` opens the region named by the handle *h*; everything constructed under the open belongs to it, and the population is reclaimed wholesale at the handle's

death: on the native substrate, a bump-allocated arena freed in one motion. Inside an open there is no per-value discipline; the checker polices only escapes across the boundary. The handle is itself an ownable, sendable value: moved across a thread boundary it carries its whole population without a copy, and a walker's abilities may grow the region of the node they fire on. Defined, as topology-aligned regions, in Chapter 11; emission mechanics in Chapter 12; the substrate-varying discharge of a region's reclamation in Chapter 13.

root node The distinguished node that anchors every topology, in scope everywhere as root. Reachability from a root confers persistence (Chapter 10), and under service deployment each user is issued a root of their own, so isolation between users is root multiplicity rather than tenancy code (Chapter 14). The root is not itself an archetype the programmer declares; it is supplied by the runtime as the fixed point from which every graph grows. Introduced in Chapter 7.

scale invariance (coined term) Program semantics are invariant under deployment-scale transformation: one user to N users, one machine to M machines, transient to persistent. Etymology: borrowed from mathematics and physics, where a law is scale-invariant when rescaling its variables leaves it unchanged; here the rescaled variable is the deployment, and the invariant is the meaning of the program text. Defined in Chapter 14, which also marks the physical limits of the property.

self-hosting frontier The division running through the toolchain artifact between components written in Jac (the compiler and runtime, the type checker, the formatter and test tooling, the linkers) and external engines carried in binary form (the CPython interpreter, the Bun JavaScript runtime, LLVM, the C runtime archives, the editor). The carried engines are included for feature completeness; each is a discontinuity in the sense of Definition 1.1, managed by digest pinning and removed only by reimplementation. The record to date moves in one direction: the compiler crossed first, then the linkers, and the stated design goal is for every component above the operating system to cross, with gradual borrow checking supplying the memory discipline the remaining crossings require (Chapter 5). Introduced in Chapter 4.

sendable subgraph A region whose handle carries no live borrows. By the extent-soundness clauses of Chapter 11 such a region has no external references at all, so moving its handle across a `flow/wait` boundary transfers exclusive access to the entire live subgraph in constant time, with no copy and no synchronization. A handle with live borrows is not sendable, and an individual region-rooted value is an escape: the subgraph crosses as a unit or not at all. Defined in Chapter 11.

single system image The presentation of a collection of processes, machines, and users as one continuous machine. The term originates in distributed operating systems research (Buyya et al. 2001); this book uses it for what the Jac runtime presents to programs: the same semantics at service scale as in a single-process script, with the essential boundaries of distribution surfacing only as lawful

boundaries. Developed in Chapter 14; its OS-layer ancestry is part of the history in Appendix F.

substrate A triple of runtime, ecosystem, and toolchain: the executor that gives compiled artifacts meaning, the corpus of libraries importable at that executor, and the apparatus that builds, packages, and ships artifacts for it. The managed, web, and native tiers of an application are substrates; so is the toolchain itself (Chapter 4), and so is the stochastic executor of Chapter 6. Etymology: Latin *substratum*, “that which is spread beneath.” Defined in Chapter 2.

substrate transparency (coined term) A language exhibits substrate transparency when the identity of the underlying runtime, ecosystem, and toolchain at any program point has no bearing on how the program is expressed. The name generalizes the classical family of distribution transparencies (Raymond 1995): where those held program text fixed and quantified over runtime configurations, substrate transparency moves the quantifier to the text itself, demanding invariance of expression rather than of behavior. Defined in Chapter 2.

synechic (coined term) A language is synechic if it presents a single continuous semantic medium across conventionally discrete ecosystems, tiers, and toolchains, such that no program point requires glue code, marshaling, or interop scaffolding to cross a substrate boundary. Etymology: Greek *synecheia*, continuity, reaching this book by way of Peirce’s *synechism* (Peirce 1892). Register: the property adjective for languages; see *synechism* and *synechistic* for the noun and the ethic. Defined in Chapter 2.

synechism In Peirce, the tendency of philosophical thought that insists on continuity as of prime importance (Peirce 1892). In this book, the name of the theory developed in Part II: the definition of substrate transparency, the cost model for glue, and the principle of lawful boundaries, taken together as an account of continuity across substrates. Register: names the theory or doctrine, never the language property. The philosophical resonance is taken up in Chapter 19.

synechistic The adjective reserved for the design ethic: the disposition to treat continuity as a governing value in the construction of languages and systems, beyond any single checkable property. A language is *synechic*; a designer, a culture, or a philosophy of construction may be *synechistic*. The term appears only in Chapter 19.

tier A substrate distinguished chiefly by its runtime component within a single application: client, server, and sometimes database. The tierless languages of the multitier literature are, in this book’s terms, substrate-transparent over a family of tiers: a fixed, finite family with a closed ecosystem and an external toolchain. Chapter 2 develops this placement of tierlessness as restricted synechism; Appendix F recounts the lineage.

topokinesis The phenomenon of which *topokinetic* is the adjective: motion of the locus of computation over a topology of data. Where the inherited convention moves data to a stationary site of compute, topokinesis moves the computation through

the stationary shape of the data. Register: the noun names the phenomenon; the adjective names the language property. Named in Chapter 7.

topokinetic (coined term) A language is topokinetic if the mobile locus of computation over a topology of data is a first-class semantic construct, inverting the von Neumann convention of streaming data to a fixed site of computation. Etymology: Greek *topos*, place, and *kinesis*, motion. The convention it inverts is the one Backus (1978) indicted as the von Neumann bottleneck. Defined in Chapter 7; the paradigm realizing the property is Object-Spatial Programming.

topology The graph of typed nodes and edges over which walkers travel. In Jac the topology is a live, persistent structure anchored at roots: it is at once the program's data model and its store, standing where a conventional stack places schema, object-relational mapping, and database. Introduced in Chapter 7; its persistence semantics are the subject of Chapter 10.

topology-aligned regions (coined term) Reclamation as a property of place: a first-class, ownable handle owns an arena; nodes, edges, and objects constructed under its open allocate into it; allocations made by a walker's abilities anchor to the region of the node being visited, so traversal grows the region it stands in; and the handle's death reclaims the entire subgraph at once, destructors first, then one bulk free. The name records the alignment the design rests on: the region's contents are decided by the shape of the data, where the classic region systems decided them by scope, inference, or annotation. Defined in Chapter 11; measured in Chapter 16.

traversal The movement of a walker through a topology. Traversal is queue-based: visit enqueues destinations rather than transporting the walker, the default order is breadth-first, and an insertion index (`visit :0: [->]`) makes depth-first and custom orders expressible. Entry abilities run as each node is reached; exit abilities unwind afterward in stack order. Traversal is always explicit: an ability that issues no `visit` ends the journey at that node. Informal treatment in Chapter 7; scheduling and determinism in Chapter 8.

visit The statement by which a walker extends its itinerary. `visit [->]` enqueues the nodes across outgoing edges; typed and filtered references narrow the set, as in `visit [->:Follow:->]`; and an `else` clause runs when the reference matched nothing, which turns find-or-create from a query plus a conditional insert into a single traversal with a fallback. Introduced in Chapter 7.

walker The mobile locus of computation in Object-Spatial Programming: an archetype declaring carried state with `has` and location-triggered abilities with `can`. A walker begins as an ordinary value and starts moving when spawned onto a node, as in `root spawn get_profile()`; it moves with `visit`, sends values back with `report`, and halts with `disengage`. When a program is served rather than run, its public walkers are projected as API endpoints, each request spawning a fresh walker on the caller's own root (Chapter 14). Introduced in Chapter 7; formalized in Chapter 8.

zero-RC compilation The native compilation mode in which the emitted artifact contains no reference counting and no collector: a module held to full ownership coverage lowers with allocation and release at statically proven points, and the build can assert structurally that no memory-management machinery survived into the binary. The mode turns the absence of a runtime into a checkable property of the artifact rather than a claim about it. Defined in Chapter 5 as the verifiable endpoint of the ownership dial, with the emission mechanics in Chapter 12; its bearing on hard-real-time and embedded targets is taken up in Chapter 17.

A HISTORICAL TIMELINE OF JASECI AND JAC

The body of this book presents Jac as a finished argument: two theses, a formal development, and a machine that realizes them. The history is messier, and more instructive. The language did not begin synechic. It became synechic through four distinct eras of construction, each of which dissolved a discontinuity that the previous era had been forced to institutionalize. This appendix narrates those eras in order.

A note on method is owed up front. This account is written from within the project, and we hold it to the same evidentiary standard as the rest of the book. Where a date can be anchored in the archival literature (Dantanarayana et al. 2025; Mars 2022; Mars et al. 2023; Mars 2025) or in the repository’s own record (release tags, release notes, and the source tree itself), we state it plainly. Where it cannot, we say “circa” and mean it. The repository is unusually good at preserving its own past: its curated release notes reach back through every renaming and rewrite, and its tag namespace still carries the parallel version streams of components that have since fused. We treat that record as the primary artifact, and Appendix E describes how to obtain it.

D.1 The Jaseci Runtime Era, circa 2019 to 2023

The idea for Jac and Jaseci arrived at the turn of 2020. The building began as a solo effort and expanded as collaborators joined over time. The work was motivated by an observation that Chapter 1 develops at length: the production AI applications of that period were not programs so much as federations, graphs of microservices, models, queues, and databases whose architecture diagram was more expressive than any single source file. The founding premise of Jaseci was that the diagram should be the program.

The first realization of that premise was the Jaseci runtime stack, released as open source circa 2021 and described in the archival literature by Mars et al. (2023). A general-audience account of the stack and its aims appeared in Forbes in May 2021

(Mars 2021), a dated public record that anchors the era’s start in print. The philosophy behind it, that the programmer should work at the level of the problem while the runtime owns the full stack beneath, is argued contemporaneously in Mars (2022). The stack comprised an interpreter for a domain-specific language (Jac, in its 1.x form) together with an API server and a scale-out orchestration layer. The essential vocabulary of this book was present from the first release: programs declared nodes and edges, dispatched walkers across them, and attached abilities that fired on arrival. Persistence was ambient rather than requested, and a walker could be exposed as a service endpoint without web scaffolding. In other words, the topokinetic core of Chapter 7 and the scale-invariant serving model of Chapter 14 are the oldest ideas in the lineage, not the newest. Commercial products were built and operated on this stack: conversational AI platforms and productivity applications among them, several of which appear in the project’s records to this day.

A complete appraisal must also record what the first stack was not. Jac 1.x was an interpreted orchestration language that delegated substantial computation to Python “actions”. The runtime was middleware that applications lived inside rather than a compiler target they passed through. The stack dissolved the database and API discontinuities while leaving a new one at its own boundary: crossing between Jac and the Python ecosystem required exactly the kind of binding code the language was meant to abolish. That contradiction set the agenda for the second era.

D.2 The jaclang Rewrite, circa 2023 to 2024

Circa 2023 the language was rebuilt from the ground up as jaclang: no longer an interpreted DSL riding on a runtime, but a compiled language whose semantics contain Python’s, lowering to Python bytecode and entering the CPython runtime as a first-class citizen. This is the supersetting strategy of Chapter 3, and it inverted the earlier relationship between Jac and its host ecosystem. Where Jac 1.x had reached into Python through actions, Jac now subsumed Python: any library on PyPI became a native import, and the object-spatial constructs became extensions of a general-purpose language rather than the whole of a special-purpose one.

The project’s curated release record opens in this era, at version 0.8.0, and reading it forward one can watch the modern surface crystallize: the `impl` keyword and the declaration/implementation split, the separation of `def` for functions from `can` for object-spatial abilities, parenthesized inheritance, and successive revisions of the typed edge-reference notation. The era also fixed the project’s engineering shape for the next several years: a small compiler core surrounded by a plugin architecture, with cloud serving, client tooling, and AI integration delivered as separately versioned packages. The tag namespace of the repository still preserves these parallel streams (`jaclang`, `jac-client`, `jac-scale`, `jac-mcp`, `byllm`, and a `jaseci` umbrella package, each with its own cadence), and their eventual convergence is the story of the fourth era.

D.3 The Meaning-Typed Era, circa 2024 to 2025

The third era is best described as the crystallization of the research claims. Large language models had been part of Jaseci applications from early on, but they entered

the language itself through the work that Chapter 6 presents: the observation that the prompt is glue code, and that a function signature already carries the meaning a model needs. The `by llm()` construct and its automatic prompt synthesis were developed under the name Meaning-Typed Programming, circulated in preprint form by 2024 (Mars et al. 2024), and published archivally by Dantanarayana et al. (2025). The implementation history is legible in the release record, which is continuous even across the renaming: the integration package’s notes pass from `mtllm` 0.4.0 to `byllm` 0.4.1 without interruption, and the older name survives only as a deprecation notice.

In the same period the object-spatial core received its own formal statement. Mars (2025) isolates Object-Spatial Programming from the runtime stack that first carried it, defining walkers, nodes, edges, and abilities as language semantics rather than middleware behavior, the position this book develops into the topokinetic thesis. By the close of this era the two pillars of the book, meaning types and OSP, both existed as peer-reviewed claims attached to a shipping implementation.

D.4 The Fused-Binary Era, 2025 to the Present

The final era widens the ambition from the language to everything the language touches, and it is the era in which the synechic claim of Chapter 2 became defensible in full. Three developments define it.

First, the surface grew until it covered the whole application. The client codespace (`cl`) compiles Jac to the web substrate, making the frontend part of the same program rather than a second project. The native codespace (`na`) lowers Jac through LLVM to machine code and WebAssembly. The scale subsystem folds Kubernetes deployment into a single flag on `jac start`. One source file could now span tiers that had previously been separate repositories. Late in the era the surface grew along a further axis: an opt-in ownership discipline and first-class allocation regions, through which a module moves by degrees between collector-backed and statically proven memory (Chapter 12).

Second, the compiler became self-hosted. The compiler that ships today is written in Jac (its passes, its type system, its code intelligence) and is brought up by `jac0`, a deliberately small bootstrap transpiler that reads the subset of Jac used in the compiler’s core and emits equivalent Python at import time. The language now stands on itself, with precompiled compiler images carried in the distributed artifact, and Chapter 12 gives the technical account.

Third, the toolchain fused into one file. The distributed `jac` is a native launcher stub, written in Zig, with the entire runtime payload appended: a private CPython, the precompiled compiler, the Bun JavaScript runtime, a statically linked LLVM, Jac’s own linker, vendored type stubs, and an embedded editor. In Chapter 4 we argue that the build system is a substrate boundary in disguise. This artifact is the argument made executable.

The repository dates this consolidation precisely. Through the first half of 2026 the components still released in parallel (the final combined release pairs `jaclang` 0.16.7 with independently versioned `scale` and `serving` packages) until the streams converge on a unified version line at `v0.30.0`, tagged in June 2026, with `jaclang` itself leaping from 0.16.7 to 0.30.1 to join it. The 0.31.0 release, weeks later, completes the fusion

Table D.1: The four eras of the Jaseci and Jac lineage. Spans marked circa are reconstructed from published work and the release record rather than from precise dates.

Era	Span	Defining artifacts
Jaseci runtime	c. 2019–2023	Jac 1.x interpreter; runtime stack
jaclang rewrite	c. 2023–2024	Python-superset compiler; plugins
Meaning-typed	c. 2024–2025	by <code>llm()</code> ; MTP and OSP papers
Fused binary	2025–present	Self-hosted compiler; one artifact

by removing the plugin system entirely: the capabilities that had been separately shipped packages are called directly by the core, every project kind builds through one command into a sealed application image, and the command-line surface contracts to a small set of intent-named verbs. At the time of writing the current release is v0.34.0, tagged in July 2026. It is also in this era that the language turned to face machine authors as deliberately as human ones, shipping a built-in MCP server and coding agent so that the synechic medium is available to both.

D.5 Reading the Record

Read as a whole (Table D.1 and Figure D.1), the record exhibits a single repeated motion. Each era shipped a genuine dissolution of some discontinuity and, in doing so, exposed the next one: the runtime era dissolved the database and API boundaries but made the language itself a discontinuity; the rewrite dissolved the language boundary but left the toolchain and the AI substrate outside; the meaning-typed era brought the stochastic substrate inside the type system; and the fused-binary era absorbed the toolchain, the last boundary the programmer had been operating by hand. The record also dates the self-hosting frontier of Chapter 4: the compiler crossed over the jaclang and fused-binary eras, the linkers crossed when the toolchain internalized linking, and the carried engines of that chapter’s inventory mark where the frontier stands at the time of writing. None of this was executed against a master plan drawn in the late 2010s. It was the same pressure, the cost of the next-most-painful boundary, applied repeatedly over roughly a decade, and the theory of this book is in large part that history systematized. The discontinuities went first where they hurt most. The definitions of Chapter 2 and Chapter 7 are what the survivors of that process have in common.

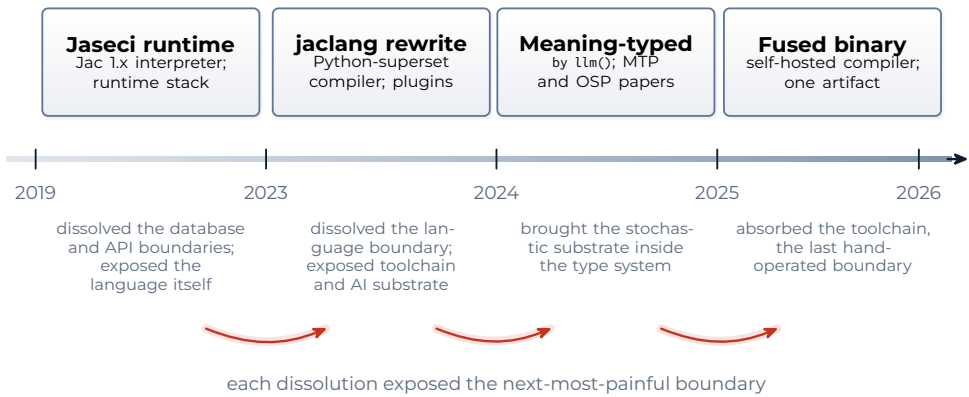


Figure D.1: Four eras, one repeated motion. Each era shipped a genuine dissolution of some discontinuity and, in doing so, exposed the next one; the fused-binary era absorbed the toolchain, the last boundary the programmer had been operating by hand. Spans and defining artifacts follow Table D.1.

ARTIFACT AND REPRODUCIBILITY GUIDE

The evidentiary chapters of this book rest on artifacts that any reader can obtain, run, and count. Chapter 15 inventories three applications from the public Jac repository; Chapter 16 reports an inventory of one of them and commits the remaining claims to protocols fixed in advance. This appendix is the practical companion to both: it states how to acquire the toolchain, how to run each case-study artifact, and exactly how every number printed in those chapters was produced, so that reproducing or disputing them costs a clone and a handful of shell commands. Throughout, commands are given for macOS and Linux. On Windows, the same commands run under WSL.

E.1 Obtaining the Toolchain

The toolchain is one file. The installer downloads the self-contained jac binary from the project's release page and places it on the path. No Python, pip, Node, or C toolchain is required, because the binary carries its own (Chapter 4).

```
curl -fsSL https://raw.githubusercontent.com/jaseci-labs/jac/main/scripts/install.sh  
| bash  
jac --version
```

Listing E.1: Installing the toolchain and verifying the installation.

The installer places the binary in `~/.local/bin` and accepts a `-version` flag for pinning, to which we return in Appendix E.4. What arrives is the fused artifact anatomized in Chapter 4. It is a native launcher stub with the runtime payload appended: a private CPython, the sealed compiler and runtime, the Bun JavaScript runtime, a statically linked LLVM, the toolchain's own linker with its vendored C floor, the package manager, the type checker, and the development tooling (`jac fmt`,

`jac test`, `jac lsp`, `jac mcp`). On first run the launcher unpacks the payload into a per-version cache. Every later run starts instantly. The meaning-typed subsystem, the deployment subsystem, and the MCP server all ride in the payload as well. Only their third-party dependencies (the model libraries behind by `llm()`, the deployment stack's cloud clients) install on demand, resolved like any project dependency by `jac install` (Chapter 6, Chapter 14).

For a reader who wants to try the language before installing anything, the hosted playground at playground.jaseci.org runs Jac in the browser and suffices for the language-level material of Parts II and III. The case studies below, which serve web applications and compile native executables, require the binary. A quick end-to-end check that an installation works is the repository's showcase application: clone github.com/jaseci-labs/this_is_jac, run `jac install` and then `jac start` inside it, and open <http://localhost:8000>.

One further note for readers who work alongside a coding agent, as Chapter 6 anticipates. The binary ships an MCP server exposing Jac validation, formatting, documentation, and examples. `claude mcp add jac - jac mcp` wires it into Claude Code, and any other MCP client can launch the same server with the command `jac mcp` (or `jac mcp -mode lite` for smaller models). The binary also carries its own terminal agent, `jac ai`, and curated references under `jac guide`.

E.2 Running the Case Studies

All three artifacts of Chapter 15 live in the repository's examples tree. Clone it once:

```
git clone https://github.com/jaseci-labs/jac
cd jac/jac/examples
```

Listing E.2: Obtaining the artifacts.

The full-stack social application is `littleX`. One command synchronizes its Python and npm dependencies against the single `jac.toml` manifest, and one command builds and serves the whole application. The served instance exposes the interactive interface documentation generated from the walker declarations (Chapter 14) at `/docs`, and the test suites under `tests/` run with `jac test`.

```
cd littleX
jac install      # resolves PyPI + npm deps from jac.toml
jac start       # builds and serves on http://localhost:8000
```

Listing E.3: Running the littleX social application.

The conversational AI artifact is the day planner, carried in three variants (`basic`, `auth`, `walkers`) as Chapter 15 describes. Its meaning-typed functions delegate to a language model, so a provider credential must be in place; the model is declared in each variant's `jac.toml` (the repository pins an Anthropic model as the default, and any LiteLLM-compatible provider may be substituted by editing that stanza).

```
cd day_planner/basic      # or auth/ or walkers/
jac install               # resolve the manifest's dependencies
export ANTHROPIC_API_KEY="sk-ant-..."
```



```
jac start main.jac
```

Listing E.4: Running a day planner variant.

The served application is at <http://localhost:8000>, with the generated interface documentation at `/docs` and a live visualization of the persisted graph at `/graph`, a direct way to watch persistence by reachability (Chapter 10) at work.

The native game is `raylib_shooter`. Its driver script detects the platform, downloads the precompiled raylib release, builds the Jac shooter through the native pathway (no C compiler or system linker is invoked at any point), builds the Zig twin as the conventional-toolchain baseline, and by default benchmarks both; running the demo requires a desktop with OpenGL, plus `curl` and `tar` for the downloads. The browser delivery of the same game lives in the `web/` subdirectory as an ordinary full-stack project.

```
cd raylib_shooter
./demo.sh          # benchmark both builds, 8 seconds each
./demo.sh --jac    # build and play the Jac build interactively
cd web && jac start # same game compiled to WebAssembly, served
```

Listing E.5: Running the shooter natively and in the browser.

E.3 Reproducing the Measurements

Every structural number in Chapters 15 and 16 is a raw physical line count or a file count, and the methodology is stated in full by the following sentence: line counts are `wc -l` over the named files, with comments and blank lines included; file counts are enumerations of `*.jac` files under the project directory. No discounting convention is applied. Listing E.6 reproduces the littleX inventory exactly as tabulated in both chapters; the expected totals appear as comments.

```
cd jac/examples/littleX
find . -name '*.jac' | wc -l          # 37 Jac source files
find . -name '*.jac' -print0 | xargs -0 wc -l # 4,432 lines total
wc -l social_graph.jac main.jac      # 466 + 9   graph and service tier
wc -l frontend.cl.jac frontend.impl.jac \
    components/*.cl.jac              # 2,177    client application tier
wc -l components/ui/* lib/utls.cl.jac # 1,365    component kit + utility
wc -l tests/*                        # 415      tests
wc -l jac.toml global.css             # 65 + 173 config + stylesheet
```

Listing E.6: Reproducing the littleX inventory of chapters 15 and 16.

The dependency-surface count of Chapter 16, twenty-five third-party packages across two foreign ecosystems, is read directly from the five dependency stanzas of the same `jac.toml`: two packages under `[dependencies]` and one under `[dev-dependencies]` from PyPI; seventeen under `[dependencies.npm]` and five under `[dependencies.npm.dev]` from npm. The remaining artifact sizes quoted in Chapter 15 reproduce the same way, from the examples directory:

```
wc -l day_planner/basic/main.jac    # 283
```

```

wc -l day_planner/walkers/main.jac      # 159
wc -l raylib_shooter/shooter.na.jac     # 627
wc -l raylib_shooter/shooter.zig        # 579
wc -l raylib_shooter/web/raylib_shim.cl.jac # 425
wc -l chess/chess.jac chess/chess.impl.jac # 220 + 782
wc -l notes-app/main.jac                # 664

```

Listing E.7: Reproducing the remaining artifact measurements of chapter 15.

These counts hold at the repository revision this book describes (Appendix E.4); at any later revision the artifacts may have grown or shrunk, which is one reason the revision is pinned. The performance suite and developer-study instruments of Chapter 16 stand on different footing, as that chapter is explicit about: their protocols are fixed, their result cells are marked *[to be measured]*, and their harnesses, including the FastAPI reimplementation of the littleX walker surface offered for third-party audit, are to be distributed with the archived artifact when the runs are discharged, together with the pinned hardware and software specification under which the numbers were taken. Until then, the only measurements this book reports as final are the structural counts above, and they are checkable in the time it takes to run two listings.

E.4 Version Pinning and Where the Artifacts Live

Reproducibility in a fused toolchain reduces to two version numbers: the revision of the repository containing the artifacts, and the version of the jac binary that runs them. The second pins everything beneath it: as Chapter 4 details, one binary version fixes the embedded interpreter, the JavaScript runtime, the LLVM, the typeshed commit, and the compiler itself under a single payload digest, so there is no toolchain vector to reconstruct. The installer accepts an explicit version, and installed versions coexist in separate caches:

```

git clone https://github.com/jaseci-labs/jac
cd jac && git checkout <tag>      # the revision archived with this book
curl -fsSL https://raw.githubusercontent.com/jaseci-labs/jac/main/scripts/install.sh
\
| bash -s -- --version <V>      # the matching jac release

```

Listing E.8: Pinning the repository revision and the binary version.

The tagged revision and matching binary release archived with the published edition are *[to be fixed at press time]*; the counts of Appendix E.3 were taken from the repository state this draft describes (revision 1f0021855) and will be re-verified against the frozen tag.

Within the repository, the artifacts and sources cited throughout the book live at stable paths. The case studies are under `jac/examples/`: the social application at `littleX/`, the day planner variants at `day_planner/basic`, `day_planner/auth`, and `day_planner/walkers`, the native game at `raylib_shooter/` with its browser delivery in `raylib_shooter/web/`, and the further examples noted in Chapter 15 at `chess/`, `notes-app/`, `mini_todo/`, and `mobui/`. The compiler and runtime studied in Chapters 12 and 13 are the `jaclang` package under `jac/`; the launcher sources behind Chapter 4 are under `jac/launcher/`; the authoritative language

reference is under `docs/docs/reference/`; and the installer of Appendix E.1 is `scripts/install.sh`. The hosted documentation lives at `docs.jaseci.org` and the playground at `playground.jaseci.org`. Nothing this book measures lies outside the public tree, which is the point: the argument of Parts II through V is about what a reader can verify, and this appendix has tried to leave the verification no harder than the claim.

A HISTORY OF ATTEMPTS AT CONTINUITY

All problems in computer science can be solved by another level of indirection.

attributed to David Wheeler

This appendix is a reference survey of the prior art behind the book's first thesis. The fragmented stack anatomized in Chapter 1 is not news to the field: every generation of systems builders has looked at the discontinuities running through its software (between machines, between languages, between tiers, between processes, between hosts) and mounted a serious attempt to dissolve them. The attempts include some of the most consequential artifacts in the history of computing, from the Java Virtual Machine to Plan 9. What follows is a history of *continuity*: the recurring ambition to make software one connected medium rather than an assembly of disconnected parts. The theory of synechism in Part II is built on this record's lessons, and the main text cites them at each point of contact. A companion survey, Appendix G, compiles the corresponding record for the book's second thesis, the ambition to set computation itself in motion, behind the theory of topokinesis in Part III.

Wheeler's aphorism, which heads this appendix, is usually quoted as a celebration of indirection, and it deserves the reputation: nearly every attempt we examine works by inserting a level of indirection between the programmer and some fractured substrate. But the aphorism carries a folkloric corollary: the resulting problem is too many levels of indirection. The corollary is the deeper lesson of this history. Each level of indirection dissolved the discontinuity it targeted while hardening into a new fixed structure, a new medium with its own edge, around which the remaining discontinuities calcified. We trace five families of attempts, in roughly the order the field produced them, and close with an analysis of why the pattern repeats: what we call the partial-continuity trap.

We read each attempt along two axes. The first axis is the *layer* at which the attempt installed its continuity, whether the hardware interface, the operating system, the middleware, or the language. The second axis is what the attempt implicitly declared to be *out of scope*. Both readings, tabulated at the appendix's end, form the requirements list for the language this book presents.

F.1 Portability by Imposition

The first discontinuity the field tried to dissolve was the machine itself. In 1958, with programming languages barely born and machine architectures proliferating, Strong et al. (1958) proposed UNCOL, a universal computer-oriented language, to break the multiplicative cost of retargeting. With M source languages and N machines, the field faced $M \times N$ translators. A shared intermediate language would reduce the bill to $M + N$. UNCOL was never built. The committee's design collapsed under the diversity of the machines it meant to unify, and the acronym survives mainly as a cautionary shorthand. But the idea proved unkillable, because the economics that motivated it never went away.

The idea's first durable success was modest in ambition and pragmatic in execution. The Pascal-P system distributed the Pascal compiler as code for a small synthetic stack machine, p-code, so that porting Pascal to a new computer required only an interpreter for that machine: weeks of work where a compiler port took months (Nori et al. 1981). Pascal's remarkable geographic spread in the late 1970s owed much to this maneuver. The strategy generalizes, and we name it *portability by imposition*: rather than translating among real machines, impose a synthetic machine above all of them, and let the synthetic machine be the program's true home.

The strategy reached commercial scale with the Java Virtual Machine. The JVM specification defines a complete synthetic computer: instruction set, memory model, class file format, verification discipline. The accompanying slogan, "write once, run anywhere," is the imposition strategy stated as a promise to the working programmer (Lindholm and Yellin 1999). Microsoft's Common Language Runtime followed the same architecture while relaxing one axis: where the JVM began as one language on one synthetic machine, the CLR was designed from the outset as a multi-language target above a shared runtime. Later descendants refined the idea at other layers: LLVM succeeded where UNCOL failed partly by scoping the shared intermediate representation to compiler infrastructure, forgoing any claim to be a universal program medium (Lattner and Adve 2004), and WebAssembly imposed a portable, sandboxed synthetic machine on the one platform (the browser) that had until then admitted only JavaScript (Haas et al. 2017).

Judged by its own goal, imposition worked. A Java program genuinely does run unmodified across operating systems and architectures, and the hardware discontinuity that consumed the field's attention in 1958 is, for most programmers, gone. But the synthetic machine became a new enclosed medium with a well-defended edge. Code inside the JVM moves freely. Code at its edge crosses through the Java Native Interface, a marshaling layer as ceremonious as any foreign function interface it replaced. The ecosystems of other languages sit on the far side of that marshaling layer. The tiers of an application (the browser, the database) were never within the

synthetic machine’s remit at all, and the toolchain that builds, packages, and deploys the portable artifact remained entirely external practice. Imposition dissolved the discontinuity between machines by creating a medium whose own boundary became the new discontinuity. Programs achieved continuity *within* the walls, and the walls were real.

F.2 Portability by Inversion

Half a century after UNCOL, the imposition strategy was inverted. Where the JVM offered one language a home above many machines, the Truffle framework on the Graal virtual machine offered many languages a home above one substrate. Würthinger et al. (2013) state the ambition in their title, one VM to rule them all. The mechanism is elegant. A guest language is implemented as a self-optimizing interpreter over abstract syntax trees, and the shared runtime partially evaluates that interpreter into compiled code, so that every language hosted on the substrate inherits a production-grade optimizing compiler it did not have to build. Languages hosted this way can interoperate within the shared runtime, passing objects across what were previously ecosystem borders.

This is a genuine second strategy, and it attacks a different discontinuity. Imposition dissolves the boundary between machines beneath one language. Inversion dissolves the boundary between languages above one runtime. Both collapse the $M \times N$ matrix, but along orthogonal axes. And inversion’s achievement should not be understated: it demonstrated that language boundaries, among the hardest discontinuities in Chapter 1’s inventory, are an artifact of separate runtimes, not a semantic necessity.

Yet the unification has a characteristic shape, which we might call annexation. Each guest language must be reimplemented as a Truffle interpreter. The unification extends only as far as the set of languages someone has ported, and each port is a substantial engineering artifact that must then track its language’s evolution forever. The programmer’s experience remains plural: a polyglot program is still written in several languages, and the substrate smooths the crossings without removing the need to cross. The substrate itself is one runtime with one edge. The tiers of an application, the native world outside the VM, and the toolchain all remain where imposition left them.

The language this book presents inverts the inversion: Jac is one surface language that compiles onto many substrates (the managed runtime, the web tier, native code), entering each as a first-class citizen. We develop this design in full in Chapter 3. For now we note only its position in the design space this section has laid out. Imposition: one language, one synthetic substrate, many machines. Inversion: many languages, one substrate. The move still missing from the record: one language, many substrates, with the substrates’ native ecosystems inherited rather than annexed.

F.3 Tierless and Multitier Programming

The web gave the field a new discontinuity and, with it, the most direct ancestors of this book’s continuity thesis. A web application is one program in the mind of its author and at least three in its execution: markup and script in the browser, application

logic on the server, queries in the database. By the mid-2000s the research community had named this fracture the *tier* problem and begun to attack it at the right layer, the language.

Links was the movement's manifesto. Cooper et al. (2006) present a single functional language for the whole web application, in which the compiler sends each part of the program to its proper tier: client-side code is compiled to JavaScript, database queries to SQL, and the crossings between tiers are typed function calls. Serrano et al. (2006) arrived at the same destination from Scheme, structuring one program into cooperating server and client strata. Chlipala (2015) pushed the discipline furthest: Ur/Web is a statically typed language in which the entire application (pages, server logic, database schema) inhabits one type system, so that cross-tier inconsistencies that conventionally surface as runtime failures in production are compile-time errors. Radanne et al. (2016) brought the approach to an industrial-strength host, extending OCaml with client and server sections and typed communication between them. And Weisenburger et al. (2018) generalized the idea past the client-server pair entirely: ScalaLoc's placement types let a program specify arbitrary distributed architectures (peers, pipelines, rings) while remaining one compilation unit. Weisenburger et al. (2020) survey the field this line of work became.

We regard multitier programming as the most important prior art for the continuity half of this book, and Chapter 2 gives it a formal placement. Its central insight is correct: the discontinuity between tiers is not a fact of nature but a failure of language design, and a compiler that sees the whole program can eliminate the marshaling, the API scaffolding, and the duplicate type declarations that Chapter 1 cataloged. Within their chosen scope, these languages achieved genuine continuity.

The instructive point is where the scope ended. Three boundaries recur across the family. First, the unification covered the tiers of a web application and stopped at the language's own borders. Client, server, and query became continuous. The world of software outside the new language did not. Second, the host ecosystems were excluded. A Links or Ur/Web program could not reach into the npm registry or the accumulated libraries of its deployment platforms as a native citizen. A new language that inherits no ecosystem must recreate every library, and this burden, more than any semantic shortcoming, confined most of the family to research use. Eliom and ScalaLoc mitigated the problem by extending established host languages, inheriting one ecosystem, though the other tiers' ecosystems remained across a boundary. Third, the toolchain was untouched. Compilation produced artifacts for each tier, and everything downstream (bundling, packaging, serving, deploying) remained external practice, as fragmented as before. The tierless languages dissolved the discontinuity between tiers and left the discontinuities around the program intact. The lineage did not end with the research family. At the time of writing, the mainstream JavaScript ecosystem is rebuilding tier splitting inside its frameworks: React Server Components mark the boundary with string directives, compute the split in the bundler, and police what crosses with serialization rules enforced by convention and lint tooling (React Team 2020). The capability the research languages demonstrated is being reconstructed piecewise, inside the toolchain, without a language to hold it, and Chapter 12 reads this convergence as evidence for the diagnosis that membership, not splitting, was the missing requirement.

F.4 Distribution Transparencies

While language designers attacked the discontinuity between tiers, the distributed-systems community spent two decades attacking the one between machines with a different instrument: the *transparency*. The founding move was remote procedure call. Birrell and Nelson (1984) built machinery (stubs, binding, marshaling) whose explicit purpose was to make a call across the network indistinguishable, at the call site, from a call across the stack frame. The programmer writes an invocation. The crossing is handled below.

The idea scaled into an industry. CORBA wrapped it in an interface definition language and object request brokers, promising invocation across machines, operating systems, and languages at once (Vinoski 1997), and a broader middleware layer grew up around the same contract (Bernstein 1996). The RM-ODP standardization effort made the ambition taxonomic: it enumerated a family of transparencies (access, location, migration, replication, failure) as named engineering targets, each a specific respect in which a distributed system could be made to present itself as a single one (Raymond 1995). The transparency family is the intellectual ancestor of the property this book will define in Chapter 2, and we owe the lineage explicit acknowledgment.

We owe equal acknowledgment to the critique. Waldo et al. (1994) argued that the entire program of unifying local and remote objects rests on a category error: local and remote invocation differ in latency by orders of magnitude, differ in their access to memory, and, decisively, differ in their failure modes. A local call either completes or the program fails with it. A remote call can partially fail, leaving the caller ignorant of whether the callee acted. Concurrency without shared fate compounds the problem. A system that syntactically erases these differences does not thereby remove them. It removes the programmer's ability to see where they apply. The critique was borne out empirically: the RPC lineage's characteristic production failures (timeouts treated as errors, retries duplicating effects, cascading unavailability) are failures at discontinuities the syntax had rendered invisible.

We treat Waldo's argument as a constraint on the book's thesis. What the critique demolishes is a specific conflation: the assumption that continuity of *expression* must entail identity of *behavior*. RPC promised both (write the same call, get the same semantics) and could not deliver the second. But the two are separable. A medium can be continuous in how programs are written while stating plainly where latency, partial failure, and cost apply. The discontinuity in the physics can surface as visible, typed, lawful properties of the program rather than as either syntax (the pre-RPC world) or silent surprise (the RPC world). Whether that separation can be engineered is a central burden of this book, taken up in Chapter 2 and discharged for the distributed case in Chapter 14. The point of record for this appendix is that the transparency tradition identified the right goal, pursued it at the middleware layer with an instrument, pretense, that could not support it, and received a critique that any successor must answer.

F.5 Single System Images at the Operating System Layer

The most literal attempts at continuity did not modify the language or the middleware. They rebuilt the operating system so that many machines would present themselves as one. Amoeba pooled the processors of a network into a single system: a user ran a command, and the operating system chose a machine for it, the file system and services being uniformly accessible from anywhere in the pool (Tanenbaum et al. 1990). Plan 9 pursued the same unity through a different discipline: every resource (files, devices, network connections, the windowing system) is presented through one file-like protocol, and every process assembles its own namespace from local and remote resources alike, so that the distinction between this machine and that machine dissolves into the composition of a namespace (Pike et al. 1995). MOSIX made the unity dynamic: processes on a cluster migrate preemptively and transparently between machines for load balancing, a running program discovering nothing about its own relocation (Barak and La’adan 1998). Buyya et al. (2001) survey the design space under the name that stuck, the *single system image*, and observe that the image can be constructed at any layer of the stack, from the hardware to the application, with distinct costs at each.

These systems worked, and their engineering repays study today. We draw on it directly in Chapter 14. But two structural limits confined them. The first is the familiar edge problem: the single system image stopped at the boundary of the cluster under one administrative domain, typically requiring homogeneous machines and a single kernel lineage, and the boundary between the cluster and the wider heterogeneous world was as hard a discontinuity as any. The commodity internet, which is nothing but that heterogeneous wider world, is where the SSI operating systems failed to follow.

The second limit matters more for this book’s argument. The continuity these systems achieved lived *below* the layer at which programmers express meaning. To the application, the single system image arrived as the same system-call interface it already knew. Programs remained conventional programs, written in conventional languages, seeing files and processes. Nothing in the program’s semantics (its types, its calls, its data structures) could refer to, exploit, or reason about the continuity beneath it. The operating system had become a continuous medium, and the programs running on it remained programs whose semantics ended at the process boundary, because the medium was installed at a layer the program’s own constructs cannot see. Continuity at the wrong layer is invisible to the very artifact, the program, whose fragmentation motivated the effort. The lesson, which Part II takes as a design premise, is that the single system image must be relocated to the layer where programs mean things: the language and its runtime.

F.6 The Partial-Continuity Trap

Five families, one pattern. Each attempt selected a discontinuity, inserted Wheeler’s level of indirection (a synthetic machine, a shared runtime, a whole-program compiler, a stub layer, a distributed kernel), and dissolved the discontinuity it targeted. Each succeeded, on its own terms, to a degree that shaped the industry. And each left the

total fragmentation of software substantially intact, because the new medium calcified into a structure around which the remaining discontinuities hardened. We call this recurrence the *partial-continuity trap*, and it is worth spelling out its mechanisms, because a design that does not understand them will repeat them.

The first mechanism is *boundary displacement*. Indirection does not remove a boundary. It relocates the boundary to the edge of the new medium. For example, the JVM's hardware discontinuity reappeared at the JNI, the tierless languages' tier discontinuity reappeared as the border between the new language and every ecosystem outside it, and the SSI cluster's machine discontinuity reappeared at the cluster's administrative perimeter. The discontinuity is conserved unless the medium's edge is pushed outward far enough to enclose what programs actually need. Every attempt in this appendix drew its edge well inside that territory.

The second mechanism is *ecosystem exclusion*. A new medium that does not inherit the working world's libraries forces its adopters to recreate them, and the recreation never completes. Meanwhile, every practical program must reach across the medium's edge to the excluded ecosystems, reintroducing at the border the glue the medium abolished in its interior. Imposition suffered this at the native boundary, inversion at the boundary of the unported languages, and the tierless family most acutely of all. It is the single largest reason languages of demonstrated semantic superiority remained research artifacts.

The third mechanism is *layer mismatch*. Continuity installed at a layer other than the one where programmers express meaning cannot participate in that meaning. The SSI operating systems are the sharpest case: the medium was genuinely continuous, and programs could not say so. The converse mismatch also occurs: the tierless languages installed continuity at the semantic layer but left the layers below (build, packaging, deployment) fragmented, so the continuous program was still produced and operated by discontinuous means. We argue in Chapter 4 that the toolchain is itself a substrate boundary in disguise. None of the attempts in this appendix treated it as one.

The fourth mechanism is *scope closure*. Each medium unified what its designers enumerated (machines, languages, tiers), and the enumeration functioned as a declaration that everything else was out of scope. Discontinuities do not respect these declarations. An application crosses machine, language, tier, process, and toolchain boundaries in a single feature, and a medium that dissolves one crossing while fixing its own edges may leave the feature's total boundary-crossing cost unchanged, the glue merely redistributed to the medium's border.

The record is summarized in Table F.1. Read as a whole, it is not a catalog of failures (every row names a success) but a proof by exhaustion of a narrower claim: partial continuity does not compose into full continuity. Fifty years of dissolving one discontinuity at a time has produced a stack in which every layer contains someone's triumph over fragmentation and the whole remains the assembly anatomized in Chapter 1.

The table also functions as a requirements list, and we close by reading it that way. A full attempt at continuity must be installed at the language layer, where programs mean things, because continuity below that layer is invisible to the program (the SSI lesson). It must inherit the existing ecosystems as first-class territory rather than

Table F.1: Five families of continuity attempts, the discontinuity each dissolved, and the discontinuities that calcified at the edge of the resulting medium.

Attempt	Discontinuity dissolved	Discontinuities calcified
Bytecode virtual machines (p-code, JVM, CLR)	Machine heterogeneity beneath one language	Native boundary (FFI); foreign ecosystems; tiers; toolchain and deployment
Polyglot runtimes (Truffle/GraalVM)	Language boundaries above one runtime	Edge of the shared runtime; per-language porting burden; tiers; toolchain
Tierless languages (Links, Hop, Ur/Web, Eliom, ScalaLoci)	Client, server, and database tiers within one program	Host-ecosystem exclusion; toolchain and deployment; scope fixed to the web
Distribution middleware (RPC, CORBA, RM-ODP)	Syntactic locality of remote invocation	Behavioral discontinuity of latency and partial failure; IDL and stub glue; vendor and language borders
SSI operating systems (Amoeba, Plan 9, MOSIX)	Machine multiplicity at the system-call interface	Cluster and administrative perimeter; homogeneity requirement; language semantics untouched

annexing or excluding them, because the medium’s edge otherwise becomes the new discontinuity (the imposition and tierless lessons). It must internalize the toolchain, because a continuous program built and shipped by discontinuous means has merely hidden a tier (the lesson every family teaches by omission). And it must keep faith with Waldo: continuity of expression, never pretense about physics. These four requirements shape Part II of this book. What they do not yet address is the second silent assumption named in Chapter 1: that computation stands still while data is delivered to it. That assumption has its own history of challenges and near-misses, which Appendix G surveys.

Bibliographic Notes

The portability lineage runs from the UNCOL proposal (Strong et al. 1958) through the Pascal-P implementation notes (Nori et al. 1981) to the JVM specification (Lindholm and Yellin 1999). Lattner and Adve (2004) and Haas et al. (2017) are the modern descendants at the compiler and web layers respectively, and Würthinger et al. (2013) is the definitive statement of the polyglot inversion. For multitier programming, the survey by Weisenburger et al. (2020) is the field’s map and the natural starting point. The primary sources remain worth reading in the original, both for the designs and for what each paper’s introduction admits about scope: Links (Cooper et al. 2006), Hop (Serrano et al. 2006), Ur/Web (Chlipala 2015), Eliom (Radanne et al. 2016), and ScalaLoci (Weisenburger et al. 2018).

On distribution, Birrell and Nelson (1984) is the founding engineering paper and still the clearest exposition of what stub-based transparency involves. Vinoski (1997) and Bernstein (1996) document the middleware industry it spawned, and Raymond (1995) introduces the standardized transparency taxonomy. The essential counterweight is Waldo et al. (1994), which we regard as required reading for anyone

proposing to unify local and remote computation (the authors of this book especially). For the single-system-image tradition, Tanenbaum et al. (1990) and Pike et al. (1995) describe the two most fully realized distributed operating systems in their designers' own voices, Barak and La'adan (1998) documents transparent process migration in production use, and Buyya et al. (2001) surveys the layer-by-layer design space. The synthesis offered in this appendix's final section, the partial-continuity trap and its four mechanisms, is our own reading of this record, and the sources above are the evidence on which the reader may check it.

A HISTORY OF MOBILE COMPUTATION

Conventional programming languages are growing ever more enormous, but not stronger.

John Backus, 1978

This appendix surveys the prior art behind the book's second thesis. It is the companion of Appendix F. In Chapter 1 we charge the von Neumann inheritance with two silent assumptions: that a program is a bounded unit whose semantics end at its own edge, and that computation stands still while data is carried to it. The first assumption has a history of attempted dissolutions, surveyed in Appendix F. That history is one of partial successes, each removing one discontinuity while fixing the others in place, and the theory of synechism in Part II builds on its lessons. This appendix compiles the record behind the second assumption, the record on which Part III's theory of topokinesis draws. That record has a stranger shape. The stationarity of computation was diagnosed sharply, and early, in a Turing Award lecture the field still quotes. Formal calculi then established that motion could be given rigorous semantics. Industrial systems established that computation could travel physical networks and survive the trip. Data systems established that topology could persist, and that computation could be sited on its elements. Every part of what this book calls topokinesis was discovered, several parts more than once, and some were shipped at scale.

What never occurred is the assembly. No general-purpose language made the mobile locus of computation a first-class semantic construct over a topology of data that persists. Mobility was always someone else's department: a calculus without a compiler, a middleware without semantics, an engine without a language. This appendix therefore reads its history as an inventory of parts, asking of each episode what it contributed and what substrate it lacked. The answers accumulate into a design brief, and Chapter 7 discharges it.

G.1 Backus's Turing Lecture and the von Neumann Bottleneck

John Backus delivered his Turing Award lecture in 1977 and published it the following August under a title that was itself a provocation: "Can Programming Be Liberated from the von Neumann Style?" (Backus 1978). The lecture is remembered as the founding polemic of functional programming, and its middle sections (the FP system, the algebra of programs) belong to that story. In this section, we take up a different part of it. Our concern is the opening move, which is a diagnosis, and the curious fact that the prescription that follows does not treat the disease the diagnosis names.

The machine Backus indicted was drawn up in the 1945 report on the EDVAC (Neumann 1945): a single store holding both data and instructions, an arithmetic organ where computation happens, a control organ sequencing it, and a channel between store and organ through which every word of a computation must pass, one at a time. The arrangement was economic before it was philosophical. Processing hardware was scarce and expensive. Memory was passive and comparatively cheap. It followed that a machine should have one site of computation and deliver data to it. Backus named the channel the von Neumann bottleneck. Had he meant only the hardware, the lecture would be a footnote, because caches, pipelines, and prefetchers were already softening the physical constraint. His claim was harsher: the tube is "an intellectual bottleneck that has kept us tied to word-at-a-time thinking," a habit of mind that survives intact in every layer built above the hardware.

The lecture's most durable observation is that conventional languages are not abstractions over the von Neumann machine so much as enlargements of it. A variable is a storage cell with a name. An assignment statement re-enacts the fetch-compute-store cycle. Control statements dress up the machine's test-and-jump instructions. Programming in such a language is, in Backus's telling, the planning of word traffic through the bottleneck, with the programmer as traffic controller for a tube. He called these von Neumann languages, and the epigraph of this appendix records his verdict on their trajectory.

What interests us is the asymmetry between diagnosis and cure. The disease, as named, has two symptoms: the mutable store, and the fixed site of computation the store feeds. Backus's prescription (functional programming, the elimination of assignment, an algebra of whole programs) attacks the store. The fixed site it does not attack. It abolishes the notion of site instead. A functional program executes nowhere in particular. Evaluation has no location, so nothing about location can be expressed, moved, or reasoned over. The imperative mainstream kept the fixed site and the store. The functional reaction kept neither. Between them lies a fork Backus did not name: keep the site of computation as a semantic concept, and let it move. The decades after 1978 explored that fork only in the margins, and this appendix is the story of those margins.

The habit, meanwhile, outlived its economics. Hardware architects spent the following decades mitigating the physical bottleneck (memory hierarchies, prefetching, out-of-order execution) while languages preserved the conceptual one. At datacenter scale the discipline rebuilt the original arrangement in the large. Data lakes feed stationary compute clusters, services stand still while payloads shuttle among them, and the cost of moving data to the site of computation dominates system design

at every tier. The premise that data travels and computation does not is enforced nowhere and assumed everywhere. It is an inheritance mistaken for a law of nature.

G.2 Actors, CSP, and Message Passing

This section takes up the concurrency tradition that answered the same moment, and committed the discipline more deeply to standing still. The same August 1978 issue of *Communications of the ACM* that carried Backus's lecture also carried Hoare's "Communicating Sequential Processes" (Hoare 1978). The coincidence is instructive: in the very issue where stationary computation was indicted, the discipline's canonical response to concurrency committed to it more deeply. Hoare proposed synchronized input and output as primitive commands, with a program a fixed parallel composition of named sequential processes exchanging values in unbuffered rendezvous. Five years earlier, Hewitt, Bishop, and Steiger had proposed the actor model (Hewitt et al. 1973), a universal formalism in which every computational entity is an actor reachable by address. Agha's systematization (Agha 1986) fixed the canon: on receiving a message, an actor may send messages, create new actors, and designate the behavior with which it will meet the next message.

Both models shatter the single von Neumann site into many, and it is tempting to read that as a loosening of the stationary habit. It is closer to the opposite. Each actor and each CSP process is a von Neumann machine in miniature: a private store, a fixed identity, a site to which data must be delivered before anything can happen. The store is partitioned and the site is multiplied, but no site moves. What moves is data, in messages, so the bottleneck is replicated: one small tube per mailbox or channel. Message passing is the von Neumann convention made distributed: compute stands still, and data travels, now in envelopes.

There is a topology in these systems (who holds whose address, which channels join which processes), but it lives in the values of variables, implicit and uninspectable, dissolving with the processes that hold it. Nothing in either model lets a program treat the communication graph as data, still less as data that persists. The shape of the system is an emergent side effect of its execution rather than an object of it.

None of this prevented success. It produced it. Erlang carried message passing into production telephony and demonstrated that stationary processes exchanging messages are a superb discipline for fault tolerance (Armstrong 2003), and the virtual actors of Orleans (Bykov et al. 2011) carried the model into cloud services. The success matters to our story because it normalized the pattern. By the 1990s, concurrency and distribution meant more stationary sites and more moving data, and the question of moving the computation itself had come to sound exotic.

G.3 Process Calculi with Location

In this section, we take up the calculi that gave motion a formal home, and we ask of each exactly what it sets moving. The word "mobile" entered the formal mainstream in 1992 with the calculus of mobile processes of Milner, Parrow, and Walker (Milner et al. 1992). The π -calculus is a genuine watershed, and what it sets moving is names. Channels are values. A process can send a channel over a channel, and the recipient

may thereafter communicate along what it received, the scope of the name extruding to meet its new holder. The topology of communication reconfigures dynamically, which is the calculus's great advance over CSP's fixed wiring. Processes themselves, though, go nowhere, and the calculus has no notion of a place to go to or from. Mobility in the π -calculus is rewiring: motion of linkage, not of locus.

The join calculus of Fournet and Gonthier (Fournet and Gonthier 1996) sharpened the account in the direction of implementation realism. By tying the reception of every name to a single defining site, it yielded a calculus a distributed machine could execute without global agreement about where a message should land, and its successors extended it with explicit locations and migration. Locality had become a semantic notion. The motion of computation was within reach of the theory.

The step was taken by Cardelli and Gordon's Mobile Ambients (Cardelli and Gordon 2000), which we regard as the closest formal ancestor of the paradigm this book develops. An ambient is a named, bounded place. It contains processes and other ambients, so the spatial structure of a system is a tree of nested places. The primitives are movements: the capability $\text{in } n$ moves the enclosing ambient inside a sibling named n , $\text{out } n$ moves it out of its parent, and $\text{open } n$ dissolves a boundary. Little else happens in the calculus: reduction is motion, and a step of computation is the crossing of a boundary by a located bundle of computation, carrying everything it contains. Where the π -calculus moved linkage, ambients move the computation itself through an explicit topology of places. The motivating images were administrative domains, firewalls, and portable devices. The ambient boundary models the very obstacle that the distribution transparencies of Appendix F pretended away.

From the vantage of this book, ambients are half of an answer. The topology an ambient moves through is the program. The tree of places is itself the reducing term, so structure and computation share one syntax, and when reduction pauses the spatial structure has no independent existence. Nothing persists, and nothing is data. There is no separation between a stable topology of information and the mobile loci of computation that traverse it. Everything is an ambient: elegance in a calculus, punishing as a programming surface. No production language carried the semantics to working programmers. The ambient calculus flourished as an analytical instrument, in typed variants and security analyses, while remaining essentially unprogrammable in the wild.

By 2000, then, the theory shelf was stocked: rigorous accounts of mobile linkage, of locality, and of located computation whose every step is movement through a topology. Formal possibility was settled. What was missing was not a calculus but a substrate: a language and runtime in which such semantics could govern ordinary programs over data that outlives them. Industry, as it happens, had been running that experiment, with instructive results.

G.4 Mobile Agents: The Rise and Fall of Traveling Computation

The idea of computation that travels did not wait for its calculi. As early as 1985, Gelernter's Linda (Gelernter 1985) performed a complementary inversion on the data side: in generative communication, a tuple, once emitted, exists in a shared space independently of its producer, persisting until some process withdraws it by pattern.

The eval primitive even deposits live computations into the space. Data at rest in a commons, decoupled in time and space from the parties that produce and consume it, is a first sketch of topology as habitat. Linda's space, though, is an unstructured bag, and the processes that reach into it stand as still as any actor.

The Emerald language (Black et al. 2007), built in the mid-1980s, is the most direct precedent for mobility as language semantics. Emerald objects, including active ones, move between machines under program control. Location is a concept of the language, and a parameter can be passed call-by-move, shipping the argument to the callee's site for the duration of the call. But an Emerald object moves in order to be invoked more cheaply on a local network. The model of computation remains invocation upon objects, motion is an optimization of placement rather than a paradigm of traversal, and the objects live no longer than the system that hosts them. A decade later, Cardelli's Obliq (Cardelli 1995) distilled the semantic core of migration itself: distributed lexical scope, under which a closure may travel to another site while its free identifiers remain bound, through network references, to their sites of origin. Migration acquired a meaning as clean as scoping.

The commercial wave believed in far more than placement optimization. Telescript (White 1997) was a complete, purpose-built language for a network imagined as a society of places. An agent executes go with a ticket naming a destination. Execution suspends. The agent (code, data, and execution state entire) travels. The same program resumes at its next instruction, elsewhere. Agents meet other agents at places, and permits issued by authorities meter what a visitor may do. White's economic argument anticipated the shape of the networked decade to come: remote procedure call generates traffic for every interaction, while remote programming sends the interaction loop itself to the data and returns only the outcome. IBM's Aglets (Lange and Oshima 1998) translated the vision into Java: an aglet is a serializable object dispatched between hosts, resuming not mid-stack but in an event handler on arrival. Mobility weakened into a library discipline. For a few years the mobile agent was to be the architecture of electronic commerce, with platforms, workshops, and interoperability standards to match.

The wave collapsed within a decade, and the customary explanations (a market unready, a pioneering vendor that folded, taking Telescript with it) are true but insufficient. Three deficits mattered more. First, with Telescript the exception, mobility was middleware bolted onto stationary languages. Java's semantics know nothing of place, so agenthood lived in serialization formats, class loaders, and transfer protocols. The language could not say what became of open files, running threads, or shared references when an agent departed, because departure was invisible to the language. Second, security. A mobile agent is code arriving from elsewhere and asking to run with local authority, and the 1990s host had no isolation story worthy of the request. Operators saw an automated intruder with a business card, and the fear ran in both directions, since the agent's owner had to entrust code and accumulated state to every host on the itinerary. Telescript's permits and authorities sketched a genuine answer, but the sketch died with the platform. Third, least remarked and in our reading most decisive, there was no persistence story. An agent traveled to a machine, not to data. Arrival purchased a process slot, while the information the agent came for still lived behind the same database interfaces any remote client faces.

Mobility saved round trips but dissolved no discontinuities: the fragmented stack of Chapter 1 stood intact at every destination.

The web's stateless request to a stationary server won the decade. The one unambiguous mass deployment of mobile code (scripts delivered to browsers) moves code a single hop, in a single direction, into an aggressively confined interpreter, and is never called agent technology. But the mobile-agent episode should not be filed as a refuted idea. It is an idea that arrived before its substrate. The itinerant locus of computation had no persistent topology to travel, no isolation model to make arrival safe, and no language in which its motion was part of the semantics. Each of these is a property a substrate can supply, and Chapter 10 and Chapter 14 will supply them.

G.5 Thinking Like a Vertex: Traversal as Trapped Topokinesis

While the agent wave receded, the data systems community was rediscovering the other half of the synthesis. Graph data models have a lineage reaching back to the network and semantic models of the 1980s, surveyed by Angles and Gutiérrez (2008) at the moment of their resurgence. By the late 2000s property-graph stores had made persistent topology (nodes and edges as durable, first-class objects) an ordinary storage abstraction. The shape of the data could now outlive every program that touched it. It could not, however, be inhabited. The topology was a citizen of the database, not of any programming language, and computation reached it as queries across a connection.

Pregel (Malewicz et al. 2010) located computation on the topology itself. In its bulk-synchronous model a program is written from the perspective of a single vertex (the discipline summarized in the slogan “think like a vertex”), executing a compute function at each superstep, exchanging messages along its edges, and voting to halt. This is computation sited at the elements of a topology, with locality made programmable: a vertex sees its own state and its own edges, and the framework carries the consequences. For whole-graph analytics the model proved durable, spawning a family of vertex-centric systems. The model also runs in a cage. The vertex program is a callback surrendered to a batch engine. It exists only inside the framework, for the duration of a job, over a graph loaded for the purpose. Nothing about vertexhood is available to the surrounding language between jobs. Thinking like a vertex was possible only while the engine was thinking.

Gremlin (Rodriguez 2015) comes nearer still to the missing paradigm. Rodriguez presents it as a graph traversal machine as much as a language: a traversal is a swarm of traversers (small mobile loci, each carrying its own state) propagating over the graph under the direction of a pipeline of steps, and the machine is presented as computationally universal. This is motion of computation through persistent topology, named as such and shipped in production systems. It is also, decisively, a query language. A Gremlin traversal is embedded in a host program as a fluent expression or a string and executed on the far side of a driver. The traverser cannot carry the application's own abstractions (its types, its effects, its libraries) along on the walk, and whatever the traversers discover is serialized back across the very discontinuity the traversal was meant to spare us. Cypher (Francis et al. 2018), the declarative language grown at Neo4j, chose pattern over path: `MATCH` describes a subgraph, and traversal is

the optimizer's private affair. That is the right choice for a query language, and that is the point: traversal semantics lived inside engines, on the far side of a border.

The situation by the mid-2010s is worth stating plainly, because it is strange. Both halves of the missing synthesis now ran in production at scale: persistent topology in graph stores, and located, moving computation in vertex programs and traversers. But they ran only inside engines. Topokinesis existed as captive semantics: available only through a connection string, expressed in a guest language, and paid for at the application-database boundary whose costs Chapter 1 itemized. A programmer who wished to compute by traversal had to leave her language to do it, and to marshal everything learned back across the border on return. The paradigm was not missing. It was confined.

G.6 The Missing Synthesis

We can now assemble the inventory this history has produced.

1. Stationarity is a habit, not a law. Backus named the bottleneck as intellectual rather than physical in 1977; the habit has outlived every hardware rationale that produced it.
2. Multiplying sites is not mobility. Actors and CSP distribute the bottleneck without dissolving it, and their success normalized the convention that data moves while compute stands still.
3. Motion has rigorous semantics. The π -calculus formalized mobile linkage; Mobile Ambients formalized located computation whose every reduction step is movement through a topology of places.
4. Motion is implementable and economically motivated. Emerald moved objects under language semantics; Telescript moved executing programs, execution state included, across real networks, with an argument for sending the interaction to the data.
5. Topology can persist, and computation can live on its elements, but only inside engines. Graph stores made the shape of data durable; Pregel and Gremlin sited and moved computation over it, behind a query interface.

Every part exists, and the reason the assembly never happened is now visible: each artifact possessed one part and lacked a substrate for the rest. The calculi had semantics without persistence or production embodiment. The agents had motion without persistence, isolation, or language status: middleware riding on stationary semantics. The engines had persistence and located motion without generality: semantics in captivity behind a discontinuity. The missing artifact is a general-purpose language of three properties. In it, the topology of data is a first-class, persistent structure of the language itself. In it, the locus of computation is a first-class, mobile construct whose motion is a semantic primitive. And in it, arrival triggers computation, with the substrate obligations the agents died for want of (durability

and isolation and distribution among them) discharged by the runtime rather than delegated to middleware.

That is the design brief for a topokinetic language, and this book's answer is Object-Spatial Programming: nodes and edges as archetypes of the language, walkers as the mobile loci of computation, abilities as computation triggered by arrival. The paradigm is presented informally in Chapter 7, where the term topokinetic receives its formal definition. The operational semantics its formal ancestors would demand appears in Chapter 8. The property whose absence recurs in every episode above, a topology that outlives the program, is supplied in Chapter 10. One condition precedes all of this. Mobility cannot live on a fractured substrate: a walker that halts at an ecosystem border, or serializes itself across a marshaling layer to reach the next tier, is a 1990s agent by another name. This survey and its companion therefore converge on a single requirement: the continuity whose pursuit Appendix F chronicles is what motion needs to survive, and motion is much of what that continuity is for. Part II, beginning with Chapter 2, builds the continuous medium first. Part III sets computation moving through it.

Bibliographic Notes

The diagnosis is best read at the source. Backus (1978) rewards rereading with the mobility question in mind, alongside the EDVAC report (Neumann 1945) whose economics hardened into the habit Backus indicts. The same journal issue carries Hoare (1978), making the two poles of the era's debate available side by side. For the actor lineage, the original formulation of Hewitt et al. (1973) is completed by the systematic treatment of Agha (1986). Armstrong (2003) documents the model's industrial vindication in Erlang, and Bykov et al. (2011) its managed-cloud descendant.

The located calculi are compact and readable. Milner et al. (1992) sets out the mobility of names. Fournet and Gonthier (1996) grounds reception in locality. Cardelli and Gordon (2000) makes movement through a topology of places the sole engine of computation, and remains the closest formal ancestor of the paradigm developed in Part III. Cardelli (1995) bridges calculus and language with distributed lexical scope. On the agent wave, White (1997) states the vision of places and traveling agents in its original voice. Lange and Oshima (1998) records the Java translation. Black et al. (2007) is the retrospective account of the language that made object mobility a semantic matter two decades ahead of demand. Gelernter (1985) supplies the data-side inversion of the same period.

For trapped topokinesis, Angles and Gutiérrez (2008) surveys graph data models at the hinge of their revival. Malewicz et al. (2010) defines vertex-centric computation. Rodriguez (2015) describes the traversal machine whose traversers come nearest to walkers among prior systems. Francis et al. (2018) documents the declarative alternative. The paradigm this history anticipates is specified in Mars (2025) and developed from Chapter 7 onward.

REVISION HISTORY

This book is released in numbered editions, and this appendix is the ledger of what changed between them. Its audience is the reader of a prior edition who wants to know what to revisit, so the entries record what such a reader would notice: chapters added or renumbered, terms coined or renamed, sections added or retitled, results that moved from protocol to measurement. Typographical corrections and line-level prose revisions are not itemized; the complete history lives with the book's source. Every released edition is archived on Zenodo with a document identifier of its own, and the concept identifier doi:10.5281/zenodo.21464040 resolves permanently to the newest, so each entry below can be checked against the exact text it describes.

H.1 Edition 1.1.0 (July 2026)

The edition in hand.

- **New chapter: topology-aligned regions.** Chapter 11, “Transience by Ownership: The Region Is the Subgraph,” joins Part III as the transient complement to persistence by reachability: first-class region handles and opens, the growth rule, the connect membrane, borrow-collapse escape checking, and sendable subgraphs, with the construct located in the region tradition and measured in Chapter 16. The insertion renumbers the chapters after it: Chapters 11 through 18 of edition 1.0.0 are Chapters 12 through 19 here. The reading map and the dependency notes of How to Read This Book were updated to match.
- **Renamed coined term: gradual borrow checking.** What edition 1.0.0 called *gradual ownership* (the chapter title of Chapter 5, its defining termdef, and the entries of the lexicon and Appendix C) is now *gradual borrow checking*. The rename makes the chapter's own precision claim typographically visible: the literature's one prior use of “gradual ownership” gradualizes the aliasing discipline of ownership types, while this construct gradualizes the affine discipline a borrow checker enforces, the question of when memory dies. Contract-flavored wording

in the chapter (“contract position,” “the contract alternative”) was revised to interface and guarantee wording at the same time.

- **The self-hosting frontier.** Chapter 4 now divides the artifact’s payload into components written in Jac and external engines carried in binary form, names the division the *self-hosting frontier*, adds a constitution column to the anatomy table and a figure drawing the frontier through the payload, and states the design goal of moving every component above the operating system across it. The frontier is taken up where it bears: Chapter 12 distinguishes the frontier from the compiler’s own self-hosted fraction, Chapter 5 identifies the fourth dial position as the mechanism the remaining crossings require, Appendix D dates the crossings to date, and the closing of Chapter 19 applies the synechistic ethic to the language’s own implementation through it.
- **Project kinds.** Chapter 4 gains a section assembling the toolchain’s product space into a catalog: twelve project kinds, from command-line tool to mobile application, factored as compositions of the three codespaces under four delivery treatments, with kind inference from the entry point and the near-empty scaffolds that follow.
- **The design space of polypilation.** Chapter 12 gains a section mapping the space the polypiler occupies: six decisions every polypiler must make (node-set design, fact placement, per-target typing, the semantic contract, boundary reification, the delivery unit), the assignment this compiler chose at each, and the alternatives a successor could take, collected in a table. The two-camps analysis earlier in the chapter now reads React Server Components as the membership camp reaching for the split from the framework side.
- **The native topology store.** Chapter 13 now documents the native substrate’s topology store: a per-node adjacency index over flat integer arrays, replacing an original global edge table whose linear scans the region benchmark suite exposed at full size, with row retraction coupled to region reclamation so the index follows the live graph rather than the graph’s history. The walker kernel’s dispatch schedule and the persistence engine’s commit-and-replay path each gain a figure, and the WalkScope listing gains the queue cursor the kernel actually uses.
- **First measured benchmark family.** The evaluation suite of Chapter 16 grows from six families to seven, and one family arrives measured: the topology-lifetime family reports final numbers over two tables and two plots (build-and-discard time, footprint plateau against unbounded growth, drop and reclamation counts, and constant-time subgraph transfer against copies that grow with the subgraph), under the mechanical-erasure protocol the section defines. The remaining families keep their *[to be measured]* cells.
- **The ledger, settled.** Chapter 19 gains a table reading the eight numbered crossings of Figure 1.1 back line by line: the mechanism that dissolves each crossing, and the lawful residue (latency, partial failure, model cost) left by the three that were facts of the world rather than accidents of description.

- **Advice to successors, extended.** The commitments of Chapter 18 gain two entries: gradualize every discipline the medium internalizes, and absorb the language-shaped components of the toolchain first while carrying the hardened engines visibly. The open problems gain the minimal fact set a shared representation must carry and the artifact-floor question the self-hosting frontier poses.
- **Seventeen new figures.** Fifteen across existing chapters (the three ecosystem doorways of Chapter 3; the delegation ladder of Chapter 6; the edge-reference anatomy and the turnstile of Chapter 7; declared endpoints in Chapter 9; the ownership membrane of Chapter 5; the self-hosting frontier of Chapter 4; codespace pull in Chapter 12; walker dispatch, the topology store, and commit-and-replay in Chapter 13; the dual motions of Chapter 14; the production twins of Chapter 15; and the transfer and footprint plots of Chapter 16) and two in the new Chapter 11.
- **Reference matter.** The lexicon and Appendix C add entries for topology-aligned regions (a coined term, bringing the count from eight to nine), the growth rule, borrow-collapse, the connect membrane, sendable subgraphs, and the self-hosting frontier. The bibliography adds twelve sources: the region tradition (Tofte and Talpin, the MLKit retrospective, Cyclone, Gay and Aiken), arena practice and graph-system allocation, the reference-counting and cycle-collection literature, and React Server Components.
- **Wording.** A recurring reliance on “honest” and “honesty” as evaluative shorthand was retired in favor of the specific property each use meant (disclosure, rigor, completeness, physical limits). The most visible instance: the section of Chapter 14 titled “The Honest Limits: Latency, Failure, and Cost” in edition 1.0.0 is now “The Physical Limits: Latency, Failure, and Cost.”

H.2 Edition 1.0.0 (July 2026)

First public release: eighteen chapters in six parts, seven appendices, 308 pages. Archived July 21, 2026, as doi:10.5281/zenodo.21464041.

BIBLIOGRAPHY

- Agha, G. (1986). *Actors: A Model of Concurrent Computation in Distributed Systems*. MIT Press.
- Angles, R. and C. Gutiérrez (2008). “Survey of Graph Database Models”. In: *ACM Computing Surveys* 40.1, 1:1–1:39.
- Armstrong, J. (2003). “Making Reliable Distributed Systems in the Presence of Software Errors”. PhD thesis. Royal Institute of Technology (KTH), Stockholm.
- Arvidsson, E., E. Castegren, S. Clebsch, S. Drossopoulou, J. Noble, M. J. Parkinson, and T. Wrigstad (2023). “Reference Capabilities for Flexible Memory Management”. In: *Proceedings of the ACM on Programming Languages* 7.OOPSLA2, pp. 1363–1393. doi: [10.1145/3622846](https://doi.org/10.1145/3622846).
- Atkinson, M. P., P. J. Bailey, K. J. Chisholm, P. W. Cockshott, and R. Morrison (1983). “An Approach to Persistent Programming”. In: *The Computer Journal* 26.4, pp. 360–365.
- Atkinson, M. P. and R. Morrison (1995). “Orthogonally Persistent Object Systems”. In: *The VLDB Journal* 4.3, pp. 319–401.
- Backus, J. (1978). “Can Programming Be Liberated from the von Neumann Style? A Functional Style and Its Algebra of Programs”. In: *Communications of the ACM* 21.8, pp. 613–641.
- Bacon, D. F. and V. T. Rajan (2001). “Concurrent Cycle Collection in Reference Counted Systems”. In: *ECOOP 2001 — Object-Oriented Programming, 15th European Conference*. Vol. 2072. Lecture Notes in Computer Science. Springer, pp. 207–235. doi: [10.1007/3-540-45337-7_12](https://doi.org/10.1007/3-540-45337-7_12).
- Barak, A. and O. La’adan (1998). “The MOSIX Multicomputer Operating System for High Performance Cluster Computing”. In: *Future Generation Computer Systems* 13.4–5, pp. 361–372.
- Berger, E. D., B. G. Zorn, and K. S. McKinley (2002). “Reconsidering Custom Memory Allocation”. In: *Proceedings of the 17th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA 2002)*. ACM, pp. 1–12. doi: [10.1145/582419.582421](https://doi.org/10.1145/582419.582421).
- Bernstein, P. A. (1996). “Middleware: A Model for Distributed System Services”. In: *Communications of the ACM* 39.2, pp. 86–98.
- Bierman, G., M. Abadi, and M. Torgersen (2014). “Understanding TypeScript”. In: *Proceedings of the 28th European Conference on Object-Oriented Programming (ECOOP)*, pp. 257–281.

- Birrell, A. D. and B. J. Nelson (1984). “Implementing Remote Procedure Calls”. In: *ACM Transactions on Computer Systems* 2.1, pp. 39–59.
- Black, A. P., N. C. Hutchinson, E. Jul, and H. M. Levy (2007). “The Development of the Emerald Programming Language”. In: *Proceedings of the Third ACM SIGPLAN Conference on History of Programming Languages (HOPL III)*.
- Brewer, E. (2012). “CAP Twelve Years Later: How the “Rules” Have Changed”. In: *IEEE Computer* 45.2, pp. 23–29.
- Brooks, F. P. (1975). *The Mythical Man-Month: Essays on Software Engineering*. Addison-Wesley.
- (1987). “No Silver Bullet: Essence and Accidents of Software Engineering”. In: *IEEE Computer* 20.4, pp. 10–19.
- Brown, T. B. et al. (2020). “Language Models are Few-Shot Learners”. In: *Advances in Neural Information Processing Systems* 33 (*NeurIPS*).
- Burns, B., B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes (2016). “Borg, Omega, and Kubernetes”. In: *Communications of the ACM* 59.5, pp. 50–57.
- Buyya, R., T. Cortes, and H. Jin (2001). “Single System Image”. In: *International Journal of High Performance Computing Applications* 15.2, pp. 124–135.
- Bykov, S., A. Geller, G. Kliot, J. R. Larus, R. Pandya, and J. Thelin (2011). “Orleans: Cloud Computing for Everyone”. In: *Proceedings of the 2nd ACM Symposium on Cloud Computing (SoCC)*.
- Cardelli, L. (1995). “A Language with Distributed Scope”. In: *Computing Systems* 8.1, pp. 27–59.
- Cardelli, L. and A. D. Gordon (2000). “Mobile Ambients”. In: *Theoretical Computer Science* 240.1, pp. 177–213.
- Chlipala, A. (2015). “Ur/Web: A Simple Model for Programming the Web”. In: *Proceedings of the 42nd ACM SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pp. 153–165.
- Clarke, D. G., J. M. Potter, and J. Noble (1998). “Ownership Types for Flexible Alias Protection”. In: *Proceedings of the 13th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pp. 48–64.
- Clebsch, S., J. Franco, S. Drossopoulou, A. M. Yang, T. Wrigstad, and J. Vitek (2017). “Orca: GC and Type System Co-Design for Actor Languages”. In: *Proceedings of the ACM on Programming Languages* 1.OOPSLA. doi: [10.1145/3133896](https://doi.org/10.1145/3133896).
- Codd, E. F. (1970). “A Relational Model of Data for Large Shared Data Banks”. In: *Communications of the ACM* 13.6, pp. 377–387.
- Cooper, E., S. Lindley, P. Wadler, and J. Yallop (2006). “Links: Web Programming Without Tiers”. In: *Proceedings of the 5th International Symposium on Formal Methods for Components and Objects (FMCO)*, pp. 266–296.
- Dantanarayana, J. L., Y. Kang, K. Sivasothyathan, et al. (2025). “MTP: A Meaning-Typed Language Abstraction for AI-Integrated Programming”. In: *Proceedings of the ACM on Programming Languages* 9.OOPSLA.
- Dijkstra, E. W. (1972). “The Humble Programmer”. In: *Communications of the ACM* 15.10, pp. 859–866.
- Felleisen, M. (1991). “On the Expressive Power of Programming Languages”. In: *Science of Computer Programming* 17.1–3, pp. 35–75.

- Fournet, C. and G. Gonthier (1996). “The Reflexive CHAM and the Join-Calculus”. In: *Proceedings of the 23rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pp. 372–385.
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
- Francis, N., A. Green, P. Guagliardo, L. Libkin, T. Lindaaaker, V. Marsault, S. Plantikow, M. Rydberg, P. Selmer, and A. Taylor (2018). “Cypher: An Evolving Query Language for Property Graphs”. In: *Proceedings of the 2018 ACM SIGMOD International Conference on Management of Data*, pp. 1433–1445.
- Gay, D. and A. Aiken (1998). “Memory Management with Explicit Regions”. In: *Proceedings of the ACM SIGPLAN 1998 Conference on Programming Language Design and Implementation (PLDI 1998)*. ACM, pp. 313–323. doi: [10.1145/277650.277748](https://doi.org/10.1145/277650.277748).
- Gelernter, D. (1985). “Generative Communication in Linda”. In: *ACM Transactions on Programming Languages and Systems* 7.1, pp. 80–112.
- Gilbert, S. and N. Lynch (2002). “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services”. In: *ACM SIGACT News* 33.2, pp. 51–59.
- Google (2006). GWT: the Google Web Toolkit. <https://www.gwtproject.org/>.
- (2025). The Dart Programming Language. <https://dart.dev/>.
- Gray, J. and A. Reuter (1993). *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann.
- Grossman, D., G. Morrisett, T. Jim, M. Hicks, Y. Wang, and J. Cheney (2002). “Region-Based Memory Management in Cyclone”. In: *Proceedings of the ACM SIGPLAN 2002 Conference on Programming Language Design and Implementation (PLDI 2002)*. ACM, pp. 282–293. doi: [10.1145/512529.512563](https://doi.org/10.1145/512529.512563).
- Haas, A., A. Rossberg, D. L. Schuff, B. L. Titzer, M. Holman, D. Gohman, L. Wagner, A. Zakai, and J. Bastien (2017). “Bringing the Web up to Speed with WebAssembly”. In: *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pp. 185–200.
- Hellerstein, J. M. and P. Alvaro (2020). “Keeping CALM: When Distributed Consistency Is Easy”. In: *Communications of the ACM* 63.9, pp. 72–81.
- Hewitt, C., P. Bishop, and R. Steiger (1973). “A Universal Modular ACTOR Formalism for Artificial Intelligence”. In: *Proceedings of the 3rd International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 235–245.
- Hoare, C. A. R. (1978). “Communicating Sequential Processes”. In: *Communications of the ACM* 21.8, pp. 666–677.
- (1981). “The Emperor’s Old Clothes”. In: *Communications of the ACM* 24.2, pp. 75–83.
- JetBrains (2025). *Kotlin Multiplatform*. <https://kotlinlang.org/docs/multiplatform.html>.
- Jonas, E. et al. (2019). *Cloud Programming Simplified: A Berkeley View on Serverless Computing*. Tech. rep. UCB/EECS-2019-3. UC Berkeley EECS.
- Jung, R., J.-H. Jourdan, R. Krebbers, and D. Dreyer (2018). “RustBelt: Securing the Foundations of the Rust Programming Language”. In: *Proceedings of the ACM on Programming Languages* 2.POPL, 66:1–66:34.
- Kay, A. C. (1993). “The Early History of Smalltalk”. In: *ACM SIGPLAN Notices* 28.3, pp. 69–95.

- Kiczales, G., J. Lamping, A. Mendhekar, C. Maeda, C. Lopes, J.-M. Loingtier, and J. Irwin (1997). "Aspect-Oriented Programming". In: *Proceedings of the 11th European Conference on Object-Oriented Programming (ECOOP)*, pp. 220–242.
- Lamport, L. (1978). "Time, Clocks, and the Ordering of Events in a Distributed System". In: *Communications of the ACM* 21.7, pp. 558–565.
- Lange, D. B. and M. Oshima (1998). *Programming and Deploying Java Mobile Agents with Aplets*. Addison-Wesley.
- Lattner, C. and V. Adve (2004). "LLVM: A Compilation Framework for Lifelong Program Analysis and Transformation". In: *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*, pp. 75–86.
- Lindholm, T. and F. Yellin (1999). *The Java Virtual Machine Specification*. 2nd ed. Addison-Wesley.
- Malewicz, G., M. H. Austern, A. J. C. Bik, J. C. Dehnert, I. Horn, N. Leiser, and G. Czajkowski (2010). "Pregel: A System for Large-Scale Graph Processing". In: *Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data*, pp. 135–146.
- Manolescu, D., B. Beckman, and B. Livshits (2008). "Volta: Developing Distributed Applications by Recompiling". In: *IEEE Software* 25.5, pp. 53–59.
- Mars, J. (2021). *What's In Store for the Next Generation of AI? The Jaseci Perspective*. Forbes. URL: <https://www.forbes.com/sites/forbesbooksauthors/2021/05/18/whats-in-store-for-the-next-generation-of-ai-the-jaseci-perspective/>.
- (2022). *The Case for a Wholistic Serverless Programming Paradigm and Full Stack Automation for AI and Beyond: The Philosophy of Jaseci and Jac*. arXiv:2206.08434.
- Mars, J. et al. (2023). "The Jaseci Programming Paradigm and Runtime Stack: Building Scale-Out Production Applications Easy and Fast". In: *IEEE Computer Architecture Letters* 22.2.
- Mars, J. (2025). *Object-Spatial Programming*. arXiv:2503.15812.
- Mars, J., Y. Kang, J. Dantanarayana, C. Irugalbandara, K. Sivasothyathan, and L. Tang (2024). *LLMs are Meaning-Typed Code Constructs*. arXiv:2405.08965v1.
- Matsakis, N. D. and F. S. Klock II (2014). "The Rust Language". In: *Proceedings of the 2014 ACM SIGAda Annual Conference on High Integrity Language Technology (HILT)*, pp. 103–104.
- McIlroy, M. D. (1968). "Mass Produced Software Components". In: *Software Engineering: Report of a Conference Sponsored by the NATO Science Committee*. Garmisch, pp. 138–155.
- Meteor Software (2012). *Meteor: A Full-Stack JavaScript Platform*. <https://www.meteor.com/>.
- Meyer, B. (1992). "Applying "Design by Contract"". In: *IEEE Computer* 25.10, pp. 40–51.
- Miller, M. S. (2006). "Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control". PhD thesis. Johns Hopkins University.
- Milner, R., J. Parrow, and D. Walker (1992). "A Calculus of Mobile Processes, I and II". In: *Information and Computation* 100.1, pp. 1–77.
- Neumann, J. von (1945). *First Draft of a Report on the EDVAC*. Tech. rep. Moore School of Electrical Engineering, University of Pennsylvania.

- Nori, K. V., U. Ammann, K. Jensen, H.-H. Nägeli, and C. Jacobi (1981). “Pascal-P Implementation Notes”. In: *Pascal: The Language and its Implementation*. Ed. by D. W. Barron. John Wiley.
- OpenAI (2023). *GPT-4 Technical Report*. arXiv:2303.08774.
- Osvald, L., G. Essertel, X. Wu, L. I. González Alayón, and T. Rompf (2016). “Gentrification Gone Too Far? Affordable 2nd-Class Values for Fun and (Co-)Effect”. In: *Proceedings of the ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pp. 234–251.
- Peirce, C. S. (1892). “The Law of Mind”. In: *The Monist* 2.4, pp. 533–559.
- Perlis, A. J. (1982). “Epigrams on Programming”. In: *ACM SIGPLAN Notices* 17.9, pp. 7–13.
- Pike, R., D. Presotto, S. Dorward, B. Flandrena, K. Thompson, H. Trickey, and P. Winterbottom (1995). “Plan 9 from Bell Labs”. In: *Computing Systems* 8.3, pp. 221–254.
- Plotkin, G. D. (1981). *A Structural Approach to Operational Semantics*. Tech. rep. DAIMI FN-19. Aarhus University.
- Radanne, G., J. Vouillon, and V. Balat (2016). “Eliom: A Core ML Language for Tierless Web Programming”. In: *Proceedings of the 14th Asian Symposium on Programming Languages and Systems (APLAS)*, pp. 377–397.
- Raymond, K. (1995). “Reference Model of Open Distributed Processing (RM-ODP): Introduction”. In: *Open Distributed Processing: Experiences with Distributed Environments*. Springer, pp. 3–14.
- React Team (2020). *Introducing Zero-Bundle-Size React Server Components*. Meta Open Source. URL: <https://legacy.reactjs.org/blog/2020/12/21/data-fetching-with-react-server-components.html>.
- Reinking, A., N. Xie, L. de Moura, and D. Leijen (2021). “Perceus: Garbage Free Reference Counting with Reuse”. In: *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*, pp. 96–111.
- Rodriguez, M. A. (2015). “The Gremlin Graph Traversal Machine and Language”. In: *Proceedings of the 15th Symposium on Database Programming Languages (DBPL)*, pp. 1–10.
- Sergey, I. and D. Clarke (2012). “Gradual Ownership Types”. In: *Proceedings of the 21st European Symposium on Programming (ESOP)*, pp. 579–599.
- Serrano, M., E. Galesio, and F. Loitsch (2006). “Hop: A Language for Programming the Web 2.0”. In: *Companion to the 21st ACM SIGPLAN Symposium on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*.
- Shun, J. and G. E. Blelloch (2013). “Ligra: A Lightweight Graph Processing Framework for Shared Memory”. In: *Proceedings of the 18th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP 2013)*. ACM, pp. 135–146. doi: [10.1145/2442516.2442530](https://doi.org/10.1145/2442516.2442530).
- Siek, J. G. and W. Taha (2006). “Gradual Typing for Functional Languages”. In: *Proceedings of the Scheme and Functional Programming Workshop*, pp. 81–92.
- Siek, J. G., M. M. Vitousek, M. Cimini, and J. T. Boyland (2015). “Refined Criteria for Gradual Typing”. In: *1st Summit on Advances in Programming Languages (SNAPL)*, pp. 274–293.

- Spolsky, J. (2002). *The Law of Leaky Abstractions*. URL: <https://www.joelonsoftware.com/2002/11/11/the-law-of-leaky-abstractions/>.
- Steele, G. L. (1999). "Growing a Language". In: *Higher-Order and Symbolic Computation* 12.3, pp. 221–236.
- Stoldt, F. P., G. B. Bucher II, S. Clebsch, M. A. Johnson, M. J. Parkinson, G. van Rossum, E. Snow, and T. Wrigstad (2025). "Dynamic Region Ownership for Concurrency Safety". In: *Proceedings of the ACM on Programming Languages* 9.PLDI, pp. 1565–1590. doi: [10.1145/3729313](https://doi.org/10.1145/3729313).
- Strong, J., J. Wegstein, A. Tritter, J. Olsztyn, O. Mock, and T. Steel (1958). "The Problem of Programming Communication with Changing Machines: A Proposed Solution". In: *Communications of the ACM* 1.8, pp. 12–18.
- Takikawa, A., D. Feltey, B. Greenman, M. S. New, J. Vitek, and M. Felleisen (2016). "Is Sound Gradual Typing Dead?" In: *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pp. 456–468.
- Tanenbaum, A. S., R. van Renesse, H. van Staveren, G. J. Sharp, S. J. Mullender, J. Jansen, and G. van Rossum (1990). "Experiences with the Amoeba Distributed Operating System". In: *Communications of the ACM* 33.12, pp. 46–63.
- Tofte, M., L. Birkedal, M. Elsmann, and N. Hallenberg (2004). "A Retrospective on Region-Based Memory Management". In: *Higher-Order and Symbolic Computation* 17.3, pp. 245–265. doi: [10.1023/B:LISP.0000029446.78563.a4](https://doi.org/10.1023/B:LISP.0000029446.78563.a4).
- Tofte, M. and J.-P. Talpin (1997). "Region-Based Memory Management". In: *Information and Computation* 132.2, pp. 109–176. doi: [10.1006/inco.1996.2613](https://doi.org/10.1006/inco.1996.2613).
- Unison Computing (2024). *The Unison Programming Language*. URL: <https://www.unison-lang.org>.
- van Kempen, N. and E. D. Berger (2026). "Reconsidering Custom Memory Allocation". In: *Proceedings of the 2026 ACM SIGPLAN International Symposium on Memory Management (ISMM 2026)*. ACM, pp. 14–24. doi: [10.1145/3814942.3816132](https://doi.org/10.1145/3814942.3816132).
- Vaswani, A., N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin (2017). "Attention Is All You Need". In: *Advances in Neural Information Processing Systems* 30 (NeurIPS).
- Vinoski, S. (1997). "CORBA: Integrating Diverse Applications Within Distributed Heterogeneous Environments". In: *IEEE Communications Magazine* 35.2, pp. 46–55.
- Vogels, W. (2009). "Eventually Consistent". In: *Communications of the ACM* 52.1, pp. 40–44.
- Waldo, J., G. Wyant, A. Wollrath, and S. Kendall (1994). *A Note on Distributed Computing*. Tech. rep. SMLI TR-94-29. Sun Microsystems Laboratories.
- Weisenburger, P., M. Köhler, and G. Salvaneschi (2018). "Distributed System Development with ScalaLoc". In: *Proceedings of the ACM on Programming Languages* 2.OOPSLA, 129:1–129:30.
- Weisenburger, P., J. Wirth, and G. Salvaneschi (2020). "A Survey of Multitier Programming". In: *ACM Computing Surveys* 53.4, 81:1–81:35.
- White, J. E. (1997). "Mobile Agents". In: *Software Agents*. Ed. by J. M. Bradshaw. AAAI Press / MIT Press.
- Wirth, N. (1995). "A Plea for Lean Software". In: *IEEE Computer* 28.2, pp. 64–68.

- Wright, A. K. and M. Felleisen (1994). “A Syntactic Approach to Type Soundness”. In: *Information and Computation* 115.1, pp. 38–94.
- Würthinger, T., C. Wimmer, A. Wöß, L. Stadler, G. Duboscq, C. Humer, G. Richards, D. Simon, and M. Wolczko (2013). “One VM to Rule Them All”. In: *Proceedings of the 2013 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)*, pp. 187–204.



A SYNECHIC AND TOPOKINETIC PROGRAMMING LANGUAGE

For seventy years, programming languages have inherited two silent assumptions from the von Neumann tradition. The first is that a program's semantics end at the edge of its process, so that frontend and backend, managed and native, script and service are separate programs, joined by glue code, marshaling layers, and toolchains the programmer must operate by hand. The second is that computation stands still while data is delivered to it. This book presents Jac, a production language that discards both assumptions, and develops the theory, design, and implementation of the two language classes it defines: the *synechic* languages, which present a single continuous semantic medium across the substrates convention treats as separate, and the *topokinetic* languages, in which the locus of computation moves, as a first-class construct, through a topology of data. The two classes are independent, and Jac is the first language that is a member of both.

The book draws on the language's evolution across a decade of research and production deployment. Jac is its existence proof: one production system in which both inherited assumptions are discarded at once. It is written not just for programming-language researchers and graduate students, but also systems builders, and passionate developers of every kind who want a rigorous account of what programming looks like when the discontinuities are removed.

The Jac language and the Jaseci runtime stack are open source.

github.com/jaseci-labs/jac

